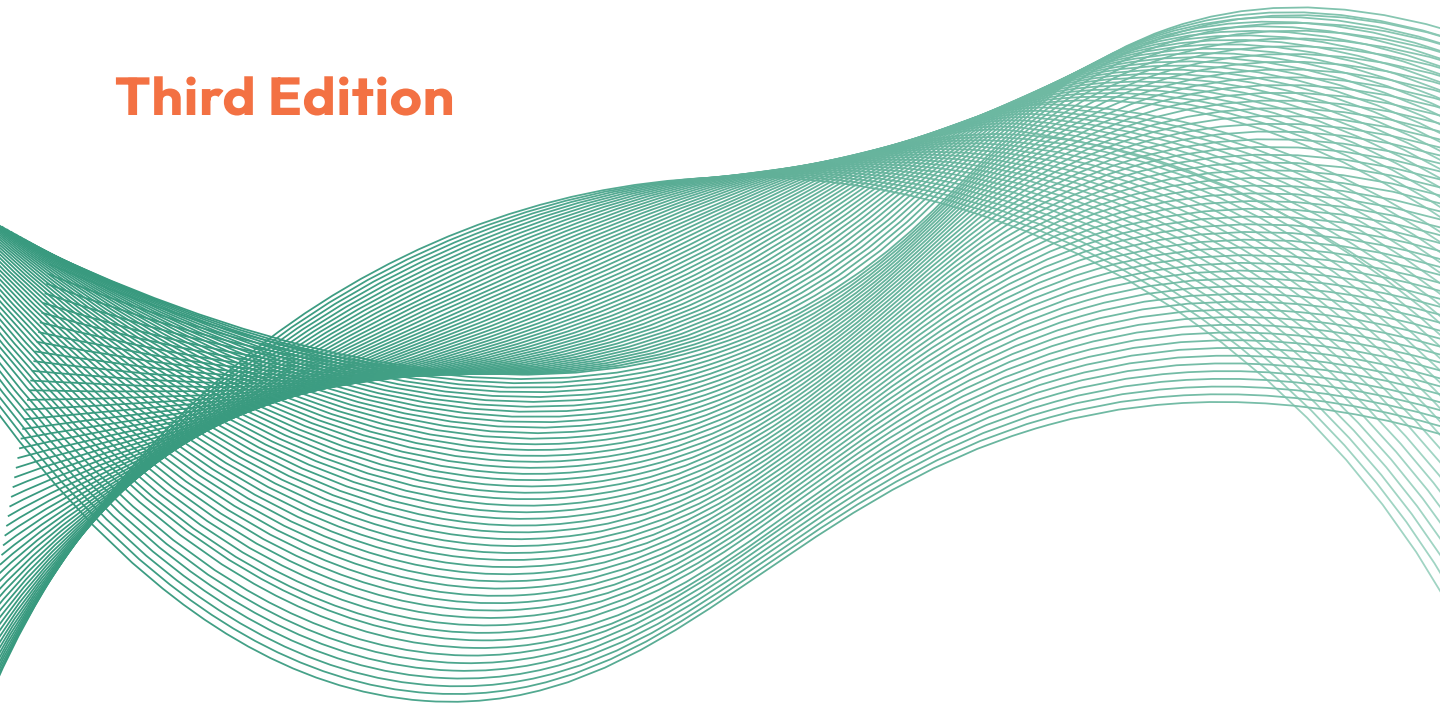


EXPERT INSIGHT

Web Development with Django 6

**A definitive guide to building modern
Python web applications using Django 6**

Third Edition



Chris Guest | Mark Walker | Ben Shaw
Saurabh Badhwar | Bharath Chandra K S

<packt>

전문가 통찰

Django 6을 이용한 웹 개발

Django 6을 사용하여 현대적인 Python 웹 애플리케이션을 구축하기 위한 확실한 가이드

제3판

Chris Guest | Mark Walker | Ben Shaw |
Saurabh Badhwar | Bharath Chandra K S

◁packt>

Web Development with Django 6

Third Edition

A definitive guide to building modern Python web applications using Django 6

**Chris Guest, Mark Walker, Ben Shaw, Saurabh Badhwar,
Bharath Chandra K S**

<packt>

장고6을 이용한 웹 개발

제3판

장고 6을 사용하여 현대적인 파이썬 웹 애플리케이션을 구축하기 위한 확실한 가이드

크리스 게스트, 마크 워커, 벤 쇼, 사우라브 바드와르, 바라트 찬드라 K S

<packt>

Web Development with Django

Third Edition

Copyright © 2026 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Portfolio Director: Ashwin Nair

Relationship Lead: Sneha Shinde

Content Engineer: Kinnari Chohan

Project Manager: Vishnu Priya R

Technical Editor: Arjun Varma

Copy Editor: Safis Editing

Indexer: Rekha Nair

Proofreader: Kinnari Chohan

Production Designer: Ponraj Dhandapani

Growth Lead: Anamika Singh

First published: January 2021

Second edition: May 2023

Third edition: March 2026

Production reference: 1130326

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83620-207-3

www.packtpub.com

장고를 이용한 웹 개발

제3판

저작권 © 2026 팩트 출판사

모든 권리 보유. 이 책의 어떤 부분도 출판사의 사전 서면 허가 없이 복제, 저장, 전송할 수 없습니다. 단, 비판적인 기사나 리뷰에 삽입된 간단한 인용의 경우는 예외입니다.

이 책의 준비 과정에서 제공된 정보의 정확성을 보장하기 위해 모든 노력을 기울였습니다. 그러나 이 책에 포함된 정보는 명시적이거나 묵시적인 보증 없이 판매됩니다. 저자, 팩트 출판사 또는 그 대리인 및 유통업체는 이 책으로 인해 직접적 또는 간접적으로 발생한 손해에 대해 책임을 지지 않습니다.

팩트 출판사는 이 책에 언급된 모든 회사 및 제품에 대한 상표 정보를 적절한 대문자 사용으로 제공하기 위해 노력했습니다. 그러나 팩트 출판사는 이 정보의 정확성을 보장할 수 없습니다.

포트폴리오 디렉터: 아쉬윈 나이
어 관계 리드: 스네하 신데 콘텐츠
엔지니어: 키나리 초한 프로젝트
매니저: 비슈누 프리야 R 기술 편
집자: 아르준 바르마 교정 편집자:
사피스 편집 색인자: 레카 나이어

교정자: 키나리 초한
제작 디자이너: 폰라즈 단다파니 성장
리드: 아나미카 싱 첫 출판: 2021년 1월

제2판: 2023년 5월
세 번째 판: 2026년 3월
제작 참조: 1130326
Packt Publishing Ltd.에서 발행함.

그로스베너 하우스
11 세인트 폴스 스
퀘어
버밍엄
B3 1RB, 영국.
ISBN 978-1-83620-207-
3
www.packtpub.com

I would like to dedicate this book to Karen. I have immense gratitude for the patience, support, and understanding of a fellow writer.

– Chris Guest

이 책을 카렌에게 바치고 싶습니다. 동료 작가의 인내, 지원, 그리고 이해에 대해 엄청난 감사의 마음을 가지고 있습니다.

- 크리스 게스트

Contributors

About the authors

Chris Guest is an Australian software developer who has worked with Python for 26 years in health, manufacturing, publishing, and logistics. He uses Django as his go-to platform for web development. He is currently researching natural language processing with ancient Anatolian languages.

I am thankful to all my loved ones for being understanding of the time that I have taken from them in order to write this book. It would not be possible without the prior work of Ben Shaw, Saurabh Badhwar, Andrew Bird, and Bharath Chandra K S—my co-authors of the first and second editions. Lastly, I am forever grateful to the editorial team at Packt for honing my skills as a writer.

Mark Walker has been building production Django systems since 2012. As technical lead of Django CMS and an active open source contributor for over a decade, he has helped shape tools used by developers worldwide. He has architected high-traffic platforms for some of the world's largest events and now builds large-scale fantasy sports games serving millions of users, with a focus on scalability, performance, and pragmatic system design.

Ben Shaw is a software engineer based in Auckland, New Zealand. He has worked as a developer for over 16 years and has been building websites with Django since 2007. In that time, his experience has helped many different types of companies, ranging in size from start-ups to large enterprises. He is also interested in machine learning, data science, automating deployments, and DevOps. When not programming, Ben enjoys outdoor sports and spending time with his partner and son.

Saurabh Badhwar is an infrastructure engineer who builds tools and frameworks to enhance developer productivity. A major part of his work involves using Python to develop services that scale to thousands of concurrent users. He is currently employed at LinkedIn, where he works on infrastructure performance tools and services.

Bharath Chandra K S is a passionate software developer with over 14 years of experience in the industry, currently residing in Sydney, Australia. He specializes in Python stack development, including frameworks such as Flask and Django, and has extensive experience with both monolithic and microservice architectures. Bharath has developed various public-facing applications and data-processing backend systems. When not creating software, he enjoys cooking delicious food.

기여자

저자 소개

크리스 게스트는 건강, 제조, 출판 및 물류 분야에서 26년 동안 파이썬으로 작업해온 호주 소프트웨어 개발자입니다. 그는 웹 개발을 위해 Django를 주 플랫폼으로 사용합니다. 현재 그는 고대 아나톨리아 언어로 자연어 처리에 대해 연구하고 있습니다.

이 책을 쓰기 위해 그들에게서 빼앗은 시간에 대해 이해해 준 모든 사랑하는 이들에게 감사드립니다. 벤 쇼, 사우라브 바드와르, 앤드류 버드, 그리고 바라트 찬드라KS—첫 번째 및 두 번째 판의 공동 저자들—의 이전 작업 없이는 불가능했을 것입니다. 마지막으로, 저자의 기술을 연마해 준 Packt의 편집 팀에게 영원히 감사드립니다.

마크 워커는 2012년부터 프로젝트 Django 시스템을 구축해왔습니다. Django CMS의 기술 리더이자 10년 이상 활동적인 오픈 소스 기여자로서, 그는 전 세계 개발자들이 사용하는 도구를 형성하는 데 도움을 주었습니다. 그는 세계에서 가장 큰 이벤트 중 일부를 위한 고트래픽 플랫폼을 설계했으며, 현재는 수백만 사용자를 위한 대규모 판타지 스포츠 게임을 구축하고 있으며, 확장성, 성능 및 실용적인 시스템 설계에 중점을 두고 있습니다.

벤 쇼는 뉴질랜드 오클랜드에 거주하는 소프트웨어 엔지니어입니다. 그는 16년 이상 개발자로 일해왔으며 2007년부터 Django로 웹사이트를 구축해왔습니다. 그동안 그의 경험은 스타트업부터 대기업까지 다양한 유형의 회사에 도움을 주었습니다. 그는 또한 머신 러닝, 데이터 과학, 배포 자동화 및 DevOps에 관심이 있습니다. 프로그래밍을 하지 않을 때, 벤은 야외 스포츠를 즐기고 파트너 및 아들과 시간을 보냅니다.

사우라브 바드와르는 개발자 생산성을 향상시키기 위해 도구와 프레임워크를 구축하는 인프라 엔지니어입니다. 그의 작업의 주요 부분은 수천 명의 동시 사용자가 사용할 수 있도록 서비스를 개발하기 위해 파이썬을 사용하는 것입니다. 그는 현재 LinkedIn에서 인프라 성능 도구 및 서비스에서 일하고 있습니다.

바라트 찬드라KS는 14년 이상의 업계 경험을 가진 열정적인 소프트웨어 개발자로 현재 호주 시드니에 거주하고 있습니다. 그는 Flask 및 Django와 같은 프레임워크를 포함한 파이썬 스택 개발을 전문으로 하며, 단일체 및 마이크로서비스 아키텍처 모두에 대한 광범위한 경험을 가지고 있습니다. 바라트는 다양한 공개 애플리케이션 및 데이터 처리 백엔드 시스템을 개발했습니다. 소프트웨어를 만들지 않을 때, 그는 맛있는 음식을 요리하는 것을 즐깁니다.

About the reviewers

Anna Makarudze is a software engineer, technical writer, and longtime contributor to the Django community. She has worked with the Django Girls Foundation since 2017 as fundraising coordinator and, since 2020, has served as the lead maintainer of the Django Girls repository and as a trustee of the foundation. She is a former president of the Django Software Foundation. Anna chairs the organizing committee for DjangoCon Africa. She specializes in web development with Python, Django, Django REST Framework, HTML, and JavaScript. She has organized numerous Django Girls events to introduce women to programming with Python and Django. She is committed to helping newcomers learn web development through structured, accessible tutorials.

Abigail Afi Gbadago is vice president of the Django Software Foundation Board and a fellow of the Python Software Foundation, shaping technologies used by millions of developers worldwide. As a senior developer advocate for Django at MongoDB, she drives global adoption and community growth. With over seven years of experience across backend engineering, open source governance, and strategic partnerships, Abigail operates at the intersection of technical systems and business strategy. She served as program chair for DjangoCon US 2025 and regional executor for Black Python Devs, is a Microsoft MVP, and is enrolled in the Stanford LEAD Executive Program (Stanford GSB). Outside work, Afi recharges by returning to her roots in Ghana, spending time with loved ones, and drawing on the vibrant community's good vibes.

Anthony Addae is a Python and Django engineer who cares deeply about open source and the communities that make it thrive. His journey has taken him from mentoring and speaking to contributing more actively within the Django ecosystem across conferences and community initiatives in Ghana and beyond. As a member of the Django Software Foundation and a Djangonaut Space graduate, Anthony enjoys building reliable backend systems while helping create spaces where more people can learn, grow, and contribute.

리뷰어에 대하여

안나 마카루제는 소프트웨어 엔지니어, 기술 작가이며, Django 커뮤니티의 오랜 기여자입니다. 그녀는 2017년부터 Django Girls 재단의 기금 모금 코디네이터로 일해왔으며, 2020년부터는 Django Girls 저장소의 주요 유지 관리자로 재단의 이사로 활동하고 있습니다. 그녀는 Django Software Foundation의 전 회장입니다. 안나는 DjangoCon Africa의 조직 위원회를 주재하고 있습니다. 그녀는 Python, Django, Django REST Framework, HTML 및 JavaScript를 사용한 웹 개발을 전문으로 합니다. 그녀는 여성들에게 Python과 Django로 프로그래밍을 소개하기 위해 수많은 Django Girls 이벤트를 조직했습니다. 그녀는 구조화되고 접근 가능한 튜토리얼을 통해 신입들이 웹 개발을 배우도록 돕는 데 전념하고 있습니다.

아비가일 아피 그바다고는 Django Software Foundation 이사회의 부회장이자 Python Software Foundation의 펠로우로, 전 세계 수백만 개발자들이 사용하는 기술을 형성하고 있습니다. MongoDB에서 Django의 수석 개발자 옹호자로서 그녀는 글로벌 채택과 커뮤니티 성장을 이끌고 있습니다. 7년 이상의 백엔드 엔지니어링, 오픈 소스 거버넌스 및 전략적 파트너십 경험을 바탕으로 아비가일은 기술 시스템과 비즈니스 전략의 교차점에서 활동하고 있습니다. 그녀는 DjangoCon US 2025의 프로그램 의장을 역임했으며 Black Python Devs의 지역 집행자이며, Microsoft MVP로 등록되어 있고 Stanford LEAD Executive Program (Stanford GSB)에 등록되어 있습니다. 일 외에는 아피가 가나의 뿌리로 돌아가 사랑하는 사람들과 시간을 보내고 활기찬 커뮤니티의 좋은 기운을 느끼며 재충전합니다.

앤서니 아다에는 오픈 소스와 그것이 번창하게 만드는 커뮤니티에 깊은 관심을 가진 Python 및 Django 엔지니어입니다. 그의 여정은 멘토링과 강연에서 시작하여 가나 및 그 너머의 회의와 커뮤니티 이니셔티브 내에서 Django 생태계에 더 적극적으로 기여하는 것으로 이어졌습니다. Django Software Foundation의 회원이자 Djangonaut Space 졸업생인 앤서니는 더 많은 사람들이 배우고 성장하며 기여할 수 있는 공간을 만드는 데 도움을 주면서 신뢰할 수 있는 백엔드 시스템을 구축하는 것을 즐깁니다.

Table of Contents

Preface	xxiii
Free benefits with your book.....	xxvii
Chapter 1: Introduction to Django	1
Technical requirements • 2	
Your purchase includes a free PDF copy + code bundle	3
Creating a virtual environment with pyenv	3
Installing pyenv on Windows • 3	
Installing pyenv on macOS and Linux • 4	
Creating a virtual environment with pyenv • 4	
Scaffolding a Django project and app	5
Exercise 1.01 – creating a project and app, and starting the development server • 7	
Understanding the model-view-template paradigm	10
Models • 10	
Views • 11	
Templates • 12	
MVT in practice • 12	
An introduction to HTTP • 15	
Processing a request • 20	
Exploring the Django project structure	22
The myproject directory • 23	
Django development server • 24	
Django apps • 25	
PyCharm setup • 26	
Exercise 1.02 – project setup in PyCharm • 26	
Introducing Django views.....	35
Exploring URL mapping details	36

목차

서문 xxiii

책과 함께 제공되는 무료 혜택..... xxvii

1장: Django 소개 1

기술 요구 사항 • 2

구매 시 무료 PDF 사본 + 코드 번들 포함 3

pyenv로 가상 환경 만들기 3

Windows에 pyenv 설치 • 3

macOS 및 Linux에 pyenv 설치 • 4

pyenv로 가상 환경 만들기 • 4

장고 프로젝트 및 앱 설정 5

연습 1.01 – 프로젝트 및 앱 생성, 개발 서버 시작 • 7

모델-뷰-템플릿 패러다임 이해하기 10

모델 • 10

뷰 • 11

템플릿 • 12

실제 MVT • 12

HTTP 소개 • 15

요청 처리하기 • 20

장고 프로젝트 구조 탐색하기 22

myproject 디렉토리 • 23

Django 개발 서버 • 24

Django 앱 • 25

PyCharm 설정 • 26

연습 1.02 – PyCharm에서 프로젝트 설정 • 26

Django 뷰 소개..... 35

URL 매핑 세부사항 탐색 36

Exercise 1.03 – writing a view and mapping a URL to it • 37	
Working with GET, POST, and QueryDict objects	39
Exercise 1.04 – exploring GET values and QueryDict objects • 42	
Exploring Django settings.....	44
Referring to Django settings in your code • 45	
Finding HTML templates in app directories.....	47
Exercise 1.05 – creating a templates directory and a base template • 47	
Rendering a template with the render function • 49	
Exercise 1.06 – rendering a template in a view • 49	
Rendering variables in templates • 50	
Exercise 1.07 – using variables in templates • 51	
Debugging and dealing with errors	53
Exceptions • 53	
Exercise 1.08 – generating and viewing exceptions • 55	
Debugging • 57	
Exercise 1.09 – debugging your code • 58	
Activity 1.01 – creating a site welcome screen	63
Activity 1.02 – a book search scaffold.....	63
Summary	65
Get this book's PDF copy, code bundle, and more • 65	
 Chapter 2: Models and Migrations	 67

Technical requirements	67
Exploring Django ORM	68
Database configuration and creating Django applications • 68	
Django migration • 70	
Creating Django models and migrations.....	73
Field options • 74	
Primary keys • 77	
Relationships • 79	
<i>Many to one</i> • 80	
<i>Many to many</i> • 81	

연습 1.03 – 뷰 작성 및 URL 매핑 • 37	
GET, POST 및 QueryDict 객체 작업 39	
연습 1.04 – GET 값 및 QueryDict 객체 탐색 • 42	
장고 설정 탐색..... 44	
코드에서 장고 설정 참조하기 • 45	
앱 디렉토리에서 HTML 템플릿 찾기..... 47	
연습 1.05 – 템플릿 디렉토리 및 기본 템플릿 만들기 • 47	
렌더 함수로 템플릿 렌더링하기 • 49	
연습 1.06 – 뷰에서 템플릿 렌더링 • 49	
템플릿에서 변수 렌더링 • 50	
연습 1.07 – 템플릿에서 변수 사용하기 • 51	
디버깅 및 오류 처리 53	
예외 • 53	
연습 1.08 – 예외 생성 및 보기 • 55	
디버깅 • 57	
연습 1.09 – 코드 디버깅하기 • 58	
활동 1.01 – 사이트 환영 화면 만들기 63	
활동 1.02 – 도서 검색 스캐폴드..... 63	
요약 65	
이 책의 PDF 복사본, 코드 번들 및 기타 • 65	

2장: 모델 및 마이그레이션 67

기술 요구 사항 67	
Django ORM 탐색 68	
데이터베이스 구성 및 Django 애플리케이션 생성 • 68	
Django 마이그레이션 • 70	
Django 모델 및 마이그레이션 생성..... 73	
필드 옵션 • 74	
기본 키 • 77	
관계 • 79	
다대일 • 80	
다대다 • 81	

<i>One-to-one relationships</i> • 83	
Adding the Review model • 84	
Model methods • 85	
Migrating the reviews app • 87	
Django's database CRUD operations	89
Exercise 2.01 – Creating an entry in the bookr database • 90	
Exercise 2.02 – Using the create() method to create an entry • 92	
Creating an object with a foreign key • 92	
Exercise 2.03 – Creating records for a many-to-one relationship • 93	
Exercise 2.04 – Creating records with many-to-many relationships • 94	
Exercise 2.05 – A many-to-many relationship using the add() method • 95	
Using the create() and set() methods for many-to-many relationships • 96	
Read operations • 97	
Exercise 2.06 – Using the get() method to retrieve an object • 97	
Returning an object using the get() method • 98	
Exercise 2.07 – Using the all() method to retrieve a set of objects • 99	
Retrieving objects by filtering • 100	
Exercise 2.08 – Using the filter() method to retrieve objects • 100	
Filtering by field lookups • 101	
Using pattern matching for filtering operations • 102	
Retrieving objects by using the exclude() method • 103	
Retrieving objects using the order_by() method • 103	
Querying across relationships • 106	
Querying using foreign keys • 107	
Querying using the model name • 107	
Querying across foreign key relationships using the object instance • 107	
Exercise 2.09 – Querying across a many-to-many relationship using the field lookup •	
108	
Exercise 2.10 – A many-to-many query using objects • 109	
Exercise 2.11 – A many-to-many query using the set() method • 109	
Exercise 2.12 – Using the update() method • 110	
Exercise 2.13 – Using the delete() method • 111	

일대일 관계 • 83	
리뷰 모델 추가하기 • 84	
모델 메서드 • 85	
리뷰 앱 마이그레이션 • 87	
장고의 데이터베이스 CRUD 작업 89	
연습 2.01 – bookr 데이터베이스에 항목 만들기 • 90	
연습 2.02 – create() 메서드를 사용하여 항목 만들기 • 92	
외래 키로 객체 만들기 • 92	
연습 2.03 – 다대일 관계를 위한 레코드 생성 • 93	
연습 2.04 – 다대다 관계로 레코드 생성 • 94	
연습 2.05 – add() 메서드를 사용한 다대다 관계 • 95	
다대다 관계를 위한 create() 및 set() 메서드 사용 • 96	
읽기 작업 • 97	
연습 2.06 – get() 메서드를 사용하여 객체 검색 • 97	
get() 메서드를 사용하여 객체 반환 • 98	
연습 2.07 – all() 메서드를 사용하여 객체 집합 검색 • 99	
필터링을 통한 객체 검색 • 100	
연습 2.08 – filter() 메서드를 사용하여 객체 검색 • 100	
필드 조회로 필터링하기 • 101	
필터링 작업을 위한 패턴 매칭 사용하기 • 102	
exclude() 메서드를 사용하여 객체 검색하기 • 103	
order_by() 메서드를 사용하여 객체 검색하기 • 103	
관계를 통한 쿼리 • 106	
외래 키를 사용한 쿼리 • 107	
모델 이름을 사용한 쿼리 • 107	
객체 인스턴스를 사용하여 외래 키 관계를 통한 쿼리 • 107	
연습 2.09 – 필드 조회를 사용하여 다대다 관계를 통한 쿼리 • 108	
연습 2.10 – 객체를 사용한 다대다 쿼리 • 109	
연습 2.11 – set() 메서드를 사용한 다대다 쿼리 • 109	
연습 2.12 – update() 메서드 사용하기 • 110	
연습 2.13 – delete() 메서드 사용하기 • 111	

Bulk create and bulk update operations	112
Exercise 2.14 – Create multiple records using bulk_create • 112	
Exercise 2.15 – Update multiple records using bulk_update • 113	
Performing complex lookups using Q objects	114
Exercise 2.16 – Perform a complex query using a Q object • 114	
Exercise 2.17 – Verify whether a queryset contains a given object • 115	
Activity 2.01 – Create models for a project management application	116
Populating the Bookr project's database	117
Summary	118
Get this book's PDF copy, code bundle, and more	118
 Chapter 3: Django Views, URL Configuration, and Templates	 119
Technical requirements	120
Working with Django views.....	120
Understanding function-based views • 120	
Understanding class-based views • 121	
URL configuration	122
Exercise 3.01 – Implementing a simple function-based view • 124	
Working with Django templates	127
Exercise 3.02 – Using templates to display a greeting message • 129	
Django template language • 132	
<i>Template variables</i> • 132	
<i>Template tags</i> • 132	
<i>Comments</i> • 133	
<i>Filters</i> • 133	
Exercise 3.03 – Displaying a list of books and reviews • 134	
Template inheritance • 138	
Template styling with Bootstrap • 139	
Exercise 3.04 – Adding template inheritance and a Bootstrap navigation bar • 141	
Activity 3.01 – Implementing the book details view	144
Summary	145
Get this book's PDF copy, code bundle, and more	146

대량 생성 및 대량 업데이트 작업	112
연습 2.14 – bulk_create를 사용하여 여러 레코드 생성 •	112
연습 2.15 – bulk_update를 사용하여 여러 레코드 업데이트 •	113
Q 객체를 사용한 복잡한 조회 수행	114
연습 2.16 – Q 객체를 사용하여 복잡한 쿼리 수행 •	114
연습 2.17 – 쿼리셋에 주어진 객체가 포함되어 있는지 확인 •	115
활동 2.01 – 프로젝트 관리 애플리케이션을 위한 모델 생성	116
Bookr 프로젝트의 데이터베이스 채우기	117
요약	118
이 책의 PDF 복사본, 코드 번들 및 기타 자료 받기	118
3장: 장고 뷰, URL 구성 및 템플릿	119
기술 요구 사항	120
장고 뷰 작업하기	120
함수 기반 뷰 이해하기 •	120
클래스 기반 뷰 이해하기 •	121
URL 구성	122
연습 3.01 – 간단한 함수 기반 뷰 구현하기 •	124
장고 템플릿 작업하기	127
연습 3.02 – 템플릿을 사용하여 인사 메시지 표시 •	129
장고 템플릿 언어 •	132
템플릿 변수 •	132
템플릿 태그 •	132
주석 •	133
필터 •	133
연습 3.03 – 책과 리뷰 목록 표시 •	134
템플릿 상속 •	138
부트스트랩을 이용한 템플릿 스타일링 •	139
연습 3.04 – 템플릿 상속 및 부트스트랩 내비게이션 바 추가 •	141
활동 3.01 – 책 세부 정보 보기 구현	144
요약	145
이 책의 PDF 복사본, 코드 번들 및 기타 자료 받기	146

Chapter 4: An Introduction to Django Admin	147
Technical requirements	147
Introducing the Django admin app	148
Creating a superuser account.....	149
Exercise 4.01 – Creating a superuser account •	149
CRUD operations using the Django admin app	152
Create •	152
<i>Activity 4.01 – Creating a user account •</i>	<i>154</i>
Retrieve •	155
Update •	156
Delete •	158
Managing Django users and groups.....	159
Exercise 4.02 – Adding and modifying users and groups through the admin app •	160
Registering models with the admin app	165
Registering the reviews model •	166
Change lists •	167
The Change publisher page •	168
The book change page •	173
Exercise 4.03 – Foreign keys and deletion behavior in the admin app •	177
Customizing the admin interface	179
Site-wide Django admin customizations •	179
<i>Examining the AdminSite object from the Python shell •</i>	<i>181</i>
<i>Subclassing AdminSite – a first approach •</i>	<i>182</i>
<i>Subclassing AdminSite – a better way •</i>	<i>185</i>
<i>Activity 4.02 – Customizing the AdminSite object •</i>	<i>186</i>
Customizing the ModelAdmin classes	190
The list display fields •	191
The display decorator •	197
The filter •	198
Exercise 4.04 – Adding the date_list_filter and date_hierarchy filters •	200
The search bar •	203

4장: 장고 관리자 소개 147

기술 요구 사항	147
장고 관리자 앱 소개	148
슈퍼유저 계정 만들기	149
연습 4.01 – 슈퍼유저 계정 만들기 • 149	
장고 관리자 앱을 사용한 CRUD 작업	152
생성 • 152	
활동 4.01 – 사용자 계정 만들기 • 154	
가져오기 • 155	
업데이트 • 156	
삭제 • 158	
장고 사용자 및 그룹 관리.....	159
연습 4.02 – 관리자 앱을 통해 사용자 및 그룹 추가 및 수정 • 160	
관리자 앱에 모델 등록	165
리뷰 모델 등록 • 166	
변경 목록 • 167	
변경 게시자 페이지 • 168	
책 변경 페이지 • 173	
연습 4.03 – 외래 키 및 관리 앱의 삭제 동작 • 177	
관리 인터페이스 사용자 정의	179
사이트 전체 Django 관리 사용자 정의 • 179	
Python 셸에서 AdminSite 객체 조사하기 • 181	
AdminSite 서브클래싱 – 첫 번째 접근 방식 • 182	
AdminSite 서브클래싱 – 더 나은 방법 • 185	
활동 4.02 – AdminSite 객체 사용자 정의 • 186	
ModelAdmin 클래스 사용자 정의	190
목록 표시 필드 • 191	
표시 데코레이터 • 197	
필터 • 198	
연습 4.04 – 날짜 list_filter 및 date_hierarchy 필터 추가 • 200	
검색창 • 203	

Excluding and grouping fields • 206	
Activity 4.03 – Customizing the model admins • 209	
Summary	212
Get this book's PDF copy, code bundle, and more	212
Chapter 5: Serving Static Files	213
Technical requirements	215
Django and static file serving	215
Introduction to static files finders	216
Static file finders – used during a request • 217	
AppDirectoriesFinder • 217	
<i>Static file namespacing • 218</i>	
Exercise 5.01 – serving a file from an app directory • 220	
Generating static URLs with the static template tag	224
The static template tag • 225	
An example of the static template tag • 226	
Exercise 5.02 – using the static template tag • 229	
FileSystemFinder.....	232
Exercise 5.03 – serving from a project static directory • 233	
Static file finders – use during collectstatic.....	236
Exercise 5.04 – collecting static files for production • 238	
STATICFILES_DIRS prefixed mode	240
The findstatic command.....	243
Exercise 5.05 – finding files using findstatic • 244	
Serving the latest files (for cache invalidation)	248
Exercise 5.06 – exploring the ManifestFilesStorage storage engine • 251	
Custom storage engines.....	255
Activity 5.01 – adding a Reviews logo • 258	
Activity 5.02 – CSS enhancements • 260	
Activity 5.03 – adding a global logo • 262	
Summary	264
Get this book's PDF copy, code bundle, and more	265

필드 제외 및 그룹화 • 206	
활동 4.03 – 모델 관리자 사용자 정의 • 209	
요약	212
이 책의 PDF 복사본, 코드 번들 및 기타 자료 받기	212

5장: 정적 파일 제공 213

기술 요구 사항	215
장고와 정적 파일 제공	215
정적 파일 탐색기 소개	216
정적 파일 탐색기 – 요청 중 사용 • 217	
AppDirectoriesFinder • 217	
정적 파일 네임스페이스 • 218	
연습 5.01 – 앱 디렉토리에서 파일 제공하기 • 220	
정적 템플릿 태그로 정적 URL 생성하기	224
정적 템플릿 태그 • 225	
정적 템플릿 태그의 예 • 226	
연습 5.02 – 정적 템플릿 태그 사용하기 • 229	
FileSystemFinder.....	232
연습 5.03 – 프로젝트 정적 디렉토리에서 제공하기 • 233	
정적 파일 찾기 – collectstatic 중 사용	236
연습 5.04 – 프로덕션을 위한 정적 파일 수집 • 238	
STATICFILES_DIRS 접두사 모드	240
findstatic 명령어	243
연습 5.05 – findstatic을 사용하여 파일 찾기 • 244	
최신 파일 제공 (캐시 무효화를 위해)	248
연습 5.06 – ManifestFilesStorage 스토리지 엔진 탐색 • 251	
사용자 정의 스토리지 엔진	255
활동 5.01 – 리뷰 로고 추가하기 • 258	
활동 5.02 – CSS 개선 • 260	
활동 5.03 – 글로벌 로고 추가 • 262	
요약	264
이 책의 PDF 복사본, 코드 번들 및 기타 자료 받기	265

Chapter 6: Forms	267
Technical requirements	268
Background to forms	268
The form element • 268	
Form security	270
Input validation • 270	
Cross-site request forgery protection • 270	
SQL injection prevention • 271	
Content Security Policy (CSP) and XSS attacks • 272	
HTTPS and security headers • 272	
Exercise 6.01 – Building a form in HTML • 273	
Accessing data in the view • 280	
Exercise 6.02 – Working with POST data in a view • 281	
Choosing between GET and POST • 286	
Why use GET when we can put parameters in the URL mappings? • 288	
The Django Forms library	289
Defining a form • 290	
Rendering a form in a template • 301	
Exercise 6.03 – Building and rendering a Django form • 307	
Validating forms and retrieving Python values	311
Exercise 6.04 – Validating forms in a view • 315	
Built-in field validation • 318	
Exercise 6.05 – Adding extra field validation • 319	
Activity 6.01 – Book Search	322
Summary	326
Get this book's PDF copy, code bundle, and more	327
Chapter 7: Advanced Form Validation and Model Forms	329
Technical requirements	330
Custom field validation and cleaning	330
Custom validators • 331	

6장: 양식 267

기술 요구 사항	268
양식의 배경	268
양식 요소 • 268	
양식 보안	270
입력 검증 • 270	
교차 사이트 요청 위조 방지 • 270	
SQL 주입 방지 • 271	
콘텐츠 보안 정책 (CSP) 및 XSS 공격 • 272	
HTTPS 및 보안 헤더 • 272	
연습 6.01 – HTML로 폼 만들기 • 273	
뷰에서 데이터 접근하기 • 280	
연습 6.02 – 뷰에서 POST 데이터 작업하기 • 281	
GET과 POST 중 선택하기 • 286	
URL 매핑에 매개변수를 넣을 수 있을 때 왜 GET을 사용할까? • 288	
장고 폼 라이브러리	289
폼 정의하기 • 290	
템플릿에서 폼 렌더링하기 • 301	
연습 6.03 – Django 폼 만들기 및 렌더링 • 307	
폼 유효성 검사 및 Python 값 검색	311
연습 6.04 – 뷰에서 폼 유효성 검사 • 315	
내장 필드 유효성 검사 • 318	
연습 6.05 – 추가 필드 유효성 검사 추가 • 319	
활동 6.01 – 도서 검색	322
요약	326
이 책의 PDF 사본, 코드 번들 및 기타 자료 받기	327

7장: 고급 폼 유효성 검사 및 모델 폼 329

기술 요구 사항	330
사용자 정의 필드 검증 및 정리	330
사용자 정의 검증기 • 331	

Cleaning methods • 332	
Multi-field validation • 334	
Exercise 7.01 – Custom clean and validation methods • 338	
Adding placeholders and initial values	346
Exercise 7.02 – Placeholders and initial values • 348	
Creating or editing Django models	350
The ModelForm class • 352	
Exercise 7.03 – Creating and editing a publisher • 357	
Activity 7.01 – Styling and integrating the publisher form • 363	
Activity 7.02 – Review creation UI • 366	
Summary	372
Get this book's PDF copy, code bundle, and more	373
 Chapter 8: Media Serving and File Uploads	 375
Technical requirements	376
Settings for media uploads and serving	376
Serving media files in development • 376	
Exercise 8.01 – configuring media storage and serving • 377	
Context processors and using MEDIA_URL in templates • 381	
Exercise 8.02 – template settings and using MEDIA_URL in templates • 384	
File uploads using HTML forms.....	386
Working with uploaded files in a view • 388	
<i>Security and trust of browsers' sent values • 390</i>	
Exercise 8.03 – file upload and download • 392	
File uploads with Django forms	395
Exercise 8.04 – file uploads with a Django form • 397	
Image uploads with Django forms.....	402
Resizing an image with Pillow • 404	
Exercise 8.05 – image uploads using Django forms • 405	
Serving uploaded (and other) files using Django.....	408
Storing files on model instances • 410	
Storing images on model instances • 413	

청소 방법 • 332	
다중 필드 검증 • 334	
연습 7.01 – 사용자 정의 청소 및 검증 방법 • 338	
자리 표시자 및 초기 값 추가	346
연습 7.02 – 자리 표시자 및 초기 값 • 348	
Django 모델 생성 또는 편집	350
ModelForm 클래스 • 352	
연습 7.03 – 출판사 생성 및 편집 • 357	
활동 7.01 – 퍼블리셔 양식 스타일링 및 통합 • 363	
활동 7.02 – 리뷰 생성 UI • 366	
요약	372
이 책의 PDF 복사본, 코드 번들 및 기타 자료 받기	373
 8장: 미디어 제공 및 파일 업로드 375	
기술 요구 사항	376
미디어 업로드 및 제공 설정	376
개발 중 미디어 파일 제공 • 376	
연습 8.01 – 미디어 저장소 및 제공 구성 • 377	
컨텍스트 프로세서 및 템플릿에서 MEDIA_URL 사용 • 381	
연습 8.02 – 템플릿 설정 및 템플릿에서 MEDIA_URL 사용 • 384	
HTML 양식을 사용한 파일 업로드.....	386
뷰에서 업로드된 파일 작업하기 • 388	
<i>브라우저가 보낸 값의 보안 및 신뢰</i> • 390	
연습 8.03 – 파일 업로드 및 다운로드 • 392	
Django 양식을 사용한 파일 업로드	395
연습 8.04 – Django 양식을 사용한 파일 업로드 • 397	
Django 양식을 사용한 이미지 업로드.....	402
Pillow로 이미지 크기 조정하기 • 404	
연습 8.05 – Django 양식을 사용한 이미지 업로드 • 405	
업로드된 (및 기타) 파일을 Django를 사용하여 제공하기.....	408
모델 인스턴스에 파일 저장하기 • 410	
모델 인스턴스에 이미지 저장하기 • 413	

Working with FieldFile • 414	
<i>Custom storage engines • 415</i>	
<i>Reading a stored FieldFile • 415</i>	
<i>Storing existing files or content on FileField • 416</i>	
<i>Writing PIL images to ImageField • 418</i>	
Referring to media in templates • 421	
Exercise 8.06 – FileField and ImageField on models • 421	
ModelForm and file uploads 426	
Exercise 8.07 – file and image uploads using ModelForm • 429	
Handling file saving • 432	
Activity 8.01 – image and PDF upload for books..... 433	
Activity 8.02 – displaying the cover and sample link..... 438	
Summary 440	
Get this book's PDF copy, code bundle, and more 441	
 Chapter 9: Sessions and Authentication 443	
Technical requirements 444	
Middleware..... 444	
Middleware modules • 445	
Implementing authentication views and templates • 447	
Exercise 9.01 – Repurposing the Admin app login template • 453	
Password storage in Django 457	
The profile page and the request.user object • 457	
Exercise 9.02 – Adding a profile page • 457	
Authentication decorators and redirection 459	
Exercise 9.03 – Adding authentication decorators to the views • 463	
Enhancing templates with authentication data..... 466	
Exercise 9.04 – Toggling login and logout links in the base template • 466	
Activity 9.01 – Authentication-based content using conditional blocks in templates • 468	
Sessions 470	
The session engine • 470	

FieldFile 작업하기 • 414	
사용자 정의 저장 엔진 • 415	
저장된 FieldFile 읽기 • 415	
FileField에 기존 파일 또는 콘텐츠 저장하기 • 416	
ImageField에 PIL 이미지 쓰기 • 418	
템플릿에서 미디어 참조하기 • 421	
연습 8.06 – 모델의 FileField 및 ImageField • 421	
ModelForm 및 파일 업로드 426	
연습 8.07 – ModelForm을 사용한 파일 및 이미지 업로드 • 429	
파일 저장 처리 • 432	
활동 8.01 – 책을 위한 이미지 및 PDF 업로드..... 433	
활동 8.02 – 표지 및 샘플 링크 표시 438	
요약 440	
이 책의 PDF 복사본, 코드 번들 및 기타 자료 받기 441	
 9장: 세션 및 인증 443	
기술 요구 사항 444	
미들웨어..... 444	
미들웨어 모듈 • 445	
인증 뷰 및 템플릿 구현 • 447	
연습 9.01 – 관리자 앱 로그인 템플릿 재사용 • 453	
Django에서의 비밀번호 저장 457	
프로필 페이지 및 request.user 객체 • 457	
연습 9.02 – 프로필 페이지 추가 • 457	
인증 데코레이터 및 리디렉션 459	
연습 9.03 – 뷰에 인증 데코레이터 추가 • 463	
인증 데이터로 템플릿 향상..... 466	
연습 9.04 – 기본 템플릿에서 로그인 및 로그아웃 링크 전환 • 466	
활동 9.01 – 조건부 블록을 사용한 인증 기반 콘텐츠 템플릿 • 468	
세션 470	
세션 엔진 • 470	

Do you need to flag cookie content? • 471	
Pickle or JSON storage? • 472	
Exercise 9.06 – Examining the session key • 473	
Storing data in sessions • 478	
Exercise 9.06 – Storing recently viewed books in sessions • 479	
Exercise 9.07 – Using session storage for the Search for Books page • 484	
Summary	485
Get this book's PDF copy, code bundle, and more	486
 Chapter 10: Advanced Django Admin and Customizations	 487
Technical requirements	488
Customizing the admin site	488
A customized user model	488
Extending the user model • 489	
Subclassing AbstractUser.....	490
<i>Scenario 1: Adding additional fields to the User class • 492</i>	
<i>Scenario 2: Replacing the Username key • 493</i>	
<i>Scenario 3: Modifying the password • 493</i>	
<i>Steps in replacing the auth user model • 493</i>	
Exercise 10.1: Replacing User with a TitledUser model • 494	
Activity 10.1: Implementing the reviewer profile model • 497	
Form enhancements with ModelAdmin attributes • 498	
<i>filter_horizontal / filter_vertical • 498</i>	
<i>Inline forms • 499</i>	
<i>radio_fields • 503</i>	
<i>readonly_fields • 503</i>	
<i>get_readonly_fields • 503</i>	
<i>list_editable • 504</i>	
Exercise 10.2 • 504	
Pagination customizations • 506	
Search and filter customizations • 507	
Activity 10.2 • 507	

쿠키 내용을 플래그해야 합니까? • 471	
피클 또는 JSON 저장소? • 472	
연습 9.06 – 세션 키 검사하기 • 473	
세션에 데이터 저장하기 • 478	
연습 9.06 – 최근에 본 책을 세션에 저장하기 • 479	
연습 9.07 – 책 검색 페이지에 세션 저장소 사용하기 • 484	
요약	485
이 책의 PDF 복사본, 코드 번들 및 기타 자료 받기	486
10장: 고급 Django 관리자 및 사용자 정의 487	
기술 요구 사항	488
관리 사이트 사용자 정의	488
사용자 정의 사용자 모델	488
사용자 모델 확장 • 489	
AbstractUser 서브클래싱.....	490
시나리오 1: 사용자 클래스에 추가 필드 추가하기 • 492	
시나리오 2: 사용자 이름 키 교체하기 • 493	
시나리오 3: 비밀번호 수정하기 • 493	
인증 사용자 모델 교체 단계 • 493	
연습 10.1: User를 TitledUser 모델로 교체하기 • 494	
활동 10.1: 리뷰어 프로필 모델 구현하기 • 497	
모델 관리 속성을 통한 폼 개선 • 498	
<i>filter_horizontal / filter_vertical</i> • 498	
인라인 폼 • 499	
<i>radio_fields</i> • 503	
<i>readonly_fields</i> • 503	
<i>get_readonly_fields</i> • 503	
<i>list_editable</i> • 504	
연습 10.2 • 504	
페이지 매김 사용자 정의 • 506	
검색 및 필터 사용자 정의 • 507	
활동 10.2 • 507	

Adding views to the admin site	508
Creating the view function • 509	
Accessing common template variables • 510	
Mapping URLs for the custom view • 510	
Restricting custom views to the admin site • 511	
Exercise 10.03 – adding custom views to the admin site • 511	
Passing additional keys to templates using template variables • 515	
Configuring an email server with Django	516
Automating admin with Django Tasks	518
Activity 10.03 – building a custom admin dashboard with a built-in search • 521	
Exercise 10.5 • 523	
Summary	528
Get this book's PDF copy, code bundle, and more • 529	
 Chapter 11: Advanced Templating and Class-Based Views	 531
Technical requirements • 532	
Template filters	532
Custom template filters	534
Creating custom template filters • 534	
<i>Setting up the directory for storing template filters • 534</i>	
<i>Setting up the template library • 535</i>	
Implementing the custom filter function • 535	
Using custom filters inside templates • 536	
String filters	536
Exercise 11.01 – Creating a custom template filter • 537	
Template tags.....	541
The types of template tags • 542	
Simple tags • 542	
Creating a simple template tag • 542	
<i>Setting up the directory • 543</i>	
<i>Setting up the template library • 543</i>	
<i>Implementing a simple template tag • 543</i>	

관리자 사이트에 뷰 추가하기	508
뷰 함수 생성하기 • 509	
공통 템플릿 변수 접근하기 • 510	
커스텀 뷰에 대한 URL 매핑 • 510	
관리자 사이트에 커스텀 뷰 제한하기 • 511	
연습 10.03 – 관리자 사이트에 커스텀 뷰 추가하기 • 511	
템플릿 변수를 사용하여 템플릿에 추가 키 전달하기 • 515	
장고로 이메일 서버 구성하기	516
Django 작업으로 관리 자동화	518
활동 10.03 – 내장 검색 기능이 있는 맞춤형 관리 대시보드 구축 • 521	
연습 10.5 • 523	
요약	528
이 책의 PDF 사본, 코드 번들 및 기타 • 529	

11장: 고급 템플릿 및 클래스 기반 뷰 531

기술 요구 사항 • 532	
템플릿 필터	532
사용자 정의 템플릿 필터	534
사용자 정의 템플릿 필터 만들기 • 534	
<i>템플릿 필터를 저장할 디렉토리 설정 • 534</i>	
<i>템플릿 라이브러리 설정 • 535</i>	
사용자 정의 필터 함수 구현 • 535	
템플릿 내에서 사용자 정의 필터 사용 • 536	
문자열 필터	536
연습 11.01 – 사용자 정의 템플릿 필터 만들기 • 537	
템플릿 태그.....	541
템플릿 태그의 종류 • 542	
간단한 태그 • 542	
간단한 템플릿 태그 만들기 • 542	
<i>디렉토리 설정 • 543</i>	
<i>템플릿 라이브러리 설정 • 543</i>	
<i>간단한 템플릿 태그 구현 • 543</i>	

<i>Using simple tags inside templates • 544</i>	
Exercise 11.02 – Creating a custom simple tag • 544	
<i>Passing the template context in a custom template tag • 547</i>	
Inclusion tags • 548	
<i>Implementing inclusion tags • 548</i>	
<i>Using an inclusion tag inside a template • 549</i>	
Exercise 11.03 – Building a custom inclusion tag • 549	
Activity 11.01 – Rendering details on the user profile page using inclusion tags • 552	
Template partials	554
Activity 11.02 – Rendering a book list item with template partials • 556	
Django views.....	556
Class-based views.....	557
Exercise 11.04 – Creating a book catalog using a CBV • 559	
CRUD operations with CBVs • 565	
<i>The Create view • 565</i>	
The Read view • 566	
<i>The Update view • 567</i>	
<i>The Delete view • 569</i>	
Summary	570
Get this book's PDF copy, code bundle, and more • 571	
 Chapter 12: Building a REST API	 573

Technical requirements	574
Understanding REST APIs.....	574
DRF	575
Installation and configuration • 575	
Functional API views • 576	
Exercise 12.01 – Creating a simple REST API • 576	
Understanding serializers.....	578
Exercise 12.02 – Creating an API view to display a list of books • 579	
Class-based API views and generic views • 582	
Model serializers • 583	

템플릿 내에서 간단한 태그 사용하기 • 544	
연습 11.02 – 사용자 정의 간단한 태그 만들기 • 544	
사용자 정의 템플릿 태그에서 템플릿 컨텍스트 전달하기 • 547	
포함 태그 • 548	
포함 태그 구현하기 • 548	
템플릿 내에서 포함 태그 사용하기 • 549	
연습 11.03 – 사용자 정의 포함 태그 만들기 • 549	
활동 11.01 – 포함 태그를 사용하여 사용자 프로필 페이지에 세부정보 렌더링하기 • 552	
템플릿 부분 554	
활동 11.02 – 템플릿 부분을 사용하여 책 목록 항목 렌더링 • 556	
장고 뷰 556	
클래스 기반 뷰 557	
연습 11.04 – CBV를 사용하여 책 카탈로그 만들기 • 559	
CBV를 사용한 CRUD 작업 • 565	
생성 뷰 • 565	
읽기 뷰 • 566	
업데이트 뷰 • 567	
삭제 뷰 • 569	
요약 570	
이 책의 PDF 복사본, 코드 번들 및 기타 • 571	
제12장: REST API 구축 573	
기술 요구 사항 574	
REST API 이해하기 574	
DRF 575	
설치 및 구성 • 575	
기능적 API 뷰 • 576	
연습 12.01 – 간단한 REST API 만들기 • 576	
직렬 변환기 이해하기 578	
연습 12.02 – 책 목록을 표시하는 API 뷰 만들기 • 579	
클래스 기반 API 뷰 및 제네릭 뷰 • 582	
모델 직렬 변환기 • 583	

Exercise 12.03 – Creating class-based API views and model serializers •	584
Activity 12.01 – Creating an API endpoint for a top contributors pageShould •	587
Simplifying the code using ViewSets	588
URL configuration using routers and ViewSets •	588
Exercise 12.04 – Using ViewSets and routers •	589
Implementing authentication.....	592
Token-based authentication •	593
Exercise 12.05 – Implementing token-based authentication for bookr APIs •	593
Summary	599
Get this book's PDF copy, code bundle, and more •	599
 Chapter 13: Generating CSV, PDF, and Other Binary Files	 601
Working with CSV files in Python	602
Working with Python's csv module.....	602
Writing to CSV files using Python •	602
Exercise 13.01 – Create a review summary CSV file •	604
Binary file formats for data exports •	607
Working with XLSX files using the XlsxWriter package •	608
<i>XLSX files •</i>	<i>608</i>
<i>The XlsxWriter Python package •</i>	<i>609</i>
<i>Creating a workbook •</i>	<i>610</i>
<i>Creating a worksheet •</i>	<i>610</i>
<i>Writing data to the worksheet •</i>	<i>610</i>
<i>Writing data to the workbook •</i>	<i>611</i>
<i>Formatting a spreadsheet •</i>	<i>611</i>
<i>Exercise 13.02 – Developing a formatted spreadsheet •</i>	<i>615</i>
ZIP files	619
Working with PDF files in Python	621
Exercise 13.03 – Generating a PDF version of a web page in Python •	622
Activity 13.01 – Adding additional functionality to a View •	626
Generating graphs with Plotly	628
<i>Installation •</i>	<i>629</i>

연습 12.03 – 클래스 기반 API 뷰 및 모델 직렬 변환기 만들기 • 584	
활동 12.01 – 상위 기여자 페이지를 위한 API 엔드포인트 만들기 • 587	
ViewSets를 사용하여 코드 단순화	588
라우터 및 ViewSets를 사용한 URL 구성 • 588	
연습 12.04 – ViewSets 및 라우터 사용하기 • 589	
인증 구현.....	592
토큰 기반 인증 • 593	
연습 12.05 – bookr API를 위한 토큰 기반 인증 구현 • 593	
요약	599
이 책의 PDF 복사본, 코드 번들 및 기타 • 599	

13장: CSV, PDF 및 기타 이진 파일 생성 601

Python에서 CSV 파일 작업하기	602
Python의 csv 모듈 작업하기	602
Python을 사용하여 CSV 파일에 쓰기 • 602	
연습 13.01 – 리뷰 요약 CSV 파일 만들기 • 604	
데이터 내보내기를 위한 이진 파일 형식 • 607	
XlsxWriter 패키지를 사용하여 XLSX 파일 작업하기 • 608	
<i>XLSX 파일</i> • 608	
<i>XlsxWriter 파이썬 패키지</i> • 609	
<i>워크북 만들기</i> • 610	
<i>워크시트 만들기</i> • 610	
<i>워크시트에 데이터 쓰기</i> • 610	
<i>워크북에 데이터 쓰기</i> • 611	
<i>스프레드시트 서식 지정</i> • 611	
연습 13.02 – 서식이 지정된 스프레드시트 개발 • 615	
ZIP 파일	619
파이썬에서 PDF 파일 작업하기	621
연습 13.03 – 파이썬에서 웹 페이지의 PDF 버전 생성하기 • 622	
활동 13.01 – 뷰에 추가 기능 추가 • 626	
Plotly로 그래프 생성	628
<i>설치</i> • 629	

<i>Setting up a figure</i> • 629	
<i>Generating a plot</i> • 630	
<i>Rendering a plot on a web page</i> • 631	
Advanced graphs – bubble plots • 631	
Exercise 13.4 – Generating graphs in Python • 633	
Generating images in Python with Pillow • 636	
Activity 13.02 – Creating dynamic graphics in Django with Pillow • 638	
Summary	641
 Chapter 14: Testing Your Django Applications	 643
Importance of testing	644
Automation testing	644
Testing in Django	645
Implementing test cases • 646	
Unit testing in Django • 646	
Utilizing assertions • 646	
Exercise 14.01 – Writing a simple unit test • 648	
<i>Types of assertions</i> • 649	
Performing pre-test setup and cleanup after every test case run • 650	
Testing Django models	652
Exercise 14.02 – Testing Django models • 652	
Testing Django views.....	656
Exercise 14.03 – Writing unit tests for Django views • 657	
Testing views with authentication • 661	
Exercise 14.04 – Writing test cases to validate authenticated users • 661	
Django RequestFactory	666
Exercise 14.05 – Using RequestFactory to test views • 666	
Testing class-based views • 669	
Test case classes in Django	670
The SimpleTestCase class • 670	
The TransactionTestCase class • 671	
The LiveServerTestCase class • 671	

그림 설정 • 629	
플롯 생성 • 630	
웹 페이지에 플롯 렌더링 • 631	
고급 그래프 – 버블 플롯 • 631	
연습 13.4 – Python에서 그래프 생성 • 633	
Pillow로 Python에서 이미지 생성 • 636	
활동 13.02 – Pillow로 Django에서 동적 그래픽 만들기 • 638	
요약	641
14장: 장고 애플리케이션 테스트 643	
테스트의 중요성	644
자동화 테스트	644
장고에서의 테스트	645
테스트 케이스 구현 • 646	
장고에서의 단위 테스트 • 646	
어설션 활용 • 646	
연습 14.01 – 간단한 단위 테스트 작성 • 648	
어설션의 종류 • 649	
각 테스트 케이스 실행 후 사전 테스트 설정 및 정리 수행 • 650	
장고 모델 테스트	652
연습 14.02 – 장고 모델 테스트 • 652	
장고 뷰 테스트	656
연습 14.03 – 장고 뷰에 대한 단위 테스트 작성 • 657	
인증이 있는 뷰 테스트 • 661	
연습 14.04 – 인증된 사용자를 검증하기 위한 테스트 케이스 작성 • 661	
장고 RequestFactory	666
연습 14.05 – RequestFactory를 사용하여 뷰 테스트 • 666	
클래스 기반 뷰 테스트 • 669	
장고의 테스트 케이스 클래스	670
SimpleTestCase 클래스 • 670	
TransactionTestCase 클래스 • 671	
LiveServerTestCase 클래스 • 671	

Modularizing test code • 671	
Running tests faster using parallelization	672
Activity 14.01 – Testing models and views in Bookr	673
Summary	674
Get this book's PDF copy, code bundle, and more	675
 Chapter 15: Deploying a Django Project	 677
Technical requirement	677
Server architecture	677
Gunicorn • 679	
Whitenoise • 679	
The deployment checklist.....	679
Run manage.py check --deploy • 680	
Switch away from manage.py runserver • 681	
Critical settings • 681	
<i>SECRET_KEY</i> • 682	
<i>DEBUG</i> • 682	
Environment-specific settings • 682	
<i>ALLOWED_HOSTS</i> • 682	
<i>CACHES</i> • 683	
<i>DATABASES</i> • 683	
<i>EMAIL_BACKEND</i> and related settings • 683	
<i>HTTPS</i> • 684	
<i>Performance optimizations</i> • 684	
<i>Error reporting</i> • 685	
<i>Customize the default error views</i> • 685	
Exercise 15.01 – Configuring the Bookr project for deployment • 686	
Deployment options for Django	690
Manual deployment • 690	
Containerized deployment • 691	
Platform-as-a-service • 691	
Exercise 15.02 – Deploying the Bookr project • 692	

테스트 코드 모듈화 • 671	
병렬화를 사용하여 테스트를 더 빠르게 실행하기	672
활동 14.01 – Bookr에서 모델 및 뷰 테스트하기	673
요약	674
이 책의 PDF 사본, 코드 번들 및 기타 자료 받기	675

15장: Django 프로젝트 배포 677

기술 요구 사항	677
서버 아키텍처	677
건이콘 • 679	
화이트노이즈 • 679	
배포 체크리스트.....	679
manage.py check --deploy 실행 • 680	
manage.py runserver에서 전환 • 681	
중요 설정 • 681	
비밀 키 • 682	
디버그 • 682	
환경별 설정 • 682	
허용된 호스트 • 682	
캐시 • 683	
데이터베이스 • 683	
이메일 백엔드 및 관련 설정 • 683	
HTTPS • 684	
성능 최적화 • 684	
오류 보고 • 685	
기본 오류 보기 사용자 정의 • 685	
연습 15.01 – Bookr 프로젝트 배포 구성 • 686	
장고를 위한 배포 옵션	690
수동 배포 • 690	
컨테이너화된 배포 • 691	
서비스로서의 플랫폼 • 691	
연습 15.02 – Bookr 프로젝트 배포하기 • 692	

Summary	710
Get this book's PDF copy, code bundle, and more • 711	
Other Books You May Enjoy	714

Index	717
--------------	------------

요약	710
----------	-----

이 책의 PDF 복사본, 코드 번들 및 기타 • 711

당신이 즐길 수 있는 다른 책들	714
-------------------	-----

색인	717
----	-----

Preface

Do you want to develop reliable and secure applications that stand out from the crowd without spending hours on boilerplate code? You've made the right choice by trusting the Django framework, and this book will tell you why. Often referred to as a "batteries included" web development framework, Django comes with all the core features needed to build a standalone application. *Web Development with Django 6* will take you through all the essential concepts and help you explore its power to build real-world applications using Python.

Throughout the book, you'll get to grips with the major features of Django by building a website called Bookr – a repository for book reviews. This end-to-end case study is split into a series of bite-sized projects presented as exercises and activities, allowing you to challenge yourself in an enjoyable and attainable way. As you advance, you'll acquire various practical skills, including how to serve static files to add CSS, JavaScript, and images to your application, implement forms to accept user input, and manage sessions to ensure a reliable user experience. You'll cover everyday tasks that are part of the development cycle of a real-world web application.

By the end of this Django book, you'll have the skills and confidence to creatively develop and deploy your own projects.

Who this book is for

This book is for programmers looking to enhance their web development skills using the Django framework. To fully understand the concepts explained in this book, basic knowledge of Python programming, as well as familiarity with JavaScript, HTML, and CSS, is assumed.

What this book covers

Chapter 1, An Introduction to Django, starts by getting a Django project set up almost immediately. You'll learn how to bootstrap a Django project, respond to web requests, and use HTML templates.

Chapter 2, Models and Migrations, introduces Django data models, the method of persisting data to a SQL database.

Chapter 3, URL Mapping, Views, and Templates, builds on the techniques that were introduced in *Chapter 1, An Introduction to Django*, and explains in greater depth how to route web requests to Python code and render HTML templates.

서문

신뢰할 수 있고 안전한 애플리케이션을 개발하고 싶으신가요? 수많은 보일러플레이트 코드에 시간을 낭비하지 않고도 돋보이게 만들고 싶으신가요? Django 프레임워크를 신뢰한 것은 올바른 선택입니다. 이 책이 그 이유를 알려드릴 것입니다. 종종 "배터리가 포함된" 웹 개발 프레임워크로 언급되는 Django는 독립형 애플리케이션을 구축하는 데 필요한 모든 핵심 기능을 제공합니다. Django 6로 웹 개발을 통해 필수 개념을 배우고 Python을 사용하여 실제 애플리케이션을 구축하는 힘을 탐구할 수 있습니다.

이 책 전반에 걸쳐, 여러분은 Bookr라는 웹사이트를 구축함으로써 Django의 주요 기능을 익히게 될 것입니다. Bookr는 도서 리뷰를 위한 저장소입니다. 이 종합 사례 연구는 연습과 활동으로 제시된 일련의 소규모 프로젝트로 나뉘어 있어, 즐겁고 달성 가능한 방식으로 스스로 도전할 수 있게 해줍니다. 진행하면서 정적 파일을 제공하여 CSS, JavaScript 및 이미지를 애플리케이션에 추가하는 방법, 사용자 입력을 수용하기 위한 양식을 구현하는 방법, 신뢰할 수 있는 사용자 경험을 보장하기 위해 세션을 관리하는 방법 등 다양한 실용적인 기술을 습득하게 될 것입니다. 실제 웹 애플리케이션의 개발 주기의 일상적인 작업을 다룰 것입니다.

이 Django 책의 끝에 이르면, 여러분은 창의적으로 자신의 프로젝트를 개발하고 배포할 수 있는 기술과 자신감을 갖게 될 것입니다.

이 책은 누구를 위한 것인가요

이 책은 Django 프레임워크를 사용하여 웹 개발 기술을 향상시키고자 하는 프로그래머를 위한 것입니다. 이 책에서 설명하는 개념을 완전히 이해하기 위해서는 Python 프로그래밍에 대한 기본 지식과 JavaScript, HTML 및 CSS에 대한 친숙함이 필요합니다.

이 책에서 다루는 내용

1장, Django 소개에서는 거의 즉시 Django 프로젝트를 설정하는 방법을 시작합니다. Django 프로젝트를 부트스트랩하고, 웹 요청에 응답하며, HTML 템플릿을 사용하는 방법을 배우게 됩니다.

2장, 모델 및 마이그레이션에서는 Django 데이터 모델과 SQL 데이터베이스에 데이터를 지속적으로 저장하는 방법을 소개합니다.

3장, URL 매핑, 뷰 및 템플릿에서는 1장, Django 소개에서 소개된 기술을 바탕으로 웹 요청을 Python 코드로 라우팅하고 HTML 템플릿을 렌더링하는 방법을 더 깊이 설명합니다.

Chapter 4, An Introduction to Django Admin, shows how to use Django's built-in Admin GUI to create, update, and delete data stored by your models.

Chapter 5, Serving Static Files, explains how to enhance your website with styles and images and how Django makes managing these files easier.

Chapter 6, Forms, shows you how to collect user input through your website by using Django's Forms module.

Chapter 7, Advanced Form Validation and Model Forms, builds upon *Chapter 6, Forms*, by adding more advanced validation logic to make your forms more powerful.

Chapter 8, Media Serving and File Uploads, shows how to further enhance sites by allowing your users to upload files and serve them with Django.

Chapter 9, Sessions and Authentication, introduces the Django session and shows you how to use it to store user data and authenticate users.

Chapter 10, Advanced Django Admin and Customization, continues from *Chapter 4, An Introduction to Django Admin*. Now that you know more about Django, you can customize Django admin with advanced features. You will also learn about configuring SMTP for Django and Django's Tasks framework.

Chapter 11, Advanced Templating and Class-Based Views, lets you see how to reduce the amount of code you need to write using some of Django's advanced templating features and classes.

Chapter 12, Building a REST API, looks at how to add a REST API to Django to allow programmatic access to your data from different applications.

Chapter 13, Generating CSV, PDF, and Other Binary Files, further expands the capabilities of Django by showing how you can use it to generate more than just HTML.

Chapter 14, Testing Your Django Applications, is an important part of real-world development. This chapter shows how to use the Django and Python testing frameworks to validate your code.

Chapter 15, Deploying a Django Project, finishes the project by showing you how to deploy your project to a virtual server.

To get the most out of this book

In order to follow the instructions in this book, it is assumed that you will be acquainted with writing simple Python code and debugging errors. Also, a basic familiarity with HTML, CSS, and JavaScript is assumed.

4장, Django Admin 소개에서는 Django의 내장 Admin GUI를 사용하여 모델에 저장된 데이터를 생성, 업데이트 및 삭제하는 방법을 보여줍니다.

5장, 정적 파일 제공에서는 웹사이트에 스타일과 이미지를 추가하는 방법과 Django가 이러한 파일을 관리하는 방법을 설명합니다.

6장, 폼에서는 Django의 Forms 모듈을 사용하여 웹사이트를 통해 사용자 입력을 수집하는 방법을 보여줍니다.

7장, 고급 폼 검증 및 모델 폼에서는 6장, 폼을 기반으로 하여 폼을 더 강력하게 만들기 위해 더 고급 검증 논리를 추가합니다.

8장, 미디어 제공 및 파일 업로드에서는 사용자가 파일을 업로드하고 Django로 제공할 수 있도록 하여 사이트를 더욱 향상시키는 방법을 보여줍니다.

9장, 세션 및 인증에서는 Django 세션을 소개하고 이를 사용하여 사용자 데이터를 저장하고 사용자를 인증하는 방법을 보여줍니다.

10장, 고급 Django Admin 및 사용자 정의에서는 4장, Django Admin 소개에서 계속됩니다. 이제 Django에 대해 더 많이 알게 되었으므로 고급 기능으로 Django admin을 사용자 정의할 수 있습니다. 또한 Django의 SMTP 구성 및 Django의 작업 프레임워크에 대해 배우게 됩니다.

11장, 고급 템플릿 및 클래스 기반 뷰에서는 Django의 고급 템플릿 기능과 클래스를 사용하여 작성해야 하는 코드의 양을 줄이는 방법을 보여줍니다.

12장, REST API 구축, 다양한 애플리케이션에서 데이터에 프로그래밍 방식으로 접근할 수 있도록 Django에 REST API를 추가하는 방법을 살펴봅니다.

13장, CSV, PDF 및 기타 이진 파일 생성, HTML 이상의 것을 생성하는 방법을 보여줌으로써 Django의 기능을 더욱 확장합니다.

14장, Django 애플리케이션 테스트, 실제 개발의 중요한 부분입니다. 이 장에서는 Django 및 Python 테스트 프레임워크를 사용하여 코드를 검증하는 방법을 보여줍니다.

15장, Django 프로젝트 배포, 가상 서버에 프로젝트를 배포하는 방법을 보여주며 프로젝트를 마무리합니다.

이 책을 최대한 활용하기 위해

이 책의 지침을 따르기 위해서는 간단한 Python 코드를 작성하고 오류를 디버깅하는데 익숙하다고 가정합니다. 또한 HTML, CSS 및 JavaScript에 대한 기본적인 친숙함이 있다고 가정합니다.

The code in this book was tested with Python 3.12 and Django 6.0. For best results, work with these versions.

Download the example code files

This book includes a complete downloadable code bundle containing all the example projects and files used throughout the chapters. We recommend downloading the bundle so you can follow along smoothly and experiment with the examples.

Use the bundle as a practical starting point. Modify it, extend it, and apply what you learn by creating your own variations as you progress through the chapters.

Get the code bundle

If you bought the book directly from Packt:

1. Go to packtpub.com
2. Click your profile picture and select Your Orders
3. Find this book and click Download Code

If you bought this book from Amazon or any other channel partner:

1. Go to packtpub.com/unlock or scan the following QR code:



2. Search for this book
3. Sign up or log in to your free Packt account
4. Upload your proof of purchase and download the code bundle locally

Usage note: You're free to use and modify this code for personal learning and non-commercial projects.

Download the color images

Your purchase includes a color, DRM-free PDF copy of this book, ideal for viewing color images, screenshots, and diagrams. Refer to *Free benefits with your book section* at the end of the *Preface* to unlock your PDF copy.

Conventions used

There are a number of text conventions used throughout this book.

이 책의 코드는 Python 3.12와 Django 6.0에서 테스트되었습니다. 최상의 결과를 위해 이 버전으로 작업하세요.

예제 코드 파일 다운로드

이 책에는 각 장에서 사용된 모든 예제 프로젝트와 파일이 포함된 완전한 다운로드 가능한 코드 번들이 포함되어 있습니다. 번들을 다운로드하여 원활하게 따라하고 예제를 실험하는 것을 권장합니다.

번들을 실용적인 출발점으로 사용하세요. 수정하고 확장하며, 장을 진행하면서 배운 내용을 바탕으로 자신의 변형을 만들어 적용하세요.

코드 번들 받기

Packt에서 직접 책을 구매한 경우:

1. packtpub.com으로 이동
2. 프로필 사진을 클릭하고 '내 주문'을 선택하세요.
3. 이 책을 찾아서 다운로드 코드를 클릭하세요

아마존이나 다른 채널 파트너에서 이 책을 구매한 경우:

1. packtpub.com/unlock으로 가거나 다음 QR 코드를 스캔하세요:



2. 이 책을 검색하세요
3. 무료 Packt 계정에 가입하거나 로그인하세요
4. 구매 증명을 업로드하고 코드를 로컬로 다운로드하세요

사용 참고: 개인 학습 및 비상업적 프로젝트를 위해 이 코드를 자유롭게 사용하고 수정할 수 있습니다.

색상 이미지를 다운로드하세요

귀하의 구매에는 이 책의 색상, DRM-free PDF 사본이 포함되어 있으며, 색상 이미지, 스크린샷 및 다이어그램을 보기 위해 이상적입니다. PDF 사본을 잠금 해제하려면 서문 끝의 책과 함께 제공되는 무료 혜택 섹션을 참조하세요.

사용된 규칙

이 책 전반에 걸쳐 사용되는 여러 텍스트 규칙이 있습니다.

CodeInText: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: "Execute the `manage.py runserver` command."

A block of code is set as follows:

```
from django.apps import AppConfig

class ReviewsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'reviews'
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    authentication_classes = [TokenAuthentication]
    permission_classes = [IsAuthenticated]
```

Any command-line input or output is written as follows:

```
django-admin startproject bookr
```

Bold: Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: "Open the **Preferences** (macOS) or **Settings** (Windows/Linux) window inside PyCharm."

Note

Warnings or important notes appear like this.

Tip

Tips and tricks appear like this.

CodeInText: 텍스트 내의 코드 단어, 데이터베이스 테이블 이름, 폴더 이름, 파일 이름, 파일 확장자, 경로 이름, 더미 URL, 사용자 입력 및 트위터 핸들을 나타냅니다. 예: "manage.py runserver 명령을 실행하십시오."

코드 블록은 다음과 같이 설정됩니다:

```
from django.apps import AppConfig

class ReviewsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'reviews'
```

코드 블록의 특정 부분에 주의를 끌고자 할 때, 관련된 줄이나 항목은 굵게 설정됩니다:

```
class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    authentication_classes = [TokenAuthentication]
    permission_classes = [IsAuthenticated]
```

명령줄 입력 또는 출력은 다음과 같이 작성됩니다:

```
django-admin startproject bookr
```

굵게: 새로운 용어, 중요한 단어 또는 화면에서 볼 수 있는 단어를 나타냅니다. 예를 들어, 메뉴나 대화 상자에 있는 단어는 다음과 같이 텍스트에 나타납니다. 예: "PyCharm에서 Preferences (macOS) 또는 Settings (Windows/Linux) 창을 엽니다."

참고

경고 또는 중요한 참고 사항은 다음과 같이 나타납니다.

팁

팁과 요령은 이렇게 나타납니다.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book or have any general feedback, please email us at customercare@packt.com and mention the book's title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit <http://www.packt.com/submit-errata>, click **Submit Errata**, and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packt.com/>.

Free benefits with your book

This book includes free benefits designed to support your learning and help you apply what you learn effectively. Activate them now for instant access (see the *How to unlock* section for instructions).

Here's a quick overview of what you can instantly unlock with your purchase:

연락하기

독자들의 피드백은 항상 환영합니다.

일반 피드백: 이 책의 어떤 부분에 대해 질문이 있거나 일반적인 피드백이 있으시면 customercare@packt.com으로 이메일을 보내주시고 메시지 제목에 책 제목을 언급해 주시기 바랍니다.

정오표: 저희는 콘텐츠의 정확성을 보장하기 위해 모든 주의를 기울였지만, 실수가 발생할 수 있습니다. 이 책에서 실수를 발견하셨다면 저희에게 보고해 주시면 감사하겠습니다. <http://www.packt.com/submit-errata>를 방문하시고, 정오표 제출을 클릭한 후 양식을 작성해 주십시오.

해적 행위: 인터넷에서 저희 작품의 불법 복사본을 발견하신 경우, 저희에게 위치 주소나 웹사이트 이름을 제공해 주시면 감사하겠습니다. 자료에 대한 링크와 함께 copyright@packt.com으로 연락해 주십시오.

저자에 관심이 있으신 경우: 전문 지식이 있는 주제가 있고 책을 쓰거나 기여하는 데 관심이 있으시면 <http://authors.packt.com/>를 방문해 주십시오.

책과 함께 제공되는 무료 혜택

이 책에는 학습을 지원하고 배운 내용을 효과적으로 적용하는 데 도움이 되는 무료 혜택이 포함되어 있습니다. 즉시 액세스를 위해 지금 활성화하세요(지침은 잠금 해제 방법 섹션을 참조하십시오).

구매를 통해 즉시 잠금 해제할 수 있는 내용에 대한 간단한 개요입니다:



Complete Code Bundle

Download the full source code for this book's examples and projects.



DRM-Free PDF Version

Download DRM-free PDF and ePub copies of this book.



7-Day Packt Library Access

Get 7-day unlimited access to 8,000+ books and videos. No credit card required.

Available for first-time Packt+ trial users only.



Next-Gen Reader Access

Read this book on Packt Reader with progress sync, dark mode and note-taking.

How to unlock

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.





Complete Code Bundle

Download the full source code for this book's examples and projects.



DRM-Free PDF Version

Download DRM-free PDF and ePub copies of this book.



7-Day Packt Library Access

Get 7-day unlimited access to 8,000+ books and videos. No credit card required.

Available for first-time Packt+ trial users only.



Next-Gen Reader Access

Read this book on Packt Reader with progress sync, dark mode and note-taking.

잠금 해제 방법

QR 코드를 스캔하세요 (또는 packtpub.com/unlock으로 가세요). 이 책의 이름으로 검색하고,판을 확인한 후 페이지의 단계를 따르세요.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

Share your thoughts

Once you've read *Web development with Django 6*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback.



<https://packt.link/r/1836202075>

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

UNLOCK NOW

참고: 송장을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

당신의 생각을 공유하세요

Django 6로 웹 개발을 읽으신 후, 당신의 생각을 듣고 싶습니다! 아래의 QR 코드를 스캔하여 이 책의 아마존 리뷰 페이지로 바로 가서 피드백을 공유하세요.



<https://packt.link/r/1836202075>

당신의 리뷰는 우리와 기술 커뮤니티에 중요하며, 우리가 우수한 품질의 콘텐츠를 제공하고 있는지 확인하는 데 도움이 될 것입니다.

1

Introduction to Django

"The web framework for perfectionists with deadlines." It's a tagline that aptly describes Django, a framework that has been around for nearly 20 years. It is battle-tested and widely used, with more and more people using it every day. All this might make you think that Django is old and no longer relevant. On the contrary, its longevity has proved that its API is reliable and consistent, and even those who learned Django v1.0 in 2007 can mostly write the same code for Django 6 today. Django is still in active development, with bug fixes and security patches being released regularly.

Like Python, the language in which it is written, Django is easy to learn, yet powerful and flexible enough to grow with your needs. It is a "batteries-included" framework – in other words, you do not have to find and install many other libraries or components to get your application up and running. Other frameworks, such as Flask or Pylons, require manually installing third-party frameworks for database connections or template rendering. Instead, Django has built-in support for database querying, URL mapping, and template rendering (we'll go into detail on what these mean soon). However, just because Django is easy to use doesn't mean it is limited. Django is used by many large sites, including Disqus, Instagram, Mozilla, Spotify, Open edX, OpenStack, and National Geographic.

Where does Django fit into the web? When talking about web frameworks, you might think of frontend JavaScript frameworks such as ReactJS, Angular, or Vue. These frameworks are used to enhance or add interactivity to already-generated web pages. Django sits at the layer beneath these tools and is instead responsible for routing a URL, fetching data from databases, rendering templates, and handling form input from users. However, this does not mean you must pick one or the other; JavaScript frameworks can be used to enhance the output from Django or interact with a REST API generated by Django.

In this book, we will build a Django project using the same methods that professional Django developers use every day. The project is called **Bookr** and allows you to browse and add books

1

장고 소개

"마감이 있는 완벽주의자를 위한 웹 프레임워크." 이는 거의 20년 동안 존재해온 장고를 적절히 설명하는 태그라인입니다. 장고는 전투 테스트를 거쳤고 널리 사용되며, 매일 더 많은 사람들이 사용하고 있습니다. 이 모든 것이 장고가 오래되었고 더 이상 관련이 없다고 생각하게 만들 수 있습니다. 반대로, 그 지속성은 API가 신뢰할 수 있고 일관성이 있음을 증명했으며, 2007년에 장고 v1.0을 배운 사람들도 오늘날 장고 6을 위해 대부분 같은 코드를 쓸 수 있습니다. 장고는 여전히 활발히 개발되고 있으며, 버그 수정 및 보안 패치가 정기적으로 출시되고 있습니다.

파이썬처럼, 장고가 작성된 언어는 배우기 쉽지만, 강력하고 유연하여 필요에 따라 성장할 수 있습니다. 장고는 "배터리 포함" 프레임워크입니다. 즉, 애플리케이션을 실행하기 위해 많은 다른 라이브러리나 구성 요소를 찾고 설치할 필요가 없습니다. Flask나 Pylons와 같은 다른 프레임워크는 데이터베이스 연결이나 템플릿 렌더링을 위해 서드파티 프레임워크를 수동으로 설치해야 합니다. 대신, 장고는 데이터베이스 쿼리, URL 매핑 및 템플릿 렌더링에 대한 내장 지원을 제공합니다(이것들이 의미하는 바에 대해서는 곧 자세히 설명하겠습니다). 그러나 장고가 사용하기 쉽다고 해서 제한적이라는 의미는 아닙니다. 장고는 Disqus, Instagram, Mozilla, Spotify, Open edX, OpenStack 및 National Geographic을 포함한 많은 대형 사이트에서 사용됩니다.

장고는 웹에서 어디에 적합합니까? 웹 프레임워크에 대해 이야기할 때, ReactJS, Angular 또는 Vue와 같은 프론트엔드 자바스크립트 프레임워크를 생각할 수 있습니다. 이러한 프레임워크는 이미 생성된 웹 페이지에 상호작용을 추가하거나 향상시키는 데 사용됩니다. 장고는 이러한 도구 아래의 레이어에 위치하며, 대신 URL 라우팅, 데이터베이스에서 데이터 가져오기, 템플릿 렌더링 및 사용자로부터의 폼 입력 처리를 담당합니다. 그러나 이것이 하나를 선택해야 한다는 의미는 아닙니다. 자바스크립트 프레임워크는 장고의 출력을 향상시키거나 장고가 생성한 REST API와 상호작용하는 데 사용할 수 있습니다.

이 책에서는 전문 장고 개발자들이 매일 사용하는 방법을 사용하여 장고 프로젝트를 구축할 것입니다. 이 프로젝트는 Bookr라고 하며, 책을 탐색하고 추가할 수 있습니다.

and book reviews. This book is divided into four parts. In the first part, we'll start with the basics of scaffolding a Django app, quickly build some pages, and serve them with the Django development server. You'll be able to add data to the database using the Django admin site.

This chapter introduces you to Django and its role in web development. You will begin by understanding how the model-view-template paradigm works and how Django processes HTTP requests and responses. Equipped with the basic concepts, you'll create your first Django project, called *Bookr*, an application for adding, viewing, and managing book reviews. It's an application you'll keep enhancing and adding features to throughout this book. You will then learn about the `manage.py` command (used to orchestrate Django actions). You will use this command to start the Django development server and test whether the code you wrote so far works as expected. You will also learn how to work with PyCharm, a popular Python IDE, which you'll be using throughout this book. You will use it to write code that returns a response to your web browser. Finally, you'll learn how to use PyCharm's debugger to troubleshoot problems with your code. By the end of this chapter, you'll have the necessary skills to start creating projects using Django.

We will cover the following topics in this chapter:

- Scaffolding a Django project and app
- Understanding the model-view-template paradigm
- Exploring the Django project structure
- Introducing Django views
- Exploring URL mapping details
- Exploring Django settings
- Finding HTML templates in app directories
- Debugging and dealing with errors
- Activity 1.01 – creating a site welcome screen
- Activity 1.02 – a book search scaffold

By the end of the book, you will have enough experience to design and build your own Django project from start to finish.

Technical requirements

Throughout this book, you will be writing code. If you need to refer to the completed code for this chapter, you can find it in the `Chapter01` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

그리고 책 리뷰. 이 책은 네 부분으로 나뉘어 있습니다. 첫 번째 부분에서는 Django 앱의 기본을 시작하고, 빠르게 몇 개의 페이지를 만들고, Django 개발 서버로 제공할 것입니다. Django 관리 사이트를 사용하여 데이터베이스에 데이터를 추가할 수 있습니다.

이 장에서는 Django와 웹 개발에서의 역할을 소개합니다. 모델-뷰-템플릿 패러다임이 어떻게 작동하는지, Django가 HTTP 요청과 응답을 어떻게 처리하는지 이해하는 것으로 시작할 것입니다. 기본 개념을 갖춘 후, Bookr라는 첫 번째 Django 프로젝트를 만들 것입니다. 이는 책 리뷰를 추가하고, 보고, 관리하는 애플리케이션입니다. 이 책을 통해 계속해서 기능을 향상시키고 추가할 애플리케이션입니다. 그런 다음 Django 작업을 조정하는 데 사용되는 `manage.py` 명령에 대해 배우게 됩니다. 이 명령을 사용하여 Django 개발 서버를 시작하고 지금까지 작성한 코드가 예상대로 작동하는지 테스트할 것입니다. 또한 이 책 전반에 걸쳐 사용할 인기 있는 Python IDE인 PyCharm을 사용하는 방법도 배울 것입니다. 이를 사용하여 웹 브라우저에 응답을 반환하는 코드를 작성할 것입니다. 마지막으로, PyCharm의 디버거를 사용하여 코드 문제를 해결하는 방법을 배울 것입니다. 이 장이 끝날 때쯤이면 Django를 사용하여 프로젝트를 시작하는 데 필요한 기술을 갖추게 될 것입니다.

이 장에서는 다음 주제를 다룰 것입니다

- 다: Django 프로젝트 및 앱의 스캐폴딩
- 모델-뷰-템플릿 패러다임
- 해 Django 프로젝트 구조 탐색
- Django 뷰 소개
- URL 매핑 세부정보 탐색
- Django 설정 탐색
- 앱 디렉토리에서 HTML 템플릿 찾기 오류
- 디버깅 및 처리활동 1.01 – 사이트 환영
- 화면 만들기활동 1.02 – 도서 검색 스캐폴드

책의 끝에 이르면, 당신은 처음부터 끝까지 자신의 Django 프로젝트를 설계하고 구축할 수 있는 충분한 경험을 갖게 될 것입니다.

기술 요구 사항

이 책 전반에 걸쳐 코드를 작성하게 됩니다. 이 장의 완성된 코드를 참조해야 하는 경우, 이 책의 GitHub 리포지토리의 Chapter01 폴더에서 찾을 수 있습니다. 리포지토리 링크에 접근하려면, 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르세요.

Your purchase includes a free PDF copy + code bundle

Your purchase includes a DRM-free PDF copy of this book, the code bundle, and additional exclusive extras. See the *Free benefits with your book* section in the *Preface* to unlock them instantly and maximize your learning.

Creating a virtual environment with pyenv

Before diving deeply into the theory behind Django paradigms and HTTP requests, we'll show you how easy it is to get a Django project up and running. After this first section and exercise, you will have created a Django project, made a request to it with your browser, and seen the response.

As a Python developer, you may be working with many applications on your system that have different and even conflicting dependencies. The examples in this book use Python 3.12 and Django 6.0. However, you may need to maintain other versions of Python for your other projects that may not even be compatible with Django 6.0.

In addition, operating systems such as macOS and Linux ship with Python installed, and it is frequently used as a scripting language by operating systems. For this reason, it is best practice to develop with a different Python installation that will not disrupt operating system functionality if conflicts are introduced.

For these reasons, virtual environments are an ideal technique for developing with multiple environments. When you install Python, Django, and its dependencies, they are quarantined in their own user-specific environment and do not affect the behavior of other Python software that you are developing and using.

In this book, we will use pyenv to configure Python virtual environments.

Installing pyenv on Windows

pyenv for Windows is available from <https://github.com/pyenv-win/pyenv-win>. It can be installed from the PowerShell with this command:

```
Invoke-WebRequest-UseBasicParsing-Uri"https://raw.githubusercontent.com/pyenv-win/pyenv-win/master/pyenv-win/install-pyenv-win.ps1"-OutFile"./install-pyenv-win.ps1"; &"./install-pyenv-win.ps1"
```


구매 시 무료 PDF 사본 + 코드 번들이 포함됩니다.

구매 시 이 책의 DRM 없는 PDF 사본, 코드 번들 및 추가 독점 보너스가 포함됩니다. 즉시 잠금을 해제하고 학습을 극대화하려면 서문에 있는 책과 함께 제공되는 무료 혜택 섹션을 참조하세요.

pyenv로 가상 환경 만들기

장고 패러다임과 HTTP 요청 뒤에 있는 이론에 깊이 들어가기 전에, 장고 프로젝트를 쉽게 시작하고 실행하는 방법을 보여드리겠습니다. 이 첫 번째 섹션과 연습 후에는 장고 프로젝트를 만들고, 브라우저로 요청을 보내고, 응답을 확인하게 될 것입니다.

파이썬 개발자로서, 시스템에서 서로 다른 심지어 충돌하는 종속성을 가진 많은 애플리케이션을 작업하고 있을 수 있습니다. 이 책의 예제는 Python 3.12와 Django 6.0을 사용합니다. 그러나 Django 6.0과 호환되지 않을 수 있는 다른 프로젝트를 위해 다른 버전의 Python을 유지해야 할 수도 있습니다.

또한, macOS 및 Linux와 같은 운영 체제는 Python이 설치된 상태로 제공되며, 운영 체제에서 스크립팅 언어로 자주 사용됩니다. 이러한 이유로, 충돌이 발생할 경우 운영 체제 기능을 방해하지 않는 다른 Python 설치로 개발하는 것이 최선의 방법입니다.

이러한 이유로, 가상 환경은 여러 환경에서 개발하는 데 이상적인 기술입니다. Python, Django 및 그 종속성을 설치하면, 이들은 사용자 특정 환경에 격리되어 다른 Python 소프트웨어의 동작에 영향을 미치지 않습니다.

이 책에서는 pyenv를 사용하여 Python 가상 환경을 구성할 것입니다.

Windows에 pyenv 설치하기

pyenv for id x_9d81e437 Windows는 <https://github.com/pyenv-win/pyenv-win> 에서 사용할 수 있습니다. PowerShell에서 다음 명령어로 설치할 수 있습니다:

```
Invoke-WebRequest-UseBasicParsing-Uri"https://raw.githubusercontent.com/pyenv-win/pyenv-win/master/pyenv-win/install-pyenv-win.ps1"-OutFile"./install-pyenvwin.ps1"; &"./install-pyenv-win.ps1"
```

Installing pyenv on macOS and Linux

If you use the Homebrew package manager on macOS, you can install these two packages for the required functionality:

```
brew install pyenv pyenv-virtualenv
```

Otherwise, on macOS and Linux, pyenv can be installed by running the following at the command line.

```
curl https://pyenv.run | bash
```

On Linux, you will need to include these lines in your `.bashrc` file.

```
export PYENV_ROOT="$HOME/.pyenv"
[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
```

On macOS, include these lines in your `.zshrc` file.

```
export PYENV_ROOT="$HOME/.pyenv"
command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
```

Creating a virtual environment with pyenv

Now that we have installed pyenv, we can use it to create an environment for our Django project. As we are using Python 3.12 in this book, we will start by installing this version under our local pyenv environment. We do this by using the `pyenv install` command.

```
pyenv install 3.12
```

We can check the setup using the `pyenv versions` command.

```
% pyenv versions
* system (set by /Users/chrisguest/.pyenv/version)
  3.12.0
```

macOS 및 Linux에 pyenv 설치하기

macOS에서 Homebrew 패키지 관리자를 사용하는 경우, 필요한 기능을 위해 이 두 패키지를 설치할 수 있습니다:

```
brew install pyenv pyenv-virtualenv
```

그렇지 않으면, macOS 및 Linux에서 pyenv는 명령줄에서 다음을 실행하여 설치할 수 있습니다.

```
curl https://pyenv.run | bash
```

Linux에서는 .bashrc 파일에 다음 줄을 포함해야 합니다.

```
export PYENV_ROOT="$HOME/.pyenv"
[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
```

Onidix_443331b0 macOS, .zshrc 파일에 이 줄을 포함하세요.

```
export PYENV_ROOT="$HOME/.pyenv"
command -v pyenv >/dev/null || export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
```

pyenv로 가상 환경 만들기

이제 idx_a8194fe3pyenv를 설치했으므로, Django project를 위한 idx_0441550a환경을 만들기 위해 사용할 수 있습니다. 이 책에서는 Python 3.12를 사용하고 있으므로, 로컬 pyenv 환경에 이 버전을 설치하는 것으로 시작하겠습니다. 우리는 pyenv install 명령어를 사용하여 이를 수행합니다.

```
pyenv install 3.12
```

pyenv versions 명령어를 사용하여 설정을 확인할 수 있습니다.

```
% pyenv versions
* system (set by /Users/chrisquest/.pyenv/version)
3.12.0
```

With our desired Python version installed, we can create a virtual environment. We will call it `djangoenv`.

```
pyenv virtualenv 3.12 djangoenv
```

Now, with our virtual environment created, we will activate it so that subsequent commands in the terminal are interpreted using this environment using the `pyenv local` command.

```
pyenv local djangoenv
```

With our environment created, we can install the Python modules that we will need for the project. In our case, as we are developing Django projects using Django 5.1, we will install this version of Django using the `pip install` command.

```
pip install django==6.0
```

Most of the exercises and activities in this book have the necessary modules listed in a `requirements.txt` file. For example, this chapter has this file:

```
Web-Development-with-Django-Third-Edition/Chapter01/requirements.txt
```

In order to install the modules from a specific `requirements.txt` file, you can use this command:

```
pip install -r requirements.txt
```

If you need to switch back to the system version of Python, you can use this command.

```
% pyenv global system
```

With a virtual environment created and activated, we are ready to get working with Django.

Scaffolding a Django project and app

Before diving deeply into the theory behind Django paradigms and HTTP requests, we'll show you how easy it is to get a Django project up and running. After this first section and exercise, you will have created a Django project, made a request to it with your browser, and seen the response.

A Django project is a directory that contains all the data for your project – code, settings, templates, and assets. It is created and scaffolded by running the `django-admin` command on

원하는 파이썬 버전이 설치되었으므로 가상 환경을 만들 수 있습니다. 우리는 이를 `djangoenv`라고 부를 것입니다.

```
pyenv virtualenv 3.12 djangoenv
```

이제 가상 환경이 생성되었으므로 이를 활성화하여 터미널에서 이후 명령이 이 환경을 사용하여 해석되도록 하겠습니다. `pyenv local` 명령을 사용합니다.

```
pyenv local djangoenv
```

환경이 생성되었으므로 프로젝트에 필요한 파이썬 모듈을 설치할 수 있습니다. 우리의 경우, 장고 5.1을 사용하여 장고 프로젝트를 개발하고 있으므로 `pip install` 명령을 사용하여 이 버전의 장고를 설치할 것입니다.

```
pip install django==6.0
```

이 책의 대부분의 연습과 활동에는 `requirements.txt` 파일에 필요한 모듈이 나열되어 있습니다. 예를 들어, 이 장에는 이 파일이 있습니다:

```
Web-Development-with-Django-Third-Edition/Chapter01/requirements.txt
```

특정 `requirements.txt` 파일에서 모듈을 설치하려면 다음 명령어를 사용할 수 있습니다:

```
pip install -r requirements.txt
```

시스템 버전의 Python으로 다시 전환해야 하는 경우, 이 명령어를 사용할 수 있습니다.

```
% pyenv global system
```

가상 환경이 생성되고 활성화되면, Django로 작업할 준비가 완료되었습니다.

Django 프로젝트 및 앱 스캐폴딩

Django 패러다임과 HTTP 요청 뒤에 있는 이론에 깊이 들어가기 전에, Django 프로젝트를 쉽게 시작하고 실행하는 방법을 보여드리겠습니다. 이 첫 번째 섹션과 연습 후, 여러분은 Django 프로젝트를 생성하고, 브라우저로 요청을 보내고, 응답을 확인하게 될 것입니다.

Django 프로젝트는 코드, 설정, 템플릿 및 자산을 포함하는 모든 데이터를 포함하는 디렉토리입니다. 이는 `django-admin` 명령어를 실행하여 생성되고 스캐폴딩됩니다.

the command line with the `startproject` argument and providing the project name. For example, to create a Django project with the name `myproject`, the command that is run is as follows:

```
django-admin startproject myproject
```

This will create the `myproject` directory, which Django populates with the necessary files to run the project. Inside the `myproject` directory are two files (shown in *Figure 1.1*):

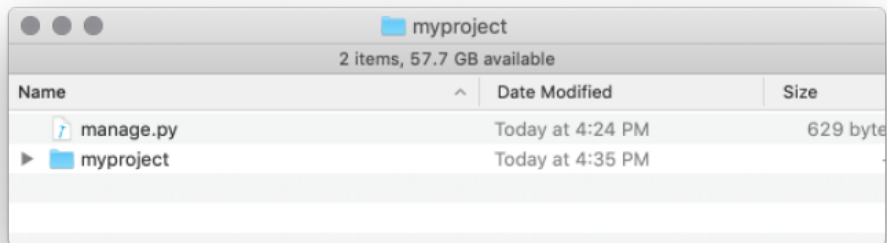


Figure 1.1 – The project directory for myproject

`manage.py` is a Python script that is executed at the command line to interact with your project. We will use it to start the **Django development server**, a development web server you will use to interact with your Django project on your local computer. Like `django-admin`, commands are passed in on the command line. Unlike `django-admin`, this script is not mapped in your system path, so we must execute it using Python. We will need to use the command line to do that. For example, inside the project directory, we run the following command:

```
python manage.py runserver
```

This passes the `runserver` command to the `manage.py` script, which starts the Django dev server. We will examine more of the commands that `manage.py` accepts later. When interacting with `manage.py` in this way, we call these management commands. For example, we might say that we are "executing the `runserver` management command."

The `startproject` command also created a directory with the same name as the project – in this case, `myproject`. This is a Python package that contains settings and some other configuration files that your project needs to run. We will examine its contents later.

startproject 인수와 프로젝트 이름을 제공하여 명령줄에서. 예를 들어, myproject라는 이름의 Django 프로젝트를 생성하려면 실행되는 명령은 다음과 같습니다:

```
django-admin startproject myproject
```

이 명령은 Django가 프로젝트를 실행하는 데 필요한 파일로 채워진 myproject 디렉토리를 생성합니다. myproject 디렉토리 안에는 두 개의 파일이 있습니다(그림 1.1에 표시됨):

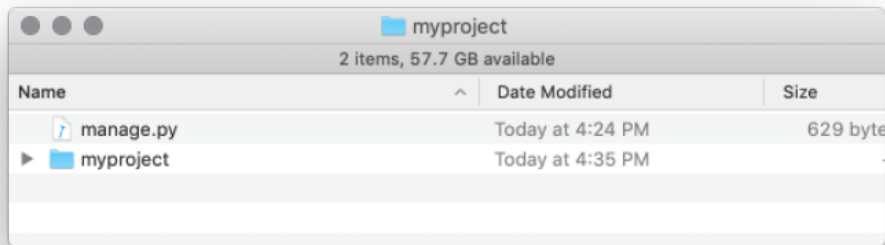


그림 1.1 - myproject의 프로젝트 디렉토리

manage.py는 명령줄에서 실행되어 프로젝트와 상호작용하는 Python 스크립트입니다. 우리는 이를 사용하여 Django 개발 서버를 시작할 것입니다. 이는 로컬 컴퓨터에서 Django 프로젝트와 상호작용하는 데 사용할 개발 웹 서버입니다. django-admin과 마찬가지로 명령은 명령줄에서 전달됩니다. django-admin과는 달리 이 스크립트는 시스템 경로에 매핑되어 있지 않으므로 Python을 사용하여 실행해야 합니다. 이를 위해 명령줄을 사용해야 합니다. 예를 들어, 프로젝트 디렉토리 안에서 다음 명령을 실행합니다:

```
python manage.py runserver
```

이 명령은 runserver 명령을 manage.py 스크립트에 전달하여 Django 개발 서버를 시작합니다. manage.py가 수용하는 명령에 대해 더 살펴보겠습니다. 이렇게 manage.py와 상호작용할 때 우리는 이를 관리 명령이라고 부릅니다. 예를 들어, 우리는 "runserver 관리 명령을 실행하고 있다"고 말할 수 있습니다.

startproject 명령은 또한 프로젝트와 동일한 이름의 디렉토리를 생성했습니다 - 이 경우 myproject입니다. 이는 프로젝트가 실행되는 데 필요한 설정 및 기타 구성 파일을 포함하는 Python 패키지입니다. 우리는 나중에 그 내용을 살펴보겠습니다.

After starting the Django project, the next thing to do is to start a Django app. We should try to segregate our Django project into different apps, grouped by functionality. For example, with Bookr, we will have a *reviews* app. This will hold all the code, HTML, assets, and database classes specific to working with book reviews. If we decide to expand Bookr to sell books as well, we might add a store application, containing the files for the bookstore. Apps are created with the `startapp` management command, passing in the application name. Here is an example:

```
python manage.py startapp myapp
```

This creates the app directory (`myapp`) inside the project directory. Django automatically populates this with files for the app that are ready to be filled in when you start developing. We'll examine these files and discuss what makes a good app in the *Django apps* section.

Now that we've introduced the basic commands to scaffold a Django project and application, let's put them into practice by starting the *bookr* project in the first exercise of this book.

Exercise 1.01 – creating a project and app, and starting the development server

As mentioned before, throughout this book, we will be building a book review website named Bookr. It will allow you to add fields for publishers, contributors, books, and reviews. A publisher will publish one or more books, and each book will have one or more contributors (author, editor, co-author, and so on). Only admin users will be allowed to modify these models. Once a user has signed up for an account on the site, they will be able to start adding reviews for a book.

In this exercise, you will scaffold the *bookr* Django project, test that Django is working by running the development server, and then create the *reviews* Django app.

You should already have a virtual environment set up with Django installed. To know how to do that, you can refer to the *Preface*. Once you're ready, let's start by creating the *bookr* project:

1. Open the terminal and run the command to create the *bookr* project directory and the default subfolders:

```
django-admin startproject bookr
```

This command does not generate any output but will create a folder called `bookr` inside the directory in which you ran the command. You can look inside this directory and see the items we described before for the `myproject` example – the `bookr` package directory and the `manage.py` file.

장고 프로젝트를 시작한 후, 다음으로 해야 할 일은 장고 앱을 시작하는 것입니다. 우리는 기능별로 그룹화된 다양한 앱으로 장고 프로젝트를 분리하려고 해야 합니다. 예를 들어, Bookr와 함께 리뷰 앱을 가질 것입니다. 이 앱은 책 리뷰 작업에 특정한 모든 코드, HTML, 자산 및 데이터베이스 클래스를 보유할 것입니다. 만약 우리가 Bookr를 확장하여 책을 판매하기로 결정한다면, 서점 파일을 포함하는 스토어 애플리케이션을 추가할 수 있습니다. 앱은 `startapp` 관리 명령어로 생성되며, 애플리케이션 이름을 전달합니다. 다음은 예입니다:

```
python manage.py startapp myapp
```

이 명령은 프로젝트 디렉토리 내에 앱 디렉토리(myapp)를 생성합니다. 장고는 이 디렉토리를 자동으로 채워 개발을 시작할 때 채울 준비가 된 파일들로 구성합니다. 우리는 이 파일들을 살펴보고 장고 앱 섹션에서 좋은 앱을 만드는 것에 대해 논의할 것입니다.

이제 장고 프로젝트와 애플리케이션을 스캐폴딩하기 위한 기본 명령어를 소개했으니, 이 책의 첫 번째 연습에서 bookr 프로젝트를 시작하여 이를 실천해 봅시다.

연습 1.01 – 프로젝트 및 앱 생성, 개발 서버 시작

앞서 언급했듯이, 이 책 전반에 걸쳐 우리는 Bookr라는 책 리뷰 웹사이트를 구축할 것입니다. 이 웹사이트는 출판사, 기여자, 책 및 리뷰를 위한 필드를 추가할 수 있게 해줄 것입니다. 출판사는 하나 이상의 책을 출판하며, 각 책은 하나 이상의 기여자(저자, 편집자, 공동 저자 등)를 가질 것입니다. 오직 관리자 사용자만이 이러한 모델을 수정할 수 있습니다. 사용자가 사이트에 계정을 등록하면, 그들은 책에 대한 리뷰를 추가할 수 있게 됩니다.

이 연습에서는 bookr 장고 프로젝트를 스캐폴딩하고, 개발 서버를 실행하여 장고가 작동하는지 테스트한 후, 리뷰 장고 앱을 생성할 것입니다.

이미 장고가 설치된 가상 환경이 설정되어 있어야 합니다. 이를 설정하는 방법은 서문을 참조할 수 있습니다. 준비가 되면, bookr 프로젝트를 생성하는 것으로 시작합시다:

1. 터미널을 열고 bookr 프로젝트 디렉토리나 기본 하위 폴더를 생성하는 명령을 실행합니다:

```
django-admin startproject bookr
```

이 명령은 출력 결과를 생성하지 않지만, 명령을 실행한 디렉토리 안에 bookr라는 폴더를 생성합니다. 이 디렉토리 안을 들여다보면 myproject 예제에서 설명한 항목들 – bookr 패키지 디렉토리와 `manage.py` 파일을 볼 수 있습니다.

We can now test that the project and Django are set up correctly by running the Django development server. Starting the server is done with the `manage.py` script.

2. In your terminal (or Command Prompt), change to the bookr project directory (using the `cd` command), and then run the `manage.py runserver` command as follows:

```
python manage.py runserver
```

This command starts the Django development server. You should get output similar to the following:

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly
until you apply the migrations for app(s): admin, auth, contenttypes,
sessions.
Run 'python manage.py migrate' to apply them.
December 06, 2025 - 13:10:57
Django version 6.0, using settings 'bookr.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

You will probably receive some warnings about unapplied migrations, but that's okay for now.

3. Open up a web browser and go to `http://127.0.0.1:8000/`, which will show you the Django welcome screen (*Figure 1.2*). If you see this, you will know that your Django project was created successfully and all is working fine for now:

이제 Djangodevelopment 서버를 실행하여 프로젝트와 Django가 올바르게 설정되었는지 테스트할 수 있습니다. 서버 시작은 manage.py 스크립트를 사용하여 수행됩니다.

2. 터미널(또는 명령 프롬프트)에서 bookr 프로젝트 디렉토리로 변경한 후(cd 명령 사용), 다음과 같이 manage.py runserver 명령을 실행합니다:

```
python manage.py runserver
```

이 명령은 Django 개발 서버를 시작합니다. 다음과 유사한 출력이 표시되어야 합니다:

```
StatReloader로 파일 변경 사항 감시 시스템 점검 수행 중...
```

```
시스템 점검에서 문제가 발견되지 않았습니다 (0 무시됨).
```

```
18개의 적용되지 않은 마이그레이션이 있습니다. 앱(admin, auth, contenttypes, sessions)의 마이그레이션을 적용하기 전까지 프로젝트가 제대로 작동하지 않을 수 있습니다.
```

```
'python manage.py migrate'를 실행하여 적용하세요. 2025년 12월 06일 - 13:10:57
```

```
Django 버전 6.0, 설정 'bookr.settings' 사용 중 개발 서버를 http://127.0.0.1:8000/에서 시작합니다. 서버를 종료하려면 CONTROL-C를 누르세요.
```

아직 적용되지 않은 마이그레이션에 대한 경고가 표시될 수 있지만, 지금은 괜찮습니다.

3. idx_1aac74e2a 웹 브라우저를 열고 http://127.0.0.1:8000/로 가면 Django 환영 화면이 표시됩니다 (그림 1.2). idx_5387cc41 이 화면이 보이면, Django 프로젝트가 idx_d681bbbed 성공적으로 생성되었고 현재 모든 것이 잘 작동하고 있음을 알 수 있습니다:

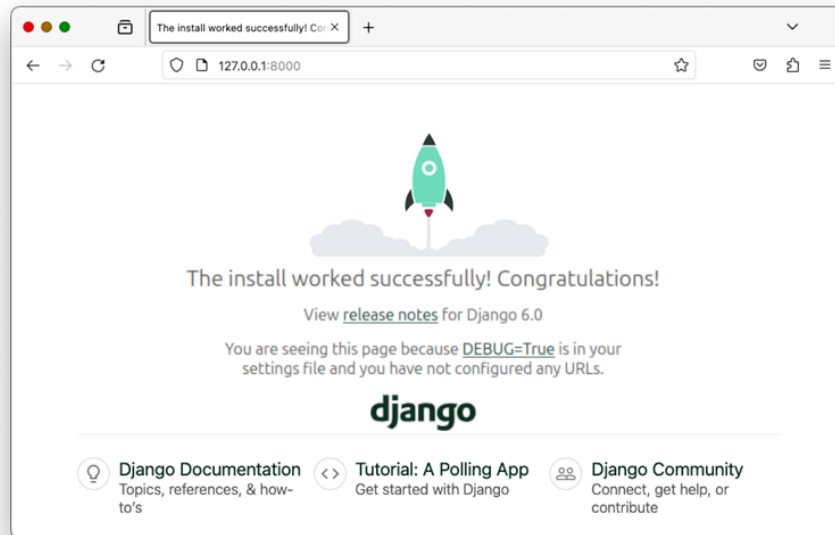


Figure 1.2 – The Django welcome screen

4. Go back to your terminal and stop the development server running by typing *Ctrl + C*.
5. We'll now create the *reviews* app for the *bookr* project. In your terminal, make sure you are in the *bookr* project directory, and then execute the following command to create the *reviews* app:

```
python manage.py startapp reviews
```

Note

After creating the *reviews* app, the files in your *bookr* project directory will look like the ones in the Chapter01 folder in this book's GitHub repo.

There is no output if the command was successful, but a *reviews* app directory has been created. You can look inside this directory to see the files that were created – the migrations directory, `admin.py`, `models.py`, and so on. We'll examine these in detail in the *Django apps* section.

In this exercise, we created the *bookr* project, tested that the project was working by starting the Django development server, and then created the *reviews* app for the project. Now that

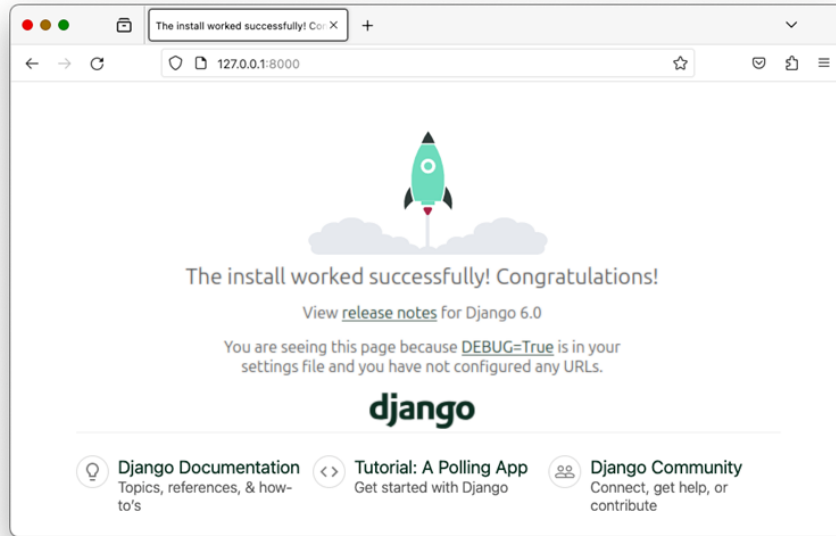


그림 1.2 - 장고 환영 화면

4. 터미널로 돌아가서 Ctrl + C를 입력하여 실행 중인 개발 서버를 중지하세요.
5. 이제 bookr 프로젝트를 위한 리뷰 앱을 생성하겠습니다. 터미널에서 bookr 프로젝트 디렉토리에 있는지 확인한 후, 다음 명령어를 실행하여 리뷰 앱을 생성하세요:

```
python manage.py startapp reviews
```

참고

리뷰 앱을 생성한 후, bookr 프로젝트 디렉토리의 파일은 이 책의 GitHub 저장소의 Chapter01 폴더에 있는 파일과 비슷하게 보일 것입니다.

명령이 성공하면 출력이 없지만, 리뷰 앱 디렉토리가 생성되었습니다. 이 디렉토리 안을 들여다보면 생성된 파일들 – migrations 디렉토리, admin.py, models.py 등이 있습니다. 장고 앱 섹션에서 이들을 자세히 살펴보겠습니다.

이 idx_472f75d8연습에서, 우리는 idx_02d28be9bookr 프로젝트를 만들고, Django 개발 서버를 시작하여 idx_5a599587프로젝트가 작동하는지 테스트한 후, 프로젝트를 위한 리뷰 앱을 만들었습니다. 이제

you've had some hands-on time with a Django project, we'll return to some of the theory behind Django's design, and HTTP requests and responses.

Understanding the model-view-template paradigm

A common design pattern in application design is **Model View Controller (MVC)**, where the model of an application (its data) is displayed in one or more views, and a controller marshals interaction between the model and view. Django follows a different, yet similar, paradigm called **Model-View-Template (MVT)**.

Like MVC, MVT also uses models for storing data. However, with MVT, a view will query a model and then render it with a template. Usually, with MVC languages, all three components need to be developed with the same language. With MVT, the template can be in a different language. In the case of Django, models and views are written in Python, and the template is written in HTML. This means that a Python developer could work on the models and views, while a specialist HTML developer works on the HTML. We'll first explain models, views, and templates in more detail and then look at some example scenarios where they are used.

Models

Django models define data for your application and provide an abstraction layer to a SQL database accessed through an **Object Relational Mapper (ORM)**. An ORM lets you define your data schema (classes, fields, and their relationships) using Python code, without needing an understanding of the underlying database. Basically, this means you can define your database layer in Python code and Django will take care of generating SQL queries for you. The ORM will be discussed in detail in *Chapter 2, Models and Migrations*.

Django 프로젝트를 직접 다뤄본 경험이 있다면, Django의 설계와 HTTP 요청 및 응답에 대한 이론으로 돌아가겠습니다.

모델-뷰-템플릿 패러다임 이해하기

애플리케이션 디자인에서 일반적인 디자인 패턴은 모델-뷰-컨트롤러(MVC)로, 애플리케이션의 모델(데이터)은 하나 이상의 뷰에 표시되고, 컨트롤러는 모델과 뷰 간의 상호작용을 조정합니다. Django는 모델-뷰-템플릿(MVT)이라는 다르지만 유사한 패러다임을 따릅니다.

MVC와 마찬가지로 MVT도 데이터를 저장하기 위해 모델을 사용합니다. 그러나 MVT에서는 뷰가 모델을 쿼리한 다음 템플릿으로 렌더링합니다. 일반적으로 MVC 언어에서는 세 가지 구성 요소가 모두 동일한 언어로 개발되어야 합니다. MVT에서는 템플릿이 다른 언어일 수 있습니다. Django의 경우 모델과 뷰는 Python으로 작성되고, 템플릿은 HTML로 작성됩니다. 이는 Python 개발자가 모델과 뷰 작업을 하고, 전문 HTML 개발자가 HTML 작업을 할 수 있음을 의미합니다. 먼저 모델, 뷰 및 템플릿에 대해 더 자세히 설명한 후, 이들이 사용되는 몇 가지 예시 시나리오를 살펴보겠습니다.

모델

Django 모델은 애플리케이션의 데이터를 정의하고 SQL 데이터베이스에 대한 추상화 계층을 제공합니다. ORM(Object Relational Mapper)을 통해 접근합니다. ORM을 사용하면 기본 데이터베이스에 대한 이해 없이도 Python 코드를 사용하여 데이터 스키마(클래스, 필드 및 그 관계)를 정의할 수 있습니다. 기본적으로 이는 데이터베이스 계층을 Python 코드로 정의할 수 있으며 Django가 SQL 쿼리를 생성하는 것을 처리한다는 것을 의미합니다. ORM에 대해서는 2장, 모델 및 마이그레이션에서 자세히 설명하겠습니다.

Note

SQL stands for **Structured Query Language** and is a way of describing a type of database that stores its data in tables, with each table having several rows. Think of each table as being like an individual spreadsheet. Unlike a spreadsheet, though, relationships can be defined between the data in each table. You can interact with data by executing SQL queries (often referred to as just queries when talking about databases). Queries allow you to retrieve data (`SELECT`), add or change data (`INSERT` and `UPDATE`, respectively), and remove data (`DELETE`). There are many SQL database servers to choose from, such as SQLite, PostgreSQL, MySQL, and Microsoft SQL Server. Much of the SQL syntax is similar between each database, but there can be some differences in dialect. Django's ORM takes care of these differences for you – when we start coding, we will use the SQLite database to store data on disk, but later, when we deploy to a server, we will switch to PostgreSQL, and won't need to make any code changes.

Normally, when querying a database, the results come back as primitive Python objects (for example, lists of strings, ints, floats, or bytes). When using the ORM, results are automatically converted into instances of the model classes you have defined. Using an ORM means that you are automatically protected from a type of vulnerability known as a SQL injection attack.

If you're more familiar with databases and SQL, you always have the option of writing your own queries too.

Views

A Django view is where most of the logic for your application is defined. When a user visits your site, their web browser will send a request to retrieve data from your site (we will go into more detail on what an HTTP request is and what information it contains in the next section). A view is a function that you write that will receive this request in the form of a Python object (specifically, a Django `HttpRequest` object). It is up to your view to decide how it should respond to the request and what it should send back to the user. Your view must return an `HttpResponse` object that encapsulates all the information being provided to the client – content, HTTP status, and other headers.

The view can also optionally receive information from the URL of the request – for example, an ID number. A common design pattern of a view is to query a database via the Django ORM, using an ID that is passed into your view. Then, the view can render a template (there'll be more on this shortly) by providing it with data from the model retrieved from the database.

노트

SQL은 구조적 쿼리 언어(Structured Query Language)의 약자로, 데이터를 테이블에 저장하는 데이터베이스 유형을 설명하는 방법입니다. 각 테이블은 여러 행을 가지고 있습니다. 각 테이블을 개별 스프레드시트처럼 생각해 보세요. 그러나 스프레드시트와는 달리, 각 테이블의 데이터 간에 관계를 정의할 수 있습니다. SQL 쿼리를 실행하여 데이터와 상호작용할 수 있습니다(데이터베이스에 대해 이야기할 때는 종종 쿼리라고만 언급됩니다). 쿼리를 사용하면 데이터를 검색(SELECT), 데이터를 추가하거나 변경(INSERT 및 UPDATE)하고, 데이터를 제거(DELETE)할 수 있습니다. SQLite, PostgreSQL, MySQL 및 Microsoft SQL Server와 같은 많은 SQL 데이터베이스 서버 중에서 선택할 수 있습니다. SQL 구문은 각 데이터베이스 간에 유사하지만 방언에 따라 약간의 차이가 있을 수 있습니다. 장고의 ORM은 이러한 차이를 처리해 줍니다. 코딩을 시작할 때는 SQLite 데이터베이스를 사용하여 디스크에 데이터를 저장하지만, 나중에 서버에 배포할 때는 PostgreSQL로 전환하며 코드 변경이 필요하지 않습니다.

일반적으로 데이터베이스를 쿼리할 때 결과는 원시 Python 객체(예: 문자열, 정수, 부동 소수점 수 또는 바이트의 리스트)로 반환됩니다. ORM을 사용할 때 결과는 자동으로 정의한 모델 클래스의 인스턴스로 변환됩니다. ORM을 사용하면 SQL 인젝션 공격으로 알려진 유형의 취약성으로부터 자동으로 보호됩니다.

데이터베이스와 SQL에 더 익숙하다면, 자신만의 쿼리를 작성할 수도 있습니다.

뷰

장고 뷰는 애플리케이션의 대부분의 로직이 정의되는 곳입니다. 사용자가 귀하의 사이트를 방문하면, 그들의 웹 브라우저는 귀하의 사이트에서 데이터를 검색하기 위한 요청을 보냅니다(HTTP 요청이 무엇인지 및 그 정보가 무엇을 포함하는지에 대한 자세한 내용은 다음 섹션에서 다룰 것입니다). 뷰는 이 요청을 Python 객체(특히 Django HttpRequest 객체)의 형태로 수신하는 함수를 작성하는 것입니다. 요청에 어떻게 응답할지와 사용자에게 무엇을 다시 보낼지를 결정하는 것은 귀하의 뷰에 달려 있습니다. 귀하의 뷰는 클라이언트에 제공되는 모든 정보를 캡슐화하는 HttpResponse 객체를 반환해야 합니다 - 콘텐츠, HTTP 상태 및 기타 헤더.

뷰는 요청의 URL에서 정보를 선택적으로 받을 수도 있습니다 - 예를 들어, ID 번호입니다. 뷰의 일반적인 디자인 패턴은 귀하의 뷰에 전달된 ID를 사용하여 Django ORM을 통해 데이터베이스를 쿼리하는 것입니다. 그런 다음 뷰는 데이터베이스에서 검색한 모델의 데이터를 제공하여 템플릿을 렌더링할 수 있습니다(곧 이에 대한 더 많은 내용이 있을 것입니다).

The rendered template becomes the content of the `HttpResponse` object and is returned from the view function. Django takes care of the communication of the data back to the browser.

Templates

A template is a **HyperText Markup Language (HTML)** file (usually, any text file can be a template) that contains special placeholders that are replaced by variables your application provides. For example, your application could render a list of items in either a gallery layout or a table layout. Your view would fetch the same models for either one, but would be able to render a different HTML file with the same information to present the data differently. Django emphasizes safety, so it will take care of automatically escaping variables for you. For example, the `<` and `>` symbols (among others) are special characters in HTML. If you try to use them in a variable, then Django automatically encodes them so that they render correctly in a browser.

MVT in practice

We'll now look at some examples to illustrate how MVT works in practice. In the examples, we have a `Book` model that stores information about different books, and a `Review` model that stores information about different reviews of books.

In the first example, we want to be able to edit the information about `Book` or `Review`. Take the first scenario of editing a book's details. We would have a view to fetch the `Book` data from the database and provide the `Book` model. Then, we would pass context information containing the `Book` object (and other data) to a template that would show a form to capture the new information. The second scenario (editing a review) is similar – fetch a `Review` model from the database and then pass the `Review` object and other data to a template to display an edit form. These scenarios might be so similar that we can reuse the same template for both, as shown in *Figure 1.3*.

렌더링된 템플릿은 HttpResponse 객체의 내용이 되며 뷰 함수에서 반환됩니다. Django는 데이터를 브라우저로 다시 전달하는 작업을 처리합니다.

템플릿

템플릿은 idx_36a9d2a2a 하이퍼텍스트 마크업 언어(HTML) 파일(일반적으로 모든 텍스트 파일이 템플릿이 될 수 있음)로, idx_e41e34ef 애플리케이션이 제공하는 변수로 대체되는 특별한 자리 표시자를 포함합니다. 예를 들어, 애플리케이션은 갤러리 레이아웃 또는 테이블 레이아웃에서 항목 목록을 렌더링할 수 있습니다. 뷰는 두 경우 모두 동일한 모델을 가져오지만, 동일한 정보를 사용하여 데이터를 다르게 표시하기 위해 다른 HTML 파일을 렌더링할 수 있습니다. Django는 안전성을 강조하므로 변수를 자동으로 이스케이프하는 작업을 처리합니다. 예를 들어, < 및 > 기호(기타 기호 포함)는 HTML에서 특별한 idx_d5b17525 문자입니다. 변수를 사용하려고 하면 Django는 이를 자동으로 인코딩하여 브라우저에서 올바르게 렌더링되도록 합니다.

실제 MVT

이제 MVT가 실제로 어떻게 작동하는지 설명하기 위해 몇 가지 idx_cccda451 예제를 살펴보겠습니다. 예제에서는 서로 다른 책에 대한 정보를 저장하는 Book 모델과 서로 다른 책 리뷰에 대한 정보를 저장하는 Review 모델이 있습니다.

첫 번째 예제에서는 Book 또는 Review에 대한 정보를 편집할 수 있어야 합니다. 책 세부 정보를 편집하는 첫 번째 시나리오를 살펴보겠습니다. 우리는 데이터베이스에서 Book 데이터를 가져오고 Book 모델을 제공하는 뷰를 가질 것입니다. 그런 다음 Book 객체(및 기타 데이터)를 포함하는 컨텍스트 정보를 템플릿에 전달하여 새로운 정보를 캡처하는 양식을 표시합니다. 두 번째 시나리오(리뷰 편집)는 유사합니다. 데이터베이스에서 Review 모델을 가져온 다음 Review 객체와 기타 데이터를 템플릿에 전달하여 편집 양식을 표시합니다. 이러한 시나리오는 너무 유사할 수 있으므로 그림 1.3에 표시된 것처럼 두 경우 모두 동일한 템플릿을 재사용할 수 있습니다.

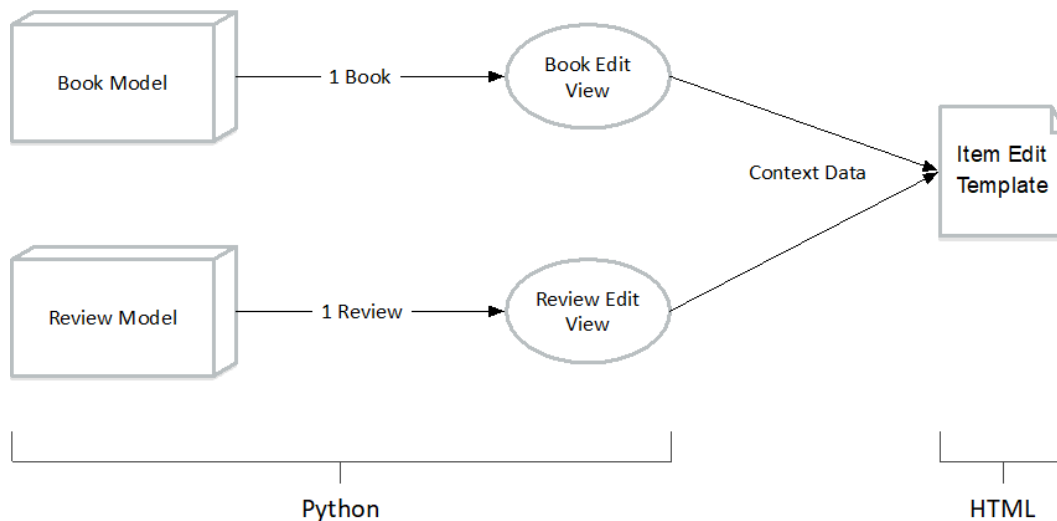


Figure 1.3 – Editing a single book or review

You can see here that we use two models, two views, and one template. Each view fetches a single instance of its associated model, but they can both use the same template, which is a generic HTML page to display a form. Views can provide extra context data to slightly alter the display of the template for each model type. Also illustrated in the diagram are the parts of the code that are written in Python and those that are written in HTML.

In the second example, we want to be able to show a user a list of the books or reviews that are stored in the application. Furthermore, we want to allow the user to search for books and get a list of all that match their criteria. We will use the same two models as the previous example (Book and Review), but we will create new views and templates. Since there are three scenarios, we'll use three views this time – the first fetches all books, the second fetches all reviews, and the last searches for books based on some search criteria. Once again, if we write a template well, we might be able to just use a single HTML template again, as shown in *Figure 1.4*.

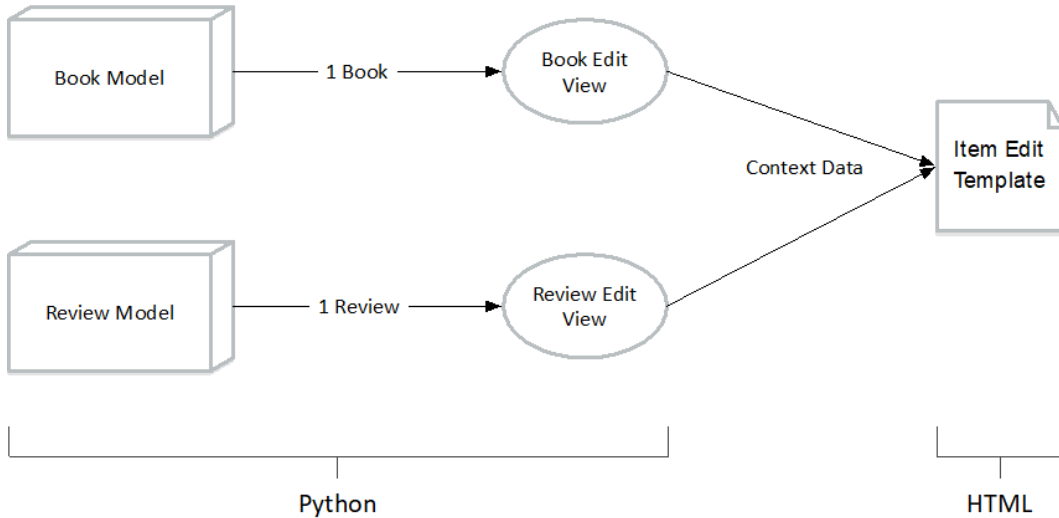


그림 1.3 - 단일 책 또는 리뷰 편집

여기에서 우리는 두 개의 모델, 두 개의 뷰 및 하나의 템플릿을 사용하고 있음을 알 수 있습니다. 각 뷰는 관련 모델의 단일 인스턴스를 가져오지만, 두 뷰 모두 동일한 템플릿을 사용할 수 있습니다. 이는 양식을 표시하기 위한 일반 HTML 페이지입니다. 뷰는 각 모델 유형에 대해 템플릿의 표시를 약간 변경하기 위해 추가 컨텍스트 데이터를 제공할 수 있습니다. 다이어그램에는 Python으로 작성된 코드 부분과 HTML로 작성된 코드 부분도 설명되어 있습니다.

두 번째 예제에서는 애플리케이션에 저장된 책 또는 리뷰 목록을 사용자에게 보여줄 수 있기를 원합니다. 또한 사용자가 책을 검색하고 자신의 기준에 맞는 모든 책 목록을 얻을 수 있도록 하고자 합니다. 우리는 이전 예제와 동일한 두 개의 모델(책과 리뷰)을 사용할 것이지만, 새로운 뷰와 템플릿을 만들 것입니다. 세 가지 시나리오가 있으므로 이번에는 세 개의 뷰를 사용할 것입니다. 첫 번째는 모든 책을 가져오고, 두 번째는 모든 리뷰를 가져오며, 마지막은 일부 검색 기준에 따라 책을 검색합니다. 다시 말해, 템플릿을 잘 작성하면 그림 1.4에 표시된 것처럼 단일 HTML 템플릿을 다시 사용할 수 있을지도 모릅니다.

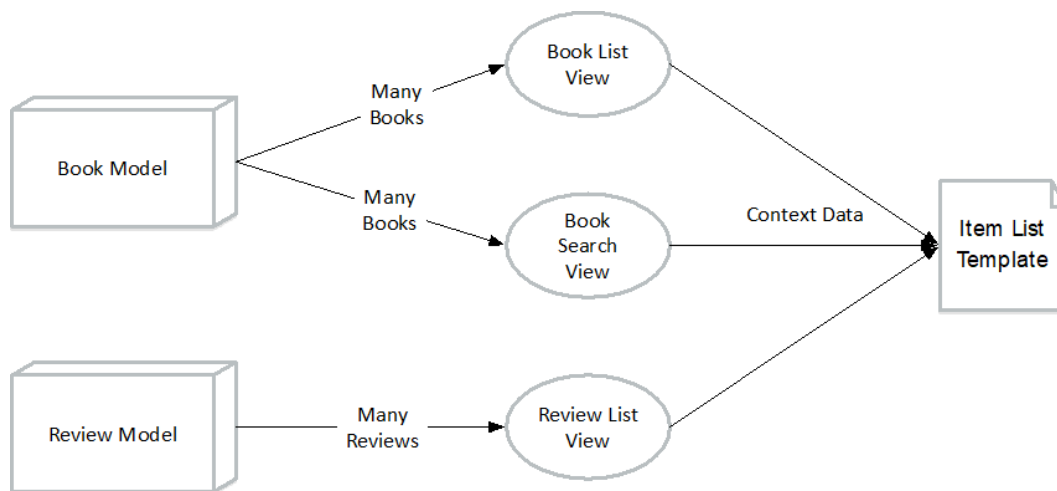


Figure 1.4 – Viewing multiple books or reviews

The Book and Review models remain unchanged from the previous example; the three views will fetch many (zero or more) books or reviews. Then, each view can use the same template, which is a generic HTML file that iterates over a list of objects it is provided and renders them. Once again, the views can send extra data in the context to alter how the template behaves, but the majority of the template will be as generic as possible.

In Django, a model does not always need to be used to render an HTML template. A view can generate the context data itself and render a template with it, without requiring any model data. *Figure 1.5* shows a view sending data straight to a template.

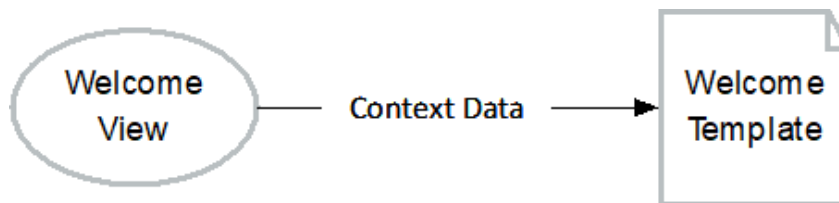


Figure 1.5 – A view sending data to a template without a model

In this example, there is a **Welcome View** to welcome a user to the site. It doesn't need any information from the database, so it can just generate the context data itself. The context data depends on the type of information you want to display – for example, you could pass the user information to greet them by name if they are logged in.

It is also possible for a view to render a template without any context data. This can be useful if you have static information in an HTML file that you want to serve.

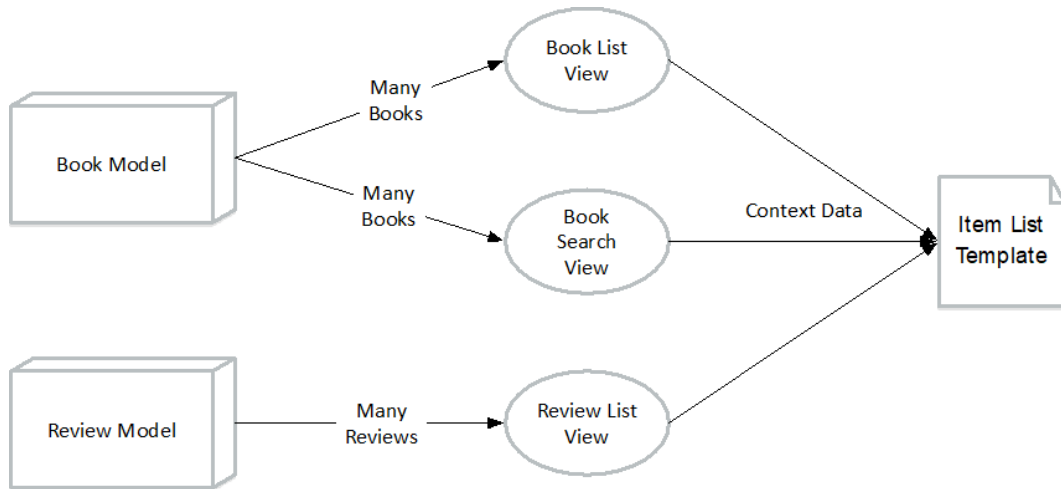


그림 1.4 - 여러 책 또는 리뷰 보기

책 및 리뷰 모델은 이전 예제와 동일하게 유지됩니다. 세 개의 뷰는 많은(0개 이상) 책 또는 리뷰를 가져옵니다. 그런 다음 각 뷰는 제공된 객체 목록을 반복하고 렌더링하는 일반 HTML 파일인 동일한 템플릿을 사용할 수 있습니다. 다시 말해, 뷰는 템플릿의 동작 방식을 변경하기 위해 컨텍스트에 추가 데이터를 보낼 수 있지만, 템플릿의 대부분은 가능한 한 일반적일 것입니다.

장고에서 모델은 HTML 템플릿을 렌더링하는 데 항상 필요하지 않습니다. 뷰는 컨텍스트 데이터를 자체적으로 생성하고 이를 사용하여 템플릿을 렌더링할 수 있으며, 모델 데이터가 필요하지 않습니다. 그림 1.5는 모델 없이 템플릿에 데이터를 직접 보내는 뷰를 보여줍니다.

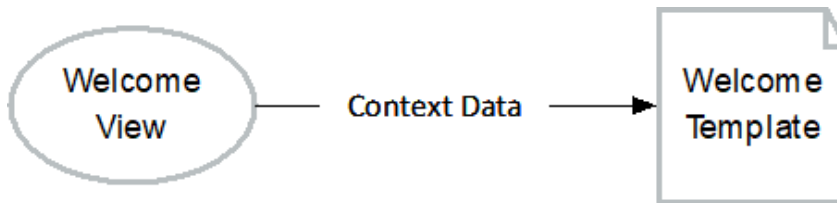


그림 1.5 - 모델 없이 템플릿에 데이터를 보내는 뷰

이 예제에서는 사용자를 사이트에 환영하는 환영 뷰가 있습니다. 데이터베이스의 정보가 필요하지 않으므로 컨텍스트 데이터를 자체적으로 생성할 수 있습니다. 컨텍스트 데이터는 표시하려는 정보의 유형에 따라 다릅니다. 예를 들어, 사용자가 로그인한 경우 이름으로 인사하기 위해 사용자 정보를 idx_d1993a69에 전달할 수 있습니다.

컨텍스트 데이터 없이 템플릿을 렌더링하는 뷰도 가능합니다. 이는 제공하려는 HTML 파일에 정적 정보가 있는 경우 유용할 수 있습니다.

Now that you have been introduced to MVT in Django, we can look at how Django processes an HTTP request and generates an HTTP response. However, we first need to explain in more detail what HTTP requests and responses are and what information they contain. We will see this in the next section.

An introduction to HTTP

Let's say someone wants to visit your web page. They type in its URL or click a link to your site from a page they are already on. Their web browser creates an HTTP request, which is sent to the server hosting your website. Once a web server receives the HTTP request from your browser, it can interpret it and then send back a response. The response that the server sends might be simple, such as just reading an HTML or image file from disk and sending it. Alternatively, the response might be more complex, maybe using server-side software (such as Django) to dynamically generate the content before sending it.

The following diagram shows the direction of the transmission of HTTP requests and HTTP responses between a browser and a web server.

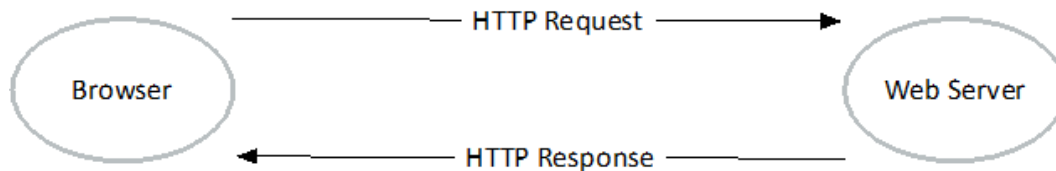


Figure 1.6 – An HTTP request and an HTTP response

The request is made up of four main parts: the method, the path, headers, and the body. Some types of requests don't have a body. If you just visit a web page, your browser will not send a body, whereas if you are submitting a form (for example, by logging into a site or performing a search), then your request will have a body containing the data you're submitting. We'll look at two example requests now to illustrate this.

The first request will be to an example page with the following URL: `https://www.example.com/page`. When your browser visits that page, behind the scenes, this is what it's sending:

```
GET /page HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/109.0
Cookie: sessid=abc123def456
```


장고에서 MVT에 대해 소개받았으니, 이제 장고가 HTTP 요청을 처리하고 HTTP 응답을 생성하는 방법을 살펴볼 수 있습니다. 그러나 먼저 HTTP 요청과 응답이 무엇인지, 그리고 그들이 포함하는 정보에 대해 더 자세히 설명해야 합니다. 다음 섹션에서 이를 살펴보겠습니다.

HTTP 소개

누군가 당신의 `webidx_7031d6ee` 페이지를 방문하고 싶다고 가정해 보겠습니다. 그들은 URL을 입력하거나 이미 보고 있는 페이지에서 당신의 사이트로 가는 링크를 클릭합니다. 그들의 웹 브라우저는 HTTP 요청을 생성하고, 이는 당신의 웹사이트를 호스팅하는 서버로 전송됩니다. 웹 서버가 당신의 브라우저로부터 HTTP 요청을 받으면, 이를 해석하고 응답을 다시 보낼 수 있습니다. 서버가 보내는 응답은 단순할 수 있으며, 예를 들어 디스크에서 HTML 또는 이미지 파일을 읽고 이를 전송하는 것일 수 있습니다. 또는 응답이 더 복잡할 수 있으며, 서버 측 소프트웨어(예: 장고)를 사용하여 콘텐츠를 동적으로 생성한 후 전송할 수 있습니다.

다음 다이어그램은 `idx_8b17da67` 브라우저와 웹 서버 간의 HTTP 요청 및 HTTP 응답 전송 방향을 보여줍니다.

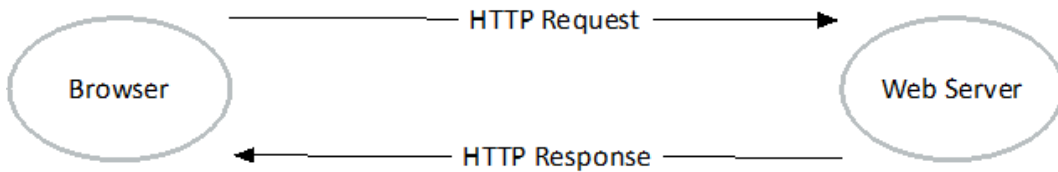


그림 1.6 - HTTP 요청 및 HTTP 응답

요청은 네 가지 주요 부분으로 구성됩니다: 메서드, 경로, 헤더 및 본문. 일부 요청 유형은 본문이 없습니다. 웹 페이지를 방문하는 경우, 브라우저는 본문을 전송하지 않지만, 양식을 제출하는 경우(예: 사이트에 로그인하거나 검색을 수행하는 경우) 요청에는 제출하는 데이터가 포함된 본문이 있습니다. 이제 이를 설명하기 위해 두 가지 예제 요청을 살펴보겠습니다.

첫 번째 요청은 다음 URL을 가진 예제 페이지로 향합니다:

`https://www.example.com/page`. 브라우저가 해당 페이지를 방문할 때, 이 뒤에서 전송되는 내용은 다음과 같습니다:

```
GET /page HTTP/1.1
```

```
호스트: www.example.com
```

```
사용자 에이전트: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/109.0
```

```
쿠키: sessid=abc123def456
```

The first line contains the method (GET) and the path (/page). It also contains the HTTP version – in this case, 1.1, although you don't have to worry about this. There are many different HTTP methods that can be used, depending on how you want to interact with the remote page. Some common ones are GET (to retrieve the remote page), POST (to send data to the remote page), PUT (to create a remote page), and DELETE (to delete the remote page). Note that the descriptions of the actions are somewhat simplified – the remote server can choose how it responds to different methods, and even experienced developers can disagree on the correct method to implement for a particular action. It's also important to note that even if a server supports a particular method, you will probably need the correct permissions to perform that action – you can't just delete a web page you don't like, for example.

When writing a web application, the vast majority of the time, you will only deal with GET requests. When you start accepting forms, you'll also have to use POST requests. It is only when you are working with advanced features, such as creating REST APIs, that you will have to worry about PUT, DELETE, and other methods.

Referring to the example request again, line 2 onwards features the headers of the request. The headers contain extra metadata about the request. Each header is on its own line, with the header name and its value separated by a colon. Most are optional (except for Host – there'll be more on that soon). Header names are not case-sensitive. For the sake of this example, we're only showing three common headers here. Let's look at the example headers in order:

- **Host:** As mentioned, this is the only header that is required (for HTTP 1.1 or later). It is needed for the web server to know which website or application should respond to the request if there are multiple sites hosted on a single server.
- **User-Agent:** Your browser usually sends to the server a string identifying its version and operating system. Your server application could use this to serve different pages to different devices (for example, a mobile-specific page for smartphones).
- **Cookie:** You have probably seen a message when visiting a web page that lets you know that it is storing a cookie in the browser. These are small pieces of information that a website can store in your browser and use to identify you or save settings for when you return to the site. If you were wondering about how your browser sends these cookies back to the server, it is through this header.

There are many other standard headers defined, and it would take up too much space to list them all. They can be used to authenticate to the server (*Authorization*), tell the server what kind of data you can receive (*Accept*), or even what language you'd like for the page (*Accept-Language*), although this will only work if the page creator has made the content available in

첫 번째 줄 `idx_b6180d5b`은 메서드(GET)와 경로(/page)를 포함합니다. 또한 HTTP 버전도 포함되어 있습니다 - 이 경우 1.1이며, 이에 대해 걱정할 필요는 없습니다. 원격 페이지와 상호작용하는 방법에 따라 사용할 수 있는 다양한 HTTP 메서드가 있습니다. 일반적인 것들로는 GET(원격 페이지를 검색하기 위해), POST(원격 페이지에 데이터를 보내기 위해), PUT(원격 페이지를 생성하기 위해), DELETE(원격 페이지를 삭제하기 위해) 등이 있습니다. 동작에 대한 설명은 다소 단순화되어 있다는 점에 유의하세요 - 원격 서버는 다양한 메서드에 대해 어떻게 응답할지 선택할 수 있으며, 심지어 경험이 많은 개발자들도 특정 동작을 구현하기 위한 올바른 메서드에 대해 의견이 다를 수 있습니다. 또한 서버가 특정 메서드를 지원하더라도 해당 동작을 수행하기 위해 올바른 권한이 필요할 수 있다는 점도 중요합니다 - 예를 들어, 마음에 들지 않는 웹 페이지를 삭제할 수는 없습니다.

웹 애플리케이션을 작성할 때, 대다수의 경우 GET 요청만 처리하게 됩니다. 양식을 수락하기 시작하면 POST 요청도 사용해야 합니다. REST API를 생성하는 것과 같은 고급 기능을 작업할 때만 PUT, DELETE 및 기타 메서드에 대해 걱정해야 합니다.

예제 요청을 다시 참조하면, 2행부터는 요청의 헤더가 포함되어 있습니다. 헤더는 요청에 대한 추가 메타데이터를 포함합니다. 각 헤더는 별도의 줄에 있으며, 헤더 이름과 값은 콜론으로 구분됩니다. 대부분은 선택 사항입니다(호스트 제외 - 곧 더 많은 내용이 있습니다). 헤더 이름은 대소문자를 구분하지 않습니다. 이 예제의 경우, `idx_6c5064c2` 세 가지 일반 헤더만 보여주고 있습니다. 예제 헤더를 순서대로 살펴보겠습니다:

- **호스트:** 언급했듯이, 이것은 필수 헤더입니다(HTTP 1.1 이상). 여러 사이트가 단일 서버에 호스팅되는 경우, 웹 서버가 요청에 응답해야 할 웹사이트나 애플리케이션을 알기 위해 필요합니다.
- **사용자 에이전트:** 브라우저는 일반적으로 서버에 자신의 버전과 운영 체제를 식별하는 문자열을 보냅니다. 서버 애플리케이션은 이를 사용하여 다양한 장치에 대해 다른 페이지를 제공할 수 있습니다(예: 스마트폰을 위한 모바일 전용 페이지).
- **쿠키:** 웹 페이지를 방문할 때 브라우저에 쿠키를 저장하고 있다는 메시지를 본 적이 있을 것입니다. 이는 웹사이트가 브라우저에 저장할 수 있는 작은 정보 조각으로, 사용자를 식별하거나 사이트에 다시 방문할 때 설정을 저장하는 데 사용됩니다. 브라우저가 이러한 쿠키를 서버에 다시 보내는 방법에 대해 궁금하다면, 바로 이 헤더를 통해 이루어집니다.

정의된 다른 많은 표준 헤더가 있으며, 모두 나열하기에는 너무 많은 공간을 차지할 것입니다. 이들은 서버에 인증(Authorization)을 하거나, 서버에 수신할 수 있는 데이터의 종류(Accept)를 알리거나, 페이지에 원하는 언어(Accept-Language)를 지정하는 데 사용될 수 있습니다(단, 이는 페이지 작성자가 콘텐츠를 해당 언어로 제공한 경우에만 작동합니다).

the particular language you request). You can even define your own headers that only your application knows how to respond to.

Now, let's look at a slightly more advanced request – one that sends some information to a server and thus (unlike the previous example) contains a body. In this example, we are logging into a web page by sending a username and password – for example, you visit `https://www.example.com/login`, and it displays a form to enter a username and password. After you click the **Login** button, this is the request that is sent to the server:

```
POST /login HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 32

username=user&password=password1
```

As you can see, this looks similar to the first example, but there are a few differences. The method is now POST, and two new headers have been introduced (you can assume that your browser will still be sending the other headers that were in the previous example too):

- **Content-Type:** This tells the server the type of data that is included in the body. In the case of `application/x-www-form-urlencoded`, the body is a set of key-value pairs. An HTTP client could set this header to tell the server if it was sending other types of data, such as JSON or XML.
- **Content-Length:** For the server to know how much data to read, the client must tell it how much data is being sent. The Content-Length header contains the length of the body. If you count the length of the body in this example, you'll see that it's 32 characters.

The headers are always separated from the body by a blank line. By looking at the example, you should be able to tell how the form data is encoded in the body – username has the user1 value and password has the password1 value.

These requests were quite simple, but most requests don't get much more complicated. They might have different methods and headers, but should follow the same format. Now that we've seen requests, we'll take a look at HTTP responses that come back from the server.

요청하는 특정 언어). 당신은 심지어 당신의 애플리케이션만 알고 있는 헤더를 정의할 수 있습니다.

이제 약간 더 고급 요청을 살펴보겠습니다. 이 요청은 서버에 정보를 보내며 (이전 예제와 달리) 본문을 포함합니다. 이 예제에서는 사용자 이름과 비밀번호를 보내 웹 페이지에 로그인하고 있습니다. 예를 들어, `https://www.example.com/login`을 방문하면 사용자 이름과 비밀번호를 입력할 수 있는 양식이 표시됩니다. 로그인 버튼을 클릭하면 서버에 전송되는 요청은 다음과 같습니다:

```
POST /login HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 32

username=user&password=password1
```

보시다시피, 이것은 첫 번째 예제와 유사하지만 몇 가지 차이점이 있습니다. 메서드는 이제 POST이며, 두 개의 새로운 헤더가 도입되었습니다 (이전 예제에 있던 다른 헤더도 여전히 브라우저가 전송하고 있다고 가정할 수 있습니다):

- **Content-Type:** 이것은 `idx_4c519086` 서버에 본문에 포함된 데이터의 유형을 알려줍니다. `application/x-www-form-urlencoded`의 경우 본문은 키-값 쌍의 집합입니다. HTTP 클라이언트는 JSON 또는 XML과 같은 다른 유형의 데이터를 전송하고 있는지 서버에 알리기 위해 이 헤더를 설정할 수 있습니다.
- **Content-Length:** 서버가 얼마나 많은 데이터를 읽어야 하는지 알기 위해 클라이언트는 얼마나 많은 데이터가 전송되고 있는지 알려야 합니다. `Content-Length` 헤더는 본문의 길이를 포함합니다. 이 예제에서 본문의 길이를 세어보면 32자라는 것을 알 수 있습니다.

헤더는 항상 본문과 빈 줄로 구분됩니다. 예제를 보면 본문에서 양식 데이터가 어떻게 인코딩되어 있는지 알 수 있어야 합니다. 사용자 이름은 `user1` 값을 가지고 있고 비밀번호는 `password1` 값을 가지고 있습니다.

이 요청들은 꽤 간단했지만 대부분의 요청은 그보다 더 복잡해지지 않습니다. 그들은 다른 방법과 헤더를 가질 수 있지만 동일한 형식을 따라야 합니다. 요청을 살펴보았으니 이제 서버에서 돌아오는 HTTP 응답을 살펴보겠습니다.

An HTTP response looks similar to a request and consists of three main parts: a status, headers, and a body. Like a request, though, depending on the type of response, it might not have a body. The first response example is a simple, successful response:

```
HTTP/1.1 200 OK
Server: nginx
Content-Length: 18132
Content-Type: text/html
Set-Cookie: sessid=abc123def46

<!DOCTYPE html><html><head>...
```

The first line contains the HTTP version, a numeric status code (200), and then a text description of what the code means (OK – the request was a success). We'll show some more statuses after the next example. Lines 2 to 5 contain headers, similar to a request. Some headers you may have seen before; we will explain them all in this context:

- **Server:** This is similar but the opposite of the User-Agent header – this is the server telling the client what software it is running.
- **Content-Length:** The client uses this value to determine how much data to read from the server to get the body.
- **Content-Type:** The server uses this header to indicate to the client what type of data it is sending. The client can then choose how it will display the data – an image must be displayed differently from HTML, for example.
- **Set-Cookie:** We saw in the first request example how a client sends a cookie to the server. This is the corresponding header that a server sends to set that cookie in the browser.

After the headers is a blank line, and then the body of the response. We haven't shown it all here, just the first few characters of the HTML that is being received, out of the 18,132 that the server has sent.

Next, we'll show an example of a response that is returned if a requested page is not found:

```
HTTP/1.1 404 Not Found
Server: nginx
Content-Length: 55
Content-Type: text/html

<!DOCTYPE html><html><body>Page Not Found</body></html>
```

HTTP 응답은 요청과 유사하게 보이며 세 가지 주요 부분으로 구성됩니다: 상태, 헤더, 본문. 그러나 요청과 마찬가지로 응답 유형에 따라 본문이 없을 수도 있습니다. 첫 번째 응답 예시는 간단하고 성공적인 응답입니다:

```
HTTP/1.1 200
OKServer:
nginxContent-Length:
18132

Content-Type: text/html
Set-Cookie: sessid=abc123def46

<!DOCTYPE html><html><head>...
```

첫 번째 줄에는 idx_d66e1859HTTP 버전, 숫자 상태 코드(200), 그리고 코드의 의미에 대한 텍스트 설명(OK – 요청이 성공했습니다)이 포함되어 있습니다. 다음 예제 후에 몇 가지 더 많은 상태를 보여드리겠습니다. 2번에서 5번 줄에는 요청과 유사한 헤더가 포함되어 있습니다. 이전에 보았던 헤더도 있을 수 있으며, 이 맥락에서 모두 설명하겠습니다:

- 서버: 이것은 사용자 에이전트 헤더와 유사하지만 반대입니다 - 이것은 서버가 클라이언트에게 어떤 소프트웨어를 실행하고 있는지 알려주는 것입니다.
- Content-Length: 클라이언트는 이 값을 사용하여 서버에서 본문을 가져오기 위해 얼마나 많은 데이터를 읽어야 하는지를 결정합니다.
- Content-Type: 서버는 이 헤더를 사용하여 클라이언트에게 어떤 유형의 데이터를 전송하고 있는지를 나타냅니다. 클라이언트는 그런 다음 데이터를 어떻게 표시할지를 선택할 수 있습니다 - 예를 들어, 이미지는 HTML과 다르게 표시되어야 합니다.
- Set-Cookie: 우리는 첫 번째 요청 예제에서 클라이언트가 서버에 쿠키를 보내는 방법을 보았습니다. 이것은 서버가 브라우저에 그 쿠키를 설정하기 위해 보내는 해당 헤더입니다.

헤더 다음에는 빈 줄이 있고, 그 다음에 응답의 본문이 있습니다. 우리는 여기에서 모든 것을 보여주지 않았고, 서버가 보낸 18,132개의 HTML 중 처음 몇 개의 문자만 보여주고 있습니다.

다음으로, 요청한 페이지를 찾을 수 없는 경우 반환되는 응답의 예를 보여드리겠습니다:

```
HTTP/1.1 404 Not
FoundServer:
nginxContent-Length:
55

Content-Type: text/html

<!DOCTYPE html><html><body>페이지를 찾을 수 없습니다</body></html>
```

It is similar to the previous example, but the status is now `404 Not Found`. If you've ever been browsing the internet and received a `404` error, this is the type of response your browser received. The various status codes are grouped by the type of success or failure they indicate:

- **100–199:** The server sends codes in this range to indicate protocol changes or that more data is required. You don't have to worry about these status codes.
- **200–299:** A status code in this range indicates successful handling of a response. As we saw, the most common one you will deal with is `200 OK`.
- **300–399:** A status code in this range means the page you are requesting has moved to another address. An example of this is a URL shortening service that would redirect you from the short URL to the full one when you visit it. Common responses are `301 Moved Permanently` or `302 Found`. When sending a redirect response, the server will also include a `Location` header that contains the URL you should be redirected to.
- **400–499:** A status code in this range means that the request could not be handled because there was a problem with what the client sent. This is in contrast to a request not being able to be handled due to a problem on the server (we will discuss those soon). We've already seen a `404 Not Found` response; this is due to a bad request because the client is requesting a document that does not exist. Some other common responses are `401 Unauthorized` (the client should log in) or `403 Forbidden` (the client is not allowed to access the specific resource). Both of these problems could be avoided by getting the client to log in, hence them being considered client-side (request) problems.
- **500–599:** Status codes in this range indicate an error on the server's side. The client shouldn't expect to be able to adjust a request to fix the problem. When working with Django, the most common server error status you will see is `500 Internal Server Error`. This will be generated if your code raises an exception. Another common one is `504 Gateway Timeout`, which might occur if your code is taking too long to run. The other variants that are common to see are `502 Bad Gateway` and `503 Service Unavailable`, which generally mean there is a problem with your application's hosting in some way.

These are only some of the most common HTTP statuses. You can find a more complete list at <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>. Like HTTP headers, though, statuses are arbitrary, and an application can return custom statuses. It is up to the server and clients to decide what these custom statuses and codes mean.

이전 예제와 유사하지만, 상태가 이제 404 Not Found입니다. 인터넷을 탐색하다가 404 오류를 받은 적이 있다면, 이것이 브라우저가 받은 응답 유형입니다. 다양한 상태 코드 idx_f964af01은 성공 또는 실패를 나타내는 유형에 따라 그룹화됩니다:

- 100–199: 서버는 이 범위의 코드를 사용하여 프로토콜 변경 또는 추가 데이터가 필요함을 나타냅니다. 이러한 상태 코드에 대해 걱정할 필요는 없습니다.
- 200–299: 이 범위의 상태 코드는 응답을 성공적으로 처리했음을 나타냅니다. 우리가 보았듯이, 가장 일반적으로 다루게 될 것은 200 OK입니다.
- 300–399: 이 범위의 상태 코드는 요청한 페이지가 다른 주소로 이동했음을 의미합니다. 이의 예로는 URL 단축 서비스가 있으며, 방문할 때 짧은 URL에서 전체 URL로 리디렉션됩니다. 일반적인 응답은 301 Moved Permanently 또는 302 Found입니다. 리디렉션 응답을 보낼 때, 서버는 또한 리디렉션할 URL이 포함된 Location 헤더를 포함합니다.
- 400–499: 이 범위의 상태 코드 idx_05796cecin은 클라이언트가 보낸 내용에 문제가 있어 요청을 처리할 수 없음을 의미합니다. 이는 서버의 문제로 인해 요청을 처리할 수 없는 경우와 대조적입니다(그에 대해서는 곧 논의할 것입니다). 우리는 이미 404 Not Found 응답을 보았으며, 이는 클라이언트가 존재하지 않는 문서를 요청했기 때문에 잘못된 요청으로 인한 것입니다. 다른 일반적인 응답으로는 401 Unauthorized(클라이언트는 로그인해야 함) 또는 403 Forbidden(클라이언트는 특정 리소스에 접근할 수 없음)이 있습니다. 이러한 문제는 클라이언트가 로그인하도록 하여 피할 수 있으며, 따라서 클라이언트 측(요청) 문제로 간주됩니다.
- 500–599: 이 범위의 상태 코드는 서버 측의 오류를 나타냅니다. 클라이언트는 문제를 해결하기 위해 요청을 조정할 수 있다고 기대해서는 안 됩니다. Django로 작업할 때 가장 일반적으로 볼 수 있는 서버 오류 상태는 500 Internal Server Error입니다. 이는 코드가 예외를 발생시킬 경우 생성됩니다. 또 다른 일반적인 오류는 504 Gateway Timeout으로, 이는 코드 실행이 너무 오래 걸릴 경우 발생할 수 있습니다. 일반적으로 볼 수 있는 다른 변형으로는 502 Bad Gateway와 503 Service Unavailable이 있으며, 이는 일반적으로 애플리케이션의 idx_1111858b 호스팅에 어떤 문제가 있음을 의미합니다.

이들은 가장 일반적인 HTTP 상태 중 일부에 불과합니다. 더 완전한 목록은 <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status> 에서 찾을 수 있습니다. 그러나 HTTP 헤더와 마찬가지로 상태는 임의적이며, 애플리케이션은 사용자 정의 상태를 반환할 수 있습니다. 이러한 사용자 정의 상태와 코드의 의미는 서버와 클라이언트가 결정해야 합니다.

If this is your first time being introduced to the HTTP protocol, there's quite a lot of information to take in. Luckily, Django does all the hard work and encapsulates the incoming data into an `HttpRequest` object. Most of the time, you don't need to know about most of the information coming in, but it's available if you need it. Likewise, when sending a response, Django encapsulates your data in an `HttpResponse` object. Normally, you just set the content to return, but you also have the freedom to set HTTP status codes and headers. We will discuss how to access and set the information in `HttpRequest` and `HttpResponse` later in this chapter. In the next section, we will look at how Django receives, parses, and responds to an HTTP request.

Processing a request

This is a basic timeline of the request and response flows so that you can get an idea of what the code you'll be writing does at each stage. In terms of writing code, the first part you will write is your view. The view you create will perform some actions, such as querying a database for data. Then, the view will pass this data to another function to render a template, finally returning the `HttpResponse` object that encompasses the data you want to send back to the client.

Next, Django needs to know how to map a specific URL to your view so that it can load the correct view for the URL it receives as part of a request. You will write this URL mapping in a URL configuration Python file.

When Django receives a request, it parses the URL config file and then finds the corresponding view. It calls the view, passing in the `HttpRequest` object that represents the request. Your view will return its `HttpResponse`, and then Django takes over again to send this data to its host web server and back out to the client that requested it.

HTTP 프로토콜에 처음 소개되는 것이라면, 받아들여야 할 정보가 꽤 많습니다. 다행히도, Django는 모든 어려운 작업을 처리하고 들어오는 데이터를 HttpRequest 객체로 캡슐화합니다. 대부분의 경우, 들어오는 정보에 대해 알 필요는 없지만, 필요할 경우 사용할 수 있습니다. 마찬가지로, 응답을 보낼 때 Django는 데이터를 HttpResponse 객체로 캡슐화합니다. 일반적으로 반환할 콘텐츠를 설정하기만 하면 되지만, HTTP 상태 코드와 헤더를 설정할 자유도 있습니다. 이 장의 후반부에서는 HttpRequest와 HttpResponse에서 정보를 액세스하고 설정하는 방법에 대해 논의할 것입니다. 다음 섹션에서는 Django가 HTTP 요청을 수신하고, 구문 분석하고, 응답하는 방법을 살펴보겠습니다.

요청 처리

다음은 idx_0bf04566 요청 및 응답 흐름의 기본 타임라인으로, 각 단계에서 작성할 코드가 수행하는 작업을 이해하는 데 도움이 됩니다. 코드를 작성하는 측면에서, 처음 작성할 부분은 뷰입니다. 생성한 뷰는 데이터베이스에서 데이터를 쿼리하는 등의 작업을 수행합니다. 그런 다음, 뷰는 이 데이터를 다른 함수에 전달하여 템플릿을 렌더링하고, 마지막으로 클라이언트에게 다시 보내고자 하는 데이터를 포함하는 HttpResponse 객체를 반환합니다.

다음으로, Django는 특정 URL을 귀하의 뷰에 매핑하는 방법을 알아야 하므로 요청의 일환으로 수신한 URL에 대해 올바른 뷰를 로드할 수 있습니다. 이 URL 매핑은 URL 구성 Python 파일에 작성합니다.

Django가 요청을 수신하면 URL 구성 파일을 구문 분석한 다음 해당 뷰를 찾습니다. 그런 다음 요청을 나타내는 HttpRequest 객체를 전달하여 뷰를 호출합니다. 귀하의 뷰는 HttpResponse를 반환하고, 그 후 Django는 다시 이 데이터를 호스트 웹 서버로 보내고 요청한 클라이언트에게 다시 보냅니다.

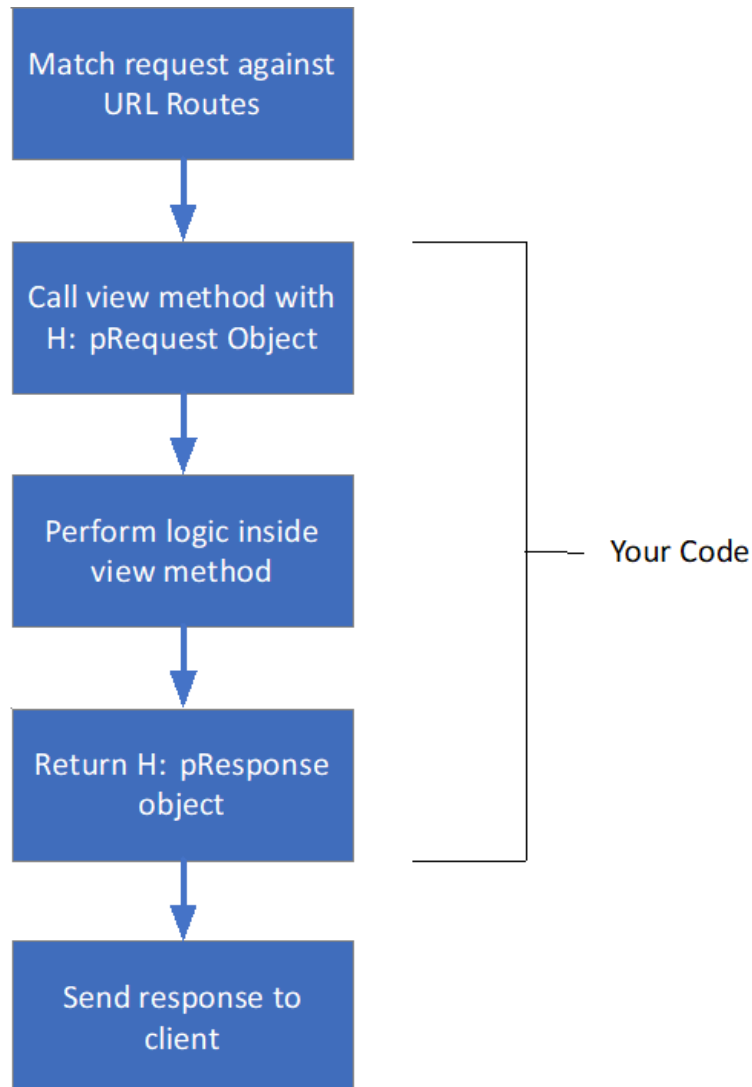


Figure 1.7 – The request and response flow

The request and response flow is illustrated in *Figure 1.7*; the sections indicated as **Your Code** are for the code that you write, and the first and last steps are taken care of by Django. Django does the URL matching for you, calls your view code, and then handles passing the response back to the client.

In this section, we learned about the structure of HTTP requests and responses, including what different types of requests are used for. We saw how HTTP status codes can be used to denote

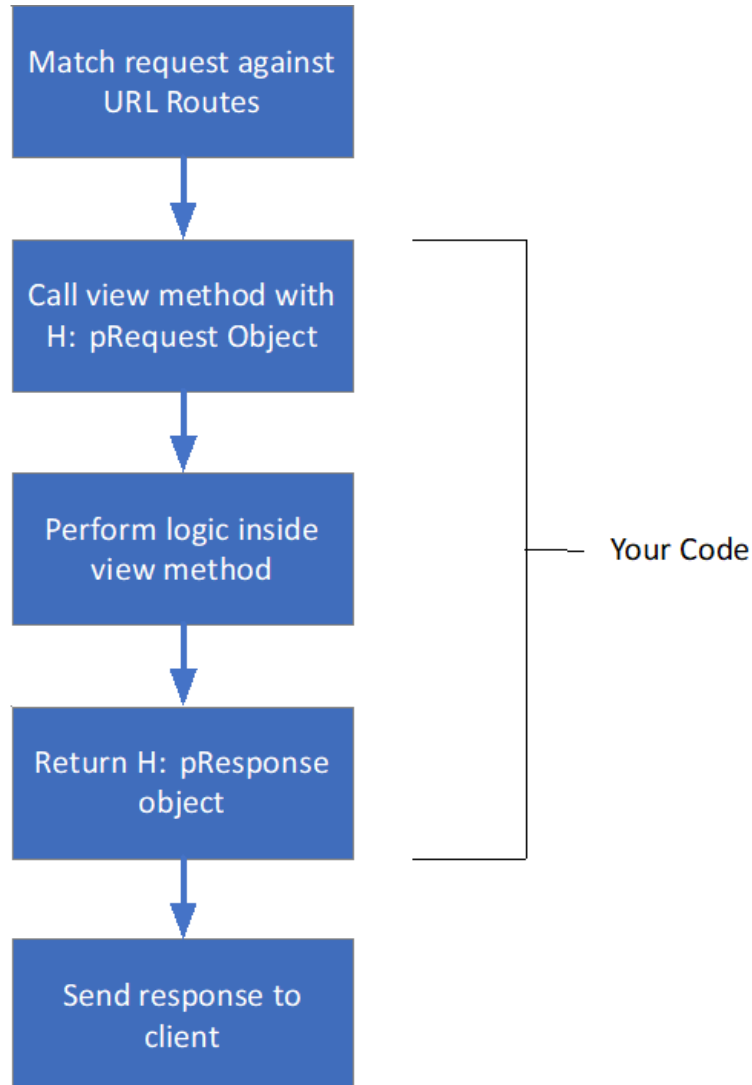


그림 1.7 - 요청 및 응답 흐름

요청 및 응답 흐름은 그림 1.7에 설명되어 있으며, Your Code로 표시된 섹션은 여러분이 작성하는 코드에 해당하고, 첫 번째 및 마지막 단계는 장고가 처리합니다. 장고는 URL 매칭을 수행하고, 여러분의 뷰 코드를 호출한 다음, 응답을 클라이언트로 다시 전달하는 작업을 처리합니다.

이 섹션에서는 HTTP 요청 및 응답의 구조에 대해 배웠으며, 다양한 유형의 요청이 사용되는 목적에 대해 알아보았습니다. HTTP 상태 코드를 사용하여 나타낼 수 있는 방법을 보았습니다.

errors in HTTP requests or responses. We also got an overview of how Django can process an HTTP request. In the next section, we'll explore the Django project structure and learn what the various files and directories are used for.

Exploring the Django project structure

We already introduced Django projects in the *Scaffolding a Django project and app* section. Let's remind ourselves of what happens when we run `startproject` (for a project named `myproject`) – the command creates a `myproject` directory with one file called `manage.py`, and a directory called `myproject` (this matches the project name; in *Exercise 1.01 – creating a project and app, and starting the development server*, this folder was called `bookr`, the same as the project). The directory layout is shown in *Figure 1.8*. We'll now examine the `manage.py` file and `myproject` package contents in more detail.

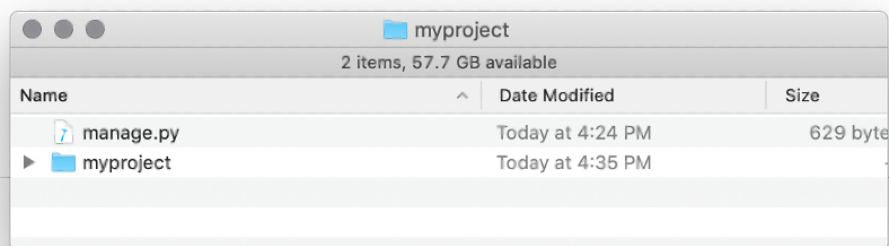


Figure 1.8 – The project directory for `myproject`

As the name suggests, `manage.py` is a script that is used to manage your Django project. Most of the commands that are used to interact with your project will be supplied to this script on the command line. The commands are supplied as an argument to this script – for example, if we want to run the `manage.py runserver` command, it would mean running the `manage.py` script like this:

```
python manage.py runserver
```

HTTP 요청 또는 응답의 오류. 우리는 또한 Django가 HTTP 요청을 처리하는 방법에 대한 개요를 얻었습니다. 다음 섹션에서는 Django 프로젝트 구조를 탐색하고 다양한 파일과 디렉토리가 무엇에 사용되는지 배울 것입니다.

Django 프로젝트 구조 탐색

우리는 이미 Django idx_28028ec4projects를 Django 프로젝트 및 앱 스캐폴딩 섹션에서 소개했습니다. startproject를 실행할 때 발생하는 일을 상기해 보겠습니다(프로젝트 이름이 myproject인 경우) – 이 명령은 manage.py라는 하나의 파일과 myproject라는 디렉토리를 포함하는 myproject 디렉토리를 생성합니다(이는 프로젝트 이름과 일치합니다; 연습 1.01 – 프로젝트 및 앱 생성, 개발 서버 시작에서 이 폴더는 프로젝트와 동일한 bookr라고 불렸습니다). 디렉토리 레이아웃은 그림 1.8에 나와 있습니다. 이제 manage.py 파일과 myproject 패키지 내용을 더 자세히 살펴보겠습니다.

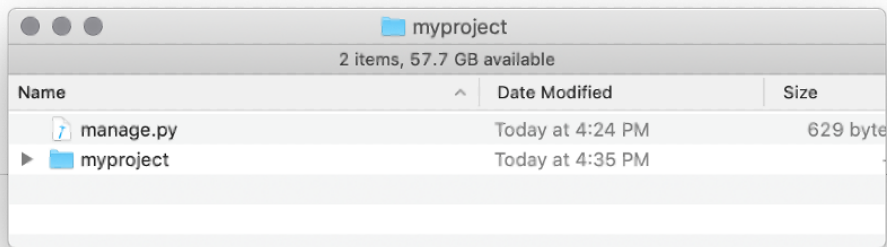


그림 1.8 – myproject의 프로젝트 디렉토리

이름 idx_61212678이 암시하듯이, manage.py는 Django 프로젝트를 관리하는 데 사용되는 스크립트입니다. 프로젝트와 상호작용하는 데 사용되는 대부분의 명령은 명령줄에서 이 스크립트에 제공됩니다. 명령은 이 스크립트에 대한 인수로 제공됩니다 – 예를 들어, manage.py runserver 명령을 실행하고 싶다면, manage.py 스크립트를 다음과 같이 실행하는 것을 의미합니다:

```
python manage.py runserver
```

There are a number of useful commands that `manage.py` provides. You will be introduced to them in more detail throughout the book; some of the common ones are as follows:

- `runserver`: This starts the Django development HTTP server to serve the Django app on your local computer.
- `startapp`: This creates a new Django app in your project. We'll talk about what apps are in more depth soon.
- `shell`: This starts a Python interpreter with the Django settings pre-loaded. This is useful for interacting with your application without having to manually load your Django settings.
- `dbshell`: This starts an interactive shell connected to your database, using the default parameters from your Django settings. You can run manual SQL queries this way.
- `makemigrations`: This generates database change instructions from your model definitions. You will learn what this means and how to use this command in *Chapter 2*.
- `migrate`: This applies migrations generated by the `makemigrations` command. You will use this in *Chapter 2*, as well.
- `test`: This runs automated tests that you have written. You'll use this command in *Chapter 14*.

A full list of all commands is available at <https://docs.djangoproject.com/en/3.0/ref/django-admin/>.

In the following sections, we will explore the contents of the project directory, learn what Django app directories contain, and see how to load up your project in PyCharm.

The myproject directory

As well as the `manage.py` file, the other file item created by `startproject` is the `myproject` directory. This is the actual Python package for your project. It contains settings for the project, some configuration files for your web server, and the global URL maps. Inside the `myproject` directory are five files:

- `__init__.py`: This is an empty file that lets Python know that the `myproject` directory is a Python module. You'll be familiar with these files if you've worked with Python before.
- `settings.py`: This contains all the Django settings for your application. We will explain the contents soon.

manage.py가 제공하는 유용한 명령어가 여러 가지 있습니다. 이 책 전반에 걸쳐 더 자세히 소개될 것입니다; 일반적인 것들은 다음과 같습니다:

- **runserver**: 이것은 장고 개발 HTTP 서버를 시작하여 로컬 컴퓨터에서 장고 앱을 제공합니다.
- **startapp**: 이것은 프로젝트에 새로운 장고 앱을 생성합니다. 곧 앱에 대해 더 깊이 이야기할 것입니다.
- **shell**: 이것은 Django 설정이 미리 로드된 상태에서 Python 인터프리터를 시작합니다. 이는 Django 설정을 수동으로 로드하지 않고도 애플리케이션과 상호작용하는 데 유용합니다.
- **dbshell**: 이것은 Django 설정의 기본 매개변수를 사용하여 데이터베이스에 연결된 대화형 셸을 시작합니다. 이렇게 하면 수동 SQL 쿼리를 실행할 수 있습니다.
- **makemigrations**: 이것은 모델 정의에서 데이터베이스 변경 지침을 생성합니다. 이것이 의미하는 바와 이 명령을 사용하는 방법은 2장에서 배울 것입니다.
- **migrate**: 이것은 makemigrations 명령으로 생성된 마이그레이션을 적용합니다. 이것도 2장에서 사용할 것입니다.
- **test**: 이것은 당신이 작성한 자동화된 테스트를 실행합니다. 이 명령은 14장에서 사용할 것입니다.

모든 명령의 전체 목록은 <https://docs.djangoproject.com/en/3.0/ref/django-admin/>에서 확인할 수 있습니다.

다음 섹션에서는 프로젝트 디렉토리의 내용을 탐색하고, Django 앱 디렉토리에 무엇이 포함되어 있는지 배우고, PyCharm에서 프로젝트를 로드하는 방법을 살펴보겠습니다.

myproject 디렉토리

manage.py 파일 외에도 startproject에 의해 생성된 다른 `idx_2cd8d104` 파일 항목은 myproject 디렉토리입니다. 이것은 프로젝트의 실제 Python 패키지입니다. 여기에는 프로젝트에 대한 설정, 웹 서버에 대한 일부 구성 파일 및 전역 URL 맵이 포함되어 있습니다. myproject 디렉토리 안에는 다섯 개의 파일이 있습니다:

- **__init__.py**: 이것은 Python에게 myproject 디렉토리가 Python 모듈임을 알리는 빈 파일입니다. 이전에 Python으로 작업한 적이 있다면 이러한 파일에 익숙할 것입니다.
- **settings.py**: 이것은 애플리케이션에 대한 모든 Django 설정을 포함하고 있습니다. 곧 내용에 대해 설명하겠습니다.

- `urls.py`: This has the global URL mappings that Django will initially use to locate views or other child URL mappings. You will add a URL map to this file soon.
- `asgi.py` and `wsgi.py`: These files are what ASGI or WSGI web servers use to communicate with your Django app when you deploy it to a production web server. You normally don't need to edit these at all, and they aren't used in day-to-day development. Their use will be discussed more in *Chapter 18*.

Django development server

You already started the Django development server in *Exercise 1.01 – creating a project and app, and starting the development server*. As we mentioned previously, it is a web server intended to only be run on a developer's machine during development. It is not intended for use in production.

By default, the server listens on port 8000 on localhost (127.0.0.1), but this can be changed by adding a port number, or address and port number, after the `runserver` argument:

```
python manage.py runserver 8001
```

This will have the server listen on port 8001 on localhost (127.0.0.1).

You can also have it listen on a specific address if your computer has more than one, or 0.0.0.0 for all addresses:

```
python manage.py runserver 0.0.0.0:8000
```

This will have the server listen on all your computer's addresses on port 8000, which can be useful if you want to test your application from another computer or your smartphone.

The development server watches your Django project directory and will restart automatically every time you save a file, so that any code changes you make are automatically reloaded into the server. However, you still have to manually refresh your browser to see changes there.

When you want to stop the `runserver` command, it can be done in the usual way for stopping processes in the terminal – by typing *Ctrl* + *C*.

- `urls.py`: 이것은 Django가 처음에 뷰 또는 다른 자식 URL 매핑을 찾기 위해 사용할 전역 URL 매핑입니다. 곧 이 파일에 URL 맵을 추가할 것입니다.`asgi.py` 및 `wsgi.py`: 이 파일들은 ASGI 또는 WSGI 웹 서버가 프로덕션 웹 서버에 배포할 때 Django 앱과 통신하는 데 사용하는 파일입니다. 일반적으로 `idx_5f51d95bd`를 전혀 수정할 필요가 없으며, 일상적인 개발에서는 사용되지 않습니다. 그 사용에 대해서는 18장에서 더 논의할 것입니다.

Django 개발 서버

당신은 이미 연습 1.01에서 Django 개발 서버 `idx_da58a3bb`를 시작했습니다 - 프로젝트와 앱을 생성하고 개발 서버를 시작했습니다. 앞서 언급했듯이, 이것은 개발 중에 개발자의 머신에서만 실행되도록 설계된 웹 서버입니다. 프로덕션에서 사용하기 위한 것이 아닙니다.

기본적으로 서버는 `localhost` (`127.0.0.1`)의 포트 8000에서 수신 대기하지만, `runserver` 인수 뒤에 포트 번호 또는 주소와 포트 번호를 추가하여 변경할 수 있습니다:

```
python manage.py runserver 8001
```

이렇게 하면 서버가 `localhost` (`127.0.0.1`)의 포트 8001에서 수신 대기하게 됩니다.

컴퓨터에 여러 개의 주소가 있는 경우 특정 주소에서 수신하도록 설정할 수 있으며, `0.0.0.0`은 모든 주소를 의미합니다:

```
python manage.py runserver 0.0.0.0:8000
```

이렇게 하면 서버가 포트 8000에서 컴퓨터의 모든 주소에서 수신하게 되며, 이는 다른 컴퓨터나 스마트폰에서 애플리케이션을 테스트하려는 경우 유용할 수 있습니다.

개발 서버는 Django 프로젝트 디렉토리를 감시하며 파일을 저장할 때마다 자동으로 재시작되므로, 변경한 코드가 서버에 자동으로 다시 로드됩니다.

그러나 변경 사항을 보려면 여전히 브라우저를 수동으로 새로 고쳐야 합니다. `runserver` 명령을 중지하려면 터미널에서 프로세스를 중지하는 일반적인 방법인 `Ctrl + C`를 입력하면 됩니다.

Django apps

Now that we've covered most of the theory about apps, we can be more specific about their purpose. An app directory contains all the models, views, templates (and more) that they need to provide application functionality. A Django project will contain at least one app (unless it has been heavily customized to not rely on a lot of Django functionality). If well-designed, an app should be able to be removed from a project and moved to another project without modification. Usually, an app will contain models for a single design domain, and this can be a useful way of determining whether your app should be split into multiple apps.

Your app can have any name as long as it is a valid Python module name (i.e., only letters, numbers, and underscores) and does not conflict with other files in your project directory. For example, as we have seen, there is already a directory called `myproject` in the project directory (containing the `settings.py` file), so you could not have an app called `myproject`. As we saw in *Exercise 1.01 – creating a project and app, and starting the development server*, creating an app uses the `manage.py startapp appname` command, as shown here:

```
python manage.py startapp myapp
```

The `startapp` command creates a directory within your project with the name of the app specified. It also scaffolds files for the app. Inside the app directory are a number of files and a folder, as shown in *Figure 1.9*:

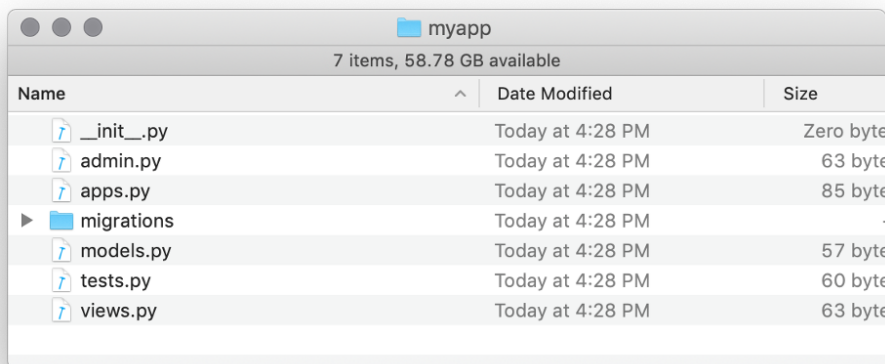


Figure 1.9 – The contents of the `myapp` app directory

- `__init__.py`: An empty file, indicating that this directory is a Python module.

장고 앱

이제 앱에 대한 이론의 대부분을 다루었으므로, 그 목적에 대해 더 구체적으로 설명할 수 있습니다. 앱 디렉토리는 애플리케이션 기능을 제공하는 데 필요한 모든 모델, 뷰, 템플릿(기타 등등)을 포함합니다. 장고 프로젝트는 최소한 하나의 앱을 포함해야 합니다 (많은 장고 기능에 의존하지 않도록 크게 사용자 정의되지 않는 한). 잘 설계된 앱은 프로젝트에서 제거되어 다른 프로젝트로 이동할 수 있어야 하며, 수정이 필요하지 않아야 합니다. 일반적으로 앱은 단일 디자인 도메인에 대한 모델을 포함하며, 이는 앱을 여러 개의 앱으로 분할해야 하는지 여부를 결정하는 데 유용한 방법이 될 수 있습니다.

앱의 이름은 유효한 파이썬 모듈 이름(즉, 문자, 숫자 및 밑줄만 포함)인 한 어떤 이름이든 가질 수 있으며, 프로젝트 디렉토리의 다른 파일과 충돌하지 않아야 합니다. 예를 들어, 우리가 보았듯이, 프로젝트 디렉토리에는 이미 settings.py 파일이 포함된 myproject라는 디렉토리가 있으므로, myproject라는 앱을 가질 수는 없습니다. 연습 1.01에서 본 것처럼 – 프로젝트와 앱을 생성하고 개발 서버를 시작하는 과정에서, 앱을 생성하는 데는 manage.py startapp appname 명령을 사용합니다. 여기서 보여준 것처럼:

```
python manage.py startapp myapp
```

startapp 명령은 idx_3480ab99 앱 이름으로 프로젝트 내에 디렉토리를 생성합니다. 또한 앱을 위한 파일을 스캐폴딩합니다. 앱 디렉토리 안에는 여러 파일과 폴더가 있으며, 그림 1.9에 나와 있습니다:

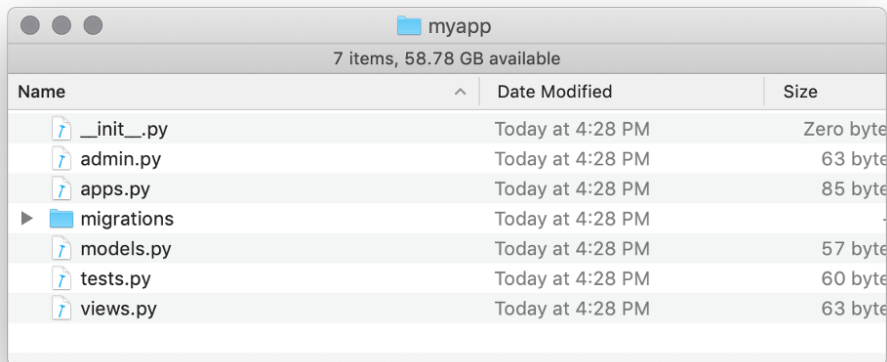


그림 1.9 - myapp 앱 디렉토리의 내용

- __init__.py: 이 디렉토리가 파이썬 모듈임을 나타내는 빈 파일입니다.

- `admin.py`: Django has a built-in admin site for viewing and editing data with a GUI. In this file, you will define how your app's models are exposed in the Django admin site. We'll cover this in more detail in *Chapter 4*.
- `apps.py`: This contains some configuration for the metadata of your app. You won't need to edit this file.
- `models.py`: This is where you will define the models for your application. You'll read about this in more detail in *Chapter 2*.
- `migrations`: Django uses migration files to automatically record changes to your underlying database as models change. They are generated by Django when you run the `manage.py makemigrations` command and stored in this directory. They do not get applied to the database until you run `manage.py migrate`. They will also be covered in *Chapter 2*.
- `tests.py`: In order to test that your code is behaving correctly, Django supports writing tests (unit, functional, or integration) and will look for them inside this file. We will write some tests throughout this book and cover testing in detail in *Chapter 14*.
- `views.py`: Your Django views (the code that responds to HTTP requests) will go in here. You will create a basic view soon, and views will be covered in more detail in *Chapter 3*.

We will examine the contents of these files in more detail later, but for now, we'll get PyCharm up and running.

PyCharm setup

We confirmed in *Exercise 1.01 – creating a project and app, and starting the development server* that the *bookr* project has been set up properly (since the development server runs successfully), so we can now start using PyCharm to run and edit our project. PyCharm is an IDE for Python development and includes features such as code completion, automatic style formatting, and a built-in debugger. We will then use PyCharm to start writing our own URL maps, views, and templates. It will also be used to start and stop the development server, which will allow debugging of your code by setting breakpoints.

Exercise 1.02 – project setup in PyCharm

In this exercise, we will open the *bookr* project in PyCharm and set up the project interpreter so that PyCharm can run and debug the project. Follow these steps:

1. Open PyCharm. When you first open PyCharm, you will be shown the **Welcome to PyCharm** screen, which asks you what you want to do:

- `admin.py`: 장고는 GUI를 사용하여 데이터를 보고 편집할 수 있는 내장 관리 사이트를 제공합니다. 이 파일에서는 앱의 모델이 장고 관리 사이트에 어떻게 노출되는지를 정의합니다. 이 내용은 4장에서 더 자세히 다룰 것입니다.
- `apps.py`: 이 파일은 앱의 메타데이터에 대한 일부 구성을 포함하고 있습니다. 이 파일을 편집할 필요는 없습니다.
- `models.py`: 이곳에서 애플리케이션의 모델을 정의합니다. 이 내용은 2장에서 더 자세히 읽을 수 있습니다.
- `migrations`: Django는 모델이 변경됨에 따라 기본 데이터베이스에 대한 변경 사항을 자동으로 기록하기 위해 마이그레이션 파일을 사용합니다. 이 파일은 `manage.py makemigrations` 명령을 실행할 때 Django에 의해 생성되며 이 디렉토리에 저장됩니다. `manage.py migrate`를 실행할 때까지 데이터베이스에 적용되지 않습니다. 이 내용은 2장에서 다룰 것입니다.
- `tests.py`: 코드가 올바르게 작동하는지 테스트하기 위해 Django는 테스트(단위, 기능 또는 통합)를 작성하는 것을 지원하며 이 파일 내에서 이를 찾습니다. 우리는 이 책 전반에 걸쳐 몇 가지 테스트를 작성하고 14장에서 테스트에 대해 자세히 다룰 것입니다.
- `views.py`: Django 뷰(HTTP 요청에 응답하는 코드)는 여기에서 작성됩니다. 곧 기본 뷰를 생성할 것이며, 뷰에 대해서는 3장에서 더 자세히 다룰 것입니다.

이 파일의 내용을 나중에 더 자세히 살펴보겠지만, 지금은 PyCharm을 시작하고 실행하겠습니다.

PyCharm 설정

우리는 연습 1.01에서 프로젝트와 앱을 생성하고 개발 서버를 시작하여 bookr 프로젝트가 제대로 설정되었음을 확인했습니다(개발 서버가 성공적으로 실행되므로), 이제 PyCharm을 사용하여 프로젝트를 실행하고 편집할 수 있습니다. PyCharm은 Python 개발을 위한 IDE로, 코드 완성, 자동 스타일 포매팅, 내장 디버거와 같은 기능을 포함합니다. 그런 다음 PyCharm을 사용하여 URL 맵, 뷰 및 템플릿을 작성하기 시작할 것입니다. 또한 개발 서버를 시작하고 중지하는 데 사용되어 코드의 중단점을 설정하여 디버깅할 수 있습니다.

연습 1.02 – PyCharm에서 프로젝트 설정

이 연습에서는 PyCharm에서 bookr 프로젝트를 열고 PyCharm이 프로젝트를 실행하고 디버깅할 수 있도록 프로젝트 인터프리터를 설정합니다. 다음 단계를 따르세요:

1. PyCharm을 엽니다. 처음 PyCharm을 열면 무엇을 할 것인지 묻는 PyCharm 환영 화면이 표시됩니다:

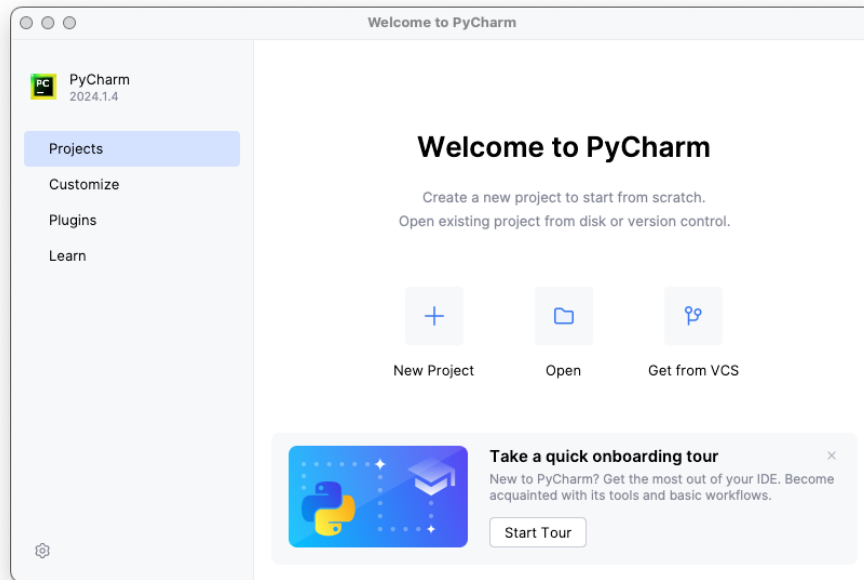


Figure 1.10 – The PyCharm welcome screen

2. Click on **Open**, then browse to the bookr project you just created, then open it. Make sure you are opening the bookr project directory and not the bookr package directory inside.
3. You will be asked if you want to trust the bookr project. Since you just created it, it's safe, so click on **Trust Project**:

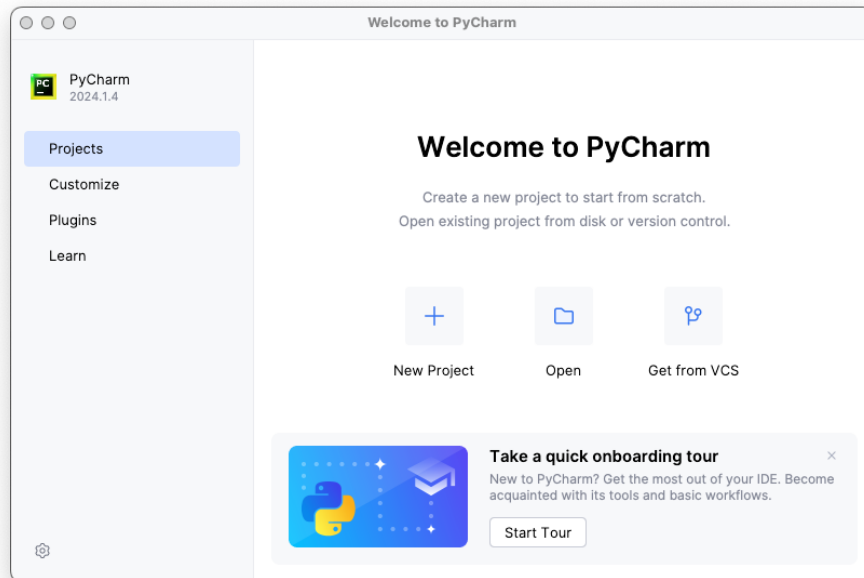


그림 1.10 - PyCharm 환영 화면

2. 열기를 클릭한 다음 방금 생성한 bookr 프로젝트로 이동하여 엽니다. bookr 프로젝트 디렉토리를 열고 bookr 패키지 디렉토리를 열지 않도록 합니다.
3. bookr 프로젝트를 신뢰할 것인지 묻는 메시지가 표시됩니다. 방금 생성했으므로 안전하니 프로젝트 신뢰를 클릭하세요:

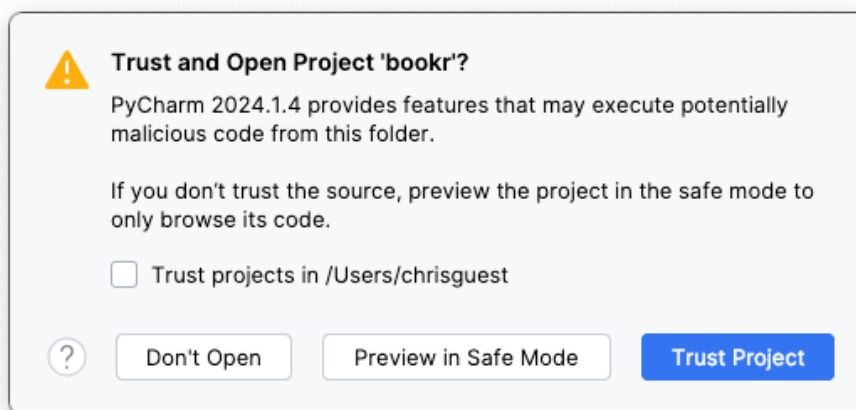


Figure 1.11 – PyCharm's Trust Project dialog

If you haven't used PyCharm before, it will ask you about what settings and themes you want to use, and once you have answered all those questions, you will see your *bookr* project structure open in the **Project** pane on the left of the window:

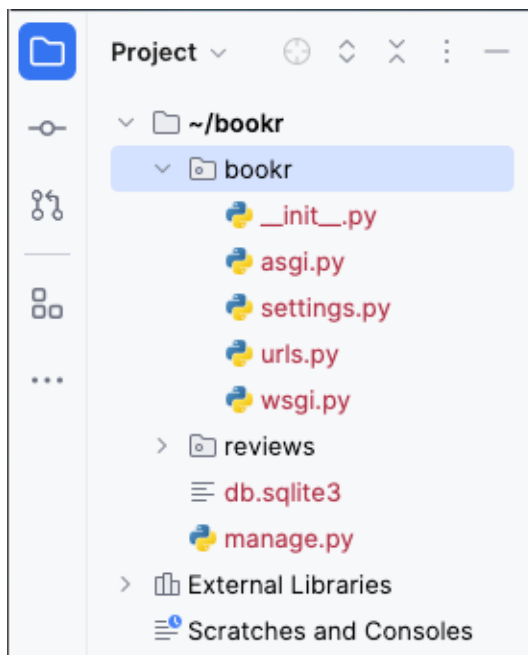


Figure 1.12 – The PyCharm Project pane

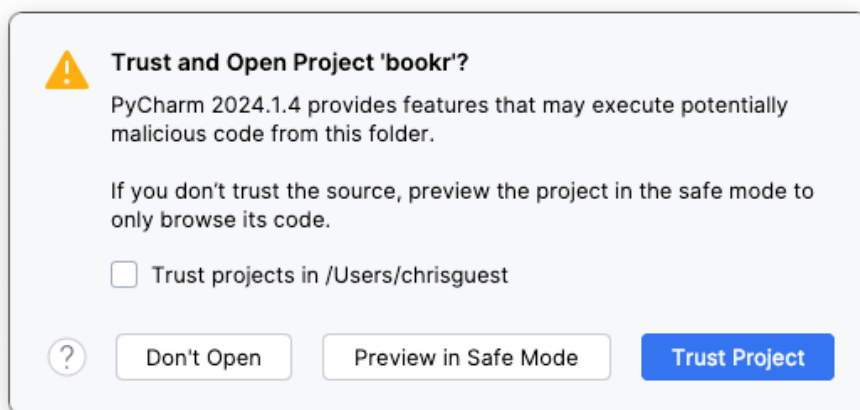


그림 1.11 - PyCharm의 신뢰 프로젝트 대화상자

idx_5eaec856PyCharm을 사용한 적이 없다면, 어떤 설정과 테마를 사용할 것인지 물어볼 것이며, 모든 질문에 답변을 완료하면, 창 왼쪽의 프로젝트 창에서 책 프로젝트 구조가 열리는 것을 볼 수 있습니다:

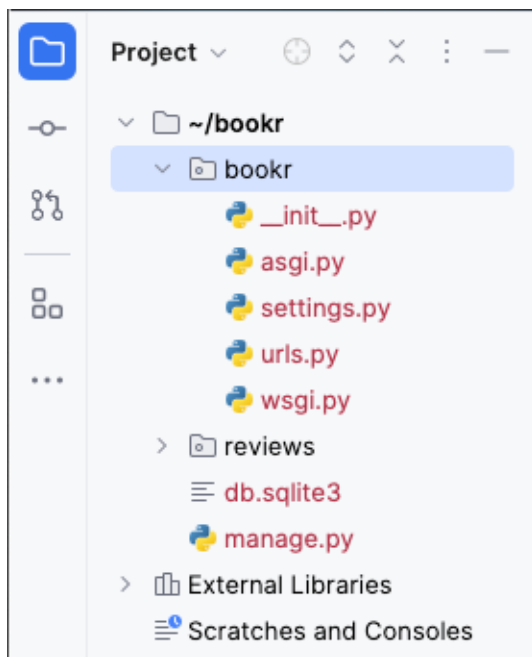


그림 1.12 - PyCharm 프로젝트 창

Your **Project** pane should look like *Figure 1.13* and show the `bookr` and `reviews` directories, and the `manage.py` file. If you do not see these and instead see `asgi.py`, `settings.py`, `urls.py`, and `wsgi.py`, then you have opened the `bookr` package directory instead. Select **File** | **Open**, and then browse and open the `bookr` project directory instead.

Before PyCharm knows how to execute your project to start the Django development server, the interpreter must be set to the Python binary inside your virtual environment. This is done first by adding the interpreter to the global interpreter settings.

4. Open the **Preferences** (macOS) or **Settings** (Windows/Linux) window inside PyCharm. Here's how you can get to these windows:
 - macOS: **PyCharm** menu | **Preferences**
 - Windows and Linux: **File** | **Settings**
5. In the preferences list pane on the left, open the **Project: bookr** item, and then click **Python Interpreter**:

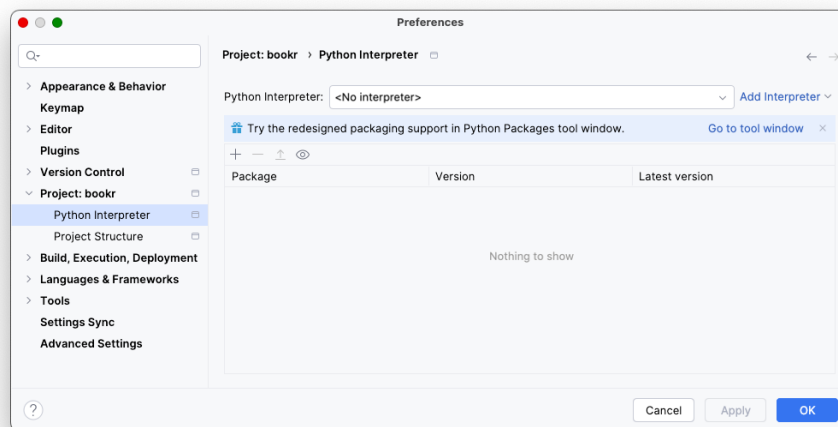


Figure 1.13 – The Python Interpreter settings

6. Sometimes, PyCharm is able to automatically determine the virtual environments, so in this case, the project interpreter may already be populated with the correct interpreter. If it has, and you see Django in the list of packages, click **OK** to close the window and complete this exercise.

프로젝트 창은 그림 1.13과 같아야 하며 bookr 및 reviews 디렉토리와 manage.py 파일을 보여야 합니다. 이러한 항목이 보이지 않고 대신 asgi.py, settings.py, urls.py 및 wsgi.py가 보인다면 bookr 패키지 디렉토리를 열었습니다. 파일 | 열기를 선택한 다음 bookr 프로젝트 디렉토리를 찾아 열어보세요.

Django 개발 서버를 시작하기 위해 프로젝트를 실행하는 방법을 PyCharm이 알기 전에, 인터프리터는 가상 환경 내의 Python 바이너리로 설정되어야 합니다. 이는 먼저 인터프리터를 전역 인터프리터 설정에 추가하여 수행됩니다.

4. PyCharm 내에서 환경설정(맥OS) 또는 설정(윈도우/리눅스) 창을 엽니다. 이 창에 접근하는 방법은 다음과 같습니다:

- 맥OS: PyCharm 메뉴 | 환경설정
- 윈도우 및 리눅스: 파일 | 설정

5. 왼쪽의 환경설정 목록 창에서 프로젝트: bookr 항목을 열고, Python 인터프리터를 클릭합니다:

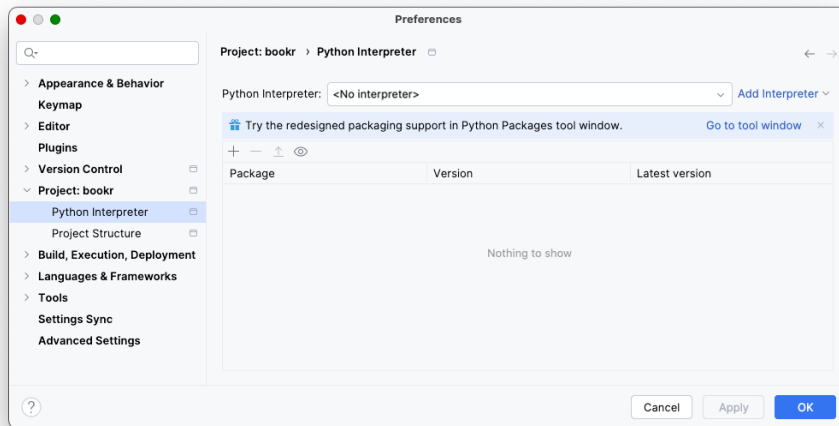


그림 1.13 - Python 인터프리터 설정

6. 가끔, PyCharm은 가상 환경을 자동으로 결정할 수 있으므로 이 경우 프로젝트 인터프리터가 이미 올바른 인터프리터로 채워져 있을 수 있습니다. 만약 그렇다면, 패키지 목록에서 Django가 보이면 창을 닫고 이 연습을 완료하기 위해 확인을 클릭하세요.

7. In most cases, though, the Python interpreter must be set manually. Click the cog icon next to the **Python Interpreter** dropdown, and then click **Add...**
8. The **Add Python Interpreter** window is now displayed. Select the **Existing Environment** radio button, and then click the ellipses (...) next to the **Interpreter** selection. You should then browse and select the Python interpreter for your virtual environment.

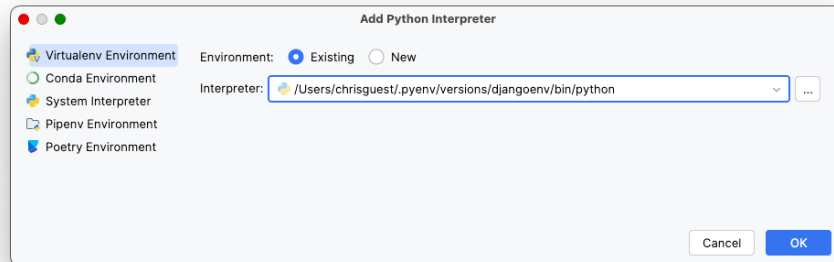


Figure 1.14 – The Add Python Interpreter window

9. On macOS (assuming you called the virtual environment `djangoenv`), the path is usually `/Users/<yourusername>/.pyenv/djangoenv/bin/python`. Similarly, in Linux, it should be in `/home/<yourusername>/.pyenv/djangoenv/bin/python`.
10. If you're unsure, you can run the `which python` command in the terminal where you previously ran the `python manage.py` command, and it will tell you the path to the Python interpreter:

```
which python
/Users/chrisguest/.pyenv/djangoenv/bin/python
```

On Windows, it will be wherever you created your virtual environment with the `pyenv` command.

After selecting the interpreter, your **Add Python Interpreter** window should look like *Figure 1.15*.

11. Click **OK** to close the **Add Python Interpreter** window. You should now see the main preferences window, and Django (and other packages in your virtual environment) will be listed (see *Figure 1.16*).

7. 대부분의 경우, Python 인터프리터는 수동으로 설정해야 합니다. Python 인터프리터 드롭다운 옆의 톱니바퀴 아이콘을 클릭한 다음, 추가(Add...)를 클릭합니다.
8. Python 인터프리터 추가 창이 이제 표시됩니다. 기존 환경(Existing Environment) 라디오 버튼을 선택한 다음, 인터프리터 선택 옆의 줄임표(...)를 클릭합니다. 그런 다음 가상 환경에 대한 Python 인터프리터를 찾아 선택해야 합니다.

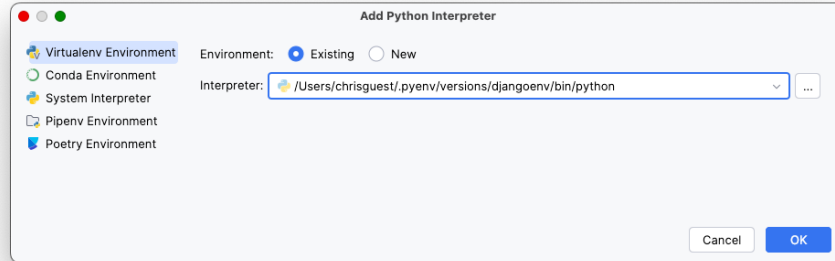


그림 1.14 - Python 인터프리터 추가 창

9. macOS에서(가상 환경을 djangoenv라고 가정할 때), 경로는 일반적으로 /Users/<yourusername>/.pyenv/djangoenv/bin/python입니다. 마찬가지로, Linux에서는 /home/<yourusername>/.pyenv/djangoenv/bin/python에 있어야 합니다.
10. 확실하지 않은 경우, 이전에 python manage.py 명령을 실행한 터미널에서 which python 명령을 실행하면 Python 인터프리터의 경로를 알려줍니다:

```
which
python/Users/chrisquest/.pyenv/djangoenv/bin/python
```

Windows에서는 pyenv 명령으로 가상 환경을 생성한 위치에 있습니다.

인터프리터를 선택한 후, Add Python Interpreter 창은 Figure 1.15와 같아야 합니다.

11. Add Python Interpreter 창을 닫으려면 OK를 클릭하세요. 이제 idx_1683f410 기본 환경 설정 창이 표시되고, Django(및 가상 환경의 다른 패키지)가 나열됩니다(그림 1.16 참조).

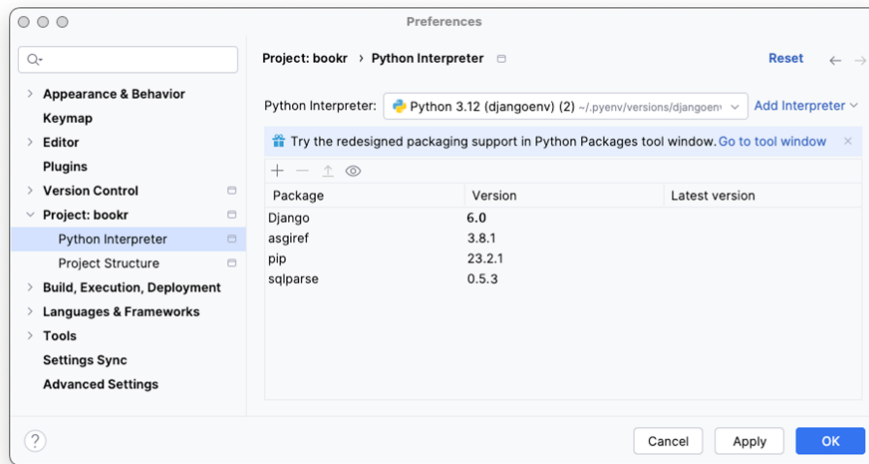


Figure 1.15 – Packages in the virtual environment are listed

12. Click **OK** in the main **Preferences** window to close it. PyCharm will now take a few seconds to index your environment and the libraries installed. You can see the process in its bottom-right status bar. Wait for this process to finish, and the progress bar will disappear.

In order to run the Django development server, Python needs to be configured with a run configuration. We will set this up now:

13. Click **Add Configuration...** at the top right of the PyCharm project window to open the **Run/Debug Configurations** window:

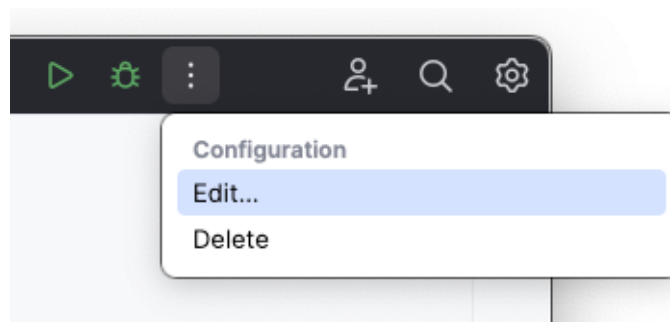


Figure 1.16 – The Add Configuration... button at the top right of the PyCharm window

14. Click the + button at the top left of this window, and select **Python** from the drop-down menu.

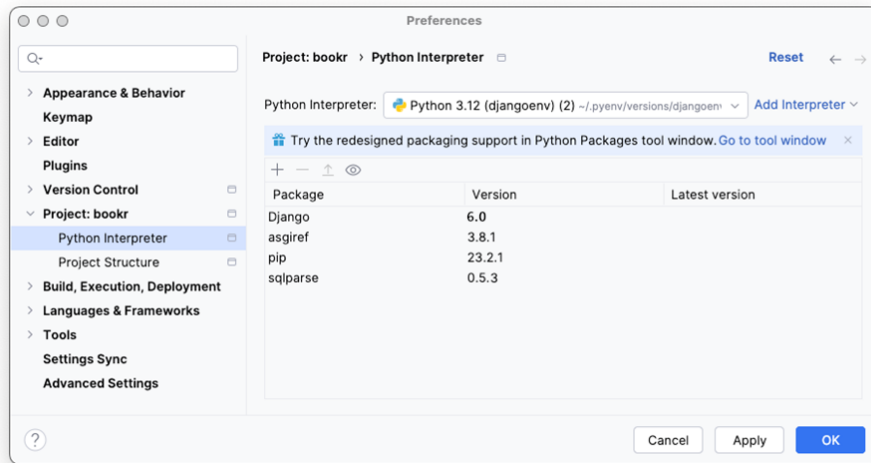


그림 1.15 - 가상 환경의 패키지가 나열됩니다

12. 주요 환경 설정 창에서 확인을 클릭하여 닫습니다. PyCharm은 이제 환경과 설치된 라이브러리를 인덱싱하는 데 몇 초가 걸립니다. 하단 오른쪽 상태 표시줄에서 이 프로세스를 볼 수 있습니다. 이 프로세스가 완료될 때까지 기다리면 진행 표시줄이 사라집니다.
장고 개발 서버를 실행하려면 Python이 실행 구성으로 설정되어야 합니다. 이제 이를 설정하겠습니다:
13. PyCharm 프로젝트 창의 오른쪽 상단에서 구성 추가...를 클릭하여 실행/디버그 구성 창을 엽니다:

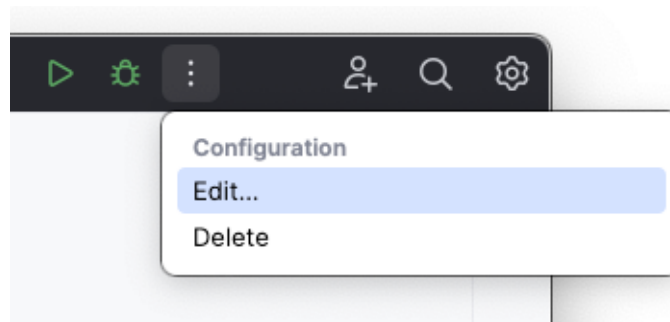


그림 1.16 - PyCharm 창 오른쪽 상단의 구성 추가... 버튼

14. 이 창의 왼쪽 상단에 있는 + 버튼을 클릭하고 드롭다운 메뉴에서 Python을 선택하세요.

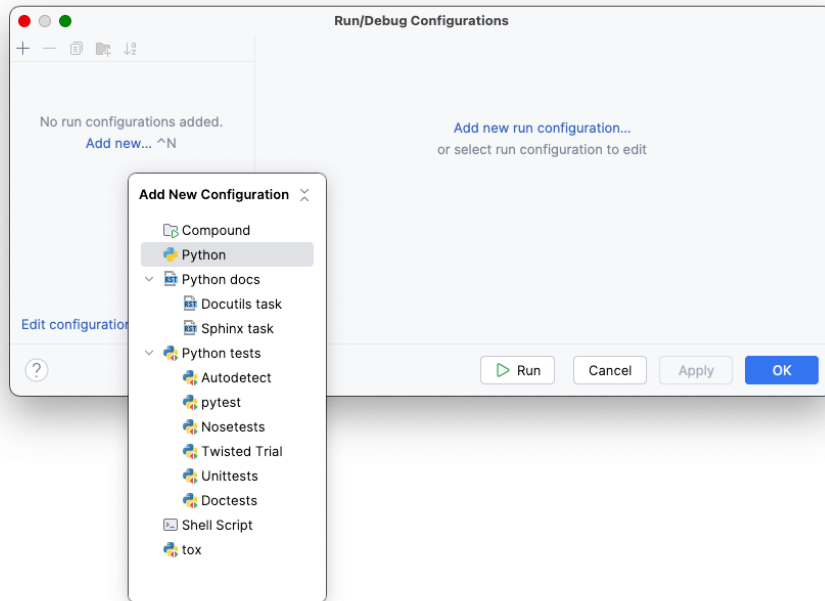


Figure 1.17 – Adding a new Python configuration in the Run/Debug Configurations window

15. A new configuration panel with fields regarding how to run your project will display on the right of the window. Go ahead and fill out the fields as follows:

- The **Name** field can be anything, but should be something understandable. Enter Django Dev Server.
- **Script path** is the path to your `manage.py` file. If you click the folder icon in this field, you can browse your filesystem to select the `manage.py` file inside the bookr project directory.
- **Parameters** refers to the arguments that come after the `manage.py` script, the same as if we were running it from the command line. We will use the same argument here to start the server, so enter `runserver`.

Note

As mentioned earlier, the `runserver` command can also accept an argument for the port or address to listen to. If you want to, you can add this argument after `runserver` in the same Parameters field.

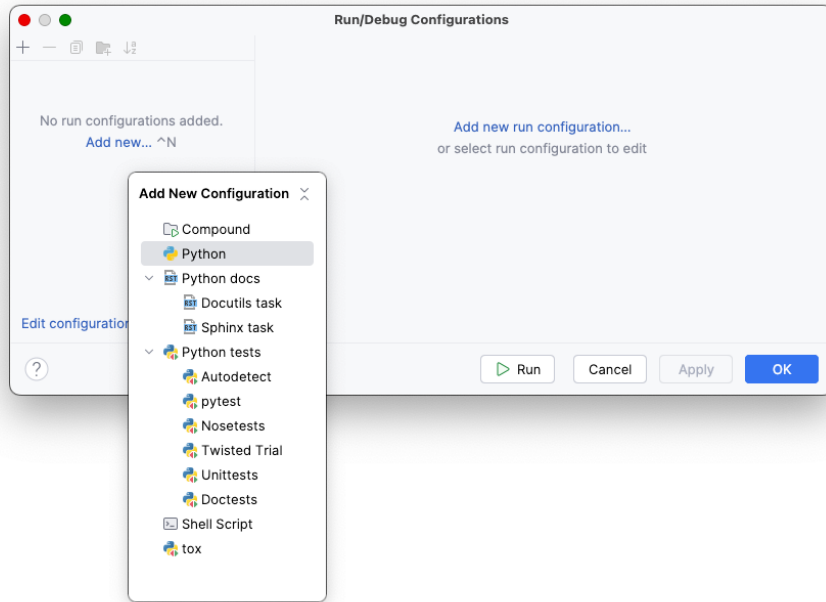


그림 1.17 - 실행/디버그 구성 창에서 새로운 Python 구성을 추가하기

15. 프로젝트를 실행하는 방법에 대한 필드가 있는 새로운 구성 패널이 창의 오른쪽에 표시됩니다. 다음과 같이 필드를 채워주세요:

- 이름 필드는 무엇이든 될 수 있지만 이해할 수 있는 것이어야 합니다. Django Dev Server를 입력하세요.
- 스크립트 경로는 manage.py 파일의 경로입니다. 이 필드에서 폴더 아이콘을 클릭하면 파일 시스템을 탐색하여 bookr 프로젝트 디렉토리 내의 manage.py 파일을 선택할 수 있습니다.
- 매개변수는 manage.py 스크립트 뒤에 오는 인수를 의미하며, 명령줄에서 실행하는 것과 동일합니다. 서버를 시작하기 위해 같은 인수를 사용할 것이므로 runserver를 입력하세요.

참고

앞서 언급했듯이, runserver 명령은 수신할 포트나 주소에 대한 인수도 받을 수 있습니다. 원하신다면, 같은 매개변수 필드에서 runserver 뒤에 이 인수를 추가할 수 있습니다.

16. The **Python Interpreter** setting should have been automatically set to the one that was set in *Exercise 1.02*. If not, click the arrow dropdown on the right to select it.
The **Working directory** setting should be the bookr project directory. This has probably already been set correctly.
17. Make sure that **Add content roots to PYTHONPATH** and **Add source roots to PYTHONPATH** are checked. This will ensure PyCharm adds your bookr project directory to PYTHONPATH (the list of paths that the Python interpreter searches when loading a module). Without these checked, the imports from your project will not work correctly.

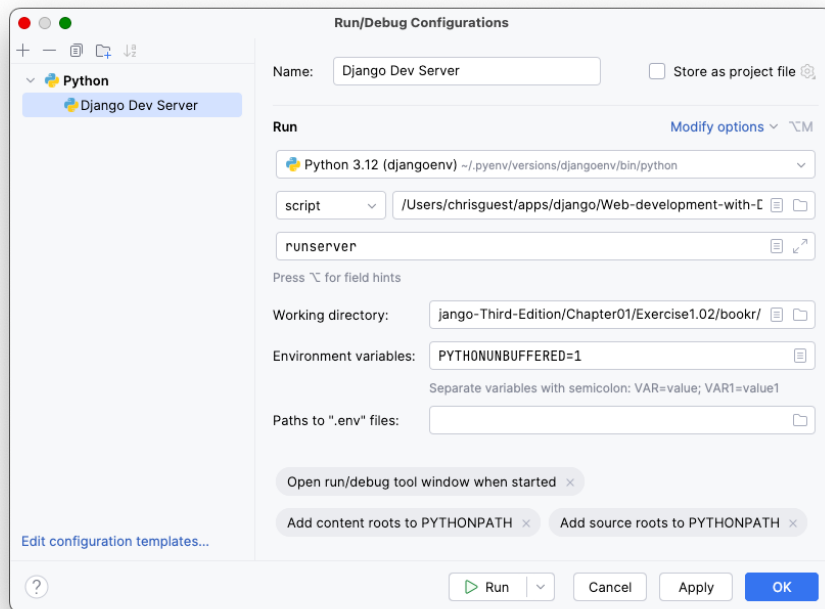


Figure 1.18 – The configuration settings

18. Check that your **Run/Debug Configurations** window looks similar to *Figure 1.19*, and then click **OK** to save the configuration.
19. Now, instead of starting the Django development server in a terminal, click the **Play** icon at the top right of the project window to start it (see *Figure 1.20*).

16. Python 인터프리터 설정은 연습 1.02에서 설정한 것으로 자동으로 설정되었어야 합니다. 그렇지 않은 경우 오른쪽의 화살표 드롭다운을 클릭하여 선택하십시오. 작업 디렉토리 설정 idx_e9a1d748은 bookr 프로젝트 디렉토리여야 합니다. 이는 아마도 이미 올바르게 설정되었을 것입니다.
17. PYTHONPATH에 콘텐츠 루트를 추가하고 PYTHONPATH에 소스 루트를 추가하는 것이 선택되어 있는지 확인하십시오. 이렇게 하면 PyCharm이 귀하의 bookr 프로젝트 디렉토리를 PYTHONPATH에 추가합니다(모듈을 로드할 때 Python 인터프리터가 검색하는 경로 목록). 이 옵션이 선택되지 않으면 프로젝트의 가져오기가 올바르게 작동하지 않습니다.

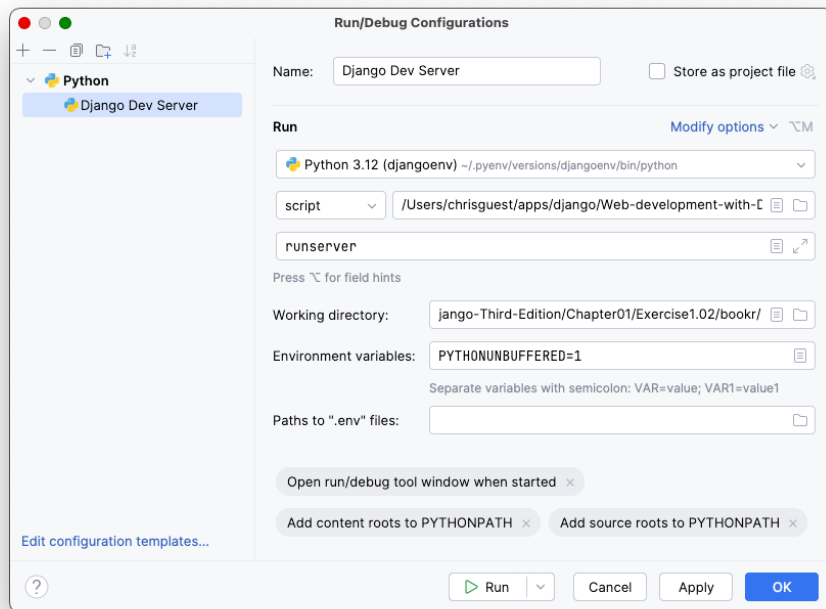


그림 1.18 - 구성 설정

18. idx_167bfc0d 귀하의 실행/디버그 구성 창이 그림 1.19와 유사한지 확인한 후 구성을 저장하려면 확인을 클릭하십시오.
19. 이제 터미널에서 Django 개발 서버를 시작하는 대신, 프로젝트 창 오른쪽 상단의 재생 아이콘을 클릭하여 시작하세요(그림 1.20 참조).



Figure 1.19 – Django development server configuration with the Play, Debug, and Stop buttons

20. Click the **Play** icon to start the Django development server.

Note

Make sure you stop any other instances of the Django development server that are running (such as in a terminal); otherwise, the one you are starting will not be able to bind to port 8000 and will fail to start.

A console will open at the bottom of the PyCharm window, which will show output indicating that the development server has started (*Figure 1.21*).

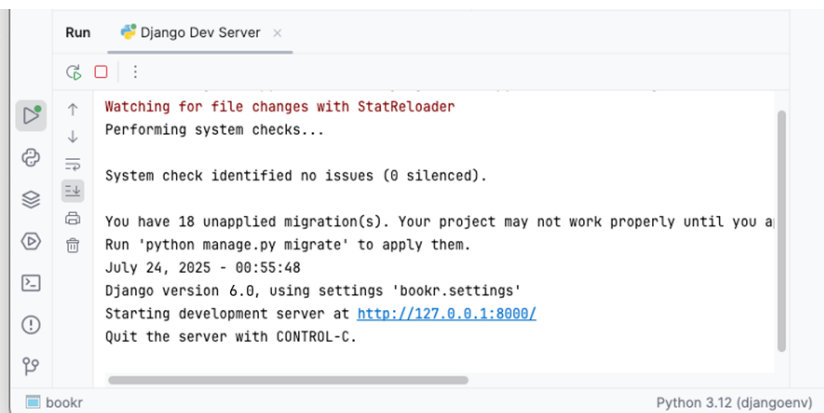


Figure 1.20 – The console with the Django development server running

21. Open a web browser and navigate to `http://127.0.0.1:8000`. You should see the same Django example screen that you saw earlier, in *Exercise 1.01 – creating a project and app, and starting the development server* (*Figure 1.21*), which will confirm that, once again, everything is set up correctly.

In this exercise, we opened the *Bookr* project in PyCharm and then set the Python interpreter for our project. We then added a run configuration in PyCharm, which allows us to start and stop the Django development server from within PyCharm. We will also be able to debug our project later by running it inside PyCharm's debugger.



그림 1.19 - Play, Debug 및 Stop 버튼이 있는 Django 개발 서버 구성

20. Play 아이콘을 클릭하여 Django 개발 서버를 시작합니다.

참고

실행 중인 Django 개발 서버의 다른 인스턴스를 중지했는지 확인하십시오(예: 터미널에서); 그렇지 않으면 시작하는 인스턴스가 포트 8000에 바인딩할 수 없고 시작에 실패합니다.

idx_720d7d8b PyCharm 창 하단에 콘솔이 열리며, 개발 서버가 시작되었음을 나타내는 출력이 표시됩니다(그림 1.21).

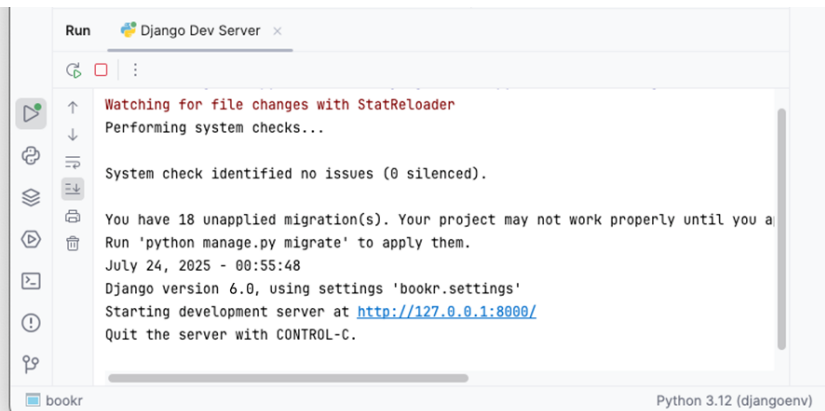


그림 1.20 - Django 개발 서버가 실행 중인 콘솔

21. 웹 브라우저를 열고 <http://127.0.0.1:8000>으로 이동합니다. 이전에 보았던 것과 동일한 Django 예제 화면이 표시되어야 하며, 연습 1.01 - 프로젝트 및 앱 생성, 개발 서버 시작(그림 1.21)에서 확인할 수 있습니다. 다시 한 번 모든 것이 올바르게 설정되었음을 확인합니다.

이 연습에서는 PyCharm에서 Bookr 프로젝트를 열고 프로젝트에 대한 Python 인터프리터를 설정했습니다. 그런 다음 PyCharm에서 실행 구성을 추가하여 PyCharm 내에서 Django 개발 서버를 시작하고 중지할 수 있게 되었습니다. 나중에 PyCharm의 디버거 내에서 실행하여 프로젝트를 디버깅할 수 있게 될 것입니다.

In this section, we explored the Django folder structure and saw the purpose of the directories and folders that were scaffolded. We also saw how to create a Django app and the files that it contains. Finally, we got a Django project up and running in PyCharm.

Introducing Django views

You now have everything set up to start writing your own Django views and configure the URLs that will map to them. As we saw earlier in this chapter, a view is simply a function that takes an `HttpRequest` instance (built by Django) and (optionally) some parameters from the URL. It will then perform some operations, such as fetching data from a database. Finally, it returns `HttpResponse`.

To use our *bookr* project as an example, we might have a view that receives a request for a certain book. It queries the database for this book and then returns a response containing an HTML page, showing information about the book. Another view could receive a request to list all the books, and then return a response with another HTML page containing this list. Views can also create or modify data; another view could receive a request to create a new book, and it would then add the book to the database and return a response with HTML that displays the new book's information.

In this chapter, we will only be using functions as views, but Django also supports class-based views that allow you to leverage object-oriented paradigms (such as inheritance). This allows you to simplify code used in multiple views that have the same business logic. For example, you might want to show all books or just books by a certain publisher. Both views need to query a list of books from the database and render them to a book list template. One `View` class could inherit from another and just implement the data fetching differently and leave the rest of the functionality (such as rendering) identical. Class-based views can be more powerful but also harder to learn. They will be introduced later, in *Chapter 11*, when you have more experience with Django.

The `HttpRequest` instance that is passed to the view contains all the data related to the request, with attributes such as the following:

- **method:** A string containing the HTTP method the browser used to request the page, usually `GET`, but it will be `POST` if a user has submitted a form. You can use this to change the flow of the view – for example, show an empty form on `GET`, or validate and process a form submission on `POST`.
- **GET:** This is a `QueryDict` class containing the parameters used in the URL query string. This is the part of the URL after `?`, if it contains one. We will go further into `QueryDict`

이 섹션에서는 장고 폴더 구조를 살펴보고 스캐폴드된 디렉토리와 폴더의 목적을 보았습니다. 또한 장고 앱을 생성하는 방법과 그 안에 포함된 파일들을 보았습니다. 마지막으로, PyCharm에서 장고 프로젝트를 설정하고 실행했습니다.

장고 뷰 소개

이제 여러분은 자신의 `idx_00d7c2d4` 장고 뷰를 작성하고 이를 매핑할 URL을 구성할 준비가 되었습니다. 이 장에서 앞서 보았듯이, 뷰는 단순히 `HttpRequest` 인스턴스(장고에 의해 생성됨)를 받고 (선택적으로) URL에서 일부 매개변수를 받는 함수입니다. 그런 다음 데이터베이스에서 데이터를 가져오는 등의 작업을 수행합니다. 마지막으로, `HttpResponse`를 반환합니다.

우리의 `bookr` 프로젝트를 예로 들면, 특정 책에 대한 요청을 받는 뷰가 있을 수 있습니다. 이 뷰는 데이터베이스에서 이 책을 쿼리한 다음, 책에 대한 정보를 보여주는 HTML 페이지를 포함한 응답을 반환합니다. 또 다른 뷰는 모든 책을 나열하라는 요청을 받을 수 있으며, 그런 다음 이 목록을 포함한 또 다른 HTML 페이지로 응답을 반환할 수 있습니다. 뷰는 데이터를 생성하거나 수정할 수도 있습니다. 또 다른 뷰는 새 책을 생성하라는 요청을 받을 수 있으며, 그러면 데이터베이스에 책을 추가하고 새 책의 정보를 표시하는 HTML로 응답을 반환합니다.

이 장에서는 뷰로서 함수만 사용할 것이지만, 장고는 객체 지향 패러다임(예: 상속)을 활용할 수 있는 클래스 기반 뷰도 지원합니다. 이를 통해 동일한 비즈니스 로직을 가진 여러 뷰에서 사용되는 코드를 단순화할 수 있습니다. 예를 들어, 모든 책을 보여주거나 특정 출판사의 책만 보여주고 싶을 수 있습니다. 두 뷰 모두 데이터베이스에서 책 목록을 쿼리하고 이를 책 목록 템플릿에 렌더링해야 합니다. 한 뷰 클래스는 다른 클래스에서 상속받아 데이터 가져오는 방식을 다르게 구현하고 나머지 기능(예: 렌더링)은 동일하게 유지할 수 있습니다. 클래스 기반 뷰는 더 강력할 수 있지만 배우기 더 어려울 수도 있습니다. 이들은 나중에 11장에서 장고에 대한 경험이 더 쌓인 후에 소개될 것입니다.

뷰에 전달되는 `HttpRequest` 인스턴스는 요청과 관련된 모든 데이터를 포함하고 있으며, 다음과 같은 속성을 가지고 있습니다:

- `method`: 페이지 요청에 사용된 HTTP 메서드를 포함하는 문자열로, 일반적으로 GET이지만 사용자가 양식을 제출한 경우 POST가 됩니다. 이를 사용하여 뷰의 흐름을 변경할 수 있습니다. 예를 들어, GET에서는 빈 양식을 보여주고, POST에서는 양식 제출을 검증하고 처리할 수 있습니다.
- `GET`: URL 쿼리 문자열에서 사용된 매개변수를 포함하는 `QueryDict` 클래스입니다. 이는 ? 이후의 URL 부분입니다. 만약 포함되어 있다면 말이죠. 우리는 `QueryDict`에 대해 더 깊이 들어갈 것입니다.

classes soon. Note that this attribute is always available, even if the request was not GET.

- **POST:** This is another `QueryDict` containing the parameters sent to the view in a POST request, such as from a form submission. Usually, you would use this in conjunction with a Django form, which will be covered in *Chapter 6, Forms*.
- **headers:** This is a case-insensitive key dictionary with the HTTP headers from the request. For example, you could vary the response with different content for different browsers based on the *User-Agent* header. We discussed some HTTP headers that are sent by the client earlier in this chapter.
- **path:** This is the path used in the request. Normally, you don't need to examine this because Django will automatically parse the path and pass it to the view function as parameters, but it can be useful in some instances.

We won't be using all these attributes yet, and there are others that will be introduced later, but you can now see what role the `HttpRequest` argument plays in your view.

Next, we'll see how Django determines what view to use for a specific URL.

Exploring URL mapping details

We briefly mentioned URL maps earlier, in the *Processing a request* section. Django does not automatically know which view function should be executed when it receives a request for a particular URL. The role of URL mapping is to build a link between a URL and a view. For example, in Bookr, you might want to map the `/books/` URL to a `books_list` view that you have created.

The URL-to-view mapping is defined in the file that Django automatically created, called `urls.py`, inside the bookr package directory (although a different file can be set in `settings.py`; there'll be more on that later).

This file contains a variable, `urlpatterns`, which is a list of paths that Django evaluates in turn until it finds a match for the URL being requested. The match will either resolve to a view function or another `urls.py` file, also containing a `urlpatterns` variable, which will be resolved in the same manner. URL files can be chained in this manner for as long as you want. This way, you can split URL maps into separate files (such as one or more per app) so that they don't become too large. Once a view has been found, Django calls it with `HttpRequest` and any parameters parsed from the URL.

클래스 곧. 이 속성은 요청이 GET이 아니더라도 항상 사용 가능합니다.

- **POST:** 이는 POST 요청에서 뷰로 전송된 매개변수를 포함하는 또 다른 QueryDict입니다. 일반적으로, 이는 장고 양식과 함께 사용되며, 이는 6장, 양식에서 다룰 것입니다.
- **headers:** 이는 요청의 HTTP 헤더가 포함된 대소문자를 구분하지 않는 키 사전입니다. 예를 들어, User-Agent 헤더에 따라 서로 다른 브라우저에 대해 서로 다른 콘텐츠로 응답을 변경할 수 있습니다. 우리는 이 장의 앞부분에서 클라이언트에 의해 전송된 일부 HTTP 헤더에 대해 논의했습니다.
- **path:** 이는 요청에 사용된 경로입니다. 일반적으로, 장고가 경로를 자동으로 구문 분석하고 이를 `idx_2a08d133` 매개변수로 뷰 함수에 전달하므로 이를 검사할 필요는 없지만, 일부 경우에는 유용할 수 있습니다.

우리는 아직 이 모든 속성을 사용하지 않을 것이며, 나중에 소개될 다른 속성들도 있지만, 이제 `HttpRequest` 인수가 뷰에서 어떤 역할을 하는지 볼 수 있습니다.

다음으로, 장고가 특정 URL에 대해 어떤 뷰를 사용할지 결정하는 방법을 살펴보겠습니다.

URL 매핑 세부사항 탐색

우리는 요청 처리 섹션에서 URL 맵 `idx_4f322667`에 대해 간략하게 언급했습니다. 장고는 특정 URL에 대한 요청을 받을 때 어떤 뷰 함수를 실행해야 하는지 자동으로 알지 못합니다. URL 매핑의 역할은 URL과 뷰 간의 링크를 구축하는 것입니다. 예를 들어, Bookr에서는 `/books/` URL을 생성한 `books_list` 뷰에 매핑하고 싶을 수 있습니다.

Django가 자동으로 생성한 파일인 `urls.py`에 URL-뷰 매핑이 정의되어 있으며, 이는 bookr 패키지 디렉토리 내에 있습니다(다른 파일을 `settings.py`에서 설정할 수 있지만, 이에 대해서는 나중에 더 설명하겠습니다).

이 파일에는 `urlpatterns`라는 변수가 포함되어 있으며, 이는 Django가 요청된 URL에 대한 일치를 찾을 때까지 차례로 평가하는 경로 목록입니다. 일치는 뷰 함수 또는 또 다른 `urls.py` 파일로 해결되며, 이 파일에도 `urlpatterns` 변수가 포함되어 있으며, 동일한 방식으로 해결됩니다. URL 파일은 원하는 만큼 이 방식으로 연결할 수 있습니다. 이렇게 하면 URL 맵을 별도의 파일(앱당 하나 이상의 파일 등)로 분할하여 너무 커지지 않도록 할 수 있습니다. 뷰가 발견되면 Django는 `HttpRequest`와 URL에서 구문 분석된 모든 매개변수로 이를 호출합니다.

Rules are set by calling the `path` function, which takes the path of the URL as the first argument. The path can contain named parameters that will be passed to a view as function parameters. Its second argument is either a view or another file also containing `urlpatterns`.

There is also the `re_path` function, which is similar to `path`, except that it takes a regular expression as the first argument for more advanced configuration. There is much more to URL mapping; it will be covered in *Chapter 3*. To illustrate these concepts, *Figure 1.22* shows the default `urls.py` file that Django generates:

```
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

Figure 1.21 – The default `urls.py` file

You can see the `urlpatterns` variable, which lists all the URLs that are set up. Currently, there is only one rule set up, which maps any path starting with `admin/` to the admin URL maps (the `admin.site.urls` module). This is not a mapping to a view; instead, it is an example of chaining URL maps together – the `admin.site.urls` module will define the remainder of the paths (after `admin/`) that map to the admin views. We will cover the Django admin site later in the book, starting in *Chapter 4*.

We will now write a view and set up a URL map to it, to see these in action.

Exercise 1.03 – writing a view and mapping a URL to it

Our first view will be very simple and will just return some static text content. In this exercise, we will see how to write a view and set up a URL map to resolve to a view:

Note

As you make changes to files in your project and save them, you might see the Django development server automatically restarting in the terminal or console in which it is running. This is normal – it automatically restarts to load any code changes that you

규칙은 URL의 경로를 첫 번째 인수로 사용하는 path 함수를 호출하여 설정됩니다. 경로에는 함수 매개변수로 뷰에 전달될 이름이 지정된 매개변수가 포함될 수 있습니다. 두 번째 인수는 뷰 또는 urlpatterns를 포함하는 다른 파일입니다.

정규 표현식을 첫 번째 인수로 사용하여 더 고급 구성을 위한 path와 유사한 re_path 함수도 있습니다. URL 매핑에 대한 내용은 훨씬 더 많으며, 3장에서 다룰 것입니다. 이러한 개념을 설명하기 위해 그림 1.22는 장고가 생성하는 기본 urls.py 파일을 보여줍니다:

```
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

그림 1.21 - 기본 urls.py 파일

설정된 모든 URL을 나열하는 urlpatterns 변수를 볼 수 있습니다. 현재 설정된 규칙은 하나뿐이며, admin/로 시작하는 모든 경로를 admin URL 맵(즉, admin.site.urls 모듈)으로 매핑합니다. 이는 뷰에 대한 매핑이 아니라 URL 맵을 함께 연결하는 예입니다. admin.site.urls 모듈은 admin/ 이후의 경로를 정의하여 admin 뷰에 매핑합니다. 이 책의 4장에서 장고 관리 사이트에 대해 다룰 것입니다.

이제 뷰를 작성하고 URL 맵을 설정하여 이를 실행해 보겠습니다.

연습 1.03 – 뷰 작성 및 URL 매핑

우리의 첫 번째 뷰는 매우 간단하며 정적 텍스트 콘텐츠를 반환합니다. 이 연습에서는 뷰를 작성하고 뷰에 해결되는 URL 맵을 설정하는 방법을 살펴보겠습니다:

참고

프로젝트의 파일을 변경하고 저장하면, Django development 서버가 자동으로 실행 중인 터미널이나 콘솔에서 재시작되는 것을 볼 수 있습니다. 이는 정상입니다 – 코드 변경 사항을 로드하기 위해 자동으로 재시작됩니다.

make. Please also note that it won't automatically apply changes to the database if you edit models or migrations – there'll be more on this in *Chapter 2*.

1. In PyCharm, expand the reviews folder in the **Project** browser on the left, and then double-click on the `views.py` file inside to open it. In the right (editor) pane in PyCharm, you should see the placeholder text automatically generated by Django:

```
from django.shortcuts import render

# Create your views here.
```

2. Remove this placeholder text from `views.py` and instead insert this content:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello, world!")
```

First, the `HttpResponse` class needs to be imported from `django.http`. This is what is used to create the response that goes back to the web browser. You can also use it to control things such as the HTTP headers or status code. For now, it will just use the default headers and the 200 Success status code. Its first argument is the string content to send as the body of the response.

Then, the view function returns an `HttpResponse` instance with the content we defined (Hello, world!).

3. We will now set up a URL pattern that maps the `index` view. This will be very simple and will not contain any parameters. Expand the `bookr` directory in the **Project** pane, and then open `urls.py`. Django has automatically generated this file. For now, we'll just add a simple URL to replace the default index that Django provides.
4. Import your views into the `urls.py` file by adding this line after the other existing imports:

```
import reviews.views
```

변경 사항이 모델이나 마이그레이션에 적용되더라도 데이터베이스에 자동으로 적용되지 않으니 참고하시기 바랍니다. 이에 대한 내용은 2장에서 더 다룰 것입니다.

1. PyCharm에서 왼쪽의 프로젝트 브라우저에서 reviews 폴더를 확장한 다음 views.py 파일을 두 번 클릭하여 엽니다. PyCharm의 오른쪽(편집기) 창에서 Django에 의해 자동으로 생성된 자리 표시자 텍스트를 볼 수 있어야 합니다:

```
from django.shortcuts import render

# 여기에 당신의 뷰를 작성하세요.
```

2. views.py에서 이 자리 표시자 텍스트를 제거하고 대신 이 내용을 삽입하세요:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello, world!")
```

먼저, HttpResponse 클래스는 django.http에서 가져와야 합니다. 이것은 웹 브라우저로 돌아가는 응답을 생성하는 데 사용됩니다. 또한 HTTP 헤더나 상태 코드와 같은 것들을 제어하는 데 사용할 수 있습니다. 지금은 기본 헤더와 200 성공 상태 코드를 사용할 것입니다. 첫 번째 인수는 응답 본문으로 보낼 문자열 내용입니다.

그런 다음, view 함수 index_cce1260a는 우리가 정의한 내용 (Hello, world!)과 함께 HttpResponse 인스턴스를 반환합니다.

3. 이제 index 뷰를 매핑하는 URL idx_ecd0c618 패턴을 설정하겠습니다. 이것은 매우 간단하며 매개변수를 포함하지 않습니다. 프로젝트 창에서 bookr 디렉토리를 확장한 다음 urls.py를 엽니다. Django가 이 파일을 자동으로 생성했습니다.

지금 Django가 제공하는 기본 인덱스를 대체할 간단한 URL을 추가하겠습니다.

4. 다른 기존 import 뒤에 이 줄을 추가하여 views를 urls.py 파일로 가져오세요:

```
import reviews.views
```

5. Add a map to the index view to the `urlpatterns` list by adding a call to the `path` function, with an empty string and a reference to the index function:

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', reviews.views.index),  
]
```

Make sure you don't add brackets after the index function (i.e., it should be `reviews.views.index` and not `reviews.views.index()`), as we are passing a reference to a function rather than calling it.

6. Switch back to your web browser and refresh. The Django default welcome screen should be replaced with the text defined in the view, **Hello, world!**.

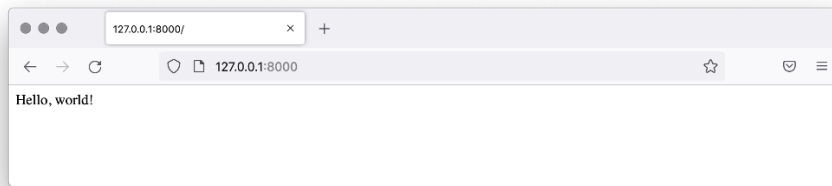


Figure 1.22 – The web browser should now display the *Hello, world!* message

We just saw how to write a view function and map a URL to it. We then tested the view by loading it in a web browser.

Next, we'll see how to work with data that is sent in an HTTP request, but not in the URL path.

Working with GET, POST, and QueryDict objects

Data can come through an HTTP request as parameters on a URL or inside the body of a POST request. You might have noticed parameters in a URL when browsing the web – the text after `?` – for example, `http://www.example.com/?parameter1=value1¶meter2=value2`. We also saw earlier in this chapter an example of form data in a POST request to log in a user (the request body was `username=user&password=password1`).

Django automatically parses these parameter strings into `QueryDict` objects. The data is then available on the `HttpRequest` object that is passed to your view – specifically, in the `HttpRequest.GET` and `HttpRequest.POST` attributes for URL parameters and body parameters,

5. 빈 문자열과 인덱스 함수에 대한 참조를 사용하여 path 함수를 호출하여 urlpatterns 목록에 인덱스 뷰에 지도를 추가합니다:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', reviews.views.index),
]
```

인덱스 함수 뒤에 괄호를 추가하지 않도록 하세요 (즉, reviews.views.index()가 아니라 reviews.views.index여야 합니다. 함수 호출이 아니라 함수에 대한 참조를 전달하고 있습니다.)

6. 웹 브라우저로 돌아가서 새로 고침하세요. Django 기본 환영 화면은 뷰에서 정의된 텍스트인 Hello, world!로 대체되어야 합니다.

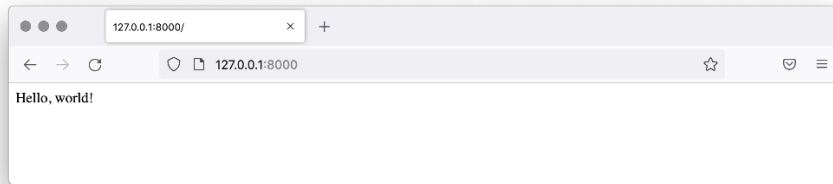


그림 1.22 - 웹 브라우저는 이제 Hello, world! 메시지를 표시해야 합니다.

우리는 방금 뷰 함수를 작성하고 URL을 매핑하는 방법을 보았습니다. 그런 다음 웹 브라우저에서 로드하여 뷰를 테스트했습니다.

다음으로, URL 경로가 아닌 HTTP 요청으로 전송된 데이터와 작업하는 방법을 살펴보겠습니다.

GET, POST 및 QueryDict 객체 작업하기

데이터는 HTTP 요청을 통해 URL의 매개변수로 또는 POST 요청의 본문 내에서 idx_0c2b3cae 전송될 수 있습니다. 웹을 탐색할 때 URL의 매개변수 - 예를 들어, <http://www.example.com/?parameter1=value1¶meter2=value2>에서 ? 뒤의 텍스트 idx_059fec93를 주목했을 것입니다. 우리는 또한 이 장의 앞부분에서 사용자를 로그인하기 위한 POST 요청의 양식 데이터 예를 보았습니다(요청 본문은 username=user&password=password1이었습니다).

Django는 이러한 매개변수 문자열을 자동으로 QueryDict 객체로 구문 분석합니다. 그런 다음 데이터는 뷰에 전달되는 HttpRequest 객체에서 사용할 수 있습니다 - 특히 URL 매개변수 및 본문 매개변수에 대한 HttpRequest.GET 및 HttpRequest.POST 속성에서.

respectively. QueryDict objects mostly behave like dictionaries, except that they can contain multiple values for a key.

To show different methods of accessing items, we'll use a simple QueryDict (the `qd` variable) with only one key (`k`) as an example. The `k` item has three values in a list – the `a`, `b`, and `c` strings. The following code snippets show output from a Python interpreter.

First, the QueryDict `qd` variable is constructed from a parameter string:

```
>>> from django.http.request import QueryDict
>>> qd = QueryDict("k=a&k=b&k=c")
```

When accessing items with square bracket notation or the `get` method, the last value for that key is returned:

```
>>> qd["k"]
'c'
>>> qd.get("k")
'c'
```

To access all the values for a key, the `getlist` should be used, as follows:

```
>>> qd.getlist("k")
['a', 'b', 'c']
```

`getlist` will always return a list; it will be empty if the key does not exist:

```
>>> qd.getlist("bad key")
[]
```

While `getlist` does not raise an exception for keys that do not exist, accessing a key that does not exist with square bracket notation will raise `KeyError`, like a normal dictionary. Use the `get` method to avoid this error.

The QueryDict objects for GET and POST are immutable (they cannot be changed), so the `copy` method should be used to get a mutable copy if you need to change its values, as follows:

```
>>> qd["k"] = "d"
AttributeError: This QueryDict instance is immutable
>>> qd2 = qd.copy()
>>> qd2
```

각각. QueryDict 객체는 대부분 사전처럼 동작하지만, 키에 대해 여러 값을 포함할 수 있습니다.

항목에 접근하는 다양한 방법을 보여주기 위해, 우리는 단일 키(k)만 있는 간단한 QueryDict(변수 qd)를 예로 사용할 것입니다. k 항목은 a, b, c 문자열이 포함된 리스트에서 세 가지 값을 가집니다. 다음 코드 조각은 Python 인터프리터의 출력을 보여줍니다.

먼저, QueryDict qd 변수는 매개변수 문자열에서 구성됩니다:

```
>>> from django.http.request import QueryDict
>>> qd = QueryDict("k=a&k=b&k=c")
```

대괄호 표기법이나 get 메서드를 사용하여 항목에 접근할 때, 해당 키의 마지막 값이 반환됩니다:

```
>>> qd["k"]
'c'
>>> qd.get("k")
'c'
```

키에 대한 모든 값을 접근하려면, getlist를 사용해야 합니다. 다음과 같이:

```
>>> qd.getlist("k")
['a', 'b', 'c']
```

getlist는 항상 리스트를 반환합니다; 키가 존재하지 않으면 빈 리스트를 반환합니다:

```
>>> qd.getlist("bad key")
[]
```

getlist는 존재하지 않는 키에 대해 예외를 발생시키지 않지만, 존재하지 않는 키에 대해 대괄호 표기법으로 접근하면 일반 사전처럼 KeyError를 발생시킵니다. 이 오류를 피하려면 get 메서드를 사용하세요.

GET 및 POST에 대한 QueryDict 객체는 불변입니다(변경할 수 없습니다), 따라서 값을 변경해야 하는 경우 가변 복사본을 얻기 위해 copymethod를 사용해야 합니다. 다음과 같이:

```
>>> qd["k"] = "d"
AttributeError: 이 QueryDict 인스턴스는 변경할 수 없습니다
>>> qd2 = qd.copy()
>>> qd2
```

```
<QueryDict: {'k': ['a', 'b', 'c']}>
>>> qd2["k"] = "d"
>>> qd2["k"]
"d"
```

To give an example of how QueryDict is populated from a URL, let's imagine an example URL: `http://127.0.0.1:8000?val1=a&val2=b&val2=c&val3`.

Behind the scenes, Django passes the query from the URL (everything after `?`) to instantiate a QueryDict object and attach it to request, which is passed to the view function. It goes something like this:

```
request.GET = QueryDict("val1=a&val2=b&val2=c&val3")
```

Remember, this is done to request before you receive it inside your view function, so you do not need to do this.

In the case of the preceding example URL, we can access the parameters inside the view function as follows:

```
request.GET["val1"]
```

Using standard dictionary access, it would return the "a" value:

```
request.GET["val2"]
```

Again, using standard dictionary access, there are two values set for the val2 key, so it would return the last value, "c":

```
request.GET.getlist("val2")
```

This would return a list of all the values for val2 – ["b", "c"]:

```
request.GET["val3"]
```

This key is in the query string but has no value set, so this returns an empty string:

```
request.GET["val4"]
```

```
<QueryDict: {'k': ['a', 'b', 'c']}>
>>> qd2["k"] = "d"
>>> qd2["k"]
"d"
```

QueryDict가 URL에서 어떻게 채워지는지 예를 들어 보겠습니다. 예제 URL을 상상해 보세요: `http://127.0.0.1:8000?val1=a&val2=b&val2=c&val3`.

뒤에서 장고는 URL의 쿼리(모든 것이? 이후에 있는)를 전달하여 QueryDict 객체를 인스턴스화하고 이를 요청에 첨부하여 뷰 함수에 전달합니다. 대략 이런 식으로 진행됩니다:

```
request.GET = QueryDict("val1=a&val2=b&val2=c&val3")
```

기억하세요, 이것은 뷰 함수 내부에서 요청을 받기 전에 수행되므로, 이 작업을 할 필요는 없습니다.

앞서 언급한 예제 URL의 경우, 다음과 같이 viewfunction 내부의 매개변수에 접근할 수 있습니다:

```
request.GET["val1"]
```

표준 사전 접근 방식을 사용하면 "a" 값을 반환합니다:

```
request.GET["val2"]
```

다시 말해, 표준 사전 접근 방식을 사용하면 val2 키에 대해 두 개의 값이 설정되어 있으므로 마지막 값인 "c"를 반환합니다:

```
request.GET.getlist("val2")
```

이것은 val2에 대한 모든 값의 목록을 반환합니다 – ["b", "c"]:

```
request.GET["val3"]
```

이 키는 쿼리 문자열에 있지만 값이 설정되어 있지 않으므로 빈 문자열을 반환합니다:

```
request.GET["val4"]
```

This key is not set, so `KeyError` will be raised. Use `request.GET.get("val4")` instead, which will return `None`:

```
request.GET.getlist("val4")
```

Since this key is not set as an empty list, `([])` will be returned.

We will now look at `QueryDict` in action using the GET parameters. We will examine POST parameters further in *Chapter 6*.

Exercise 1.04 – exploring GET values and QueryDict objects

We will now make some changes to our index view from the previous exercise to read values from the URL in the GET attribute, and then we will experiment with passing different parameters to see the result:

1. Open the `views.py` file in PyCharm. Add a new variable called `name`, which reads the user's name from the GET parameters. Add this line after the index function definition:

```
name = request.GET.get("name") or "world"
```

2. Change the return value so that the `name` is used as part of the content that is returned:

```
return HttpResponse(f"Hello, {name}!")
```

Visit `http://127.0.0.1:8000` in your browser. Note that the page still says **Hello, world!**. This is because we have not supplied a `name` parameter. You can add your name to the URL – for example, `http://127.0.0.1:8000?name=Ben`. This can be seen in the following screenshot:

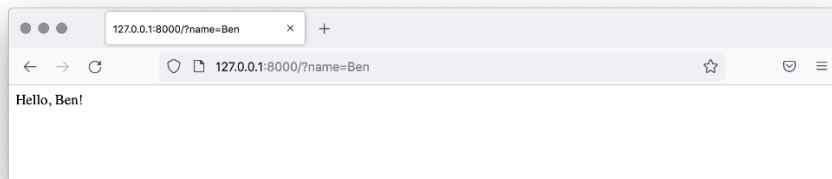


Figure 1.23 – Setting the name in the URL

이 키는 설정되지 않았으므로 `KeyError`가 발생합니다. 대신 `request.GET.get("val4")`를 사용하세요. 이는 `None`을 반환합니다:

```
request.GET.getlist("val4")
```

이 `idx_15a2f7e6key`는 `idx_182bb8d7`로 설정되지 않았으므로 `idx_8496f734` 빈 리스트 (`[]`)가 반환됩니다.

이제 GET 매개변수를 사용하여 `QueryDict`의 작동을 살펴보겠습니다. POST 매개변수는 6장에서 더 살펴보겠습니다.

연습 1.04 – GET 값 및 QueryDict 객체 탐색

이제 이전 연습에서 인덱스 뷰에 몇 가지 변경을 하여 URL의 값 `idx_adebbee3`를 GET 속성에서 읽고, 다양한 매개변수를 전달하여 결과를 확인해 보겠습니다:

1. PyCharm에서 `views.py` 파일을 엽니다. GET 매개변수에서 사용자의 이름을 읽는 `name`이라는 새 변수를 추가합니다. 인덱스 함수 정의 후에 이 줄을 추가하세요:

```
name = request.GET.get("name") or "world"
```

2. 반환 값을 변경하여 이름이 콘텐츠의 일부로 사용되도록 하십시오. 반환됨:

```
return HttpResponse(f"안녕하세요, {name}!")
```

브라우저에서 `http://127.0.0.1:8000` 을 방문하세요. 페이지에 여전히 Hello, world!라고 표시되는 것을 확인할 수 있습니다. 이는 `name` 매개변수를 제공하지 않았기 때문입니다. URL에 이름을 추가할 수 있습니다 – 예를 들어, `http://127.0.0.1:8000?name=Ben`. 이는 다음 스크린샷에서 확인할 수 있습니다:

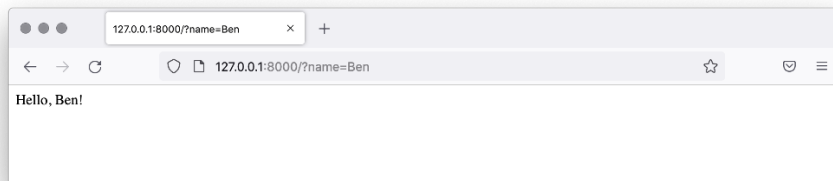


그림 1.23 – URL에서 이름 설정하기

3. Try adding two names – for example, `http://127.0.0.1:8000/?name=Ben&name=John`. As we mentioned, the last value for the parameter is retrieved with the `get` function, so you should see **Hello, John!**:

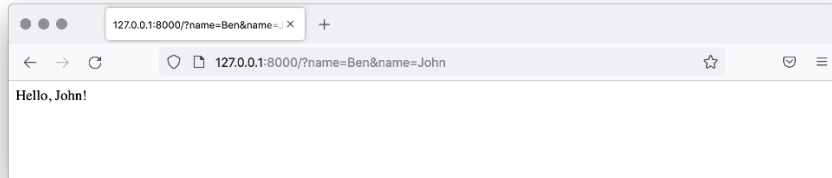


Figure 1.24 – Setting multiple names in the URL

4. Try setting no name, like this – `http://127.0.0.1:8000/?name=`. The page should go back to displaying **Hello, world!**:

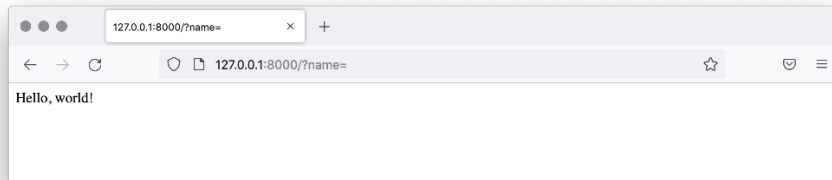


Figure 1.25 – No name set in the URL

Note

You might wonder why we set `name` to the default **world** by using `or`, instead of passing `'world'` as the default value to `get`. Consider what happened in *step 6*, when we passed in a blank value for the `name` parameter. If we had passed `'world'` as a default value for `get`, then the `get` function would still have returned an empty string. This is because a value *is* set for `name`; it's just that it's blank. Keep this in mind when developing your views, as there is a difference between no value being set and a blank value being set. Depending on your use case, you might choose to pass the default to `get`.

In this exercise, we retrieved values from the URL in our view using the `GET` attribute of the incoming request. We saw how to set default values and which value is retrieved if multiple

3. 두 개의 이름을 추가해 보세요 – 예를 들어, `http://127.0.0.1:8000/?name=Ben&name=John`. 우리가 언급했듯이, 매개변수의 마지막 값은 `get` 함수로 검색되므로 `Hello, John!`을 볼 수 있어야 합니다:

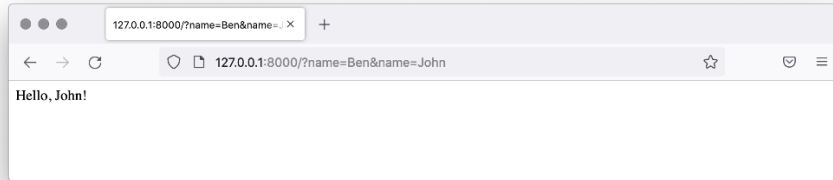


그림 1.24 - URL에 여러 이름 설정하기

4. 이름이 없는 `idx_fd0263e6`을 설정해 보세요, 예를 들어 `idx_ceb3152c`와 같이 – `http://127.0.0.1:8000/?name=`. 페이지는 `Hello, world!`를 다시 표시해야 합니다:

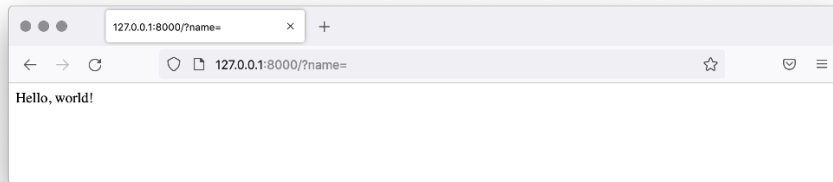


그림 1.25 - URL에 이름이 설정되지 않음

참고

이름을 기본값인 `world`로 설정한 이유가 궁금할 수 있습니다. `'world'`를 기본값으로 `get`에 전달하는 대신 `or`를 사용했습니다. 이름 매개변수에 빈 값을 전달했을 때 6단계에서 무슨 일이 일어났는지 생각해 보세요. 만약 `'world'`를 `get`의 기본값으로 전달했다면, `get` 함수는 여전히 빈 문자열을 반환했을 것입니다. 이는 이름에 값이 설정되어 있지만, 그 값이 비어 있기 때문입니다. 뷰를 개발할 때 이 점을 염두에 두세요. 값이 설정되지 않은 것과 빈 값이 설정된 것 사이에는 차이가 있습니다. 사용 사례에 따라 기본값을 `get`에 전달할 수 있습니다.

이 연습에서는 수신 요청의 `GET` 속성을 사용하여 뷰에서 URL의 값을 가져왔습니다. 기본 값을 설정하는 방법과 여러 값이 있는 경우 어떤 값이 검색되는지 보았습니다.

values are set for the same parameter. The view returned `HttpResponse`, containing the message we wanted to send to the browser.

In the next section, we'll learn about how to update and use Django's settings.

Exploring Django settings

We haven't yet looked at how Django stores its settings. Now that we've seen the different parts of Django, it is a good time to examine the `settings.py` file. There are many settings Django contains that can be used to customize it. A default `settings.py` file was created for you when you started the *bookr* project. We will discuss some of the more important settings in the file now, and a few others that might be useful as you become more fluent with Django. You should open your `settings.py` file in PyCharm and follow along so that you can see where and what the values are for your project.

Each setting in this file is just a file-global variable. The order in which we will discuss the settings is the same order in which they appear in this file, although we may skip over some – for example, there is the `ALLOWED_HOSTS` setting between `DEBUG` and `INSTALLED_APPS`, which we won't cover in this part of the book (you'll see it in *Chapter 18*):

```
SECRET_KEY = '...'
```

This is an automatically generated value that shouldn't be shared with anyone. It is used for hashing, tokens, and other cryptographic functions. If you had existing sessions in a cookie and changed this value, the sessions would no longer be valid:

```
DEBUG = True
```

With this value set to `True`, Django will automatically display exceptions to the browser to allow you to debug any problems you encounter. It should be set to `False` when deploying your app to production:

```
INSTALLED_APPS = [...]
```

When you write your own Django apps (such as the reviews app) or install third-party applications (which will be covered in *Chapter 15*), they should be added to this list. As we've seen, it is not strictly necessary to add them here (our index view worked without our reviews

같은 매개변수에 대한 값이 설정됩니다. 반환된 뷰는 브라우저에 보내고자 하는 메시지를 포함하는 `HttpResponse`입니다.

다음 섹션에서는 Django의 설정을 업데이트하고 사용하는 방법에 대해 배울 것입니다.

Django 설정 탐색하기

Djangoidx_5404d122가 설정을 어떻게 저장하는지 아직 살펴보지 않았습니다. Django의 다양한 부분을 살펴보았으니, 이제 `settings.py` 파일을 살펴볼 좋은 시점입니다. Django에는 사용자 정의에 사용할 수 있는 많은 설정이 포함되어 있습니다. 책 프로젝트를 시작할 때 기본 `settings.py` 파일이 생성되었습니다. 이제 파일에서 더 중요한 설정 몇 가지와 Django에 익숙해짐에 따라 유용할 수 있는 몇 가지 다른 설정에 대해 논의하겠습니다. PyCharm에서 `settings.py` 파일을 열고 따라가면서 프로젝트의 값이 어디에 있는지 확인해야 합니다.

이 파일의 각 설정은 단순한 파일 전역 변수입니다. 설정을 논의할 순서는 이 파일에 나타나는 순서와 동일하지만, 일부는 건너뛸 수 있습니다. 예를 들어, `DEBUG`와 `INSTALLED_APPS` 사이에 `ALLOWED_HOSTS` 설정이 있지만, 이 책의 이 부분에서는 다루지 않을 것입니다(18장에서 볼 수 있습니다):

```
SECRET_KEY = '...'
```

이것은 자동으로 생성된 값으로, 누구와도 공유해서는 안 됩니다. 해싱, 토큰 및 기타 암호화 기능에 사용됩니다. 쿠키에 기존 세션이 있고 이 값을 변경하면 세션이 더 이상 유효하지 않게 됩니다:

```
DEBUG = True
```

이 값을 `True`로 설정하면 Django는 브라우저에 예외를 자동으로 표시하여 발생하는 문제를 디버깅할 수 있도록 합니다. 프로덕션에 앱을 배포할 때는 `False`로 설정해야 합니다:

```
INSTALLED_APPS = [...]
```

자신의 Django 앱(예: 리뷰 앱)을 작성하거나 제3자 애플리케이션(15장에서 다룰 예정)을 설치할 때 이 목록에 추가해야 합니다. 우리가 보았듯이, 여기에 추가하는 것이 엄격히 필요하지는 않습니다(우리의 인덱스 뷰는 리뷰 없이도 작동했습니다).

app being in this list). However, for Django to be able to automatically find the app's templates, static files, migrations, and other configurations, it must be listed here:

```
ROOT_URLCONF = 'bookr.urls'
```

This is the Python module that Django will first load to find URLs. Note that it is the file we added our index view URL map to previously:

```
TEMPLATES = [...]
```

Right now, it's not too important to understand everything in this setting, as you won't be changing it; the important line to point out is this one:

```
'APP_DIRS': True,
```

This tells Django that it should look in a `templates` directory inside each `INSTALLED_APP` when loading a template to render. We don't have a `templates` directory for `reviews` yet, but we will add one in the next exercise.

Django has more settings available that aren't listed in the `settings.py` file, so it will use its built-in defaults in these cases. You can also use the file to set arbitrary settings that you make up for your application. Third-party applications might want settings to be added here as well. In later chapters, we will add settings here for other applications. You can find a list of all settings, and their defaults, at <https://docs.djangoproject.com/en/5.1/ref/settings/>.

Referring to Django settings in your code

It can sometimes be useful to refer to settings from `settings.py` in your own code, whether they are Django's built-in settings or ones you have defined yourself. You might be tempted to write code such as these import settings:

```
from bookr import settings

# check if running in DEBUG mode
if settings.DEBUG:
    do_some_logging()
```

This method is incorrect, for several reasons:

- It is possible to run Django and specify a different settings file to read from, in which case the previous code would cause an error, as it would not be able to find that file.

이 목록에 앱이 있어야 합니다). 그러나 장고가 앱의 템플릿, 정적 파일, 마이그레이션 및 기타 구성을 자동으로 찾으려면 여기에 나열되어 있어야 합니다:

```
ROOT_URLCONF = 'bookr.urls'
```

이것은 장고가 URL을 찾기 위해 처음 로드할 파이썬 모듈입니다. 이전에 인덱스 뷰 URL 맵을 추가한 파일이라는 점에 유의하세요:

```
TEMPLATES = [...]
```

지금은 이 설정의 모든 것을 이해하는 것이 그리 중요하지 않습니다. 왜냐하면 변경하지 않을 것이기 때문입니다. 강조할 중요한 줄은 다음과 같습니다:

```
'APP_DIRS': True,
```

이것은 idx_9ff62bccDjango에게 템플릿을 렌더링하기 위해 템플릿 디렉토리를 각 INSTALLED_APP 내에서 찾아야 한다고 알려줍니다. 리뷰를 위한 템플릿 디렉토리는 아직 없지만, 다음 연습에서 추가할 것입니다.

장고에는 settings.py 파일에 나열되지 않은 더 많은 설정이 있으며, 이러한 경우 기본값을 사용합니다. 또한 파일을 사용하여 애플리케이션을 위해 만든 임의의 설정을 설정할 수 있습니다. 서드파티 애플리케이션도 여기에 설정이 추가되기를 원할 수 있습니다. 이후 장에서는 다른 애플리케이션을 위해 여기에 설정을 추가할 것입니다. 모든 설정과 그 기본값 목록은 <https://docs.djangoproject.com/en/5.1/ref/settings/>에서 확인할 수 있습니다.

코드에서 Django 설정 참조하기

때때로 settings.py의 설정을 자신의 코드에서 참조하는 것이 유용할 수 있습니다. 그것이 Django의 내장 설정이든, 당신이 직접 정의한 것이든 상관없이 말이죠. 다음과 같은 코드를 작성하고 싶을 수도 있습니다:

```
from bookr import settings

# DEBUG 모드에서 실행 중인지 확인
if settings.DEBUG:
    do_some_logging()
```

이 방법은 여러 가지 이유로 잘못되었습니다:

- Django를 실행하고 읽을 다른 설정 파일을 지정할 수 있으며, 이 경우 이전 코드는 해당 파일을 찾을 수 없기 때문에 오류를 발생시킬 것입니다.

Alternatively, if the file exists, the import would succeed but contain the wrong settings.

- Django has settings that might not be listed in the `settings.py` file, and if they aren't, it will use its own internal default. For example, if you removed the `DEBUG = True` line from your `settings.py` file, Django would fall back to using its internal value for `DEBUG` (which is `False`). You would get an error if you tried to access it using `settings.DEBUG` directly though.
- Third-party libraries can change how your settings are defined, so your `settings.py` file would look completely different. None of the expected variables may exist at all. The behavior of all these applications is beyond the scope of this book, but it is something to be aware of.

The preferred way is to use the `django.conf` module instead, like this:

```
# import settings from here instead
from django.conf import settings

if settings.DEBUG:
    do_some_logging()
```

When importing settings from `django.conf`, Django mitigates the three issues we just discussed:

- Settings are read from whatever Django settings file has been specified
- Any default settings values are interpolated
- Django takes care of parsing any settings defined by a third-party library

In our new, shorter example code snippet, even if `DEBUG` is missing from the `settings.py` file, it will fall back to the default value that Django has internally (which is `False`). The same is true for all other settings that Django defines; however, if you define your own custom settings in this file, Django will not have internal values for them, so in your code, you should have some provision for them not existing – how your code behaves is your choice and beyond the scope of this book.

Next, we'll see how Django locates template files.

대안적으로, 파일이 존재하면 가져오기가 성공하지만 잘못된 설정을 포함할 것입니다.

- Django에는 settings.py 파일에 나열되지 않을 수 있는 설정이 있으며, 나열되지 않은 경우 자체 내부 기본값을 사용합니다. 예를 들어, settings.py 파일에서 `DEBUG = True` 줄을 제거하면 Django는 DEBUG에 대한 내부 값을 사용하게 됩니다(이는 False입니다). 그러나 settings.DEBUG를 직접 사용하여 접근하려고 하면 오류가 발생합니다.
- 타사 라이브러리는 설정이 정의되는 방식을 변경할 수 있으므로 settings.py 파일은 완전히 다르게 보일 수 있습니다. 예상되는 변수는 전혀 존재하지 않을 수 있습니다. 이러한 모든 애플리케이션의 동작은 이 책의 범위를 넘어가지만, 인지해야 할 사항입니다.

선회되는 방법은 대신 django.conf 모듈을 사용하는 것입니다. 다음과 같이:

```
# 여기에서 설정을 가져옵니다.
from django.conf import settings

if settings.DEBUG:
    do_some_logging()
```

Django는 django.conf에서 설정을 가져올 때 우리가 방금 논의한 세 가지 문제를 완화합니다:

- 설정은 지정된 Django 설정 파일에서 읽히지며, 모든 기본 설정 값은 보관됩니다.
- Django는 서드파티 라이브러리에 의해 정의된 설정을 파싱하는 것을 처리합니다. 우리의 새로운 짧은 예제 `idx_da85a9ab` 코드 스니펫에서는, settings.py 파일에 DEBUG가 없더라도 Django가 내부적으로 가지고 있는 기본값(즉, False)으로 대체됩니다. Django가 정의하는 모든 다른 설정에도 동일하게 적용됩니다. 그러나 이 파일에 사용자 정의 설정을 정의하면 Django는 그에 대한 내부 값을 가지지 않으므로, 코드에서 존재하지 않을 경우에 대한 조치를 취해야 합니다. 코드의 동작 방식은 여러분의 선택이며 이 책의 범위를 넘어섭니다.

다음으로, Django가 템플릿 파일을 찾는 방법을 살펴보겠습니다.

Finding HTML templates in app directories

There are many options that are available to tell Django how to find templates, which can be set in the `TEMPLATES` setting of `settings.py`, but the easiest one (for now) is to create a `templates` directory inside the `reviews` directory. Django will look in this (and in other apps' `templates` directories) because of `APP_DIRS` being `True` in the `settings.py` file, as we saw in the previous section. However, for Django to know that the `reviews` directory is an app, we need to configure it in the settings. We'll do that in the next exercise.

Exercise 1.05 – creating a templates directory and a base template

In this exercise, you will create a `templates` directory for the `reviews` app. Then, you will add an HTML template file that Django will be able to render to an HTTP response.

We discussed `settings.py` and its `INSTALLED_APPS` setting in the *Exploring Django settings* section. We need to add the `reviews` app to `INSTALLED_APPS` for Django to be able to find templates. Follow these steps:

1. Open `settings.py` in PyCharm. Update the `INSTALLED_APPS` setting and add `reviews` to the end. It should look like this:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'reviews',  
]
```

2. Save and close `settings.py`.
3. In the PyCharm **Project** browser, right-click the `reviews` directory and select **New | Directory**.

앱 디렉토리에서 HTML 템플릿 찾기

장고가 템플릿을 찾는 방법을 알려주는 여러 `idx_db31c958` 옵션이 있으며, 이는 `settings.py`의 `TEMPLATES` 설정에서 설정할 수 있지만, 지금 당장 가장 쉬운 방법은 `reviews` 디렉토리 안에 `templates` 디렉토리를 만드는 것입니다. 장고는 `APP_DIRS`가 `settings.py` 파일에서 `True`로 설정되어 있기 때문에 이 디렉토리(및 다른 앱의 `templates` 디렉토리)에서 찾을 것입니다. 이전 섹션에서 보았듯이, 그러나 장고가 `reviews` 디렉토리가 앱을 알기 위해서는 설정에서 이를 구성해야 합니다. 다음 연습에서 이를 수행할 것입니다.

연습 1.05 – templates 디렉토리 및 기본 템플릿 만들기

이 연습에서는 `reviews` 앱을 위한 `templates` 디렉토리를 생성할 것입니다. 그런 다음, 장고가 HTTP 응답으로 렌더링할 수 있는 HTML 템플릿 파일을 추가할 것입니다.

우리는 Exploring Django settings 섹션에서 `settings.py`와 그 `INSTALLED_APPS` 설정에 대해 논의했습니다. 장고가 템플릿을 찾을 수 있도록 `reviews` 앱을 `INSTALLED_APPS`에 추가해야 합니다. 다음 단계를 따르세요:

1. PyCharm에서 `settings.py`를 엽니다. `INSTALLED_APPS` 설정을 업데이트하고 `reviews`를 끝에 추가합니다. 다음과 같아야 합니다:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'reviews',  
]
```

2. `settings.py`를 저장하고 닫습니다.
3. PyCharm 프로젝트 브라우저에서 `reviews` 디렉토리를 마우스 오른쪽 버튼으로 클릭하고 새로 만들기 | 디렉토리를 선택합니다.

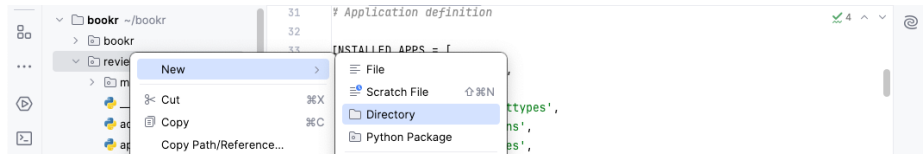


Figure 1.26 – Creating a new directory inside the reviews directory

4. Enter the name `templates` and press *Enter/Return* to create it.
5. Right-click the newly created `templates` directory and select **New | HTML File**.

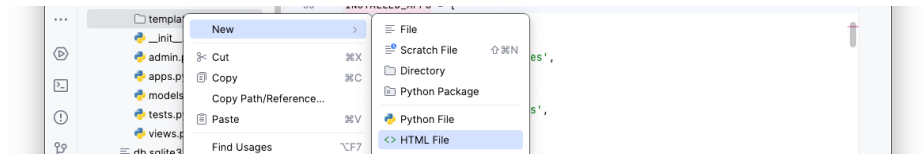


Figure 1.27 – Creating a new HTML file in the templates directory

6. In the window that appears, enter the name `base.html` and then press *Enter/Return* to create the file:
7. After PyCharm creates the file, it automatically opens it too. It will have this content:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>

</body>
</html>
```

8. Between the `<body>` and `</body>` tags, add a short message to validate that the template is being rendered: `Hello from a template!`
9. Your full code should look like this:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

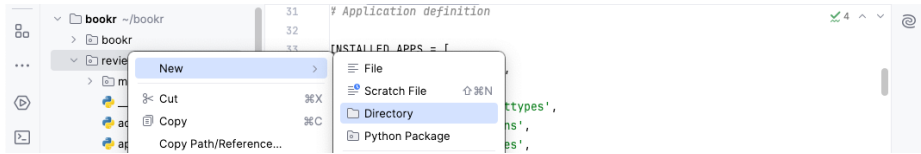


그림 1.26 - 리뷰 디렉토리 안에 새 디렉토리 만들기

4. 이름 templates를 입력하고 Enter/Return 키를 눌러 생성합니다.
5. 새로 생성된 templates 디렉토리를 마우스 오른쪽 버튼으로 클릭하고 새로 만들기 | HTML 파일을 선택합니다.

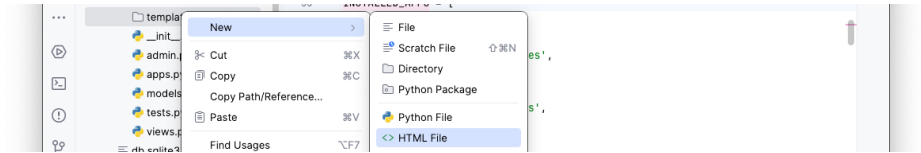


그림 1.27 - 템플릿 디렉토리에 새 HTML 파일 만들기

6. 나타나는 창에 base.html이라는 이름을 입력한 후 Enter/Return 키를 눌러 파일을 생성합니다:
7. idx_80772263PyCharm이 파일을 생성한 후 자동으로 열립니다. 그것은 idx_cc4cb9fb 이 내용을 가질 것입니다:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>제목</title>
</head>
<body>

</body>
</html>
```

8. <body>와 </body> 태그 사이에 템플릿이 렌더링되고 있음을 확인하는 짧은 메시지를 추가하세요: 템플릿에서 안녕하세요!
9. 전체 코드는 다음과 같아야 합니다:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
<title>Title</title>
</head>
<body>
    Hello from a template!
</body>
</html>
```

In this exercise, we created a `templates` directory for the `reviews` app and added an HTML template to it. The HTML template will be rendered once we implement the use of the `render` function on our view.

Rendering a template with the `render` function

We now have a template to use, but we need to update our `index` view so that it renders the template, instead of returning the *Hello (name)!* text that it is currently displaying (refer to *Figure 1.30* to see how it currently looks). We will do this by using the `render` function and providing the name of the template. `render` is a shortcut function that returns `HttpResponse`. There are other ways to render a template that provide more control over how it is rendered, but for now, this function is fine for our needs. `render` takes at least two arguments – the first is always the request that was passed to the view, and the second is the name/relative path of the template being rendered. We will also call it with a third argument, the render context that contains all the variables that will be available in the template – there'll be more on this in *Exercise 1.07 – using variables in templates*.

Exercise 1.06 – rendering a template in a view

In this exercise, you will update your `index` view function to render the HTML template you created in *Exercise 1.05 – creating a templates directory and a base template*. You will make use of the `render` function, which loads your template from disk, renders it, and sends it to the browser. This will replace the static text you are currently returning from the `index` view function:

1. In PyCharm, open `views.py` in the `reviews` directory.
2. We no longer manually create `HttpResponse`, so remove the `HttpResponse` import line:

```
from django.http import HttpResponse
```

3. Replace it with an import of the `render` function from `django.shortcuts`:

```
from django.shortcuts import render
```

```
<title>제목</title>
</head>
<body>
    템플릿에서 인사드립니다!
</body>
</html>
```

이번 연습에서는 reviews 앱을 위한 templates 디렉토리를 만들고 그 안에 idx_7a55a60dHTML 템플릿을 추가했습니다. HTML 템플릿은 뷰에서 render 함수를 사용하도록 구현하면 렌더링됩니다.

render 함수로 템플릿 렌더링하기

이제 사용할 템플릿이 있지만, 현재 표시되고 있는 Hello (name)! 텍스트 대신 템플릿을 렌더링하도록 index 뷰 idx_b5d88293를 업데이트해야 합니다(현재 모습은 Figure 1.30을 참조하세요). 우리는 render 함수를 사용하고 템플릿의 이름을 제공하여 이를 수행할 것입니다. render는 HttpResponse를 반환하는 단축키 함수입니다. 렌더링되는 방식을 더 많이 제어할 수 있는 다른 방법도 있지만, 지금은 이 함수가 우리의 필요에 적합합니다. render는 최소한 두 개의 인수를 필요로 합니다 - 첫 번째는 항상 뷰에 전달된 요청이며, 두 번째는 렌더링되는 템플릿의 이름/상대 경로입니다. 우리는 또한 템플릿에서 사용할 수 있는 모든 변수를 포함하는 렌더링 컨텍스트라는 세 번째 인수로 호출할 것입니다 - 템플릿에서 변수를 사용하는 것에 대한 내용은 연습 1.07에서 더 다룰 것입니다.

연습 1.06 – 뷰에서 템플릿 렌더링하기

이 연습에서는 인덱스 뷰 함수를 업데이트하여 연습 1.05에서 생성한 HTML 템플릿을 렌더링합니다 - 템플릿 디렉토리 및 기본 템플릿 만들기. 디스크에서 템플릿을 로드하고 렌더링하여 브라우저로 전송하는 render 함수를 사용할 것입니다. 이는 현재 인덱스 뷰 함수에서 반환하고 있는 정적 텍스트를 대체할 것입니다:

1. PyCharm에서 reviews 디렉토리의 views.py를 엽니다.
2. 이제 더 이상 HttpResponse를 수동으로 생성하지 않으므로 HttpResponse 가져오기 줄을 제거합니다:

```
from django.http import HttpResponse
```

3. django.shortcuts에서 render 함수를 가져오는 것으로 교체합니다:

```
from django.shortcuts import render
```

4. Update the `index` function so that, instead of returning `HttpResponse`, it's returning a call to `render`, passing in `request` and the template name.
5. The `views.py` file now contains this code:

```
from django.shortcuts import render

def index(request):
    return render(request, "base.html")
```

6. Start the development server if it's not already running. Then, open your web browser and refresh `http://127.0.0.1:8000`. You should see the **Hello from a template!** message rendered, as shown in *Figure 1.38*:

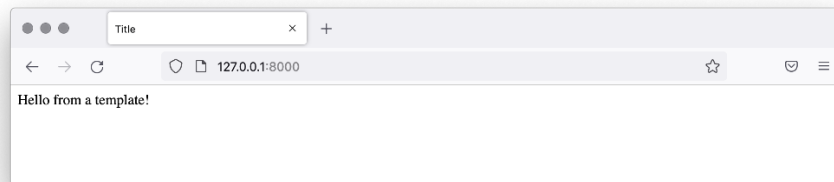


Figure 1.28 – Your first rendered HTML template

Rendering variables in templates

Templates aren't just static HTML. Most of the time, they will contain variables that are interpolated as part of the rendering process. They are passed from a view to a template using a `context` – a dictionary (or dictionary-like object) that contains names of all the variables a template can use. Let's take `Bookr` again as an example. Without variables in your template, you would need a different HTML file for each book you wanted to display. Instead, we use a variable such as `book_name` inside the template, and then the view provides the template with a `book_name` variable, set to the title of the book model it has loaded. When displaying a different book, the HTML does not need to change; the view just passes a different book to it. You can see how models, views, and templates are all now coming together.

Unlike some other languages, such as PHP, variables must be explicitly passed to a template, and variables in a view aren't automatically available to the template. This is for security reasons, as well as to avoid accidentally polluting the template's namespace (we don't want any unexpected variables in the template).

4. HttpResponse를 반환하는 대신 요청과 템플릿 이름을 전달하여 render를 호출하도록 인덱스 함수를 업데이트하세요.
5. views.py 파일에는 이제 다음 코드가 포함되어 있습니다:

```
from django.shortcuts import render

def index(request):
    return render(request, "base.html")
```

6. 개발 서버가 이미 실행 중이지 않다면 시작하세요. 그런 다음 웹 브라우저를 열고 `http://127.0.0.1:8000`을 새로 고치세요. 템플릿에서 Hello! 메시지가 렌더링된 것을 볼 수 있어야 합니다. 그림 1.38과 같이 표시됩니다:

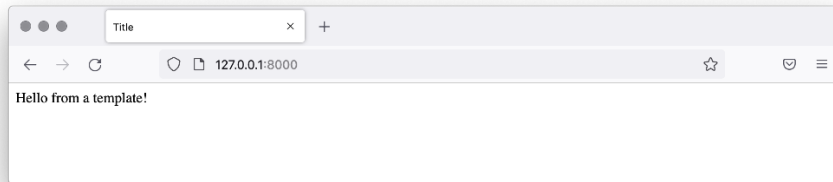


그림 1.28 - 첫 번째 렌더링된 HTML 템플릿

템플릿에서 변수 렌더링

템플릿은 단순한 정적 HTML이 아닙니다. 대부분의 경우, 렌더링 과정의 일환으로 보간되는 변수를 포함합니다. 이들은 뷰에서 템플릿으로 전달되며, 컨텍스트라는 사전(또는 사전과 유사한 객체)을 사용하여 템플릿이 사용할 수 있는 모든 변수의 이름을 포함합니다. Bookr를 다시 예로 들어 보겠습니다. 템플릿에 변수가 없다면, 표시하고자 하는 각 책마다 다른 HTML 파일이 필요할 것입니다. 대신, 템플릿 내에서 `book_name`과 같은 변수를 사용하고, 뷰는 로드한 책 모델의 제목으로 설정된 `book_name` 변수를 템플릿에 제공합니다. 다른 책을 표시할 때 HTML은 변경할 필요가 없으며, 뷰는 단지 다른 책을 전달합니다. 모델, 뷰 및 템플릿이 모두 함께 작동하는 방식을 볼 수 있습니다.

PHP와 같은 다른 언어와 달리, 변수는 템플릿에 명시적으로 전달되어야 하며, 뷰의 변수는 템플릿에서 자동으로 사용할 수 없습니다. 이는 보안상의 이유와 템플릿의 네임스페이스를 우연히 오염시키지 않기 위해서입니다(템플릿에 예상치 못한 변수가 있는 것을 원하지 않습니다).

Inside a template, variables are denoted by double braces – `{{ }}`. While not strictly a standard, this style is quite common and used in other templating tools, such as Vue.js and Mustache. Symfony (a PHP framework) also uses double braces in its Twig templating language, so you might have seen them used similarly there.

To render a variable in a template, simply wrap it with braces – `{{ book_name }}`. Django will automatically escape HTML in output so that you can include special characters (such as `<` or `>`) in your variable without worrying about it garbling your output. If a variable is not passed to a template, Django will simply render nothing at that location, instead of throwing an exception.

There are many more ways to render a variable differently using filters, but these will be covered in *Chapter 3*.

Exercise 1.07 – using variables in templates

We'll put a simple variable inside the `base.html` file to demonstrate how Django's variable interpolation works:

1. In PyCharm, open the `base.html` file.
2. Update the `<body>` element so that it contains a place to render the `name` variable:

```
<body>  
Hello, {{ name }}!  
</body>
```

3. Go back to your web browser and refresh (you should still be at `http://127.0.0.1:8000`). You will see that the page now displays **Hello, !**. This is because we have not set the `name` variable in the rendering context.

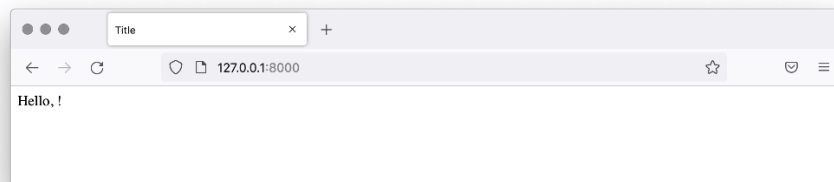


Figure 1.29 – No value is rendered in the template because no context was set

템플릿 내부에서 변수는 이중 중괄호로 표시됩니다 – {{ }}. 엄격하게 표준은 아니지만, 이 스타일은 꽤 일반적이며 Vue.js 및 Mustache와 같은 다른 템플릿 도구에서도 사용됩니다. Symfony (PHP 프레임워크) 또한 Twig 템플릿 언어에서 이중 중괄호를 사용하므로, 비슷하게 사용되는 것을 보았을 것입니다.

템플릿에서 변수를 렌더링하려면, 단순히 중괄호로 감싸면 됩니다 – {{ book_name }}. 장고는 출력에서 HTML을 자동으로 이스케이프하여 < 또는 >와 같은 특수 문자를 변수에 포함할 수 있도록 하여 출력이 손상되는 것에 대해 걱정할 필요가 없습니다. 변수가 템플릿에 전달되지 않으면, Djangoidx_e6c50d77는 해당 위치에서 아무것도 렌더링하지 않고, 대신 idx_c0859b0e 예외를 발생시키지 않습니다.

변수를 다르게 렌더링하는 방법은 필터를 사용하여 더 많이 있지만, 이는 3장에서 다룰 것입니다.

연습 1.07 – 템플릿에서 변수 사용하기

우리는 Django의 idx_6533760a 변수 보간이 어떻게 작동하는지를 보여주기 위해 base.html 파일에 간단한 idx_1673f88f 변수를 넣을 것입니다:

1. PyCharm에서 base.html 파일을 엽니다.
2. <body> 요소를 업데이트하여 name 변수를 렌더링할 수 있는 공간을 포함하도록 하세요:

```
<body>
...
안녕하세요, {{ name }}!
</body>
```

3. 웹 브라우저로 돌아가서 새로 고침하세요(여전히 http://에 있어야 합니다).
127.0.0.1:8000). 이제 페이지에 Hello, !가 표시되는 것을 볼 수 있습니다. 이는 우리가 하기 때문입니다. 렌더링 컨텍스트에서 name 변수를 설정하지 않았습니니다.

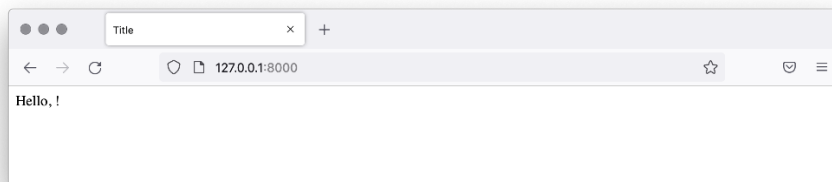


그림 1.29 – 컨텍스트가 설정되지 않아 템플릿에 값이 렌더링되지 않음

4. Open `views.py` and add a variable called `name`, set to the "world" value, inside the `index` function:

```
def index(request):  
    name = "world"  
    return render(request, "base.html")
```

5. Refresh your browser again. Note that nothing has changed – anything we want to render must be explicitly passed to the render function as **context**. This is the dictionary of variables that are made available when rendering.
6. Add the context dictionary as the third argument to the render function. Change your render line to this:

```
return render(request, "base.html", {"name": name})
```

The completed `views.py` file will contain this code:

```
from django.shortcuts import render  
  
def index(request):  
    name = "world"  
    return render(request, "base.html",  
                  {"name": name})
```

7. Refresh your browser again, and you'll see it now says **Hello, world!**.

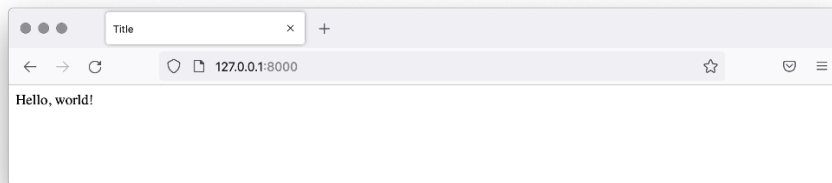


Figure 1.30 – A template rendered with a variable

In this exercise, we combined the template we created in the previous exercise with the render function to render an HTML page, with the `name` variable that was passed to it inside a context dictionary.

4. `views.py`를 열고 `index` 함수 안에 `"world"` 값으로 설정된 `name`이라는 변수를 추가하세요:

```
def index(request):
    name = "world"
    return render(request, "base.html")
```

5. 브라우저를 다시 새로 고치세요. 아무것도 변경되지 않았음을 주목하세요 – 우리가 렌더링하고자 하는 모든 것은 렌더 함수에 명시적으로 전달되어야 합니다. 이것은 렌더링할 때 사용 가능한 변수의 사전입니다.
6. 렌더 함수에 세 번째 인수로 컨텍스트 사전을 추가하세요. `yourrender` 줄을 다음과 같이 변경하세요:

```
return render(request, "base.html", {"name": name})
```

완성된 `views.py` 파일은 다음 코드를 포함할 것입니다:

```
from django.shortcuts import render

def index(request):
    name = "world"
    return render(request, "base.html",
                  {"name": name})
```

7. 브라우저를 다시 새로 고치면 이제 `Hello, world!`라고 표시됩니다.

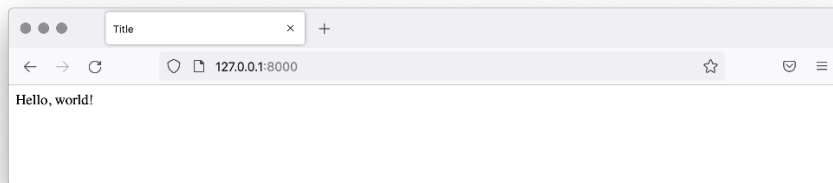


그림 1.30 - 변수를 사용하여 렌더링된 템플릿

이 연습에서는 이전 연습에서 만든 템플릿과 `renderfunction`을 결합하여 HTML 페이지를 렌더링했습니다. 이 페이지에는 컨텍스트 사전 내에 전달된 이름 변수가 포함되어 있습니다.

In the next section, we'll look at some ways you can debug your code and handle errors.

Debugging and dealing with errors

When programming, unless you're the perfect programmer who never makes mistakes, you'll probably have to deal with errors or debug your code at some point. When there is an error in your program, there are usually two ways to tell – either your code will raise an exception, or you will get unexpected output or results when viewing the page. You will probably see exceptions more often, as there are many accidental ways to cause them. If your code is generating unexpected output but not raising any exceptions, you will probably want to use the PyCharm debugger to find out why.

We'll start with an overview of some Python exceptions and how to handle them, along with an exercise that demonstrates how Django shows exceptions. Then, we'll look at running Django inside the PyCharm debugger so that you can peek inside your program while it executes.

Exceptions

If you have worked with Python or other programming languages before, you have probably come across exceptions. If not, here's a quick introduction. Exceptions are *raised* (or *thrown* in other languages) when an error occurs. The execution of a program stops at that point in the code, and the exception travels back up the function call chain until it is caught. If it is not caught, then the program will crash, sometimes with an error message describing the exception and where it occurred. There are exceptions that are raised by Python itself, and your code can raise exceptions to quickly stop execution at any point. Some common exceptions that you might see when programming Python are as follows:

- `IndentationError`: Python will raise this if your code is not correctly indented or has mixed tabs and spaces
- `SyntaxError`: Python raises this error if your code has invalid syntax:

```
>>> a == 1
      File "<stdin>", line 1
        a == 1
          ^
      SyntaxError: invalid syntax
```

다음 섹션에서는 코드를 디버깅하고 오류를 처리하는 몇 가지 방법을 살펴보겠습니다.

디버깅 및 오류 처리

프로그래밍을 할 때, 실수를 전혀 하지 않는 완벽한 프로그래머가 아니라면, 언젠가는 오류를 처리하거나 코드를 디버깅해야 할 것입니다. 프로그램에 오류가 발생하면 일반적으로 두 가지 방법으로 알 수 있습니다. 첫째, 코드에서 예외가 발생하거나, 둘째, 페이지를 볼 때 예상치 못한 출력이나 결과를 얻는 것입니다. 예외는 우연히 발생할 수 있는 여러 가지 방법이 있기 때문에 아마도 예외를 더 자주 보게 될 것입니다. 코드가 예상치 못한 출력을 생성하지만 예외를 발생시키지 않는 경우, PyCharm 디버거를 사용하여 그 이유를 알아내고 싶을 것입니다.

우리는 먼저 몇 가지 파이썬 예외와 이를 처리하는 방법에 대한 개요를 시작하고, 장고가 예외를 어떻게 보여주는지를 보여주는 연습을 진행할 것입니다. 그런 다음, PyCharm 디버거 내에서 장고를 실행하여 프로그램이 실행되는 동안 내부를 들여다볼 수 있도록 하겠습니다.

예외

파이썬이나 다른 프로그래밍 언어로 작업해본 적이 있다면, 아마도 예외를 접해보았을 것입니다. 그렇지 않다면, 간단한 소개를 하겠습니다. 예외는 오류가 발생할 때 발생(또는 다른 언어에서는 던져짐)합니다. 프로그램의 실행은 코드의 그 지점에서 중단되며, 예외는 함수 호출 체인을 따라 위로 올라가다가 잡힐 때까지 이동합니다. 잡히지 않으면 프로그램이 충돌하며, 때때로 예외와 발생한 위치를 설명하는 오류 메시지와 함께 충돌합니다. 파이썬 자체에서 발생하는 예외가 있으며, 코드에서 언제든지 실행을 빠르게 중단하기 위해 예외를 발생시킬 수 있습니다. 파이썬 프로그래밍 중에 볼 수 있는 몇 가지 일반적인 예외는 다음과 같습니다:

- `IndentationError: Pythonidx_5b3542b0`는 코드가 올바르게 들여쓰기되지 않았거나 탭과 공백이 혼합된 경우 이 오류를 발생시킵니다.
- `SyntaxError: Python idx_9463fc94`는 코드에 잘못된 구문이 있는 경우 이 오류를 발생시킵니다:

```
>>> a === 1
      File "<stdin>", line 1
        a === 1
          ^
      SyntaxError: 잘못된 구문
```

- **ImportError:** This is raised when an import fails – for example, if trying to import from a file that does not exist or trying to import a name that is not set in a file:

```
>>> import missing_file
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ImportError: No module named missing_file
```

- **NameError:** This is raised when trying to access a variable that has not yet been set:

```
>>> a = b + 5
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'b' is not defined
```

- **KeyError:** This is raised when accessing a key that is not set in a dict (or a dict-like object):

```
>>> d = {'a': 1}
>>> d['b']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'b'
```

- **IndexError:** This is raised when accessing an index outside the length of a list:

```
>>> l = ['a', 'b']
>>> l[3]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

- **TypeError:** This is raised when trying to perform an operation on an object that does not support it, or when using two objects of the wrong type – for example, trying to add a string to an int:

```
>>> 1 + '1'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

- **ImportError:** 가져오기가 실패할 때 발생하는 `idx_40def445`입니다 – 예를 들어, 존재하지 않는 파일에서 가져오려고 하거나 파일에 설정되지 않은 이름을 가져오려고 할 때 발생합니다:

```
>>> import missing_file
추적 (가장 최근 호출 마지막): 파일 "<stdin>", 1행, <module>에서
ImportError: missing_file이라는 모듈이 없습니다
```

- **NameError:** 설정되지 않은 변수를 접근하려고 할 때 발생하는 `idx_28d8f839`입니다:

```
>>> a = b + 5
Traceback (most recent call last):File
"<stdin>", line 1, in <module>
NameError: name 'b' is not defined
```

- **KeyError:** This is `idx_959814ca` raised when accessing a key that is not set in a dict (or a dict-like object):

```
>>> d = {'a': 1}
>>> d['b']
Traceback (most recent call last):File
"<stdin>", line 1, in <module>
KeyError: 'b'
```

- **IndexError:** This is `idx_f45f8e0e` raised when accessing an index outside the length of a list:

```
>>> l = ['a', 'b']
>>> l[3]
추적 (가장 최근 호출 마지막): 파일 "<stdin>", 1행, <모듈>에서
IndexError: 리스트 인덱스 범위를 벗어났습니다.
```

- **TypeError:** 이는 지원하지 않는 객체에서 작업을 수행하려고 하거나 잘못된 유형의 두 객체를 사용할 때 발생합니다 – 예를 들어, 문자열을 정수에 추가하려고 할 때:

```
>>> 1 + '1'
추적 (가장 최근 호출 마지막): 파일 "<stdin>", 1행, <모듈>에서
TypeError: +에 대한 지원되지 않는 피연산자 유형: 'int'와 'str'
```

Django also raises its own custom exceptions, and you will be introduced to them throughout the book.

When running the Django development server with `DEBUG = True` in your `settings.py` file, Django will automatically capture exceptions that occur in your code (instead of crashing). It will then generate an HTTP response, showing you a stack trace and other information to help you debug the problem. When running in production, `DEBUG` should be set to `False`. Django will then return a standard internal server error page, without any sensitive information. You also have the option to display a custom error page.

Exercise 1.08 – generating and viewing exceptions

Let's create a simple exception in our view so that you are familiar with how Django displays them. In this case, we'll try to use a variable that doesn't exist, which will raise `NameError`:

1. In PyCharm, open `views.py`. In the `index` view function, change the context being sent to the render function so that it's using a variable that doesn't exist. We'll try to send `invalid_name` in the context dictionary, instead of `name`. Don't change the context dictionary key, just its value:

```
return render(request, "base.html",  
              {"name": invalid_name})
```

2. Go back to your browser and refresh the page. You should see a screen like *Figure 1.31*:

장고는 자체 사용자 정의 예외를 발생시키며, 이 책 전반에 걸쳐 그것들을 소개할 것입니다.

settings.py 파일에서 `DEBUG = True`로 장고 개발 서버를 실행하면, 장고는 코드에서 발생하는 예외를 자동으로 캡처합니다(충돌하는 대신). 그러면 HTTP 응답을 생성하여 스택 추적 및 문제를 디버깅하는 데 도움이 되는 기타 정보를 보여줍니다. 프로덕션에서 실행할 때는 `DEBUG`를 `False`로 설정해야 합니다. 그러면 장고는 민감한 정보 없이 표준 내부 서버 오류 페이지를 반환합니다. 사용자 정의 오류 페이지를 표시할 옵션도 있습니다.

연습 1.08 – 예외 생성 및 보기

장고가 예외를 표시하는 방식을 익히기 위해 뷰에서 간단한 예외를 생성해 보겠습니다. 이 경우 존재하지 않는 변수를 사용해 보겠습니다. 그러면 `NameError`가 발생합니다:

1. PyCharm에서 `views.py`를 엽니다. `idx_6dea7614`의 인덱스 뷰 함수에서, 존재하지 않는 변수를 사용하도록 `render` 함수에 전달되는 `context`를 변경합니다. `context` 사전에서 `name` 대신 `invalid_name`을 보내도록 하겠습니다. `context` 사전의 키는 변경하지 말고 값만 변경하세요:

```
return render(request, "base.html",
               {"name": invalid_name})
```

2. 브라우저로 돌아가서 페이지를 새로 고치세요. 그림 1.31과 같은 화면이 표시되어야 합니다:

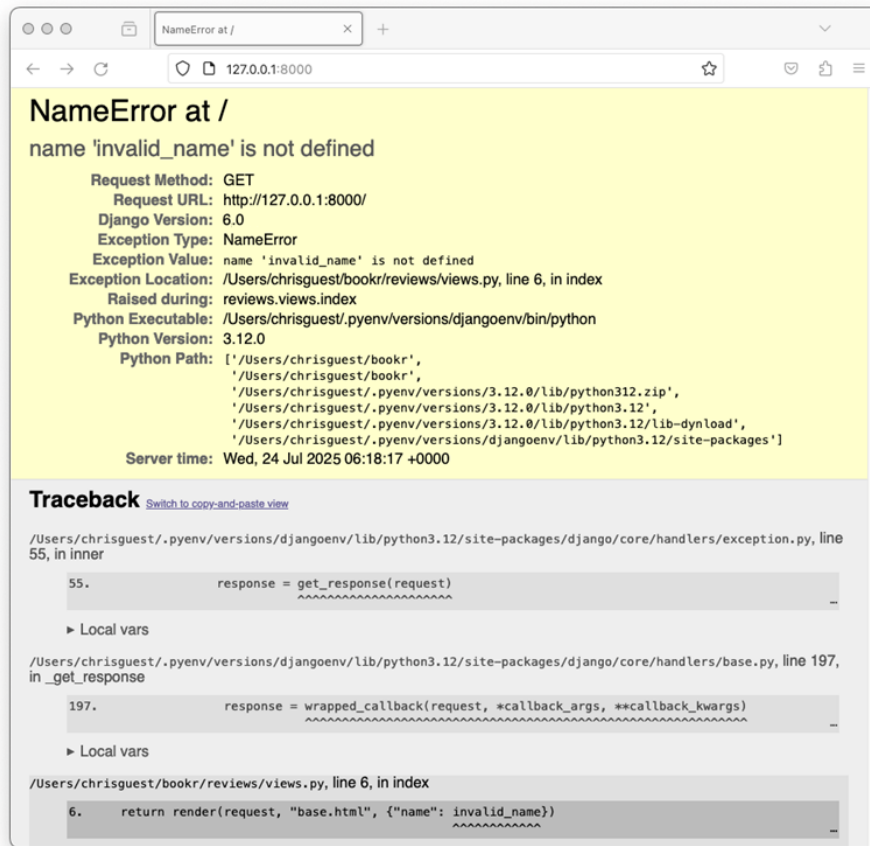


Figure 1.31 – A Django exception screen

The first two header lines on the page inform you of the error that occurred:

```
NameError at /
name 'invalid_name' is not defined
```

- Below the header is a traceback to where the exception occurred. Click on the various lines of code to expand them and see the surrounding code, or click **Local vars** for each frame to expand them and see what the values of the variables are:

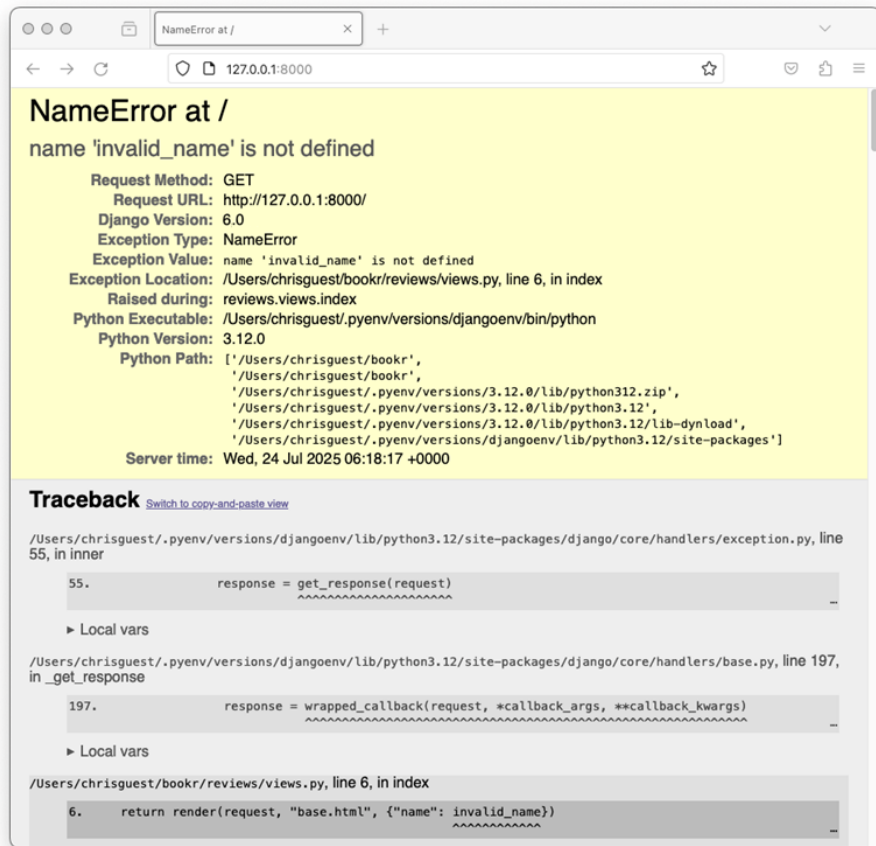


그림 1.31 - 장고 예외 화면

페이지의 첫 번째 idx_f5db7371two 헤더 라인은 발생한 오류에 대해 알려줍니다:

```
/에서 NameError
name 'invalid_name'이 정의되지 않았습니다
```

3. 헤더 아래에는 예외가 발생한 위치에 대한 추적이 있습니다. 다양한 코드 줄을 클릭하여 확장하고 주변 코드를 보거나 각 프레임의 Local vars를 클릭하여 확장하고 변수의 값을 확인하세요:

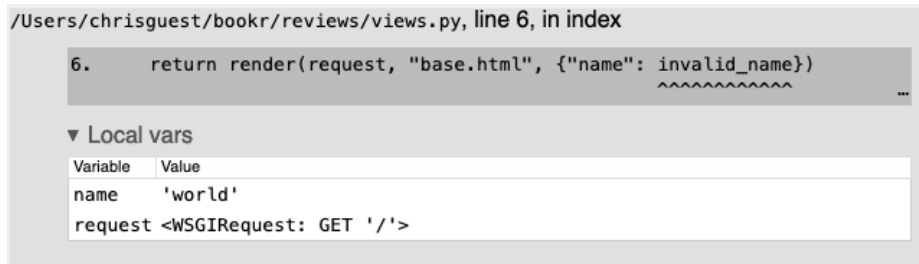


Figure 1.32 – The line that caused the exception

In our case, we can see the exception was raised on line six of our `views.py` file, and by expanding its **Local vars** dropdown, we see that `name` has the value `world`, and the only other variable is the incoming request (Figure 1.43).

4. Go back to `views.py` and fix `NameError` by renaming `invalid_name` back to `name`.
5. Save the file and refresh your browser. **Hello, world!** should display again (as shown in Figure 1.41).

In this exercise, we made our Django code raise an exception (`NameError`) by trying to use a variable that had not been set. We saw that Django automatically sent details of this exception and a stack trace to the browser to help us find the cause of the error. We then reverted our code change to make sure our view worked properly again.

Debugging

When you're trying to find problems in your code, it can help to use a debugger. This is a tool that lets you go through your code line by line, rather than executing it all at once. Each time the debugger is paused on a particular line of code, you can see the values of all the current variables. This is very useful for finding errors in your code that don't raise exceptions.

For example, in `Bookr`, we talked about having a view that fetches a list of books from the database and renders them in an HTML template. If you view the page in the browser, you might see only one book when you expect several. You could have the execution pause inside your view function and see what values were fetched from the database. If your view is only receiving one book from the database, you know there is a problem with your database querying somewhere. If your view is successfully fetching multiple books but only one is being rendered, then it's probably a problem with the template. Debugging helps you narrow down faults like this.

PyCharm has a built-in debugger to make it easy to step through your code and see what is happening on each line. To tell the debugger where to stop the execution of code, you need to

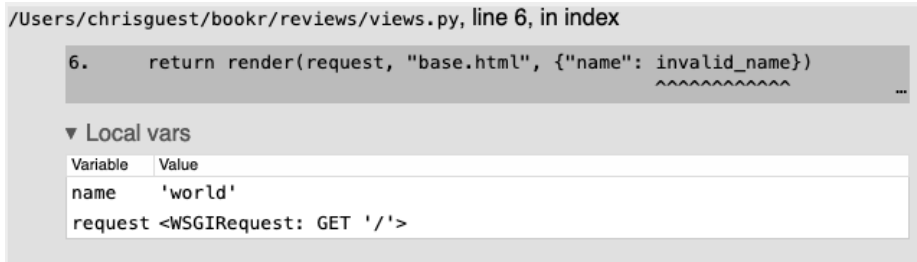


그림 1.32 - 예외를 발생시킨 줄

우리의 경우, `views.py` 파일의 6번째 줄에서 예외가 발생한 것을 볼 수 있으며, Local vars 드롭다운을 확장하면 `name`의 값이 `world`이고 유일한 다른 변수는 들어오는 요청임을 알 수 있습니다 (그림 1.43).

4. `views.py`로 돌아가서 `invalid_name`을 `name`으로 다시 이름을 바꿔 `NameError`를 수정하세요. 파일을 저장하고 브라우저를 새로 고치세요. `Hello, world!`가 다시 표시되어야 합니다 (그림 1.41에 표시된 대로).

이번 연습에서는 설정되지 않은 변수를 사용하려고 시도하여 장고 코드가 예외 (`NameError`)를 발생시키도록 했습니다. 우리는 장고가 이 예외의 세부정보와 스택 추적을 브라우저에 자동으로 전송하여 오류의 원인을 찾는 데 도움을 주는 것을 보았습니다. 그런 다음 우리의 코드 변경을 되돌려서 뷰가 다시 제대로 작동하는지 확인했습니다.

디버깅

코드에서 문제를 찾으려고 할 때, 디버거를 사용하는 것이 도움이 될 수 있습니다. 이것은 코드를 한 줄씩 실행할 수 있게 해주는 도구입니다. 디버거가 특정 코드 줄에서 일시 중지될 때마다 현재 모든 변수의 값을 볼 수 있습니다. 이는 예외를 발생시키지 않는 코드의 오류를 찾는 데 매우 유용합니다.

예를 들어, Bookr에서는 데이터베이스에서 책 목록을 가져와 HTML 템플릿에 렌더링하는 뷰에 대해 이야기했습니다. 브라우저에서 페이지를 보면 여러 권의 책을 기대할 때 한 권의 책만 보일 수 있습니다. 뷰 함수 내부에서 실행을 일시 중지하고 데이터베이스에서 가져온 값이 무엇인지 확인할 수 있습니다. 데이터베이스에서 한 권의 책만 수신하고 있다면, 데이터베이스 쿼리의 어딘가에 문제가 있다는 것을 알 수 있습니다. 뷰가 여러 권의 책을 성공적으로 가져오고 있지만 한 권만 렌더링되고 있다면, 아마도 템플릿에 문제가 있을 것입니다. 디버깅은 이러한 결함을 좁히는 데 도움이 됩니다.

PyCharm에는 코드의 각 줄에서 무슨 일이 일어나고 있는지 쉽게 확인할 수 있도록 내장된 디버거가 있습니다. 디버거에게 코드 실행을 중지할 위치를 알려주려면, 당신은 다음을 해야 합니다.

set a *breakpoint* on one or more lines of code. They are named as such because the execution of code will *break* (stop) at that *point*.

For breakpoints to be activated, PyCharm needs to be set to run your project in its debugger. There is a small performance penalty, but it is usually not noticeable, so you might choose to always run your code inside the debugger so that you can quickly set a breakpoint, without having to stop and restart the Django development server.

Running the Django development server inside the debugger is as simple as clicking the **Debug** icon instead of the **Play** icon (see *Figure 1.20*) to start it.

Exercise 1.09 – debugging your code

In this exercise, you will learn the basics of the PyCharm debugger. You will run the Django development server in the debugger and then set a breakpoint in your view function to pause execution so that you can examine the variables:

1. If the Django development server is running, stop it by clicking the **Stop** button in the top-right corner of the PyCharm window:



Figure 1.33 – The Stop button in the top-right corner of the PyCharm window

2. Start the Django development server again inside the debugger by clicking the **Debug** icon just to the left of the **Stop** button (*Figure 1.44*).
The server will take a few seconds to start, and then you should be able to refresh the page in your browser to make sure it's still loading. You shouldn't notice any changes; all the code is executed the same as before.
3. Now, we can set a breakpoint that will cause execution to stop so that we can see the state of the program. In PyCharm, click just to the right of the line numbers, on line 5, in the gutter on the left of the editor pane. A red circle will appear to indicate that the breakpoint is now active:

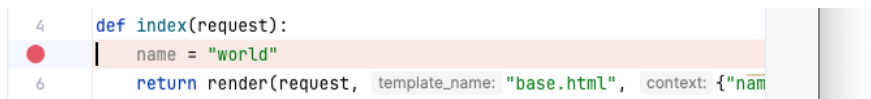


Figure 1.34 – A breakpoint on line 5

4. Go back to your browser and refresh the page. Your browser will not display any content; instead, it will just continue to try to load the page. Depending on your operating system, PyCharm should become active again; if not, bring it to the

하나 이상의 코드 줄에 중단점을 설정합니다. 중단점이라고 불리는 이유는 코드 실행이 그 지점에서 중단되기 때문입니다.

중단점이 활성화되려면 PyCharm이 디버거에서 프로젝트를 실행하도록 설정되어야 합니다. 약간의 성능 저하가 있지만 일반적으로 눈에 띄지 않으므로, Django idx_788904a8 개발 서버를 중단하고 다시 시작하지 않고도 중단점을 빠르게 설정할 수 있도록 항상 디버거 내에서 코드를 실행하도록 선택할 수 있습니다.

디버거 내에서 Django 개발 서버를 실행하는 것은 재생 아이콘 대신 디버그 아이콘을 클릭하는 것만큼 간단합니다(그림 1.20 참조).

연습 1.09 – 코드 디버깅

이 연습에서는 PyCharm 디버거의 기본 사항을 배웁니다. 디버거에서 Django 개발 서버를 실행한 다음, 실행을 일시 중지하여 변수를 검사할 수 있도록 뷰 함수에 중단점을 설정합니다:

1. Django 개발 서버가 실행 중인 경우, PyCharm 창의 오른쪽 상단 모서리에 있는 중지 버튼을 클릭하여 중지합니다:



그림 1.33 - PyCharm 창의 오른쪽 상단 모서리에 있는 중지 버튼

2. 중지 버튼(그림 1.44) 왼쪽에 있는 디버그 아이콘을 클릭하여 디버거 내에서 Django 개발 서버를 다시 시작합니다.
서버가 시작되는 데 몇 초가 걸리며, 그런 다음 브라우저에서 페이지를 새로 고쳐 여전히 로드되고 있는지 확인할 수 있어야 합니다. 변경 사항은 눈에 띄지 않아야 하며, 모든 코드는 이전과 동일하게 실행됩니다.
3. 이제 실행을 중지시켜 프로그램의 상태를 볼 수 있는 중단점을 설정할 수 있습니다. PyCharm에서 편집기 창 왼쪽의 여백에서 5행의 줄 번호 오른쪽을 클릭합니다. 중단점이 활성화되었음을 나타내기 위해 빨간 원이 나타납니다:

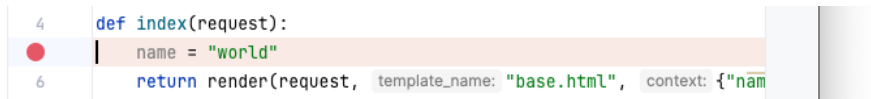


그림 1.34 - 5행의 중단점

4. 브라우저로 돌아가서 페이지를 새로 고칩니다. 브라우저는 어떤 내용도 표시하지 않으며; 대신 페이지를 계속 로드하려고 시도할 것입니다. 운영 체제에 따라 PyCharm이 다시 활성화되어야 하며; 그렇지 않으면 그것을 가져옵니다.

foreground. You should see that line 5 is highlighted, and at the bottom of the window, the debugger is shown. The stack frames (the chain of functions that were called to get to the current line) are on the left, and the current variables of the function are on the right.

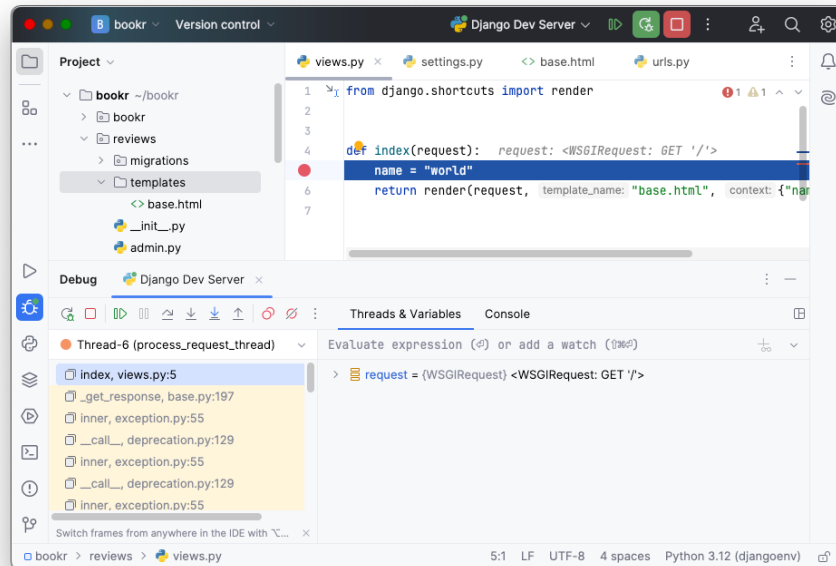


Figure 1.35 – The debugger paused with the current line (5) highlighted

There is currently one variable in scope, `request`. If you click the toggle triangle to the left of its name, you can show or hide the attributes it has set.

전경.5번 줄이 강조 표시되고, 창의 하단에 디버거가 idx.f81c16b6 표시되어야 합니다. 현재 줄에 도달하기 위해 호출된 함수의 체인인 스택 프레임은 왼쪽에 있으며, 함수의 현재 변수는 오른쪽에 있습니다.

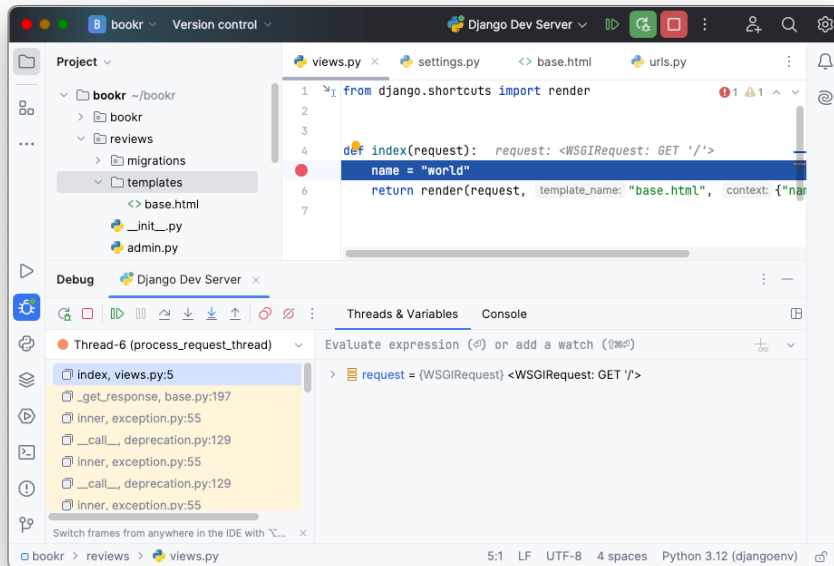


그림 1.35 - 현재 줄(5)이 강조 표시된 상태에서 디버거가 일시 중지됨

현재 범위에 하나의 변수, `request`가 있습니다. 이름 왼쪽의 톱글 삼각형을 클릭하면 설정된 속성을 표시하거나 숨길 수 있습니다.

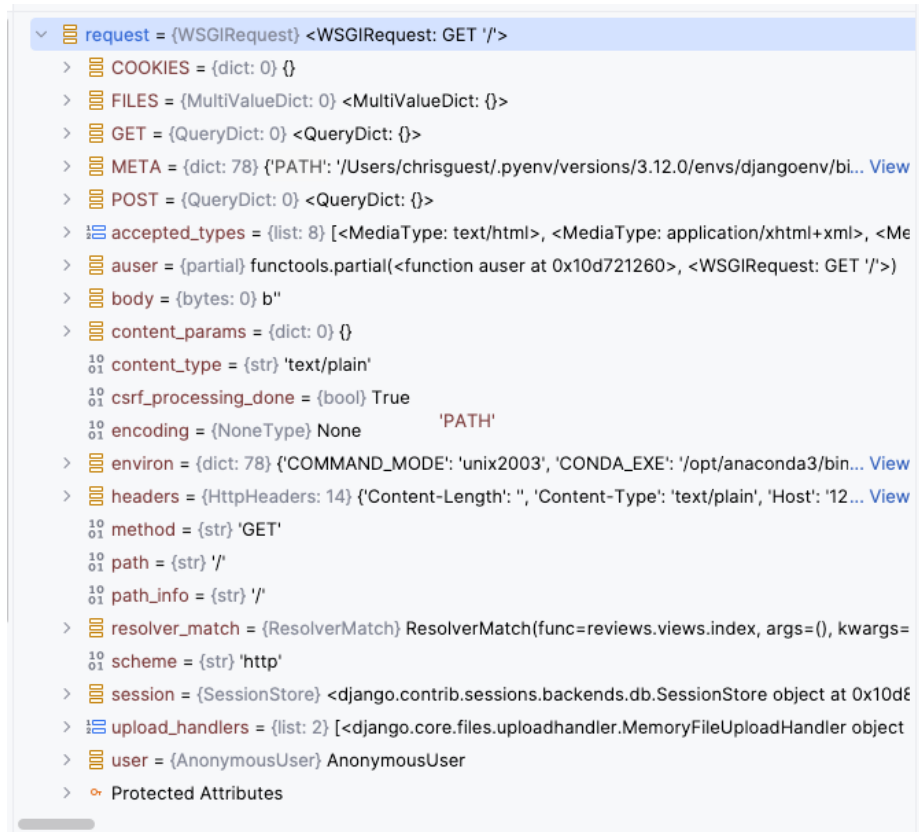


Figure 1.36 – The attributes of the request variable

For example, if you scroll down through the list of attributes, you can see that method is GET and path is /.

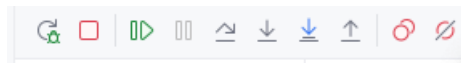


Figure 1.37 – The actions bar

5. The actions bar, shown in *Figure 1.37*, is above the stack frames and variables. Its buttons (from left to right) are as follows:
 - **Rerun:** This stops the program and restarts it.
 - **Stop:** This stops the debugger.
 - **Resume Program:** This continues running until the next breakpoint.
 - **Pause Program:** This button breaks the program at its current execution point.

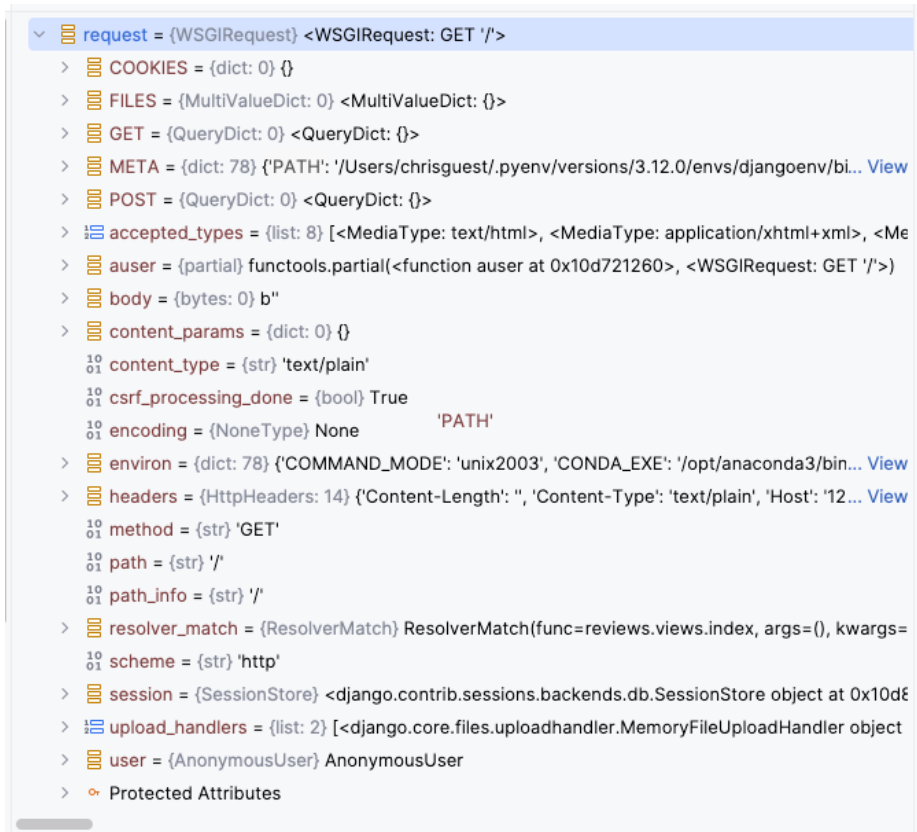


그림 1.36 - 요청 변수의 속성

예를 들어, idx_93eb6857을 통해 속성 목록을 아래로 스크롤하면 method가 GET 이고 path가 /임을 볼 수 있습니다.

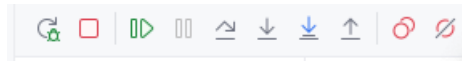


그림 1.37 - 작업 표시줄

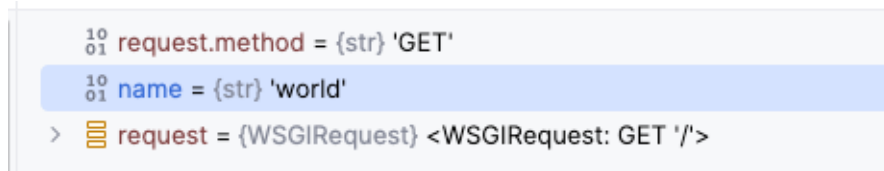
5. 그림 1.37에 표시된 작업 표시줄은 스택 프레임과 변수 위에 있습니다. 왼쪽에서 오른쪽으로 버튼은 다음과 같습니다:

- 다시 실행: 이 작업은 프로그램을 중지하고 다시 시작합니다. 중지: 이 작업은 디버거를 중지합니다.
- 프로그램 재개: 이 작업은 다음 중단점까지 계속 실행됩니다.
- 프로그램 일시 중지: 이 버튼은 현재 실행 지점에서 프로그램을 중단합니다.

- **Step Over:** This executes the current line of code and continues to the next line.
- **Step Into:** This steps into the current line. For example, if the line contained a function, it would continue with the debugger inside this function.
- **Step into My Code:** This steps into the line being executed but continues until it finds the code you have written. For example, if you're stepping into third-party library code that later calls your code, it will not show you the third-party code, instead continuing through until it returns to the code that you have written.
- **Step Out:** This returns from the current code to the function or the method that called it. It is the opposite of the **Step In** action.
- **View Breakpoints:** This opens a window to see all the breakpoints you have set.
- **Mute Breakpoints:** This toggles all breakpoints on or off but does not remove them.

Note that not all buttons are useful all the time. For example, it can be easy to step out of your view and end up confusing Django library code.

6. Click the **Step Over** button once to execute line 5.
7. You can see that the name variable has been added to the list of variables in the debugger view, and its value is world.



```
10 01 request.method = {str} 'GET'
10 01 name = {str} 'world'
> ☰ request = {WSGIRequest} <WSGIRequest: GET '/'>
```

Figure 1.38 – The new variable name is now in scope, with the world value

We are now at the end of our `index` view function, and if we were to step over this line of code, it would jump to Django library code, which we don't want to see. To continue executing and to send the response back to your browser, click the **Resume Program** button, the third from the left on the debugger toolbar (Figure 1.39). You should see that your browser has now loaded the page again:

- Step Over: 현재 코드 줄을 실행하고 다음 줄로 계속 진행합니다. Step Into: 현재 줄로 들어갑니다. 예를 들어, 줄에 함수가 포함되어 있다면, 이 함수 내부에서 디버거가 계속 진행됩니다.
- 내 코드로 들어가기: 실행 중인 줄로 들어가지만, 작성한 코드를 찾을 때까지 계속 진행합니다. 예를 들어, 서드파티 라이브러리 코드로 들어가서 나중에 내 코드를 호출하는 경우, 서드파티 코드는 보여주지 않고, 작성한 코드로 돌아올 때까지 계속 진행합니다. Step Out: 현재 코드에서 호출한 함수나 메서드로 돌아갑니다. 이는 Step In 동작의 반대입니다.
- 중단점 보기: 이 창을 열면 설정한 모든 중단점을 볼 수 있습니다. 중단점 음소거: 이 버튼을 클릭하면 모든 중단점이 켜지거나 꺼지지만 제거되지는 않습니다.

모든 버튼이 항상 유용한 것은 아닙니다. 예를 들어, 뷰에서 벗어나 Django 라이브러리 코드를 혼동할 수 있습니다.

6. 5행을 실행하려면 Step Over 버튼을 한 번 클릭하세요.

7. 이제 name 변수가 디버거 뷰의 변수 목록에 추가되었고, 그 값은 world입니다.



```

10 01 request.method = {str} 'GET'
10 01 name = {str} 'world'
> ☰ request = {WSGIRequest} <WSGIRequest: GET '/'>

```

그림 1.38 - 새로운 변수 name이 이제 범위에 있으며, 값은 world입니다.

이제 우리는 idx_623fee3a 인덱스 뷰 함수의 끝에 있으며, 이 코드 줄을 넘어가면 Django 라이브러리 코드로 점프하게 되므로 보고 싶지 않습니다. 계속 실행하고 응답을 브라우저로 보내려면 디버거 톨바에서 왼쪽에서 세 번째에 있는 Resume Program 버튼을 클릭하세요 (그림 1.39). 이제 브라우저가 페이지를 다시 로드한 것을 볼 수 있어야 합니다:

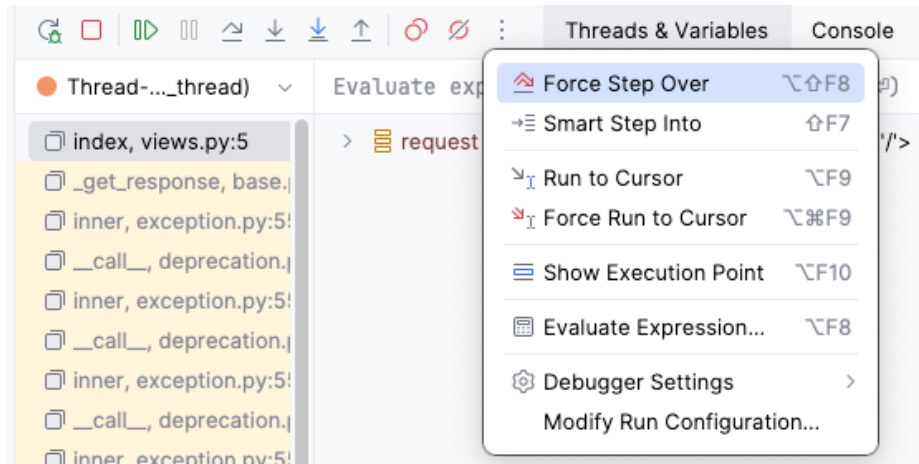


Figure 1.39 – Actions to control execution – the green play icon is the Resume Program button

Clicking on the vertical ellipsis button, `:`, reveals more buttons (see Figure 1.50); from the top, they are **Force Step Over** (steps over the current code line, ignoring any method calls to the next line), **Smart Step Into** (allows cursor selection of a method from the current line of code), **Run to Cursor** (keeps running the code to the current line under the cursor, suspending the program at any breakpoints), **Force Run to Cursor** (keeps running the code to the current line under the cursor, ignoring any breakpoints), **Show Execution Point** (focuses on the execution point in the editor window and the stack frame in the panel), **Evaluate Expression...** (evaluates an expression from the editor window), **Debugger Settings** (menu for configurable settings), and **Modify Run Configuration...** (opens the run configuration window).

8. For now, turn off the breakpoint in PyCharm by clicking on it (the red circle next to line 5).

```

4     def index(request):
5         name = "world"
6         return render(request, template_name="base.html", context={"name":

```

Figure 1.40 – Clicking the breakpoint that was on line 5 disables it

This is just a quick introduction to how to set breakpoints in PyCharm. If you have used debugging features in other IDEs, then you should be familiar with the concepts – you can step through code, step in and out of functions, or evaluate expressions. Once you have set a breakpoint, you can right-click on it to change the options. For example, you can make the breakpoint conditional so that the execution only stops under certain circumstances. All these

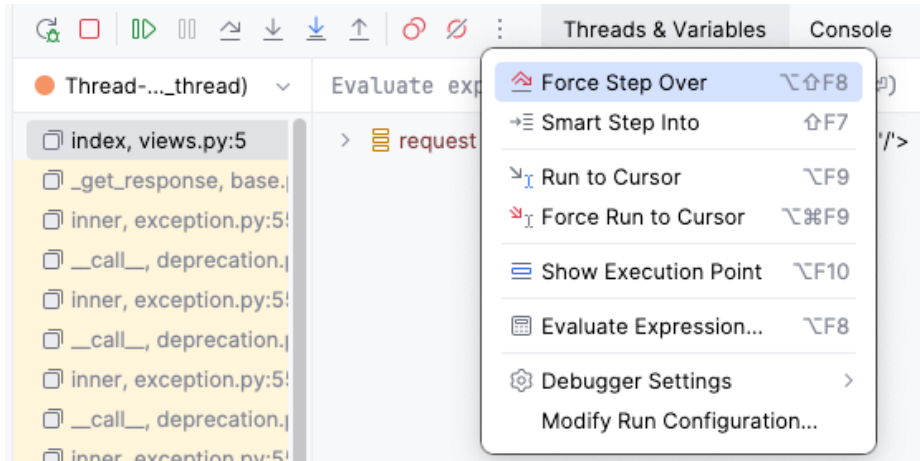


그림 1.39 - 실행 제어 작업 - 녹색 재개 프로그램 아이콘은 프로그램 재개 버튼입니다.

idx_40811387 수직 줄임표 버튼을 클릭하면, ;, 더 많은 버튼이 표시됩니다(그림 1.50 참조); 위에서부터, 강제 단계 건너뛰기(현재 코드 줄을 건너뛰고 다음 줄로의 메서드 호출을 무시함), 스마트 단계 진입(현재 코드 줄에서 메서드를 선택할 수 있음), 커서까지 실행(커서 아래의 현재 줄까지 코드를 계속 실행하며, 프로그램을 모든 중단점에서 일시 중지함), 커서까지 강제 실행(커서 아래의 현재 줄까지 코드를 계속 실행하며, 모든 중단점을 무시함), 실행 지점 표시(편집기 창의 실행 지점과 패널의 스택 프레임에 초점을 맞춤), 표현식 평가...(편집기 창에서 표현식을 평가함), 디버거 설정(구성 가능한 설정을 위한 메뉴), 실행 구성 수정...(실행 구성 창을 엽니다).

8. 지금은 PyCharm에서 5행 옆의 빨간 원을 클릭하여 중단점을 끄세요.

```

4     def index(request):
5         name = "world"
6         return render(request, template_name="base.html", context={"name":

```

그림 1.40 - 5행에 있던 중단점을 클릭하면 비활성화됩니다.

이것은 PyCharm에서 중단점을 설정하는 방법에 대한 간단한 소개입니다. 다른 IDE에서 디버깅 기능을 사용해본 적이 있다면, 개념에 익숙할 것입니다 - 코드를 단계별로 실행하고, 함수 안팎으로 이동하거나, 표현식을 평가할 수 있습니다. 중단점을 설정한 후에는 오른쪽 클릭하여 옵션을 변경할 수 있습니다. 예를 들어, 특정 상황에서만 실행이 중단되도록 중단점을 조건부로 만들 수 있습니다. 이 모든 것들

are beyond the scope of this book, but are useful to know about when trying to solve problems in your code.

We're going to finish the chapter with two activities. First, you'll build a welcome screen for Bookr, and then you'll scaffold the book-searching page.

Activity 1.01 – creating a site welcome screen

The Bookr website that we are building needs to have a splash page that welcomes users and lets them know what site they are on. It will also contain links to other parts of the site, but these will be added in later chapters. For now, you will create a page with a welcome message.

These steps will help you complete the activity:

1. In your `index` view, render the `base.html` template.
2. Update the `base.html` template to contain the welcome message. It should be in both the `<title>` tag in `<head>` and in a new `<h1>` tag in the body.

After completing the activity, you should be able to see something like this:

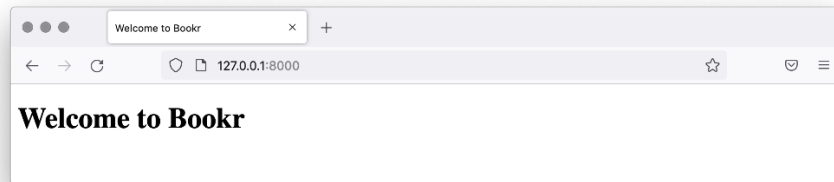


Figure 1.41 – The Bookr welcome page

In the next activity, you will build a basic page to show the results of a book search in Bookr.

Activity 1.02 – a book search scaffold

A useful feature of a site such as Bookr is the ability to search through data to find something on the site quickly. Bookr will implement book searching to allow users to find a particular book by part of its title. While we don't have any books to find yet, we can still implement a page that shows the text a user searched for. The user enters the search string as part of the URL parameters. We will implement searching and a form for easy text entry in *Chapter 6, Forms*.

이 책의 범위를 넘어서는 내용이지만, 코드 문제를 해결할 때 알아두면 유용합니다.

우리는 두 가지 활동으로 장을 마무리할 것입니다. 먼저, Bookr의 환영 화면을 만들고, 그 다음에는 책 검색 페이지를 스캐폴딩할 것입니다.

활동 1.01 – 사이트 환영 화면 만들기

우리가 만들고 있는 Bookr 웹사이트는 사용자에게 환영 메시지를 전하고 그들이 어떤 사이트에 있는지 알려주는 스플래시 페이지가 필요합니다. 또한 사이트의 다른 부분으로 연결되는 링크도 포함될 것이지만, 이는 나중에 추가될 것입니다. 지금은 환영 메시지가 있는 페이지를 생성할 것입니다.

이 단계는 활동을 완료하는 데 도움이 될 것입니다:

1. 인덱스 뷰에서 base.html 템플릿을 렌더링합니다.
2. base.html 템플릿을 업데이트하여 환영 메시지를 포함해야 합니다. 이는 `<head>`의 `<title>` 태그와 본문의 새로운 `<h1>` 태그 모두에 있어야 합니다. 활동을 완료한 후, 다음과 같은 것을 볼 수 있어야 합니다:

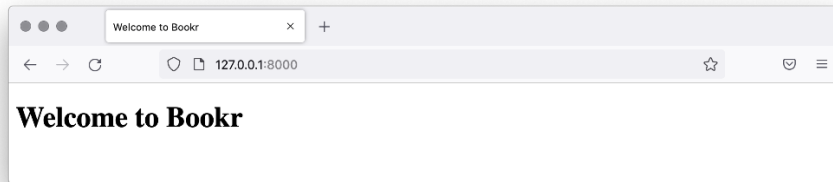


그림 1.41 – Bookr 환영 페이지

다음 활동에서는 Bookr에서 책 검색 결과를 보여주는 기본 페이지를 구축할 것입니다.

활동 1.02 – 책 검색 스캐폴드

Bookr와 같은 사이트의 유용한 기능은 데이터를 통해 사이트에서 무언가를 빠르게 찾을 수 있는 능력입니다. Bookr는 사용자가 제목의 일부로 특정 책을 찾을 수 있도록 책 검색 기능을 구현할 것입니다. 아직 찾을 책이 없지만, 사용자가 검색한 텍스트를 보여주는 페이지를 구현할 수 있습니다. 사용자는 URL 매개변수의 일부로 검색 문자열을 입력합니다. 우리는 6장, 양식에서 검색 및 텍스트 입력을 위한 양식을 구현할 것입니다.

These steps will help you complete the activity:

1. Create a search result HTML template. It should include a variable placeholder to show the search word(s) that were passed in through the render context. Show the passed-in variable in the `<title>` and `<h1>` tags. Use an `<em>` tag around the search text in the body to make it italic.
2. Add a search view function in `views.py`. The view should read a search string from the URL parameters (in the request's GET attribute). It should then render the template you created in the previous step, passing in the search value to be substituted, using the context dictionary.
3. Add a URL mapping to your new view to `urls.py`. The URL can be something such as `/book-search`.

After completing this activity, you should be able to pass in a search value through the URL's parameters and see it rendered on the resulting page. It should look like this:



Figure 1.42 – Searching Django workshop

You should also be able to pass in special HTML characters such as `<` and `>` to see how Django automatically escapes them in the template.

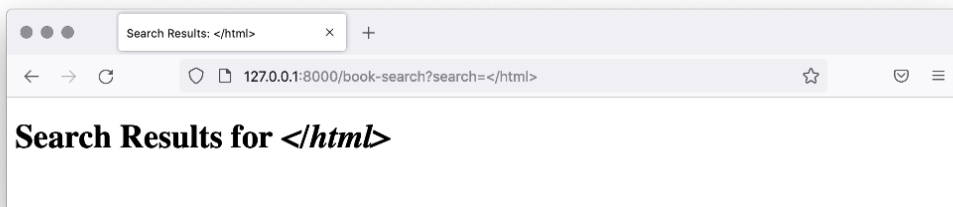


Figure 1.43 – Note how HTML characters are escaped so that we are protected from tag injection

이 단계들은 활동을 완료하는 데 도움이 될 것입니다:

1. 검색 결과 HTML 템플릿을 만듭니다. 템플릿에는 렌더링 컨텍스트를 통해 전달된 검색 단어(s)를 표시하기 위한 변수 자리 표시자가 포함되어야 합니다. 전달된 변수를 `<title>` 및 `<h1>` 태그에 표시합니다. 본문에서 검색 텍스트를 이탤릭체로 만들기 위해 `<em>` 태그를 사용합니다.
2. `views.py`에 검색 뷰 함수를 추가합니다. 뷰는 URL 매개변수(요청의 GET 속성)에서 검색 문자열을 읽어야 합니다. 그런 다음 이전 단계에서 만든 템플릿을 렌더링하고, 컨텍스트 사전을 사용하여 대체할 검색 값을 전달해야 합니다.
3. `urls.py`에 새 뷰에 대한 URL 매핑을 추가합니다. URL은 `/book-search`와 같은 것이 될 수 있습니다.

이 `idx_b31def13` 활동을 완료한 후, URL의 매개변수를 통해 검색 값을 전달하고 결과 페이지에 렌더링된 것을 볼 수 있어야 합니다. 다음과 같이 보여야 합니다:

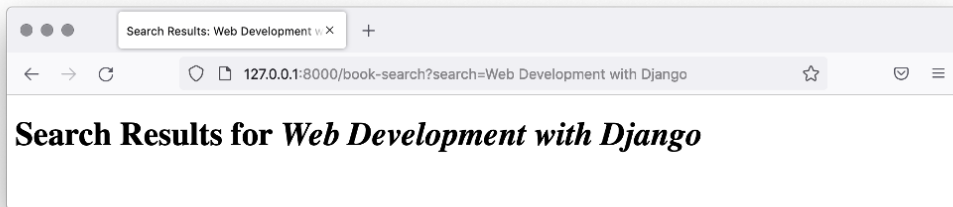


그림 1.42 - Django 워크숍 검색

또한 `<` 및 `>`와 같은 특수 HTML 문자를 전달하여 Django가 템플릿에서 자동으로 이들을 이스케이프하는 방법을 볼 수 있어야 합니다.

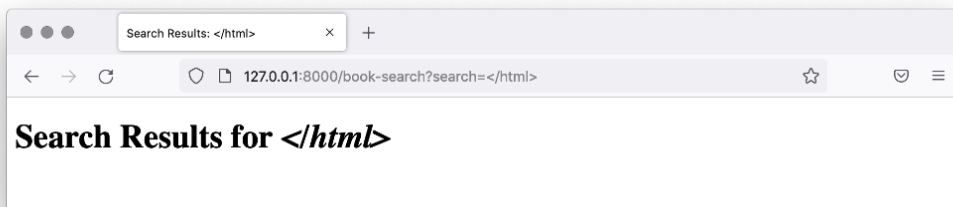


그림 1.43 - HTML 문자가 이스케이프되어 태그 주입으로부터 보호되는 방법에 주목하세요.

Summary

This chapter was a quick introduction to Django. You first got up to speed on the HTTP protocol and the structure of HTTP requests and responses. We then saw how Django uses the MVT paradigm, and then how it parses a URL, generates an HTTP request, and sends it to a view to get an HTTP response. We scaffolded the *bookr* project and then created the *reviews* app for it. We then built two example views to illustrate how to get data from a request and use it when rendering templates. You also experimented to see how Django escapes output in HTML when rendering a template.

We did all this with the PyCharm IDE, and you learned how to set it up to debug your application. The debugger will help you find out why things aren't working as they should.

In the next chapter, you will start to learn about Django's database integration and its model system, so you can start storing and retrieving real data for your application.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

요약

이 장에서는 장고에 대한 간단한 소개를 했습니다. 먼저 HTTP 프로토콜과 HTTP 요청 및 응답의 구조에 대해 빠르게 알아보았습니다. 그런 다음 장고가 MVT 패러다임을 어떻게 사용하는지, URL을 어떻게 파싱하고, HTTP 요청을 생성하여 뷰에 보내고 HTTP 응답을 받는지 살펴보았습니다. 우리는 bookr 프로젝트를 설정하고 그에 대한 리뷰 앱을 만들었습니다. 그런 다음 요청에서 데이터를 가져오고 템플릿을 렌더링할 때 사용하는 방법을 설명하기 위해 두 개의 예제 뷰를 만들었습니다. 또한 장고가 템플릿을 렌더링할 때 HTML에서 출력을 어떻게 이스케이프하는지 실험해 보았습니다.

우리는 모두 PyCharm IDE를 사용하여 이 작업을 수행했으며, 애플리케이션을 디버깅하기 위해 설정하는 방법을 배웠습니다. 디버거는 문제가 발생하는 이유를 찾는 데 도움을 줄 것입니다.

다음 장에서는 장고의 데이터베이스 통합 및 모델 시스템에 대해 배우기 시작할 것이므로 애플리케이션에 대한 실제 데이터를 저장하고 검색할 수 있습니다.

이 책의 PDF 사본, 코드 번들 및 기타 자료를 받으세요

QR 코드를 스캔하거나 packtpub.com/unlock으로 이동하세요. 이 책의 이름으로 검색하고, 판을 확인한 다음 페이지의 단계를 따르세요.



UNLOCK NOW

참고: 송장 사본을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

2

Models and Migrations

In this chapter, we will learn about Django's **object-relational mapping (ORM)**, through which our application can interact and seamlessly work with a relational database using simple Python code, eliminating the need to run complex SQL queries. We will learn about models and migrations, which are part of Django's ORM, used to propagate database schema changes from the application to the database, and also perform database **Create, Retrieve, Update, Delete (CRUD)** operations. Toward the end of the chapter, we will study the various types of database relationships and use that knowledge to perform queries across related records.

We will cover the following topics in this chapter:

- Exploring Django ORM
- Creating Django models and migrations
- Django's database CRUD operations
- Bulk create and bulk update operations
- Performing complex lookups using Q objects
- Activity 2.01 – Create models for a project management application
- Populating the Bookr project's database

By the end of this chapter, we will have a good understanding of databases and how Django can be used to implement database operations for a web application.

Technical requirements

Find the solution in the Chapter02 folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

2

모델과 마이그레이션

이 장에서는 Django의 객체-관계 매핑(ORM)에 대해 배우고, 이를 통해 우리의 애플리케이션이 관계형 데이터베이스와 상호작용하고 간단한 Python 코드를 사용하여 원활하게 작업할 수 있도록 하여 복잡한 SQL 쿼리를 실행할 필요를 없애는 방법을 배웁니다. 우리는 Django의 ORM의 일부인 모델과 마이그레이션에 대해 배우고, 이를 사용하여 애플리케이션에서 데이터베이스로 데이터베이스 스키마 변경을 전파하고 데이터베이스 생성, 검색, 업데이트, 삭제(CRUD) 작업을 수행합니다. 장의 끝부분에서는 다양한 유형의 데이터베이스 관계를 연구하고, 그 지식을 사용하여 관련 레코드 간의 쿼리를 수행할 것입니다.

이 장에서는 다음 주제를 다룰 것입니다:

- Django ORM 탐색하기
 - Django 모델 및 마이그레이션 생성하기 Django의 데이터베이스 CRUD 작업
 - 대량 생성 및 대량 업데이트 작업 복잡한
 - 조회를 Q 객체를 사용하여 수행하기
 - 활동 2.01 – 프로젝트 관리 애플리케이션을 위한 모델 생성.
- Bookr 프로젝트의 데이터베이스 채우기

이 장이 끝날 무렵, 우리는 데이터베이스와 Django가 웹 애플리케이션을 위한 데이터베이스 작업을 구현하는 데 어떻게 사용될 수 있는지에 대한 좋은 이해를 갖게 될 것입니다.

기술 요구 사항

이 책의 GitHub 리포지토리의 Chapter02 폴더에서 솔루션을 찾으십시오. 리포지토리 링크에 접근하려면 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르십시오.

.

Exploring Django ORM

Web applications constantly interact with databases, and one of the ways to do so is by using SQL. If you decide to write a web application without a web framework, such as Django, and instead use Python alone, Python libraries such as `psycopg2` (for Postgres DB) could be used to interact directly with the databases using SQL commands. But while developing a web application with multiple tables and fields, SQL commands can easily become overly complex and thus difficult to maintain. For this reason, popular web frameworks such as Django provide a level of abstraction that allows us to easily work with databases. The part of Django that helps us do this is called the ORM.

Django's ORM converts object-oriented Python code into actual database constructs, such as database tables with data type definitions, and facilitates all the database operations via simple Python code. Because of this, we do not have to deal with SQL commands while performing database operations. This helps in faster application development and ease in maintaining the application source code.

Django supports relational databases such as SQLite, PostgreSQL, Oracle Database, and MySQL. Django's database abstraction layer ensures that the same Python/Django source code can be used across any of the preceding relational databases with very little modification to the project settings. Since SQLite is part of the Python libraries and Django is configured by default to use SQLite, for the scope of this chapter, we shall use SQLite while we learn about Django models and migrations. Next, we will understand how database configuration is done with Django.

Database configuration and creating Django applications

As we have already seen in *Chapter 1*, when we create a Django project and run the Django server, the default database configuration is SQLite3. The database configuration will be present in the project directory, in the `settings.py` file.

Note

Make sure you examine the `settings.py` file for the `bookr` app. Going through the entire file once will help you understand the concepts that follow. You can find the file in the `Chapter02` folder in this book's GitHub repo.

장고 ORM 탐색하기

웹 애플리케이션은 데이터베이스와 지속적으로 상호작용하며, 이를 수행하는 방법 중 하나는 SQL을 사용하는 것입니다. Django와 같은 웹 프레임워크 없이 웹 애플리케이션을 작성하기로 결정하고 대신 Python만 사용한다면, psycopg2(포스트그레스 DB용)와 같은 Python 라이브러리를 사용하여 SQL 명령어를 사용하여 데이터베이스와 직접 상호작용할 수 있습니다. 그러나 여러 테이블과 필드가 있는 웹 애플리케이션을 개발하는 동안 SQL 명령어는 쉽게 지나치게 복잡해져 유지 관리가 어려워질 수 있습니다. 이러한 이유로 Django와 같은 인기 있는 웹 프레임워크는 데이터베이스와 쉽게 작업할 수 있는 추상화 수준을 제공합니다. Django의 이 부분을 ORM이라고 합니다.

장고의 ORM은 객체 지향 Python 코드를 데이터 유형 정의가 있는 데이터베이스 테이블과 같은 실제 데이터베이스 구조로 변환하고, 간단한 Python 코드를 통해 모든 데이터베이스 작업을 용이하게 합니다. 이로 인해 데이터베이스 작업을 수행할 때 SQL 명령어를 다룰 필요가 없습니다. 이는 애플리케이션 개발을 더 빠르게 하고 애플리케이션 소스 코드를 유지 관리하는 데 용이합니다.

장고는 SQLite, PostgreSQL, Oracle Database 및 MySQL과 같은 관계형 데이터베이스를 지원합니다. 장고의 데이터베이스 추상화 계층은 동일한 Python/Django 소스 코드를 앞서 언급한 관계형 데이터베이스에서 프로젝트 설정을 거의 수정하지 않고 사용할 수 있도록 보장합니다. SQLite는 Python 라이브러리의 일부이며 Django는 기본적으로 SQLite를 사용하도록 구성되어 있으므로, 이 장의 범위 내에서는 Django 모델과 마이그레이션에 대해 배우는 동안 SQLite를 사용할 것입니다. 다음으로, Django로 데이터베이스 구성이 어떻게 이루어지는지 이해할 것입니다.

데이터베이스 구성 및 Django 애플리케이션 생성하기

1장에서 이미 보았듯이, Django 프로젝트를 생성하고 Django 서버를 실행하면 기본 데이터베이스 구성은 SQLite3입니다. 데이터베이스 구성은 프로젝트 디렉토리의 settings.py 파일에 존재합니다.

참고

bookr 앱의 settings.py 파일을 반드시 확인하세요. 전체 파일을 한 번 살펴보는 것은 이후의 개념을 이해하는 데 도움이 될 것입니다. 이 파일은 이 책의 GitHub 리포지토리의 Chapter02 폴더에서 찾을 수 있습니다.

So, for our example project, the database configuration will be present at the following location: `bookr/settings.py`. The default database configuration present in this file when a Django project is created is as follows:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

The `DATABASES` variable is assigned a dictionary containing the database details for the project. Inside the dictionary, there is a nested dictionary with a key of `default`. This holds the configuration of a default database for the Django project. The reason we have a nested dictionary with `default` as a key is that a Django project could potentially interact with multiple databases, and the default database is the one used by Django for all operations unless explicitly specified. The `ENGINE` key represents which database engine is being used; in this case, `sqlite3`.

The `NAME` key defines the name of the database, which can have any value. But for `SQLite3`, since the database is created as a file, `NAME` can have the full path of the directory where the file needs to be created. The full path of the `db` file is processed by joining (or concatenating) the previously defined path in `BASE_DIR` with `db.sqlite3`. Note that `BASE_DIR` is the project directory as already defined in the `settings.py` file. Because `BASE_DIR` is defined as `pathlib.Path`, the `/` operator in this line of code is the overloaded `pathlib.Path.__truediv__` method, rather than a numerical operator. This operator is used to join two paths so that `pathlib.Path('a') / 'b'` is equal to `pathlib.Path('a/b')`.

If you are using other databases, such as PostgreSQL, MySQL, and so on, changes will have to be made in the preceding database settings, as shown here:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'bookr',
        'USER': 'django',
        'PASSWORD': '0b3ef4b0d80c28db',
        'HOST': '127.0.0.1',
        'PORT': '5432', }}}
```

따라서 우리의 예제 프로젝트에서 데이터베이스 구성은 다음 위치에 있습니다:
bookr/settings.py. Django 프로젝트가 생성될 때 이 파일에 있는 기본 데이터베이스
구성은 다음과 같습니다:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

DATABASES 변수는 프로젝트의 데이터베이스 세부정보를 포함하는
idx_fe4b752fdictionary에 할당됩니다. 사전 내부에는 default라는 키를 가진 중첩 사전
이 있습니다. 이는 Django 프로젝트의 기본 데이터베이스 구성을 보유합니다. default를
키로 가진 중첩 사전이 있는 이유는 Django 프로젝트가 여러 데이터베이스와 상호작용
할 수 있으며, 기본 데이터베이스는 명시적으로 지정되지 않는 한 Django가 모든 작업에
사용하는 데이터베이스이기 때문입니다. ENGINE 키는 어떤 데이터베이스 엔진이 사
용되고 있는지를 나타냅니다. 이 경우 sqlite3입니다.

NAME 키는 데이터베이스의 이름을 정의하며, 어떤 값도 가질 수 있습니다. 그러나
SQLite3의 경우 데이터베이스가 파일로 생성되기 때문에 NAME은 파일이 생성될 디렉
토리의 전체 경로를 가질 수 있습니다. db 파일의 전체 경로는 BASE_DIR에 미리 정의된
경로와 db.sqlite3를 결합(또는 연결)하여 처리됩니다. BASE_DIR은 이미 settings.py 파
일에서 정의된 프로젝트 디렉토리입니다. BASE_DIR이 pathlib.Path로 정의되었기 때문
에 이 코드 줄의 / 연산자는 숫자 연산자가 아니라 오버로드된 pathlib.Path.__truediv__ 메
서드입니다. 이 연산자는 두 경로를 결합하는 데 사용되므로 pathlib.Path('a') / 'b'는
pathlib.Path('a/b')와 같습니다.

다른 데이터베이스(예: PostgreSQL, MySQL 등)를 사용하는 경우, 앞서 언급한 데이
터베이스 설정에서 변경이 필요합니다. 다음과 같이 표시됩니다:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'bookr',
        'USER': 'django',
        'PASSWORD': '0b3ef4b0d80c28db',
        'HOST': '127.0.0.1',
        'PORT': '5432', }}
}
```

Here, changes have been made to `ENGINE` to use PostgreSQL. The host IP address and port number of the server need to be provided for `HOST` and `PORT`, respectively. As the names suggest, `USER` is the database username, and `PASSWORD` is the database password. In addition to changes in the configuration, we will have to install the database drivers or bindings along with the database host and credentials. Note that this code block is just an example to show the changes you'll need to make to use a different database, such as PostgreSQL, but since we are using SQLite, we shall use the database configuration that exists already, and there is no need to make any modifications to the database settings. Next, we will briefly read about Django apps and some of the default apps present in Django.

Django migration

As we have learned before, Django's ORM helps make database operations simpler. A major part of the operation is transforming the Python code into database structures, such as database fields with stated data types and tables. In other words, the transformation of Python code into database structures is known as **migration**. For example, instead of creating database tables by running SQL commands, you would write models for them in Python; something we'll learn to do in the upcoming *Creating models and migrations* section. These models will have **fields** that form the blueprints of database tables. The fields, in turn, will have different field types, giving us more information about the type of data stored there (recall how we specified the data type of our field as `TEXT` in *step 4 of Exercise 2.01, Creating an entry in the bookr database*).

Since we have a Django project set up, let's perform our first migration. Although we have not added any code to our project, we can migrate the applications listed in `INSTALLED_APPS`. This is necessary because Django's installed apps need to store the relevant data in the database for their operations, and migration will create the required database tables to store the data in the database.

The following is a snippet from the Bookr project's `settings.py` file:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'reviews',  
]
```

여기에서 ENGINE을 PostgreSQL을 사용하도록 변경했습니다. HOST와 PORT에 대해 각각 서버의 호스트 IP 주소와 포트 번호를 제공해야 합니다. 이름이 암시하듯이, USER는 데이터베이스 사용자 이름이고, PASSWORD는 데이터베이스 비밀번호입니다. 구성의 변경 외에도 데이터베이스 호스트 및 자격 증명과 함께 데이터베이스 드라이버 또는 바인딩을 설치해야 합니다. 이 코드 블록은 PostgreSQL과 같은 다른 데이터베이스를 사용하기 위해 필요한 변경 사항을 보여주기 위한 예제일 뿐이지만, SQLite를 사용하고 있으므로 이미 존재하는 데이터베이스 idx_37515fb1 구성을 사용할 것이며, 데이터베이스 설정을 수정할 필요는 없습니다. 다음으로, Django 앱과 Django에 존재하는 기본 앱에 대해 간략히 알아보겠습니다.

Django 마이그레이션

앞서 배운 바와 같이, Django의 ORM은 데이터베이스 작업을 더 간단하게 만들어 줍니다. 작업의 주요 부분은 Python 코드를 데이터베이스 구조로 변환하는 것입니다. 예를 들어, SQL 명령을 실행하여 데이터베이스 테이블을 생성하는 대신, Python에서 모델을 작성하게 됩니다. 이는 앞으로 '모델 및 마이그레이션 생성' 섹션에서 배우게 될 것입니다. 이러한 모델은 데이터베이스 테이블의 청사진을 형성하는 필드를 가질 것입니다. 필드는 차례로 서로 다른 필드 유형을 가지며, 그곳에 저장된 데이터의 유형에 대한 더 많은 정보를 제공합니다(2.01 연습의 4단계에서 필드의 데이터 유형을 TEXT로 지정한 방법을 기억해 보세요).

Django 프로젝트가 설정되었으므로 첫 번째 마이그레이션을 수행해 보겠습니다. 프로젝트에 코드를 추가하지 않았지만, INSTALLED_APPS에 나열된 애플리케이션을 마이그레이션할 수 있습니다. 이는 Django의 설치된 앱이 작업을 위해 데이터베이스에 관련 데이터를 저장해야 하며, 마이그레이션이 데이터베이스에 데이터를 저장하기 위한 필요한 데이터베이스 테이블을 생성할 것이기 때문에 필요합니다.

다음은 Bookr 프로젝트의 settings.py 파일에서 발췌한 내용입니다:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'reviews',
]
```

This code block contains a set of installed or default apps used for the admin site, authentication, content types, sessions, messaging, static file management, and the review app that we started in *Chapter 1*.

The following command should be entered in the terminal or shell to do this:

```
python manage.py migrate
```

By executing this command, we create all the database structures required by the installed apps.

For better visualization of the data in the SQLite database, we will use an open source tool called **DB Browser for SQLite**. This tool helps visualize the data and provides a shell to execute the SQL commands.

If you haven't done so already, visit the <https://sqlitebrowser.org> URL, and from the downloads section, install the application as per your operating system and launch it.

Let's take a look at the database for which changes have been made after executing the migrate command using DB Browser for SQLite.

The database file will be created in the project directory under the `db.sqlite3` name.

Note

Database operations can also be performed using a command-line shell by using the `sqlite3` command `sqlite3 db.sqlite3`.

Follow these steps to verify that migration has been successful and database tables have been created:

1. Open DB Browser, click **Open Database**, navigate until you find the `db.sqlite3` file, and open it. You should see a set of newly created tables created by the Django migration. It will look as follows in DB Browser:

이 코드 블록은 관리 사이트, 인증, 콘텐츠 유형, 세션, 메시징, 정적 파일 관리 및 1장에서 시작한 리뷰 앱에 사용되는 설치된 또는 기본 앱 세트를 포함합니다.

이를 수행하기 위해 터미널 또는 셸에 다음 명령을 입력해야 합니다:

```
python manage.py migrate
```

이 명령을 실행함으로써, 우리는 설치된 앱에 의해 요구되는 모든 데이터베이스 구조를 생성합니다.

idx_354e8c09SQLite 데이터베이스의 데이터를 더 잘 시각화하기 위해, 우리는 DB Browser for SQLite라는 오픈 소스 도구를 사용할 것입니다. 이 도구는 데이터를 시각화하고 SQL 명령을 실행할 수 있는 셸을 제공합니다.

아직 idx_c31733e6를 수행하지 않았다면, <https://sqlitebrowser.org> URL을 방문하고, 다운로드 섹션에서 운영 체제에 맞게 애플리케이션을 설치하고 실행하세요.

DB Browser for SQLite를 사용하여 migrate 명령을 실행한 후 변경된 데이터베이스를 살펴보겠습니다.

데이터베이스 파일은 프로젝트 디렉토리 내에 db.sqlite3라는 이름으로 생성됩니다.

메모

데이터베이스 작업은 sqlite3 명령어 sqlite3 db.sqlite3를 사용하여 명령줄 셸에서도 수행할 수 있습니다.

idx_0ef5dc83migration이 성공적으로 완료되었고 데이터베이스 테이블이 생성되었는지 확인하려면 다음 단계를 따르십시오:

1. DB Browser를 열고 데이터베이스 열기를 클릭한 다음 db.sqlite3 파일을 찾을 때까지 탐색하고 엽니다. Django 마이그레이션에 의해 생성된 새로 생성된 테이블 세트를 볼 수 있어야 합니다. DB Browser에서 다음과 같이 보일 것입니다:

Name	Type	Schema
Tables (11)		
auth_group		CREATE TABLE "auth_group" ("id" integer NOT NULL PRIMARY KEY
auth_group_permissions		CREATE TABLE "auth_group_permissions" ("id" integer NOT NULL
auth_permission		CREATE TABLE "auth_permission" ("id" integer NOT NULL PRIMAR
auth_user		CREATE TABLE "auth_user" ("id" integer NOT NULL PRIMARY KEY
auth_user_groups		CREATE TABLE "auth_user_groups" ("id" integer NOT NULL PRIMA
auth_user_user_permissions		CREATE TABLE "auth_user_user_permissions" ("id" integer NOT
django_admin_log		CREATE TABLE "django_admin_log" ("id" integer NOT NULL PRIMAR
django_content_type		CREATE TABLE "django_content_type" ("id" integer NOT NULL PRI
django_migrations		CREATE TABLE "django_migrations" ("id" integer NOT NULL PRIMAR
django_session		CREATE TABLE "django_session" ("session_key" varchar(40) NOT
sqlite_sequence		CREATE TABLE sqlite_sequence(name,seq)
Indices (15)		
Views (0)		
Triggers (0)		

Figure 2.1: Contents of the db.sqlite3 file

2. Now, browse through the newly created database structure by clicking the database tables, and we should see the following:

Name	Type	Schema
Tables (11)		
auth_group		CREATE TABLE "auth_group" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
name	varchar(150)	"name" varchar(150) NOT NULL UNIQUE
auth_group_permissions		CREATE TABLE "auth_group_permissions" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
group_id	integer	"group_id" integer NOT NULL
permission_id	integer	"permission_id" integer NOT NULL
auth_permission		CREATE TABLE "auth_permission" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
content_type_id	integer	"content_type_id" integer NOT NULL
codename	varchar(100)	"codename" varchar(100) NOT NULL
name	varchar(255)	"name" varchar(255) NOT NULL

Figure 2.2: Browsing through the newly created database structure

3. Notice that the database tables created have different fields, each with its respective data types. Click on the **Browse data** tab in DB Browser and select a table from the drop-down menu. For instance, after clicking the auth_group_permissions table, you should see something like this:

Table: auth_group_permissions		
id	group_id	permission_id
Filter	Filter	Filter

Figure 2.3: Viewing the auth_group_permissions table

Name	Type	Schema
Tables (11)		
auth_group		CREATE TABLE "auth_group" ("id" integer NOT NULL PRIMARY KEY
auth_group_permissions		CREATE TABLE "auth_group_permissions" ("id" integer NOT NULL
auth_permission		CREATE TABLE "auth_permission" ("id" integer NOT NULL PRIMAR
auth_user		CREATE TABLE "auth_user" ("id" integer NOT NULL PRIMARY KEY
auth_user_groups		CREATE TABLE "auth_user_groups" ("id" integer NOT NULL PRIM
auth_user_user_permissions		CREATE TABLE "auth_user_user_permissions" ("id" integer NOT
django_admin_log		CREATE TABLE "django_admin_log" ("id" integer NOT NULL PRIM
django_content_type		CREATE TABLE "django_content_type" ("id" integer NOT NULL PRI
django_migrations		CREATE TABLE "django_migrations" ("id" integer NOT NULL PRIM
django_session		CREATE TABLE "django_session" ("session_key" varchar(40) NOT
sqlite_sequence		CREATE TABLE sqlite_sequence(name,seq)
Indices (15)		
Views (0)		
Triggers (0)		

그림 2.1: db.sqlite3 파일의 내용

2. 이제 데이터베이스 테이블을 클릭하여 새로 생성된 데이터베이스 구조를 탐색해 보세요. 다음과 같은 내용을 볼 수 있어야 합니다:

Name	Type	Schema
Tables (11)		
auth_group		CREATE TABLE "auth_group" ("id" integer NOT NULL PRIMARY KEY
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
name	varchar(150)	"name" varchar(150) NOT NULL UNIQUE
auth_group_permissions		CREATE TABLE "auth_group_permissions" ("id" integer NOT
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
group_id	integer	"group_id" integer NOT NULL
permission_id	integer	"permission_id" integer NOT NULL
auth_permission		CREATE TABLE "auth_permission" ("id" integer NOT NULL F
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
content_type_id	integer	"content_type_id" integer NOT NULL
codename	varchar(100)	"codename" varchar(100) NOT NULL
name	varchar(255)	"name" varchar(255) NOT NULL

그림 2.2: 새로 생성된 데이터베이스 구조 탐색

3. 생성된 데이터베이스 테이블이 각각의 데이터 유형을 가진 서로 다른 필드를 가지고 있음을 주목하세요. DB idx_c3abec66Browser에서 데이터 탐색 탭을 클릭하고 드롭다운 메뉴에서 테이블을 선택하세요. 예를 들어, auth_group_permissions 테이블을 클릭한 후에는 다음과 같은 내용을 볼 수 있어야 합니다:

Table:	auth_group_permissions				
id	group_id	permission_id			
Filter	Filter	Filter			

그림 2.3: auth_group_permissions 테이블 보기

You will see that there is no data available for these tables yet because Django migration only creates the database structure or the blueprint, and the actual data in the database is stored during the operation of the application. Now that we have migrated the built-in or default Django apps, let's try to create an app and perform a Django migration.

Creating Django models and migrations

A Django model is essentially a Python class that holds the blueprint for creating a table in a database. The `models.py` file can have many such models, and each model transforms into a database table. The attributes of the class form the fields and relationships of the database table as per the model definitions.

For our reviews application, we need to create the following models and their database tables:

- **Book:** This should store information about books
- **Contributor:** This should store information about the person(s) who contributed to writing the book, such as the author, co-author, or editor
- **Publisher:** As the name implies, this refers to the book's publisher
- **Review:** This should store all the book reviews written by the users of the application

Every book in our application will need to have a publisher, so let's create `Publisher` as our first model. Enter the following code in `reviews/models.py`:

```
from django.db import models

class Publisher(models.Model):
    """A company that publishes books."""
    name = models.CharField(
        max_length=50,
        help_text="The name of the Publisher.")
    website = models.URLField(
        blank=True,
        help_text="The Publisher's website.")
    email = models.EmailField(
        blank=True,
        help_text="The Publisher's email address.")
```

이 테이블에 대한 데이터가 아직 없다는 것을 알 수 있습니다. Django 마이그레이션은 데이터베이스 구조 또는 청사진만 생성하며, 데이터베이스의 실제 데이터는 애플리케이션의 작동 중에 저장됩니다. 기본 제공 또는 기본 Django 앱을 마이그레이션했으므로, 이제 앱을 생성하고 Django 마이그레이션을 수행해 보겠습니다.

Django 모델 및 마이그레이션 생성

Django 모델 `idx_0817e184`는 본질적으로 데이터베이스에 테이블을 생성하기 위한 청사진을 담고 있는 Python 클래스입니다. `models.py` 파일에는 이러한 모델이 여러 개 있을 수 있으며, 각 모델은 데이터베이스 테이블로 변환됩니다. 클래스의 속성은 모델 정의에 따라 데이터베이스 테이블의 필드와 관계를 형성합니다.

우리의 리뷰 애플리케이션을 위해 다음 모델과 해당 데이터베이스 테이블을 생성해야

- 합니다: 책: 이 모델은 책에 대한 정보를 저장해야 합니다.
- 기여자: 이 모델은 책을 집필하는 데 기여한 사람(들)에 대한 정보를 저장해야 합니다. 예를 들어 저자, 공동 저자 또는 편집자와 같은 정보입니다.
- 출판사: 이름이 암시하듯이, 이는 책의 출판사를 의미합니다
- 리뷰: 이는 애플리케이션 사용자들이 작성한 모든 책 리뷰를 저장해야 합니다. 우리 애플리케이션의 모든 책은 출판사가 필요하므로, 출판사를 첫 번째 모델로 생성합니다. `reviews/models.py`에 다음 코드를 입력하세요:

```
from django.db import models

class Publisher(models.Model):
    """책을 출판하는 회사입니다."""
    name = models.CharField(
        max_length=50,
        help_text="출판사의 이름입니다.")
    website = models.URLField(
        blank=True,
        help_text="출판사의 웹사이트입니다.")
    email = models.EmailField(
        blank=True,
        help_text="출판사의 이메일 주소입니다.")
```

Note

You can take a look at the complete `models.py` file for the bookr app in the `Chapter02` folder in this book's GitHub repo.

The first line of code imports the Django `models` module. While this line will be autogenerated at the time of the creation of the Django app, do make sure you add it if it is not present. Following the import, the rest of the code defines a class named `Publisher`, a subclass of Django's `models.Model`. Furthermore, this class will have attributes or fields such as `name`, `website`, and `email`. The following are the field types used while creating this model.

As we can see, each of these fields is defined to have the following types:

- `CharField`: This field type stores shorter string fields, such as `Packt Publishing`. For very large strings, we use `TextField`.
- `EmailField`: This is similar to `CharField` but validates whether the string represents a valid email address – for example, `customersupport@packtpub.com`.
- `URLField`: Again, this is similar to `CharField`, but validates whether the string represents a valid URL – for example, `https://www.packtpub.com`.

Next, we will look at the field options used while creating each of these fields.

Field options

Django provides a way to define field options for a model's field. These field options are used to set a value or a constraint, and so on. For example, we can set a default value for a field using `default=<value>` to ensure that every time a record is created in the database for the field, it is set to a default value specified by us. The following are the three field options that we have used while defining the `Publisher` model:

- `blank`: The default is `False`. If this option is set to `True`, then blank values will be allowed for this field during validation of the Django form.
- `help_text`: This field option helps us add descriptive text for a field that gets automatically included for Django forms.

노트

이 책의 GitHub 저장소의 Chapter02 폴더에서 bookr 앱의 complete models.py 파일을 확인할 수 있습니다.

첫 번째 코드 줄은 Django 모델 모듈을 가져옵니다. 이 줄은 Django 앱 생성 시 자동 생성되지만, 없을 경우 반드시 추가해야 합니다. 가져오기 다음에 나오는 나머지 코드는 Publisher라는 클래스를 정의하며, 이는 Django의 models.Model의 하위 클래스입니다. 또한 이 클래스는 name, website, email과 같은 속성 또는 필드를 가집니다. 다음은 이 모델을 생성할 때 사용된 필드 유형입니다.

우리가 볼 수 있듯이, 이러한 각 필드는 다음과 같은 유형으로 정의됩니다:

- CharField: 이 필드 유형은 Packt Publishing과 같은 짧은 문자열 필드를 저장합니다. 매우 큰 문자열의 경우 TextField를 사용합니다.
- EmailField: 이것은 CharField와 유사하지만 문자열이 유효한 이메일 주소를 나타내는지 확인합니다 - 예를 들어, customersupport@packtpub.com.
- URLField: 다시 말해, 이것은 CharField와 유사하지만 문자열이 유효한 URL을 나타내는지 확인합니다 - 예를 들어, https://www.packtpub.com.

다음으로, 이러한 각 필드를 생성할 때 사용되는 필드 옵션을 살펴보겠습니다.

필드 옵션

Django는 모델의 필드에 대한 idx_df58940a 필드 옵션을 정의하는 방법을 제공합니다. 이러한 필드 옵션은 값을 설정하거나 제약 조건을 설정하는 데 사용됩니다. 예를 들어, default=<value>를 사용하여 필드에 대한 기본값을 설정할 수 있으며, 이를 통해 데이터베이스에 레코드가 생성될 때마다 우리가 지정한 기본값으로 설정되도록 할 수 있습니다. 다음은 Publisher 모델을 정의할 때 사용한 세 가지 필드 옵션입니다:

- blank: 기본값은 False입니다. 이 옵션이 True로 설정되면 Django 양식의 유효성 검사 중에 이 필드에 대해 빈 값이 허용됩니다.
- help_text: 이 필드 옵션은 Django 양식에 자동으로 포함되는 필드에 대한 설명 텍스트를 추가하는 데 도움이 됩니다.

- `max_length`: This option is provided to `CharField`, where it defines the maximum length of the field in terms of the number of characters.
- Other field options that are frequently used include the following:
 - `null`: The default is `False`. If this option is set to `True`, then a `NULL` value is allowable for the field at the database level.
 - `unique`: The default is `False`. If this option is set to `True`, then the database column corresponding to the field will be defined as unique. Duplicate values for this field will lead to a validation error.

Django has many more field types and field options that can be explored from the extensive official Django documentation. As we go about developing our sample book review application, we will learn about the types and fields used for the project. Now, let's migrate the Django models into the database by following these steps:

1. Execute the following command in the shell or terminal to do that (run it from the folder where your `manage.py` file is stored):

```
python manage.py makemigrations reviews
```

The output of the command looks like this:

```
Migrations for 'reviews':
  reviews/migrations/0001_initial.py
    - Create model Publisher
```

The `makemigrations <appname>` command creates the migration scripts for the given app – in this case, for the `reviews` app. Notice that after running `makemigrations`, there is a new file created under the `migrations` folder:

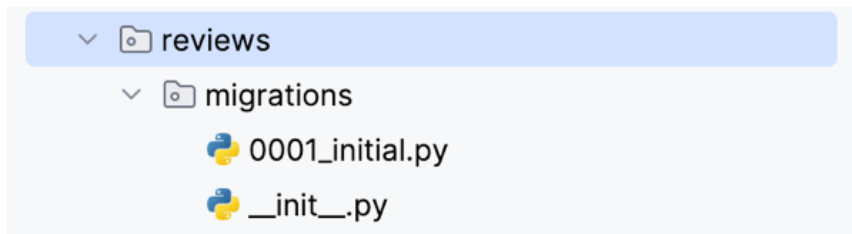


Figure 2.4: New file under the migrations folder

This is the migration script created by Django. When we run `makemigrations` without the app name, the migration scripts will be created for all the apps in the project.

- `max_length`: 이 옵션은 CharField에 제공되며, 필드의 최대 길이를 문자 수로 정의합니다.
- 자주 사용되는 다른 필드 옵션은 다음과 같습니다:
 - `null`: 기본값은 False입니다. 이 옵션이 True로 설정되면 데이터베이스 수준에서 필드에 NULL 값이 허용됩니다.
 - `unique`: 기본값은 False입니다. 이 옵션이 True로 설정되면 필드에 해당하는 데이터베이스 열이 고유하게 정의됩니다. 이 필드의 중복 값은 유효성 검사 오류를 초래합니다.

장고는 광범위한 공식 장고 문서에서 탐색할 수 있는 더 많은 필드 유형과 필드 옵션을 가지고 있습니다. 샘플 도서 리뷰 애플리케이션을 개발하면서 프로젝트에 사용되는 유형과 필드에 대해 배울 것입니다. 이제 다음 단계를 따라 장고 모델을 데이터베이스로 마이그레이션해 보겠습니다:

1. 이를 위해 셸 또는 터미널에서 다음 명령을 실행합니다 (`manage.py` 파일이 저장된 폴더에서 실행하세요):

```
python manage.py makemigrations reviews
```

명령의 출력은 다음과 같습니다:

```
'reviews'에 대한 마이그레이션:
reviews/migrations/0001_initial.py
- 모델 퍼블리셔 생성
```

`makemigrations <appname>` 명령은 주어진 앱에 대한 마이그레이션 스크립트를 생성합니다 - 이 경우 리뷰 앱에 대한 것입니다. `makemigrations`를 실행한 후 `migrations` 폴더 아래에 새 파일이 생성된 것을 확인하세요:

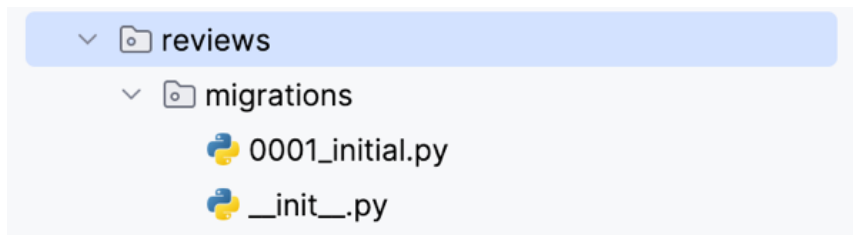


그림 2.4: migrations 폴더 아래의 새 파일

이것은 장고에 의해 생성된 마이그레이션 스크립트입니다. 앱 이름 없이 `makemigrations`를 실행하면 프로젝트의 모든 앱에 대한 마이그레이션 스크립트가 생성됩니다.

2. Next, let's list the project migration status. Remember that earlier, we applied migrations to Django's installed apps, and now we have created a new app, `reviews`. Run the following command in the shell or terminal, and it will show the status of model migrations throughout the project (run it from the folder where your `manage.py` file is stored):

```
python manage.py showmigrations
```

The output for the preceding command is as follows:

```
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
contenttypes
[X] 0001_initial
[X] 0002_remove_content_type_name
reviews
[ ] 0001_initial
sessions
[X] 0001_initial
```

Here, the `[X]` mark indicates that the migrations have been applied. Notice the difference that all the other apps' migrations have applied except that of `reviews`. The

2. 다음으로, idx_374b275eproject 마이그레이션 상태를 나열해 보겠습니다. 이전에 Django의 설치된 앱에 마이그레이션을 적용했으며, 이제 새로운 앱인 reviews를 생성했습니다. 셸이나 터미널에서 다음 명령을 실행하면 프로젝트 전반에 걸친 모델 마이그레이션 상태가 표시됩니다(manage.py 파일이 저장된 폴더에서 실행하세요):

```
python manage.py showmigrations
```

이전 명령의 출력은 다음과 같습니다:

```
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
contenttypes
[X] 0001_initial
[X] 0002_remove_content_type_name
reviews
[ ] 0001_initial
sessions
[X] 0001_initial
```

여기서 [X] 마크는 마이그레이션이 적용되었음을 나타냅니다. 리뷰를 제외한 모든 다른 앱의 마이그레이션이 적용되었다는 차이를 주목하세요.

showmigrations command can be executed to understand the migration status, but this is not a mandatory step while performing model migrations.

3. Next, let's understand how Django transforms a model into an actual database table. Django provides the sqlmigrate command to show the actual SQL generated by a migration. This isn't a mandatory step in model migrations, but it can be useful for further understanding.

We should see the following output:

```
BEGIN;
--
-- Create model Publisher
--
CREATE TABLE "reviews_publisher" ("id" integer
NOT NULL PRIMARY KEY AUTOINCREMENT, "name"
varchar(50) NOT NULL, "website" varchar(200)
NULL, "email" varchar(254) NULL);
COMMIT;
```

The preceding snippet shows the SQL command equivalent used when Django migrates the database. In this case, we are creating the reviews_publisher table with the name, website, and email fields with the defined field types. Furthermore, the name field is defined as NOT NULL, implying that the entries for this field cannot be null and should have a value. The field options null=True or unique=True will lead to a SQL definition with NULL or UNIQUE, respectively.

In the next section, we will learn about primary keys and their importance while storing data in a database.

Primary keys

Let's assume that a database table called users, as its name suggests, stores information about users. Let's say it has more than 1,000 records, and there are at least 3 users with the same name, Joe Burns. How do we uniquely identify these users from the application? The solution is to have a way to uniquely identify each record in the database. This is done using **primary keys**. A primary key is unique for a database table, and as a rule, a table cannot have two rows with the same primary key. In Django, when the primary key is not explicitly mentioned in the database models, Django automatically creates id as the primary key (of type integer), which auto-increments as new records are created.

showmigrations 명령어를 실행하여 마이그레이션 상태를 이해할 수 있지만, 모델 마이그레이션을 수행할 때 필수 단계는 아닙니다.

3. 다음으로, idx_0365f6beDjango가 모델을 실제 데이터베이스 테이블로 변환하는 방법을 이해해 봅시다. Django는 마이그레이션에 의해 생성된 실제 SQL을 보여주기 위해 sqlmigrate 명령어를 제공합니다. 이것은 모델 마이그레이션에서 필수 단계는 아니지만, 추가적인 이해를 위해 유용할 수 있습니다. 다음과 같은 출력을 볼 수 있어야 합니다:

```
BEGIN;
--
-- 모델 퍼블리셔 생성--

CREATE TABLE "reviews_publisher" ("id" integer
NOT NULL PRIMARY KEY AUTOINCREMENT, "name"
varchar(50) NOT NULL, "website" varchar(200)
NULL, "email" varchar(254) NULL);

COMMIT;
```

앞의 코드 조각은 Django가 데이터베이스를 마이그레이션할 때 사용되는 SQL 명령어와 동등한 것을 보여줍니다. 이 경우, 우리는 이름, 웹사이트 및 이메일 필드가 정의된 필드 유형으로 reviews_publisher 테이블을 생성하고 있습니다. 또한, name 필드는 NOT NULL로 정의되어 있어 이 필드의 항목은 null이 될 수 없으며 값이 있어야 함을 의미합니다. 필드 idx_ef483fff의 옵션 null=True 또는 unique=True는 각각 NULL 또는 UNIQUE로 SQL 정의를 이끌어냅니다.

다음 섹션에서는 데이터베이스에 데이터를 저장할 때 기본 키와 그 중요성에 대해 배울 것입니다.

기본 키

사용자라는 이름의 데이터베이스 테이블이 있다고 가정해 보겠습니다. 이 테이블은 사용자에 대한 정보를 저장합니다. 예를 들어, 1,000개 이상의 레코드가 있으며, 이름이 같은 사용자 Joe Burns가 최소 3명 있다고 가정해 보겠습니다. 애플리케이션에서 이러한 사용자를 어떻게 고유하게 식별할 수 있을까요? 해결책은 데이터베이스의 각 레코드를 고유하게 식별할 수 있는 방법을 갖는 것입니다. 이는 기본 키를 사용하여 이루어집니다. 기본 키는 데이터베이스 테이블에 대해 고유하며, 일반적으로 테이블은 동일한 기본 키를 가진 두 개의 행을 가질 수 없습니다. Django에서는 데이터베이스 모델에서 기본 키가 명시적으로 언급되지 않으면 Django가 자동으로 id를 기본 키(정수형)로 생성하며, 새로운 레코드가 생성될 때마다 자동으로 증가합니다.

In the previous section, notice the output of the `python manage.py sqlmigrate` command. While creating the `Publisher` table, the SQL `CREATE TABLE` command added one more field called `id` to the table. An `id` is defined as `PRIMARY KEY AUTOINCREMENT`. In relational databases, a primary key is used to uniquely identify an entry in the database. For example, the `book` table has `id` as the primary key, which has numbers starting from 1. This value increments by one as new records are created. The integer value of `id` is always unique across the `book` table. Since the migration script has already been created by executing `makemigrations`, let's now migrate the newly created model in the `reviews` app by executing the following command:

```
python manage.py migrate reviews
```

We should get the following output:

```
Operations to perform:
  Apply all migrations: reviews
Running migrations:
  Applying reviews.0001_initial... OK
```

This operation creates the database table for the `reviews` app. The following is a snippet from DB Browser indicating the new `reviews_publisher` table has been created in the database:

▼  reviews_publisher		CREATE TABLE "reviews_publisher" ("id" integer NOT NULL PRIMARY KEY AUTOIN
	id	integer
	name	varchar(50)
	website	varchar(200)
	email	varchar(254)

Figure 2.5: The `reviews_publisher` table created after executing the migration command

So far, we have explored how to create a model and migrate it into the database. Let's now work on creating the rest of the models for our book review application. As we've already seen, the application will have database tables for `Book`, `Publisher`, `Contributor`, and `Review`.

Let's add the `Book` and `Contributor` models, as shown in the following code snippet, into `reviews/models.py`:

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
        max_length=70, help_text="The title of the book.")
    publication_date = models.DateField(
```

이전 섹션에서 `python manage.py sqlmigrate` 명령의 출력을 확인하세요. Publisher 테이블을 생성하는 동안 SQL CREATE TABLE 명령이 테이블에 `id`라는 추가 필드를 추가했습니다. `id`는 PRIMARY KEY AUTOINCREMENT로 정의됩니다. 관계형 데이터베이스에서 기본 키는 데이터베이스의 항목을 고유하게 식별하는 데 사용됩니다. 예를 들어, 책 테이블은 `id`를 기본 키로 가지고 있으며, 이는 1부터 시작하는 숫자를 가집니다. 이 값은 새로운 레코드가 생성될 때마다 1씩 증가합니다. `id`의 정수 값은 항상 책 테이블에서 고유합니다. `makemigrations`를 실행하여 마이그레이션 스크립트가 이미 생성되었으므로, 이제 다음 명령을 실행하여 리뷰 앱에서 새로 생성된 모델을 마이그레이션합니다:

```
python manage.py migrate reviews
```

다음 출력을 얻어야 합니다:

```
수행할 작업:
  모든 마이그레이션 적용:
  reviews 마이그레이션 실행 중:

  reviews.0001_initial 적용 중... OK
```

이 작업은 리뷰 앱을 위한 데이터베이스 테이블을 생성합니다. 다음은 DB Browser에서 데이터베이스에 새로 생성된 `reviews_publisher` 테이블이 생성되었음을 나타내는 스니펫입니다:

▼  reviews_publisher		CREATE TABLE "reviews_publisher" ("id" integer NOT NULL PRIMARY KEY AUTO	
	id	integer	"id" integer NOT NULL
	name	varchar(50)	"name" varchar(50) NOT NULL
	website	varchar(200)	"website" varchar(200)
	email	varchar(254)	"email" varchar(254)

그림 2.5: 마이그레이션 명령을 실행한 후 생성된 `reviews_publisher` 테이블

지금까지 우리는 모델을 생성하고 이를 데이터베이스로 마이그레이션하는 방법을 탐구했습니다. 이제 책 리뷰 애플리케이션의 나머지 모델을 생성하는 작업을 진행해 보겠습니다. 이미 보았듯이, 애플리케이션은 Book, Publisher, Contributor 및 Review에 대한 데이터베이스 테이블을 가질 것입니다.

다음 코드 스니펫에 표시된 대로 Book 및 Contributor 모델을 `reviews/models.py`에 추가해 보겠습니다:

```
class Book(models.Model):
    """출판된 책입니다."""
    title = models.CharField(
        max_length=70, help_text="책의 제목입니다.")
    publication_date = models.DateField(
```

```
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="ISBN number of the book.")

class Contributor(models.Model):
    """A contributor to a Book, e.g. author, editor,
        co-author."""
    first_names = models.CharField(
        max_length=50, help_text="The contributor's first
        name or names.")
    last_names = models.CharField(
        blank=True, max_length=50,
        help_text="The contributor's last name or names.")
    email = models.EmailField(
        blank=True,
        help_text="The contact email for the contributor.")
```

The code is self-explanatory. The Book model has the title, publication_date, and isbn fields. The Contributor model has the first_names and last_names fields and the email ID of the contributor. There are also some newly added models, apart from the ones we have seen in the Publisher model. They have DateField as a new field type, which, as the name suggests, is used to store a date. A new field option called verbose_name is also used. It provides a descriptive name for the field. Next, we will see how relationships work in a relational database.

Relationships

One of the powers of relational databases is the ability to establish relationships between data stored across database tables. Relationships help maintain data integrity by establishing the correct references across tables, which in turn helps maintain the database. Relationship rules, on the other hand, ensure data consistency and prevent duplicates.

In a relational database, there can be the following types of relations:

- Many to one
- Many to many
- One to one

Let's explore each relationship in detail in the following sections.


```

        verbose_name="책이 출판된 날짜입니다.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="책의 ISBN 번호입니다.")

class Contributor(models.Model):
    """책의 기여자, 예: 저자, 편집자,
    공저자."""
    first_names = models.CharField(
        max_length=50, help_text="기여자의 이름 또는 이름들
        입니다.")
    last_names = models.CharField(
        blank=True, max_length=50,
        help_text="기여자의 성 또는 성들입니다.")
    email = models.EmailField(
        blank=True,
        help_text="기여자의 연락 이메일입니다.")

```

코드는 자명합니다. Book 모델은 제목, 출판일, 그리고 isbn 필드를 가지고 있습니다. Contributor 모델은 first_names와 last_names 필드 및 기여자의 이메일 ID를 가지고 있습니다. 또한, Publisher 모델에서 본 것 외에도 새로 추가된 모델들이 있습니다. 이들은 날짜를 저장하는 데 사용되는 새로운 필드 유형으로 DateField를 가지고 있습니다. verbose_name이라는 새로운 필드 옵션도 사용됩니다. 이는 필드에 대한 설명적인 이름을 제공합니다. 다음으로, 관계가 관계형 데이터베이스에서 어떻게 작동하는지 살펴보겠습니다.

관계

관계형 데이터베이스의 강점 중 하나는 데이터베이스 테이블 간에 저장된 데이터 간의 관계를 설정할 수 있는 능력입니다. 관계는 테이블 간의 올바른 참조를 설정하여 데이터 무결성을 유지하는 데 도움을 주며, 이는 차례로 데이터베이스를 유지하는 데 도움을 줍니다. 반면에 관계 규칙은 데이터 일관성을 보장하고 중복을 방지합니다.

관계형 데이터베이스에는 다음과 같은 유형의 관계가 있을 수 있습니다:

- 다대일 다대
- 다 일대일
-

다음 섹션에서 각 관계를 자세히 살펴보겠습니다.

Many to one

In this relationship, many records (rows/entries) from one table can refer to one record (row/entry) in another table. For example, there can be many books produced by one publisher. This is an example of a **many-to-one relationship**. To establish this relationship, we need to use the database's foreign keys. A foreign key in a relational database establishes the relationship between a field from one table and a primary key from a different table.

For example, say we have data about employees belonging to different departments stored in a table called `employee_info` with their employee ID as the primary key alongside a column that stores their department name; this table also contains a column that stores that department's department ID. Now, there's another table called `departments_info`, which has the department ID as the primary key. In this case, then, the department ID is a foreign key in the `employee_info` table.

In our bookr app, the `Book` model can have a foreign key referring to the primary key of the `Publisher` table. Since we have already created the models for `Book`, `Contributor`, and `Publisher`, let's now establish a many-to-one relationship across the `Book` and `Publisher` models. For the `Book` model, add the last line:

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
        max_length=70, help_text="The title of the book.")
    publication_date = models.DateField(
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="ISBN number of the book.")
    publisher = models.ForeignKey(
        Publisher, on_delete=models.CASCADE)
```

Now the newly added `publisher` field is establishing a many-to-one relationship between `Book` and `Publisher` using a foreign key. This relationship ensures the nature of a many-to-one relationship, which is that many books can have one publisher:

- `models.ForeignKey`: This is the field option to establish a many-to-one relationship.

다대일

이 관계에서 하나의 테이블의 여러 idx_7a1ad8a1 레코드(행/항목)는 다른 테이블의 하나의 레코드(행/항목)를 참조할 수 있습니다. 예를 들어, 하나의 출판사에서 여러 권의 책이 발행될 수 있습니다. 이것은 다대일 관계의 예입니다. 이 관계를 설정하기 위해서는 데이터베이스의 외래 키를 사용해야 합니다. 관계형 데이터베이스에서 외래 키는 하나의 테이블의 필드와 다른 테이블의 기본 키 간의 관계를 설정합니다.

예를 들어, 서로 다른 부서에 속한 직원에 대한 데이터가 직원 ID를 기본 키로 하고 부서 이름을 저장하는 열이 있는 employee_info라는 테이블에 저장되어 있다고 가정해 보겠습니다. 이 테이블에는 해당 부서의 부서 ID를 저장하는 열도 포함되어 있습니다. 이제 부서 ID를 기본 키로 하는 departments_info라는 또 다른 테이블이 있습니다. 이 경우, 부서 ID는 employee_info 테이블의 외래 키입니다.

우리의 bookr 앱에서 Book 모델은 Publisher 테이블의 기본 idx_d643d6b1 키를 참조하는 외래 키를 가질 수 있습니다. 이미 Book, Contributor 및 Publisher 모델을 생성했으므로 이제 Book과 Publisher 모델 간의 다대일 관계를 설정해 보겠습니다. Book 모델에 마지막 줄을 추가합니다:

```
class Book(models.Model):
    """출판된 책입니다."""
    title = models.CharField(
        max_length=70, help_text="책의 제목입니다.")
    publication_date = models.DateField(
        verbose_name="책이 출판된 날짜입니다.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="책의 ISBN 번호입니다.")
    publisher = models.ForeignKey(
        Publisher, on_delete=models.CASCADE)
```

이제 새로 추가된 출판사 필드는 외래 키를 사용하여 Book과 Publisher 간의 다대일 관계를 설정하고 있습니다. 이 관계는 여러 권의 책이 하나의 출판사를 가질 수 있다는 다대일 관계의 본질을 보장합니다:

- models.ForeignKey: 이것은 다대일 관계를 설정하기 위한 필드 옵션입니다.

- **Publisher:** When we establish relationships with different tables in Django, we refer to the model that creates the table; in this case, the Publisher table is created by the Publisher model (or the Publisher Python class).
- **on_delete:** This is a field option that determines the action to be taken upon the deletion of the referenced object. In this case, the on_delete option is set to CASCADE(models.CASCADE), which deletes the referenced objects.

For example, assume a publisher has published a set of books. For some reason, if the publisher has to be deleted from the application, the next action is CASCADE, which means deleting all the referenced books from the application. There are many more on_delete actions, such as the following:

- **PROTECT:** This prevents the record deletion unless all the referenced objects are deleted
- **SET_NULL:** This sets a null value if the database field has been previously configured to store null values
- **SET_DEFAULT:** Sets to a default value on the deletion of the referenced object

We will only use the CASCADE option for our book review application.

Many to many

In this relationship, multiple records in a table can have a relationship with multiple records in a different table. For example, a book can have multiple co-authors, and each author (contributor) could have written multiple books. So, this forms a **many-to-many relationship** between the Book and Contributor tables:

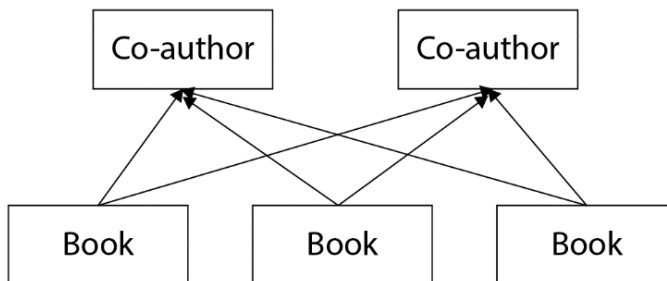


Figure 2.6: Many-to-many relationship between books and co-authors

In models.py, for the Book model, add the last line as shown here:

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
```

- 출판사: Django에서 서로 다른 테이블 간의 관계를 설정할 때, 우리는 테이블을 생성하는 모델을 참조합니다. 이 경우, 출판사 테이블은 출판사 모델(또는 출판사 Python 클래스)에 의해 생성됩니다.
- on_delete: 이는 참조된 객체가 삭제될 때 취할 행동을 결정하는 필드 옵션입니다. 이 경우, on_delete 옵션은 CASCADE(models.CASCADE)로 설정되어 있어 참조된 객체가 삭제됩니다.

예를 들어, publisher idx_1f202966이 책 세트를 출판했다고 가정해 보겠습니다. 어떤 이유로 출판사가 애플리케이션에서 삭제되어야 하는 경우, 다음 행동은 CASCADE이며, 이는 idx_6b4ab21e가 애플리케이션에서 모든 참조된 책을 삭제하는 것을 의미합니다. 다음과 같은 많은 on_delete 행동이 있습니다:

- PROTECT: 이는 모든 참조된 객체가 삭제되지 않는 한 기록 삭제를 방지합니다.
- SET_NULL: 이는 데이터베이스 필드가 이전에 null 값을 저장하도록 구성된 경우 null 값을 설정합니다.

SET_DEFAULT: 참조된 객체가 삭제될 때 기본값으로 설정합니다. 우리는 책 리뷰 애플리케이션에 대해 CASCADE 옵션만 사용할 것입니다.

다대다

이 관계에서, 하나의 테이블의 여러 레코드는 다른 테이블의 여러 레코드와 관계를 가질 수 있습니다. 예를 들어, 한 책은 여러 공동 저자를 가질 수 있으며, 각 저자(기여자)는 여러 책을 썼을 수 있습니다. 따라서, 이는 Book과 Contributor 테이블 간의 다대다 관계를 형성합니다:

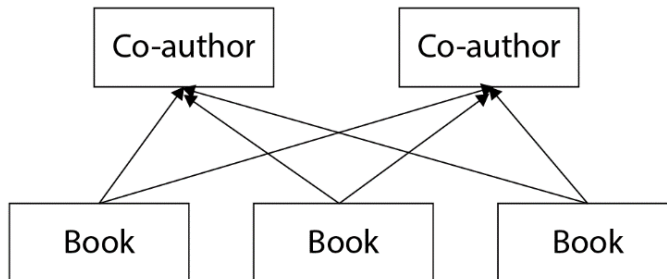


그림 2.6: 책과 공동 저자 간의 다대다 관계

models.py에서 Book 모델에 대해 마지막 줄을 다음과 같이 추가합니다:

```
class Book(models.Model):
    """출판된 책입니다."""
    title = models.CharField(
```

```

        max_length=70, help_text="The title of the book.")
    publication_date = models.DateField(
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="ISBN number of the book.")
    publisher = models.ForeignKey(
        Publisher, on_delete=models.CASCADE)
    contributors = models.ManyToManyField(
        'Contributor', through="BookContributor")

```

The newly added `contributors` field establishes a many-to-many relationship with `Book` and `Contributor` using the `ManyToManyField` field type:

- `models.ManyToManyField`: This is the field type to establish a many-to-many relationship.
- `through`: This is a special field option for many-to-many relationships. When we have a many-to-many relationship across two tables, if we want to store some extra information about the relationship, then we can use this to establish the relationship via an intermediary table.

For example, we have two tables, namely `Book` and `Contributor`, where we need to store the information on the type of contributor for the book, such as `Author`, `Co-author`, or `Editor`. Then the type of contributor is stored in an intermediary table called `BookContributor`. Here is how the `BookContributor` table/model looks. Make sure you include this model in `reviews/models.py`:

```

class BookContributor(models.Model):
    class ContributionRole(models.TextChoices):
        AUTHOR = "AUTHOR", "Author"
        CO_AUTHOR = "CO_AUTHOR", "Co-Author"
        EDITOR = "EDITOR", "Editor"

    book = models.ForeignKey(
        Book, on_delete=models.CASCADE)
    contributor = models.ForeignKey(
        Contributor, on_delete=models.CASCADE)
    role = models.CharField(
        verbose_name="The role this contributor had in"

```

```

        max_length=70, help_text="책의 제목입니다.")
    publication_date = models.DateField(
        verbose_name="책이 출판된 날짜입니다.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="책의 ISBN 번호입니다.")
    publisher = models.ForeignKey(
        Publisher, on_delete=models.CASCADE)
    contributors = models.ManyToManyField(
        '기여자', through="BookContributor")

```

새로 추가된 기여자 필드는 ManyToManyField 필드 유형을 사용하여 Book과 Contributor 간의 다대다 관계를 설정합니다:

- `models.ManyToManyField`: 이것은 다대다 관계를 설정하기 위한 필드 유형입니다.
- `through`: 이것은 다대다 관계를 위한 특별한 필드 옵션입니다. 두 테이블 간에 다대다 관계가 있을 때, 관계에 대한 추가 정보를 저장하고 싶다면, 중간 테이블을 통해 관계를 설정하는 데 사용할 수 있습니다.

예를 들어, Book과 Contributor라는 두 개의 테이블이 있으며, 책에 대한 기여자 유형에 대한 정보를 저장해야 합니다. 예를 들어, 저자, 공동 저자 또는 편집자와 같은 기여자 유형을 저장합니다. 그러면 기여자 유형은 BookContributor라는 중간 테이블에 저장됩니다. BookContributor 테이블/모델은 다음과 같습니다. 이 모델을 `reviews/models.py`에 포함시키는 것을 잊지 마세요:

```

class BookContributor(models.Model):
    class ContributionRole(models.TextChoices):
        AUTHOR = "AUTHOR", "저자"
        CO_AUTHOR = "CO_AUTHOR", "공동 저자"
        EDITOR = "EDITOR", "편집자"

    book = models.ForeignKey(
        Book, on_delete=models.CASCADE)
    contributor = models.ForeignKey(
        Contributor, on_delete=models.CASCADE)
    role = models.CharField(
        verbose_name="이 기여자가 맡았던 역할"

```

```
" the book.", choices=ContributionRole.choices,  
max_length=20)
```

An intermediary table, such as `BookContributor`, establishes relationships by using foreign keys to both the `Book` and `Contributor` tables. It can also have extra fields that can store information about the relationship the `BookContributor` model has with the following fields:

- `book`: This is a foreign key to the `Book` model. As we saw previously, `on_delete=models.CASCADE` will delete an entry from the relationship table when the relevant book is deleted from the application.
- `Contributor`: Again, this is a foreign key to the `Contributor` model/table. This is also defined as `CASCADE` upon deletion.
- `role`: This is the field of the intermediary model, which stores the extra information about the relationship between `Book` and `Contributor`.
- `class ContributionRole(models.TextChoices)`: This can be used to define a set of choices by creating a subclass of `models.TextChoices`. For example, `ContributionRole` is a subclass created out of `TextChoices`, which is used by the `roles` field to define `Author`, `Co-Author`, and `Editor` as a set of choices.
- `choices`: This refers to a set of choices defined in the models, and they are useful when creating Django Forms using the models.

Note

When the `through` field option is not provided while establishing a many-to-many relationship, Django automatically creates an intermediary table to manage the relationship.

One-to-one relationships

In this relationship, one record in a table will have a reference to only one record in a different table. For example, a person can have only one driver's license, so a person with a driver's license could form a one-to-one relationship:

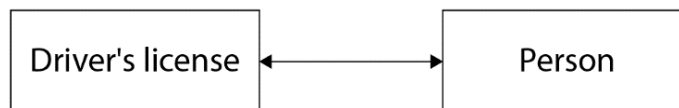


Figure 2.7: Example of a one-to-one relationship


```
" 책입니다.", choices=ContributionRole.choices,
max_length=20)
```

중간 테이블인 BookContributor는 외래 키를 사용하여 Book 및 Contributor 테이블 간의 관계를 설정합니다. 또한 BookContributor 모델이 다음 필드와의 관계에 대한 정보를 저장할 수 있는 추가 필드를 가질 수 있습니다:

- **book:** 이것은 Book 모델에 대한 idx_9e58fa18 외래 키입니다. 이전에 보았듯이, on_delete=models.CASCADE는 관련 책이 애플리케이션에서 삭제될 때 관계 테이블에서 항목을 삭제합니다.
- **기여자:** 다시 말해, 이것은 기여자 모델/테이블에 대한 외래 키입니다. 삭제 시 CASCADE로 정의됩니다.
- **역할:** 이것은 중개 모델의 필드로, 책과 기여자 간의 관계에 대한 추가 정보를 저장합니다.
- **class ContributionRole(models.TextChoices):** 이것은 models.TextChoices의 하위 클래스를 생성하여 선택 세트를 정의하는 데 사용될 수 있습니다. 예를 들어, ContributionRole은 TextChoices에서 생성된 하위 클래스로, 역할 필드에서 저자, 공동 저자 및 편집자를 선택 세트로 정의하는 데 사용됩니다.
- **선택:** 이것은 모델에서 정의된 선택 세트를 나타내며, 모델을 사용하여 Django 양식을 생성할 때 유용합니다.

참고

다대다 관계를 설정할 때 through 필드 옵션이 제공되지 않으면 Django는 관계를 관리하기 위해 자동으로 중개 테이블을 생성합니다.

일대일 관계

이 관계에서 테이블의 하나의 레코드는 다른 테이블의 하나의 레코드에만 참조를 가집니다. 예를 들어, 한 사람은 운전 면허증을 하나만 가질 수 있으므로, 운전 면허증을 가진 사람은 일대일 관계를 형성할 수 있습니다:



그림 2.7: 일대일 관계의 예

OneToOneField can be used to establish a one-to-one relationship, as shown here:

```
class DriverLicence(models.Model):
    person = models.OneToOneField(
        Person, on_delete=models.CASCADE)
    licence_number = models.CharField(max_length=50)
```

Now that we have explored database relationships, let's come back to our bookr application and add one more model there.

Adding the Review model

We've already added the Book and Publisher models to the reviews/models.py file. The last model that we are going to add is the Review model. The following code snippet should help us do this:

```
from django.contrib import auth
class Review(models.Model):
    content = models.TextField(
        help_text="The Review text.")
    rating = models.IntegerField(
        help_text="The rating the reviewer has given.")
    date_created = models.DateTimeField(
        auto_now_add=True,
        help_text="The date and time the"
        " review was created.")
    date_edited = models.DateTimeField(
        null=True,
        help_text="The date and time the review"
        " was last edited.")

    creator = models.ForeignKey(
        auth.get_user_model(), on_delete=models.CASCADE)
    book = models.ForeignKey(
        Book, on_delete=models.CASCADE,
        help_text="The Book that this review is for.")
```

OneToOneField는 일대일 관계를 설정하는 데 사용할 수 있습니다. 다음과 같이 보여집니다:

```
class DriverLicence(models.Model):
    person = models.OneToOneField(
        Person, on_delete=models.CASCADE)
    licence_number = models.CharField(max_length=50)
```

이제 데이터베이스 관계를 탐색했으니, bookr 애플리케이션으로 돌아가서 거기에 하나의 모델을 더 추가해 보겠습니다.

리뷰 모델 추가

우리는 이미 reviews/models.py 파일에 Book 및 Publisher 모델을 추가했습니다. 우리가 추가할 마지막 모델은 Review 모델입니다. 다음 코드 스니펫이 이를 도와줄 것입니다:

```
from django.contrib import auth
class Review(models.Model):
    content = models.TextField(
        help_text="리뷰 텍스트입니다.")
    rating = models.IntegerField(
        help_text="리뷰어가 준 평점입니다.")
    date_created = models.DateTimeField(
        auto_now_add=True,
        help_text="리뷰가 생성된 날짜와 시간입니다."
        " review was created.")
    date_edited = models.DateTimeField(
        null=True,
        help_text="리뷰가 마지막으로 수정된 날짜와 시간"
        "가 마지막으로 수정되었습니다.")

    creator = models.ForeignKey(
        auth.get_user_model(), on_delete=models.CASCADE)
    book = models.ForeignKey(
        Book, on_delete=models.CASCADE,
        help_text="이 리뷰가 해당하는 책입니다.")
```

The Review model/table will be used to store user-provided review comments and ratings for books. It has the following fields:

- **content:** This field stores the text for a book review; hence, the field type used is `TextField`, as this can store a large amount of text.
- **rating:** This field stores the review rating of a book. Since the rating will be an integer, the field type used is `IntegerField`.
- **date_created:** This field stores the time and date when the review was written; hence, the field type is `DateTimeField`.
- **date_edited:** This field stores the date and time whenever a review is edited. Again, the field type is `DateTimeField`.
- **creator:** This field specifies the review creator or the person who writes the book review. Notice that this is a foreign key to `auth.get_user_model()`, which refers to the `User` model from Django's built-in authentication module. It has a `on_delete=models.CASCADE` field option. This explains that when a user is deleted from the database, all the reviews written by that user will be deleted.
- **book:** Reviews have a field called `book`, a foreign key to the `Book` model. This is because reviews have to be written for a book review application, and a book can have many reviews, so this is a many-to-one relationship. This is also defined with an `on_delete=models.CASCADE` field option because once the book is deleted, there is no point in retaining the reviews in the application. So, when a book is deleted, all the reviews referring to the book will also get deleted.

In the next section, we will learn about and implement model methods.

Model methods

In Django, we can write methods inside a model class. These are called **model methods**, and they can be custom or special methods that override the default methods of Django models. One such method is `__str__()`. This method returns the string representation of the `Model` instances and can be especially useful while using the Django shell. In the following example, where the `__str__()` method is added to the `Publisher` model, the string representation of the `Publisher` object will be the publisher's name; hence, `self.name` is returned, with `self` referring to the `Publisher` object:

```
class Publisher(models.Model):
    """A company that publishes books."""
    name = models.CharField(
```

리뷰 모델/테이블은 사용자가 제공한 리뷰 댓글과 책에 대한 평가를 저장하는 데 사용됩니다. 다음과 같은 필드가 있습니다:

- **content**: 이 필드는 책 리뷰의 텍스트를 저장합니다. 따라서 사용되는 필드 유형은 `TextField`로, 이는 대량의 텍스트를 저장할 수 있습니다.
- **rating**: 이 필드는 책의 리뷰 평점을 저장합니다. 평점은 정수이므로 사용되는 필드 유형은 `IntegerField`입니다.
- **date_created**: 이 필드는 리뷰가 작성된 시간과 날짜를 저장합니다. 따라서 필드 유형은 `DateTimeField`입니다.
- **date_edited**: 이 필드는 리뷰가 편집될 때마다 날짜와 시간을 저장합니다. 다시 말해, 필드 유형은 `DateTimeField`입니다.
- **creator**: 이 필드는 리뷰 작성자 또는 책 리뷰를 작성하는 사람을 지정합니다. 이는 `auth.get_user_model()`에 대한 외래 키로, Django의 내장 인증 모듈의 `User` 모델을 참조합니다. 이 필드는 `on_delete=models.CASCADE` 필드 옵션을 가지고 있습니다. 이는 사용자가 데이터베이스에서 삭제될 때, 해당 사용자가 작성한 모든 `idx_8556afce` 리뷰가 삭제된다는 것을 설명합니다.
- **book**: 리뷰에는 `book`이라는 필드가 있으며, 이는 `Book` 모델에 대한 외래 키입니다. 이는 리뷰가 책 리뷰 애플리케이션을 위해 작성되어야 하며, 하나의 책이 여러 리뷰를 가질 수 있기 때문에 다대일 관계입니다. 또한, 책이 삭제되면 애플리케이션에서 리뷰를 유지할 이유가 없기 때문에 `on_delete=models.CASCADE` 필드 옵션으로 정의됩니다. 따라서 책이 삭제되면 해당 책을 참조하는 모든 리뷰도 삭제됩니다.

다음 섹션에서는 모델 메서드에 대해 배우고 구현할 것입니다.

모델 메서드

Django에서는 모델 `idx_ab5a7dbb` 클래스 내에 메서드 `idx_420e3af0`를 작성할 수 있습니다. 이를 모델 메서드라고 하며, Django 모델의 기본 메서드를 재정의하는 사용자 정의 또는 특수 메서드일 수 있습니다. 이러한 메서드 중 하나는 `__str__()`입니다. 이 메서드는 모델 인스턴스의 문자열 표현을 반환하며, Django 셸을 사용할 때 특히 유용할 수 있습니다. 다음 예제에서는 `__str__()` 메서드가 `Publisher` 모델에 추가되며, `Publisher` 객체의 문자열 표현은 출판사의 이름이 됩니다. 따라서 `self.name`이 반환되며, `self`는 `Publisher` 객체를 참조합니다:

```
class Publisher(models.Model):
    """책을 출판하는 회사입니다."""
    name = models.CharField(
```

```
        max_length=50,
        help_text="The name of the Publisher.")
website = models.URLField(
    help_text="The Publisher's website.")
email = models.EmailField(
    blank=True,
    help_text="The Publisher's email address.")

def __str__(self):
    return self.name
```

Add the `__str__()` methods to `Contributor` and `Book` as well, as follows:

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
        max_length=70, help_text="The title of the book.")
    publication_date = models.DateField(
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="ISBN number of the book.")
    publisher = models.ForeignKey(
        Publisher, on_delete=models.CASCADE)
    contributors = models.ManyToManyField(
        'Contributor', through="BookContributor")

    def __str__(self):
        return self.title

class Contributor(models.Model):
    """A contributor to a Book, e.g. author, editor,
    co-author."""
    first_names = models.CharField(
        max_length=50,
        help_text="The contributor's first name or names.")
    last_names = models.CharField(
        blank=True, max_length=50,
        help_text="The contributor's last name or names.")
    email = models.EmailField(
```

```

        max_length=50,
        help_text="출판사의 이름입니다.")
website = models.URLField(
    help_text="출판사의 웹사이트입니다.")
email = models.EmailField(
    blank=True,
    help_text="출판사의 이메일 주소입니다.")

def __str__(self):
    return self.name

```

Contributor와 Book에도 `_str_()` 메서드를 추가하세요, 다음과 같이:

```

class Book(models.Model):
    """출판된 책입니다."""
    title = models.CharField(
        max_length=70, help_text="책의 제목입니다.")
    publication_date = models.DateField(
        verbose_name="책이 출판된 날짜입니다.")
    isbn = models.CharField(
        blank=True, max_length=20,
        verbose_name="책의 ISBN 번호입니다.")
    publisher = models.ForeignKey(
        Publisher, on_delete=models.CASCADE)
    contributors = models.ManyToManyField(
        'Contributor', through="BookContributor")

    def __str__(self):
        return self.title

class Contributor(models.Model):
    """책의 기여자, 예: 저자, 편집자,
    공저자입니다."""
    first_names = models.CharField(
        max_length=50,
        help_text="기여자의 이름 또는 이름들입니다.")
    last_names = models.CharField(
        blank=True, max_length=50,
        help_text="기여자의 성 또는 성들입니다.")
    email = models.EmailField(

```

```
blank=True,  
help_text="The contact email for the contributor.")  
  
def __str__(self):  
    return self.first_names
```

Similarly, the string representation of book is title, so the returned value is `self.title`, with `self` referring to the Book object. The string representation of Contributor is the first name of the contributor; hence `self.first_names` is returned. Here, `self` is referring to the Contributor object. Next, we will look at migrating the reviews app.

Migrating the reviews app

Since we have the entire model file ready, let's now migrate the models into the database, similar to what we did before with the installed apps. Since the reviews app has a set of models created by us, it is important to create the migration scripts before running the migration. Migration scripts help in identifying any changes to the models and will propagate these changes into the database while running the migration. Follow these steps to create migration scripts and then migrate models into the database:

1. Execute the following command to create the migration scripts:

```
python manage.py makemigrations reviews
```

We should get an output similar to this:

```
reviews/migrations/0002_auto_20191007_0112.py  
- Create model Book  
- Create model Contributor  
- Create model Review  
- Create model BookContributor  
- Add field contributors to book  
- Add field publisher to book
```



```

        blank=True,
        help_text="기여자의 연락처 이메일입니다.")

    def __str__(self):
        return self.first_names

```

마찬가지로, 책의 문자열 표현 `idx_48b7376a`은 제목이므로 반환 값은 `self.title`이며, `self`는 `Book` 객체를 참조합니다. 기여자의 문자열 표현은 기여자의 이름이므로 `self.first_names`가 반환됩니다. 여기서 `self`는 `Contributor` 객체를 참조합니다. 다음으로, 리뷰 앱의 마이그레이션을 살펴보겠습니다.

리뷰 앱 마이그레이션

모델 파일이 준비되었으므로, 이제 설치된 앱과 유사하게 모델을 데이터베이스로 마이그레이션합니다. 리뷰 앱에는 우리가 만든 모델 세트가 있으므로, 마이그레이션을 실행하기 전에 마이그레이션 `idx_937e0fb2` 스크립트를 생성하는 것이 중요합니다. 마이그레이션 스크립트는 모델의 변경 사항을 식별하는 데 도움을 주며, 마이그레이션을 실행하는 동안 이러한 변경 사항을 데이터베이스에 전파합니다. 마이그레이션 스크립트를 생성하고 모델을 데이터베이스로 마이그레이션하기 위해 다음 단계를 따르십시오:

1. 마이그레이션 스크립트를 생성하기 위해 다음 명령을 실행하십시오:

```
python manage.py makemigrations reviews
```

우리는 이와 유사한 출력을 얻어야 합니다:

```

reviews/migrations/0002_auto_20191007_0112.py
- 모델 Book 생성
- 모델 Contributor 생성
- 모델 Review 생성
- 모델 BookContributor 생성
- 책에 contributors 필드 추가
- 책에 publisher 필드 추가

```

Migration scripts will be created in a folder named migrations in the application folder.

2. Next, migrate all the models into the database using the migrate command:

```
python manage.py migrate reviews
```

We should see the following output:

```
Operations to perform:
  Apply all migrations: reviews
Running migrations:
  Applying reviews.0002_auto_20191007_0112... OK
```

After executing this command, we successfully created the database tables defined in the reviews app. You may use DB Browser for SQLite to explore the tables you have just created after the migration.

3. To do so, open DB Browser for SQLite, click on the **Open Database** button (*Figure 2.17*), and navigate to your project directory.
4. Select the `db.sqlite3` database file to open it.

We should now be able to browse the new sets of tables created. The following screenshot shows the database tables defined in the reviews app:

마이그레이션 스크립트는 applicationfolder라는 폴더의 migrations라는 폴더에 생성됩니다.

2. 다음으로, migrate 명령어를 사용하여 모든 모델을 idx_d137cc5f 데이터베이스로 마이그레이션합니다:

```
python manage.py migrate reviews
```

다음과 같은 출력이 표시되어야 합니다:

```
수행할 작업:  
모든 마이그레이션 적용:  
reviews마이그레이션 실행 중:  
  
리뷰 적용 중.0002_auto_20191007_0112... 성공
```

이 명령을 실행한 후, 우리는 리뷰 앱에 정의된 데이터베이스 테이블을 성공적으로 생성했습니다. 마이그레이션 후 방금 생성한 테이블을 탐색하려면 DB Browser for SQLite를 사용할 수 있습니다.

3. 그렇게 하려면, DB Browser for SQLite를 열고, 데이터베이스 열기 버튼을 클릭한 후(그림 2.17), 프로젝트 디렉토리로 이동합니다.
4. 열기 위해 db.sqlite3 데이터베이스 파일을 선택합니다.

이제 생성된 새로운 테이블 세트를 탐색할 수 있어야 합니다. 다음 스크린샷은 리뷰 앱에 정의된 데이터베이스 테이블을 보여줍니다: